UF2405: Modelo de programación web y bases de datos

Presentación

Identificación de la Unidad Formativa:

Bienvenido a la Unidad Formativa **UF2405: Modelo de programación web y bases de datos**. Esta Unidad Formativa pertenece al Módulo Formativo **MF0227_3: Programación orientada a objetos** que forma parte del Certificado de Profesionalidad **IFCD0112: Programación de lenguajes orientados a objetos y bases de datos relacionales**, de la familia de **Informática y comunicaciones.**

Presentación de los contenidos:

La finalidad de esta Unidad Formativa es enseñar al alumno a manipular bases de datos a través de interfaces para integrar el lenguaje de programación con el lenguaje de acceso a datos en la construcción de una aplicación y a utilizar los componentes orientados a objeto como base en el desarrollo de aplicaciones para el modelo de programación web.

Para ello, se realizará una primera introducción al desarrollo de aplicaciones en el modelo de programación web, para después estudiar la arquitectura multicapa, la capa de presentación, el diseño de bases de datos relacionales, el acceso a bases de datos relacionales, los lenguajes de definición de datos y la manipulación de los datos.

Objetivos del Módulo Formativo:

Los objetivos generales del Módulo Formativo son:

– Implementar los componentes software encomendados de modo que cumplan las especificaciones del diseño y los niveles de calidad establecidos.

– Manipular bases de datos a través de interfaces para integrar el lenguaje de programación con el lenguaje de acceso a datos en la construcción de una aplicación.

– Probar los componentes software desarrollados para asegurar que cumplen las especificaciones recibidas.

– Utilizar los componentes orientados a objeto como base en el desarrollo de aplicaciones para el modelo de programación web.

– Elaborar la documentación del código desarrollado según los estándares de la organización.

Índice

UD1. Introduccion al desarrollo de aplicaciones en el modelo de programacion web 9

1.1. Análisis de la arquitectura web: Cliente ligero, servidor web, servidor de aplicaciones, servidor de datos..... 11

1.2. Enumeración de protocolos y tecnologías habituales..... 35

1.3. Análisis de los modelos de programación estándares de facto..... 52

1.4. Uso de componentes orientados a objetos como base de desarrollo de aplicaciones en el modelo de programación web..... 57

UD2. Arquitectura multicapa (N- tier) 75

2.1. Análisis de la arquitectura multicapa..... 77

2.2. Distinción y estudio del modelo de tres capa en Web: presentación, aplicación y datos..... 93

2.3. Diseño de arquitecturas de aplicación basadas en el modelo multicapa..... 101

2.4. Análisis del concepto de lógica de negocio y significado de la capa lógica 115

UD3. La capa de presentación 131

3.1. Descripción de la capa de presentación: El lenguaje de hipertexto 133

3.2. Descripción de la capa de presentación avanzada: El lenguaje de Scripting y lenguaje de hipertexto dinámico..... 147

3.3. Análisis de lenguajes orientados a la preparación de la capa de presentación y a la ejecución de solicitudes desde clientes ligeros Web. (JSP, SERVLETS, ASP, PHP) 163

UD4. Diseño de base de datos relacionales 177

4.1. Definición de bases de datos relacionales..... 179

4.2. Diseño de base de datos en varios niveles 195

4.3. Análisis de los distintos tipos de relaciones y su implementación en la base de datos 209

4.4. Descripción del lenguaje de acceso a base de datos 235

4.5. Descripción de correlaciones entre el modelo relacional y el modelo orientado a objetos 241

4.6. Nociones sobre el almacenamiento de objetos en las bases de datos relacionales 243

UD5. Acceso a bases de datos relacionales: capa de acceso a datos 261

5.1. Análisis del API de acceso a la base de datos..... 263

5.2. Nivel controlador 275

5.3. Interfaz de acceso a la base de datos (driver)..... 279

5.4. Análisis del nivel de aplicación 284

5.4.1. Establecimiento de la conexión con una base de datos..... 285

5.4.2. Operar sobre la base de datos. Sentencias del lenguaje de acceso a base de datos. Objetos que permiten ejecutar una consulta. Objetos que permiten manipular el resultado de una consulta..... 294

5.4.3. Integración de los tipos de datos propios del lenguaje de acceso de datos en el lenguaje de programación de la aplicación..... 307

5.4.4. Procedimientos almacenados..... 319

5.4.5. Transacciones distribuidas..... 326

UD6. Lenguajes de definición de datos 343

6.1. Conceptos básicos, nociones y estándares 345

6.2. Lenguaje de definición de datos (DDL) y aplicación en SGBD actuales 355

6.3. Discriminación de los elementos existentes en el estándar SQL-92 de otros elementos existentes en base de datos comerciales 365

6.4. Sentencias de creación. CREATE: 369
6.4.1. Bases de datos 370
6.4.2. Tablas 373
6.4.3. Vistas 385
6.4.4. Tiggers o disparadores 387
6.4.5. Procedimientos almacenados 391

6.5. Sentencia de modificación: ALTER 393
6.5.1. base de datos. 393
6.5.2. Tablas 395
6.5.3. Vistas 397
6.5.4. Disparadores o triggers 398
6.5.5. Procedimientos 399

6.6. Sentencias de borrado: DROP, TRUNCATE 400
6.6.1. Bases de datos. 401
6.6.2. Tablas 402
6.6.3. Vistas. 403
6.6.4. Disparadores o triggers 404
6.6.5. Procedimiento 405

UD7. Manipulacion de los datos 415

7.1. Lenguaje de manipulación de datos DML 417

7.2. Consulta de datos SELECT. 419

7.3. Inserción de datos INSERT 430

7.4. Modificación de datos UPDATE. 436

7.5. Eliminación de datos DELETE. 437

7.6. Agregación de conjuntos de datos para consulta: JOIN, UNION 441

7.7. Subconsultas. 447

Glosario 457

Soluciones 461

Anexo 463

UD1

Introduccion al desarrollo de aplicaciones en el modelo de programacion web

1.1. Análisis de la arquitectura web: Cliente ligero, servidor web, servidor de aplicaciones, servidor de datos

1.2. Enumeración de protocolos y tecnologías habituales

1.3. Análisis de los modelos de programación estándares de facto

1.4. Uso de componentes orientados a objetos como base de desarrollo de aplicaciones en el modelo de programación web

1.1. Análisis de la arquitectura web: Cliente ligero, servidor web, servidor de aplicaciones, servidor de datos

Antes de comenzar vamos ver un poco sobre la evolución de las aplicaciones web, las aplicaciones web se ido desarrollando más en las dos últimas décadas, en los principios de la informática se utilizaba maquinas muy grandes y eran maquinas muy costosas.

Esta maquinas eran de un tamaño muy considerable a parte era una maquinaria muy costosa, que solo lo podían tener grandes organizaciones gubernamentales. Estas máquinas no realizaban ninguna interacción con el usuario final, sino que la tarea se realizaba por medio de tarjetas perforadas, con esto la maquina podía realizar su tarea.

Este tipo de procesos podía llevar días hasta que recibiera el usuario la información resultante.

Fueron apareciendo diferentes tipos de máquinas, hasta llegar a los microcomputadores, estos utilizaban la tecnología de microchip, pero no estaban integradas en un microprocesador, esto redujo considerablemente los costos, se comercializaron, pero no tuvo mucho éxito ya que seguían siendo maquinas muy grandes.

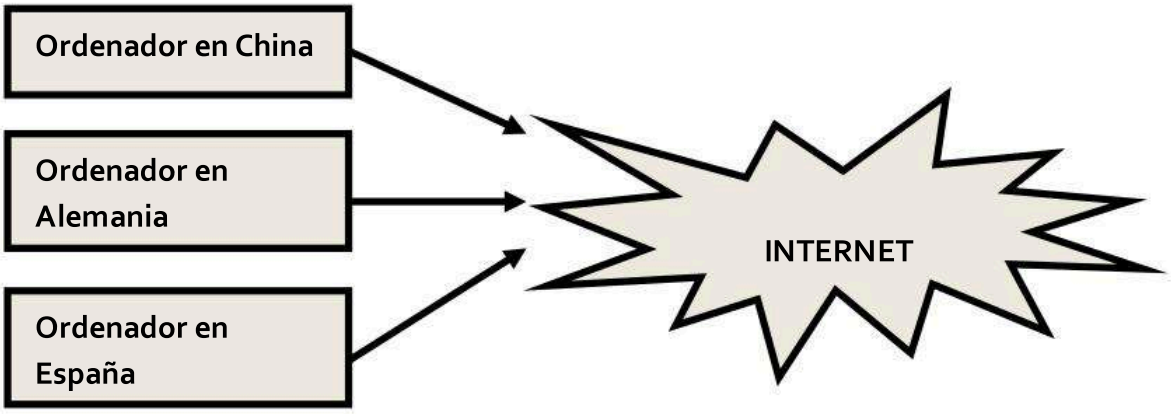
A partir de aquí fueron evolucionando los microprocesadores hasta las maquinas que hoy en día tenemos.



Hoy en día existen más de 1.00.00.00 millones de ordenadores en todo el mundo.

La aparición de Internet tuvo lugar cuando sintieron la necesidad de comunicación e interacción entre los usuarios.

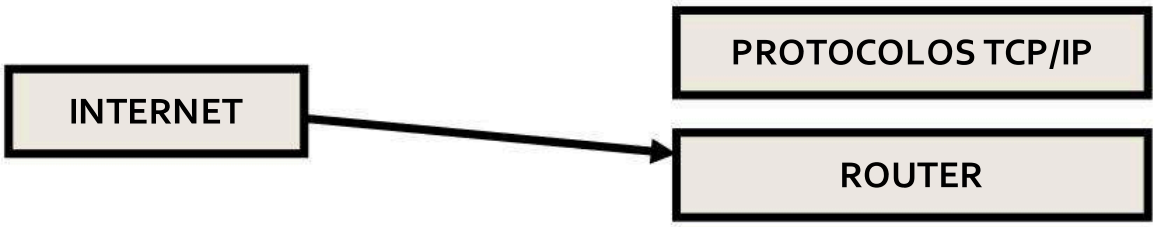
Internet apareció por primera vez en los años 50, desde sus principios ha estado evolucionando para satisfacer la gran demanda existente por parte de la población, tiene bastantes usos generalizado , proporciona que se puedan mantener relaciones sociales a larga distancia y poder obtener cualquier información aunque el usuario este en un sitio determinado y la información en otro.

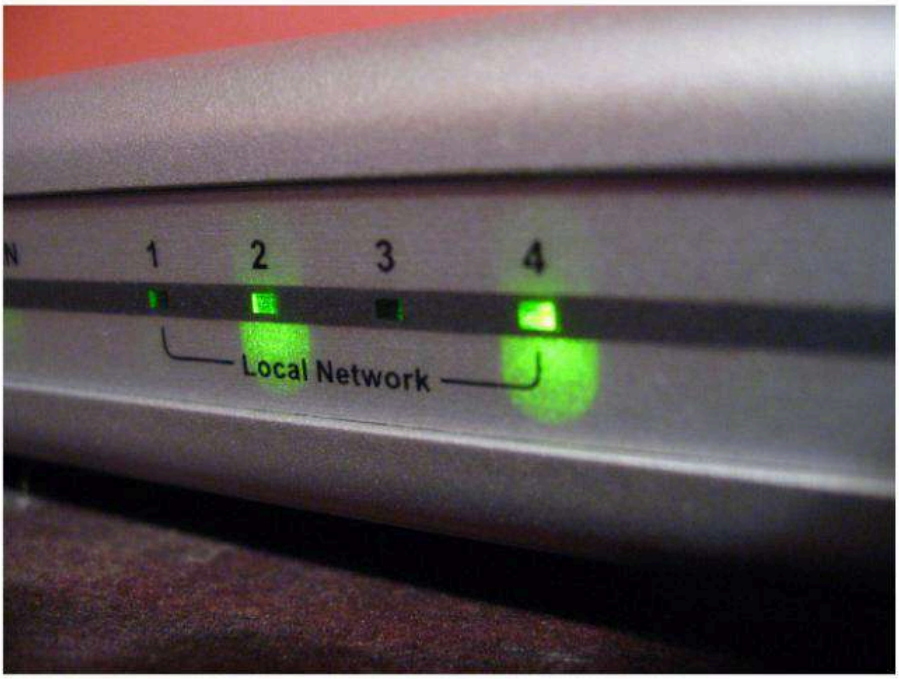


Internet fue evolucionando gracias al capitalismo, a los de sistemas de transporte y venta, con que se pudo realizar la comercialización de ordenadores a escala mundial.

A partir de aquí fueron introduciéndose las redes y aparecieron las primeras redes LAN o redes de área local, la más utilizada Ethernet.

La arquitectura de internet está compuesta por dos elementos muy importantes, uno era un grupo o conjunto de protocolos que son los llamados TCP/IP y un elemento que servía para acceder en diferentes redes, es lo que hoy denominamos Router, más tarde apareció el sistema de dominios, mediante el cual se podía saber el número de la dirección de un ordenador, y la introducción de hipervínculos.





Router

Ya en los años 80, llego la arquitectura de Internet World Wide Web, este tipo de arquitectura revoluciono la era de la tecnología e hizo que se pudiese realizar una infraestructura a nivel mundial.

Con la creación de WWW se empezaron a introducir un modelo en el que los usuarios podían elaborar páginas web de contenido de texto, en servidores y están paginas podían verse mediante un browser o ordenador, la web tendría mucha información en forma de texto y tendría hipervínculos a otras webs, así dio lugar las primeras red de conexiones de contenidos.

Con la llegado del siglo xx1, llego también la era de la comunicación por medio de redes sociales, otra forma de compartir información entre usuarios en línea.

Importante

Las aplicaciones web son todas apellas aplicaciones que son utilizadas por usuarios, accediendo a ellas a través de un servidor web mediante un navegador.

Se trata de un software elaborado en un lenguaje de programación que pueda ser soportado por los navegadores web.

Las aplicaciones web se han vuelto muy populares ya que hace que se utilice un tipo de cliente llamado cliente ligero (mas tarde hablaremos de el).

La aplicación web será totalmente independiente del sistema operativo que se utilice, además es muy fácil de mantener y realizar cambios sin tener que estar instalando software en ningún ordenador usuario.



Un navegador web es un software que se utiliza para acceder a internet. Un navegador va a permitir que un usuario puede visitar páginas web y realizar acciones en ella, como por ejemplo iniciar sesión, ver el contenido multimedia, enlazar con otras páginas, imprimir o enviar o recibir correos, entre otras.

Un navegador web puede recibir también el nombre de browser, los navegadores interpretan información de archivos y de sitios web para que estos puedan ser leídos.

Estos permitirán que se visualicen documentos de una página web y para ello se necesitara algunos recursos que estarán incrustados.

Los nombres de los navegadores más utilizados y comunes en el mercado son:

- Microsoft Internet Explorer.
- Mozilla FireFox.
- Safari.
- Opera.

La disponibilidad de estos navegadores dependerá del sistema operativo que se utilice en un ordenador.

Por ejemplo:

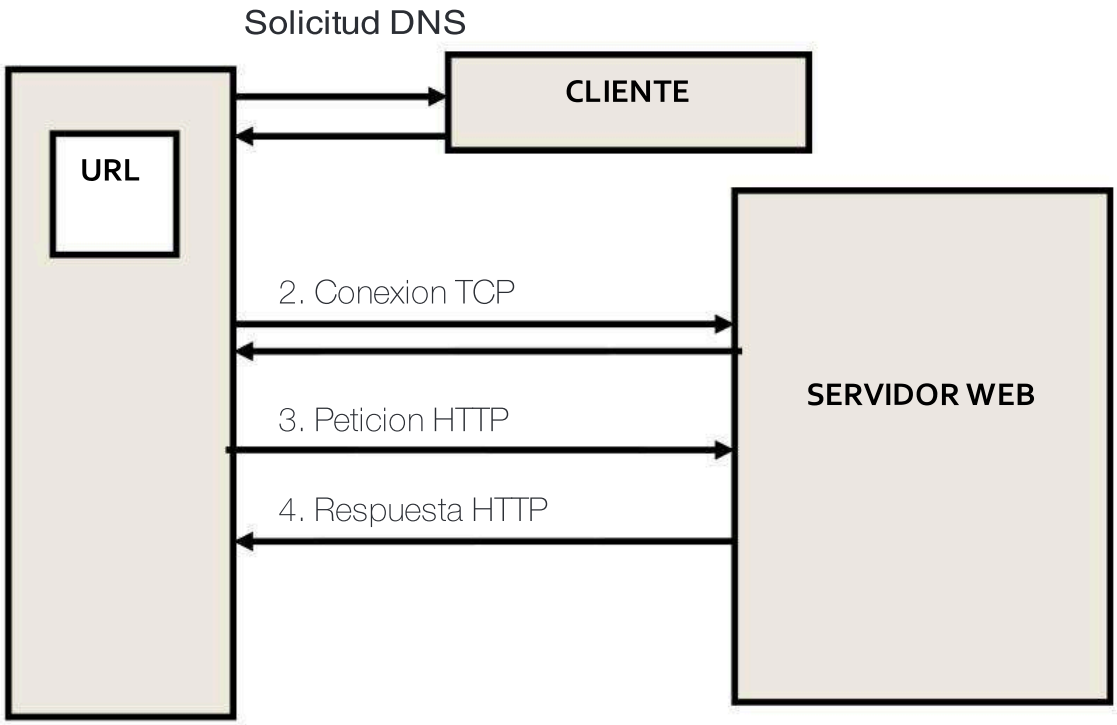
Microsoft Windows.
Linux.
Mac

Entre otros sistemas operativos que existen, pero estos son los más comunes.

Importante

El navegador es el que realiza las peticiones, a solicitud de un usuario, y además recibe, analiza y representa las respuestas.

Los pasos que se siguen son los siguientes:



Las funciones de los navegadores son:

- Construyen y mandan la petición HTTP.
- Reciben, interpretan y presentan la respuesta.
- Proporcionan el interfaz para conectarse y utilizar otros servicios: mail, news, ftp, etc.
- Caché local.
- Manejo de las Cookies.



Una página web, se define como un documento electrónico el cual contiene información en forma de texto, de elementos visuales y de elementos sonoros, que se encuentran alojados en un servidor y se pueden acceder a ellas por medio de un navegador.

Una página web forma parte de una colección de otras páginas webs dando lugar al denominado sitio web el cual se encuentra identificado bajo el nombre de un dominio.

Un a pagina web va a contener elementos que permiten la comunicación entre un usuario y la información. La pagina web responderá a todas las acciones realizadas por los usuarios, como puede ser rellenar un formulario.

Las aplicaciones web son unos elementos muy importantes en el día a día. El uso que se le vaya a dar es muy importante para el desarrollo del mismo, ya que según la dificultad en el uso de un usuario condicionara el buen desarrollo o no de la aplicación web.



La usabilidad es un factor muy importante en el desarrollo de aplicaciones web, ya que según la dificultad que experimenta el usuario, dependerá que la aplicación sea un fracaso o no.

Las aplicaciones web se han convertido en una herramienta básica para cualquier actividad empresarial y como intercambio de información, por eso se hace preciso emplear métodos que van a comprobar la usabilidad de una aplicación que son especificas para la web y que permiten el desarrollo correcto de estas aplicaciones web.

Las aplicaciones web generan de forma dinámica páginas web en formato estándar que es HTML, que es lenguaje que soportan los navegadores web.

Importante

Una aplicación web tiene una arquitectura simplificada, se trata de una arquitectura **CLIENTE/SERVIDOR,** en el que por un lado se encuentra el cliente que está constituido por browser web, que serán los encargados tanto de mostrar como de solicitar información o documentos a través de una red.

El cliente puede contener aplicaciones del cual puede usar una parte como presentación.



Diagrama CLINETE/SERVDOR

Importante

El Cliente.

Es una aplicación ejecutable que permitirá el acceso a internet y serán capaces de mostrar y solicitar documentos sobre una red.

Suelen ser las computadoras o cualquier otro dispositivo móvil que sirva como soporte para la entrada y salida de datos. Se puede decir que el cliente es el consumidor de un servicio.

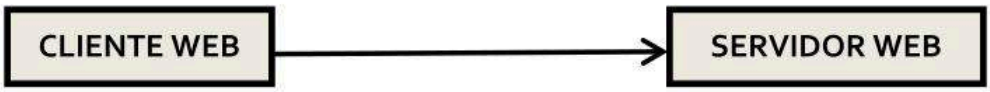
Importante

Son los ordenadores o computadoras o cualquier otro dispositivo móvil en el que se pueda instalar la aplicación, que esté en red que y tenga acceso a internet, ya que se tiene que tener acceso a un navegador web, que serán utilizados por los usuarios

Todo el procesamiento de las aplicaciones y la gestión de los datos se llevan a cabo en el servidor.

El cliente simplemente es responsable de la capa de presentación del software.

Cliente solicita petición



En el otro lado se la arquitectura web se encuentra el servidor web, la función más importante que cumple un servidor web es la de atender los pedidos del cliente web, el cual obtendrá información que tendrá almacenados en un sistema de archivos que estar instalado en la plataforma.

Una característica muy importante de la arquitectura web es que fue diseñado para funcionar con todo tipo de estructuras orientadas a internet.

Importante

Servidores Web.

Los servidores web son aquellos cuya tarea es alojar sitios web y aplicaciones web, las cuales son accedidas por los clientes utilizando un navegador web que se comunica con el servidor por medio de un protocolo llamado HTTP.

Básicamente un servidor web consta de un intérprete HTTP, el cual se mantiene a la espera de las peticiones de los clientes y le responde con el contenido de la información deseada por el cliente.

El cliente una vez que ha recibido el código, lo interpreta y lo refleja visualmente en la pantalla.

Además los servidores pueden disponer de un intérprete de otros lenguajes de programación que ejecuten código embebido dentro del código HTML de las paginas que contiene el sitio antes de enviar el resultado al cliente.

Esto se conoce como programación del lado del servidor y utiliza lenguajes de programación como:

- ASP.
- PHP.
- PERL.
- AJAX.

Las ventajas de utilizar lenguajes radica en la potencia de los mismos ejecutando tareas mas complejas, como por ejemplo acceder a una base de datos.



Un servidor web es un tipo de software que se instala y se ejecuta constantemente en un ordenador, todo el día los 365 días del año, se mantiene a las espera de las peticiones del cliente.

La principal función de un servidor Web es almacenar los archivos de un sitio y emitirlos por Internet para poder ser visitado por los usuarios.

Básicamente, un servidor Web es una gran computadora que guarda y transmite datos vía Internet. Cuando un usuario entra en una página de Internet su navegador se comunica con el servidor enviando y recibiendo datos que determinan qué es lo que ve en la pantalla.

Por eso decimos que los servidores Web están para almacenar y transmitir datos de un sitio según lo que pida el navegador de un visitante.

Hoy en día también se utiliza la definición de servidor web para referirse al ordenador en el cual funciona este software, su función principal será la de administrar los datos a otros ordenadores Ciente donde los utilizaran.

El servidor web, es el que pone a disposición contenido estático en un navegador utilizando el protocolo HTTP para enviar o cargar los archivos por medio de la red de navegación del usuario que lo solicita.

Existen varios tipos de servidores, pero lo más utilizados por los usuarios de internet son: los servidores de correo, servidores de videos, los servidores web y los servidores de base de datos.

> *Para más información, consulta "Tipos de servidores " en los anexos al final del libro.*

Recuerda

El servidor web siempre se estará ejecutando en un ordenador e ira realizando todas las peticiones de los usuarios.

En él se almacenan documentos HTML, archivos, imágenes, videos, datos, etc.

Servidor da respuesta a la petición



Importante

La web se compone de una gran cantidad de ordenadores que interactúan entre si gracias a internet, que sirve como un espacio integral en el que se permite una comunicación internacional de intercambio de información entre diversos usuarios.

La arquitectura web por todo esto es considerado un ambiente multiplataforma y distribuido.

Recuerda

Una web sigue las siguientes pautas:

- Como se realiza la transferencia de la información.
- Como se describe la información:
- Como se localiza la información.

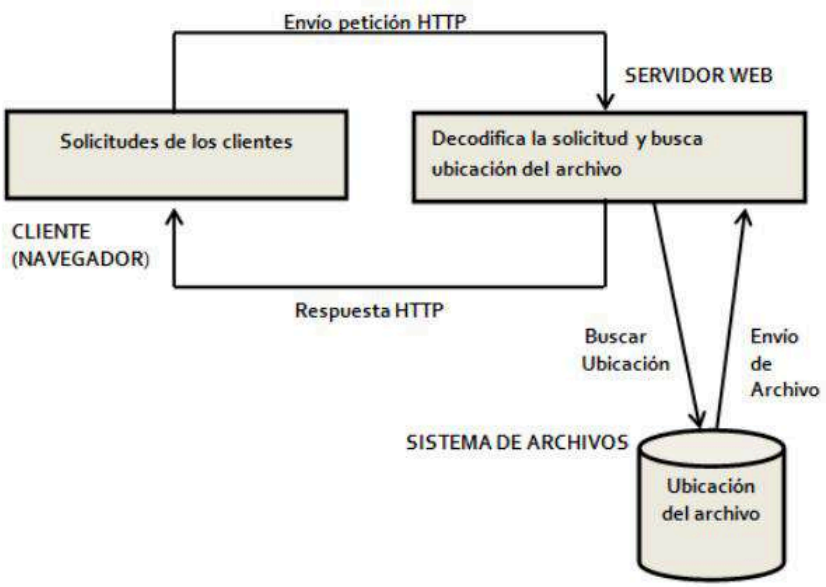
- **Como se realiza la transferencia de la información.**

  Para transferir la información desde un servidor web hacía un cliente web se utiliza un protocolo llamado HTTP

  Este protocolo es el más utilizado en internet su siglas provienen del inglés Hiper-Text Transfer Protocol, tiene como misión principal la de transferir datos a través de la red de internet.

  Principalmente datos de páginas web que hayan sido escritas en HTML, entre un navegar que sería el cliente y un servidor web que estará ubicado por medio de una dirección web URL.

  Este es un esquema de cómo se realiza la transferencia de archivos entre el navegador o cliente y el servidor web.



Cuando se realiza una solicitud HTTP, se determina qué tipo de documento es, el protocolo que se va a utilizar y el método de cómo se va a realizar.

Este protocolo se caracteriza por ser un protocolo abierto y cualificado para soportar un sistema distribuido web.

- **Como se describe la información:**



Para la presentación de la información web se utiliza el lenguaje HTML.

Este lenguaje consigue organizar los documentos web, es un lenguaje de hipervínculos y usando esos hipervínculos nos puede trasladar hacia otros documentos y demás recursos de internet.

Este lenguaje está constituido por un conjunto de etiquetas que marcaran el inicio y fin del documento. El documento de hipertexto es un documento multimedia ya que utiliza imágenes, videos, audio, etc.

Los navegadores acceden a visualizar una página web si estas son HTML.

- **Como se localiza la información.**



Para poder localizar documentos web que formen parte de internet se utiliza la URL.

Son las siglas de localizador de recurso uniforme, es una dirección general de documentos web y de diversos recursos en la Word Wide Web.

La dirección URL se divide en tres partes: en uno se pondrá el protocolo, en otra la ruta del servidor y por último el nombre del dominio.

En protocolo se pondrá el protocolo para la realización de transferencia de datos que será HTTP seguido de "://" + La ruta del servidor corresponderá con la ubicación del archivo que se ha solicitado por medio de un usuario que está en un

ordenador+ el nombre de dominio, se refiere a la IP, que será un número identificador único de un ordenador.

HTTP://ruta del servidor+ ubicación+ dominio

Que pueden ser:

Dominios	Significado
– .com	– Que se refiere a empresas y compañías.
– .ed	– Corresponderá a instituciones
– .org	– Que corresponderá a organizaciones.
– .gv	– Que corresponderá con una entidad gubernamental

También existen según el país:

Dominios	Significado
– .es	– España.
– .it	– Italia.
– .fr	– Francia

Un ejemplo de URL seria, http://www.hotmail.com.

Las primeras páginas webs que se construyeron, el contenido que formaba parte de ella, eran contenidos estáticos.

Con contenido estático nos referimos a que las páginas que se subían a Internet, tenían un contenido que permanecía sin modificaciones, una vez que se subía ya no se podía modificar, con lo que surge la necesidad de que por medio de una página web se pudieran realizar más servicios que no fueran solo de consulta, para ello se fueron creando soluciones tecnológicas, como fue las CGI.

El CGI es un tipo de tecnología utilizada en los servidores web, en inglés es Comon Gategay Interface. Es una de las primeras tecnologías utilizadas de programación web dinámica.

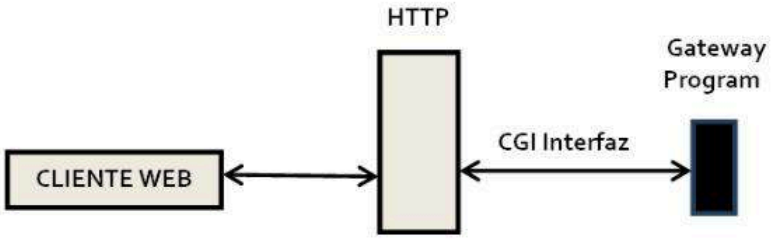
Con la introducción de las CGI cambio por completo la forma de manipular la información, las CGI en si, son métodos para facilitar la transmisión de información hacia un compilador instalado en un servidor.

Su función principal será la de añadir mas interacción a los documentos web por medio del lenguaje HTML se presentan de forma estática.

En los inicios de World Wide Web solo se podía ver texto, imágenes y enlaces, más tarde se introdujeron los pluggins en los navegadores, con lo que se daba mayor interacción entre el usuario y el cliente, pero el problema surgía en que se tenía que instalar los pluggins y en que la velocidad era bastante limitada.

Con el surgimiento de CGi, se da un cambio en la manera de manejar la información de una web. Es un método que se utiliza para la transferencia de información por medio de una petición HTTP, a la que pueden acompañar datos llegados a partir de una URL o a partir de un formulario.

El servidor será el que se ocupe de ejecutar el CGI, y a través del obtendrá todos los recursos necesarios para poder llevar a cabo la petición, para ello escribirá el resultado con etiquetas HTML, que es el lenguaje utilizado por las páginas web.



En la actualidad se ha quedado obsoleto, por su dificultad en su desarrollo y la pesada carga que tiene que soportar el servidor.

Importante

La web se debe de tener en cuenta a la hora de realizar aplicaciones web con acceso a base de datos.

La arquitectura web intenta dar solución a las necesidades que están asociadas a los negocios y a los clientes de esos negocios.

Es una mezcla de acciones dirigidas al desarrollo de páginas web. Hoy en día las páginas web están formadas por muchos elementos que facilitan la comunicación con el usuario por medio de formularios, bases de datos, etc.

La misión principal de una aplicación web en red va a ser la de desarrollar ayudas para satisfacer todas las necesidades de los negocios, como puede ser la venta de productos a través de internet.

Para llevar a cabo una aplicación web orientada al negocio, debe de diseñarse por expertos en lenguajes de programación y expertos en lenguajes de bases de datos.

La aplicación web es un sistema distribuido, ya que está formado por ordenadores que están conectados entre si por medio de una red, por la cual se comunican mediante un protocolo en un esquema Cliente/Servidor.



Se define **la arquitectura web**, como la combinación de sistemas que colaboran entre sí para dar como resultado información a los usuarios, con independencia de donde esté ubicada dicha información.

El modelo cliente /servidor es una arquitectura de software que involucra a uno o mas clientes que se encuentran solicitando servicios a uno o mas servidores. El cliente puede ser un proceso corriendo en un ordenador o cualquier dispositivo móvil que tenga acceso a internet.

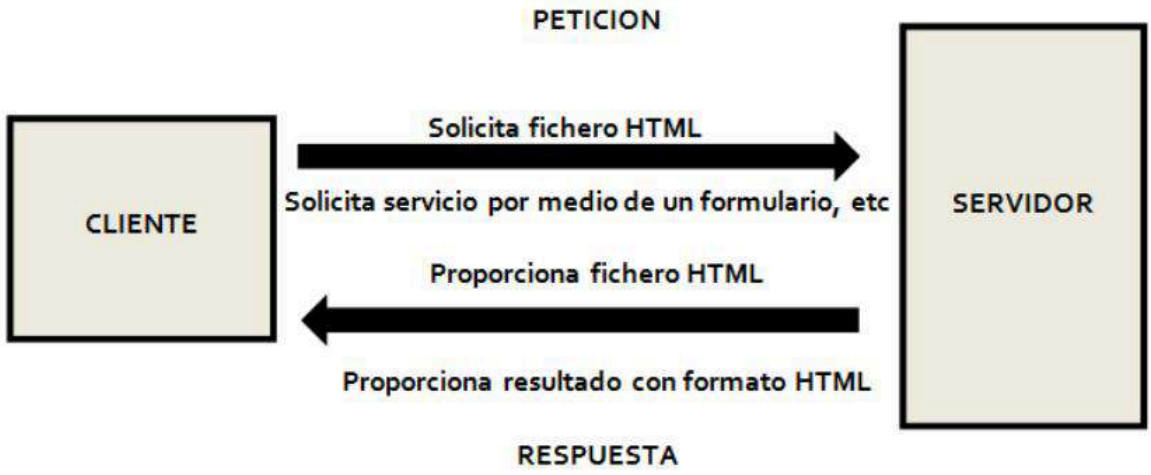
El servidor puede ser un proceso corriendo en un ordenador, en la arquitectura web actual podemos encontrar también elementos que se sitúan en medio como pueden ser proxis o cache.

Los beneficios que aporta una arquitectura web son:

- Usabilidad.
- Flexibilidad.
- Interoperabilidad.
- Escalabilidad.

Recuerda

Los elementos que forman **una arquitectura Cliente/Servidor**, son, como su propio nombre indica, el Cliente y el Servidor. Es un conjunto de componentes, que siguiendo una serie de reglas y procesos, permitirán utilizar una gran variedad de servicios informáticos que serán utilizados por una organización o empresa para un mejor rendimiento.



Recuerda

La arquitectura web utiliza conocimientos, funciones y diseño para páginas web
La construcción de una página web es un proceso muy complejo, se compone de varios sistemas relacionados entre sí como servidores, bases de datos, información, etc.

Para poder comprender en qué consiste una arquitectura web, debemos empezar explicando la arquitectura web básica, que se compone de un cliente y un servidor.

El modelo es el llamado Cliente/Servidor.

Este tipo de arquitectura consiste en la petición por parte del cliente, de una información o datos al servidor, que va a ser quien va a dar la respuesta.

Las aplicaciones web utilizan una arquitectura cliente/ servidor, esta arquitectura consta de tres capas o niveles, pero están capas o niveles se pueden encontrar tanto en un solo ordenador, como repartidas en varios ordenadores.

Lo mejor es tenerlo repartidos y así cada uno se ocupara de unas funciones determinadas.



Tipos de arquitectura Cliente/Servidor son:

- Arquitectura monocapa.
- Arquitectura Cliente/Servidor de dos capas.
- Arquitectura Cliente/Servidor de tres capas.
- Arquitectura n capas.

- La arquitectura monocapa:

  Es la estructura mas básica, en este tipo de arquitectura encontramos , en el que todas las partes involucradas se encuentran en un solo ordenador.

- Arquitectura Cliente/Servidor de dos capas:

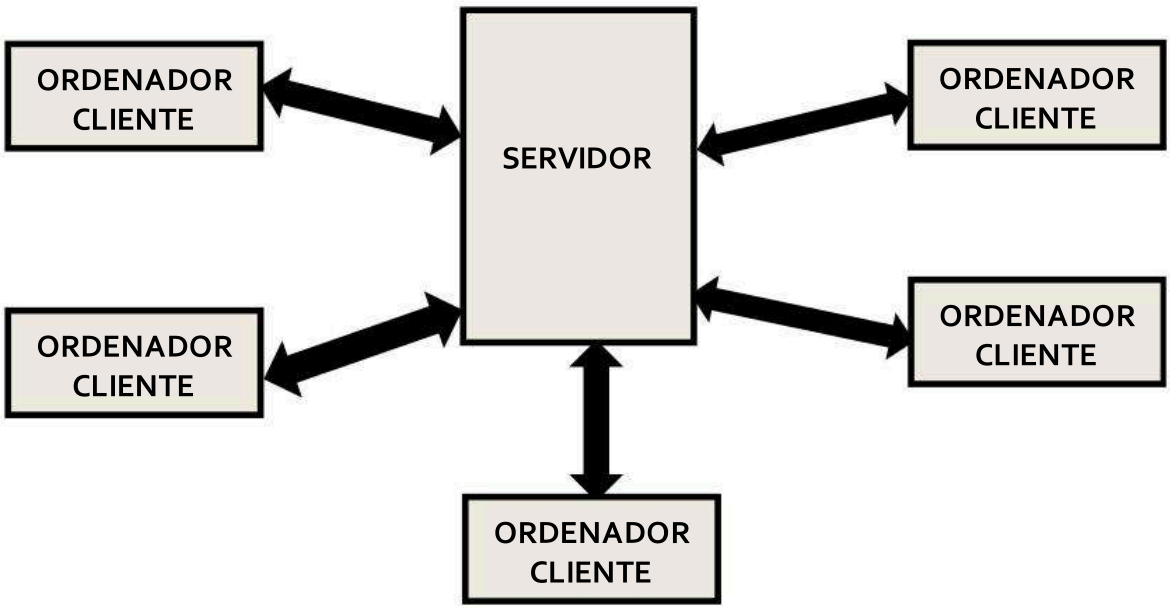
  Este tipo de arquitectura es la básica, está formado por un navegador en la parte del cliente y un servidor web en la parte del servidor, también se necesitara una red a la que conectarse.

  La arquitectura Cliente/Servidor, se divide en dos capas una la del cliente que implementa la interface y otra es donde se encuentra el sistema gestor de base de datos.

  Se compone de capa de presentación y lógica + capa de la base de datos. Este tipo de estructura de utiliza en los siguientes casos.

Pocos datos a procesar	base de datos estatica	Mantenimiento minimo	Centralizacion en servidor de Base de Datos

Estructura de una arquitectura básica Cliente/Servidor.



– Arquitectura Cliente/Servidor de tres capas:

Este tipo de arquitectura se basa en el modelo Cliente/Servidor, pero además se le añade una capa más entre el cliente y el servidor, que será un servidor de aplicaciones, que es donde se encontrara la lógica de la aplicación.

Se compone de capa de presentación, capa lógica y capa de la base de datos.

Recuerda

El principal objetivo que se intenta conseguir incorporando una arquitectura de tres capas es separar la capa del usuario (presentación), la capa de aplicación de negocio (capa lógica de negocio) y la capa de datos (capa de base de datos).



De esta manera el cliente es simplemente un interface, en la que si se cambia algo en la aplicación o en la base de datos, no cambia nada en el cliente.

Este tipo de arquitectura se utiliza en los siguientes casos:

Muchos datos a procesar	Procesos no relacionados con datos.
Separar codigo cliente para mantenimiento	Utilizar tecnologia orientada a objetos
Cambio en la funcion de las aplicaciones.	Aislar base de datos

Los usuarios que utilizan esta aplicación Cliente lo hacen por medio de un **navegador**, como por ejemplo Internet Explorer o Mozilla Firefox.



.Los usuarios serán los responsables de obtener, instalar y mantener el software que va a necesitar.

Este ordenador o dispositivos contendrán su propio sistema operativo, su propio sistema de entrada y salida de datos y sus propias conexiones de red.

Contendrá un software especial para poder conectar a través de una red de comunicación con el servidor.

El cliente es el que solicita una información o servicio a un servidor, y el servidor es que da respuesta a la solicitud del cliente.

Existen tres tipos de clientes:

- **Cliente hibrido**
- **Cliente ligero**
- **Cliente pesado**

- **Cliente pesado**, es cuando el cliente soporta la mayor parte de la lógica de la aplicación, realiza tareas de procesamiento de datos de usuarios y requiere mucha capacidad de almacenamiento.

- **Cliente ligero,** es cuando el cliente no tiene que soportar la lógica de la aplicación, simplemente será el intermediario entre el usuario y el servidor por medio de entrada de datos que serán luego enviados al servidor.

- **Cliente hibrido**, es cuando la parte lógica de la aplicación es compartida entre el cliente y el servidor.

La arquitectura de una aplicación web tiene sus cimientos en el modelo cliente servidor.

- **Servidores de Aplicaciones.**

  Con el paso del tiempo el uso de servidores Web se generalizó y se hizo necesario incrementar los servicios ofrecidos. El primer paso fue mejorar la eficiencia en el proceso de construcción de información dinámica y de hay fueron evolucionando hasta que dio lugar a los servidores de aplicaciones.

  Casi todos los servidores web que existen hoy en día son también servidores de aplicaciones, ya que incluyen todas las tecnologías que van a permitir crear aplicaciones que generan contenido dinámico.



Los servidores de aplicaciones son los servidores web de nueva generación. Se denomina servidores de aplicaciones a un software que proporciona servicio de aplicación a los clientes.

Este tipo de servidor, se suele asociar con organizaciones o empresas que necesitan servidores de alto rendimiento, que sea de alto rendimiento viene definido por ejemplo, por una afluencia masiva de visitas, por un mayor número de aplicaciones, etc.

Por ejemplo:

En un ambiente multimedia el dispositivo para escuchar música, sería el cliente y el audio, es distribuido por el servidor

El servidor de aplicación, es una aplicación que se puede utilizar de forma sencilla y eficiente por los usuarios de una organización o empresa. Además, este tipo de servidores proporciona una estructura en tres capas con lo que el sistema será aún más eficiente.

Estas aplicaciones que se utilizan en el servidor de aplicaciones, suelen ser creadas en Java, ya que son las más fáciles de instalar en cualquier computadora u ordenador, se puede crear por ejemplo con NET.framework 3.0.



Una aplicación web es la que es capaz de ofrecer al usuario una interfaz que sirva para que el usuario pueda trabajar sobre la lógica de negocio de un servidor.

La interfaz web tiene ciertas limitaciones para el usuario, los desarrolladores de aplicaciones web utilizan scripts para eliminar estas limitaciones, así será mas interactiva para el usuario.

En los últimos años se han desarrollados algunas tecnologías para coordinar los lenguajes que servirán para la creación de la interfaz del cliente y para la creación de la aplicación en el servidor.

Entendemos como lógica de negocios a una serie de procesos que se producen en un sistema que se encargara de tareas relacionadas con un negocio, como por ejemplo, las ventas de un producto

Este ejemplo consistirá, en que un usuario puede entrar en una página web para comprar un producto, cuando accedemos a aplicaciones de este tipo casi siempre nos aparece un carrito de la compra en la vamos poniendo los productos y podemos ver cómo va el proceso de envío.

Este tipo de aplicaciones que dejan interactuar con el usuario y en los que sufre modificaciones en las bases de datos a la que está conectada, recibe el nombre de aplicación web.

Tras los avances tecnológicos que se han producido en la arquitectura de las aplicaciones web, han dado como resultado la introducción de nuevos elementos, como los servidores de aplicaciones, la utilización de bases de datos, etc.

Con lo que da lugar a un avance de la arquitectura hacia una arquitectura basada en aplicaciones distribuidas.



La característica más importante en las aplicaciones web, es hacer que el cliente no realice demasiadas tareas, que el que se encargue de realizar estas operaciones de mayor importancia como son almacenar datos, Implantar reglas de negocio y la lógica de la aplicación, sea el Servidor. El tipo cliente que se utilizara será el cliente ligero.

Esquema ventajas de utilizar un cliente ligero:

Ventaja	Descripción
Información centralizada	La información se encuentra en un solo lugar, con lo que facilita la realización de copias de seguridad y se evita que se guarden archivos que no van a ser útiles para la empresa.
Menos costes en administración	Al administrar los Clientes ligeros desde servidor, hace que haya menos problemas de hardware y el acceso es restringido por lo que protegerá de malware.
Menos Coste de Hardware	El hardware de los clientes es mucho más barato ya que no va a necesitar por ejemplo tanta capacidad de disco duro, o memoria RAM para las aplicaciones o un procesador fuerte.
Consumo energético	El cliente ligero tiene un consumo eléctrico mucho menor, ya que no tienen que estar conectador las 24 horas del día, como para en el servidor.
Más Seguro	El cliente ligero será más seguro, ya que no contiene ninguna información de las aplicaciones.



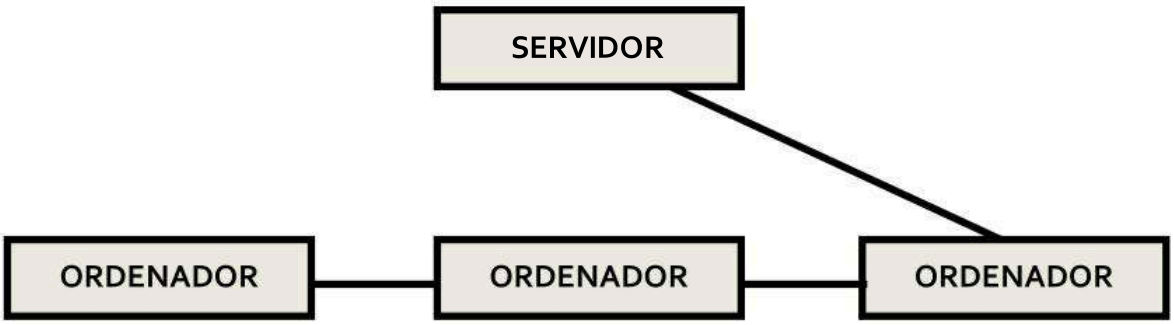
El servidor. La función del servidor va a ser la de atender las peticiones del cliente, dándole como respuesta al cliente, documentos que estarán almacenados en un sistema de archivos desde donde se encuentre ubicado.

Se puede decir que el servidor es el proveedor del servicio.

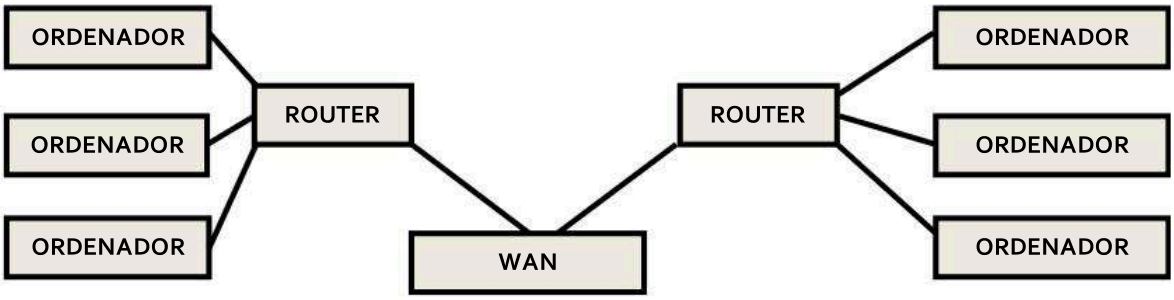
Se dice que es cualquier recurso que este destinado a responder a las necesidades del cliente.

Los servidores se conectan a los clientes por medio de redes LAN O WANs, y servirá de proveedor de multitud de funciones a los clientes y usuarios como puede ser, la impresión de un documento, tratamiento de imágenes, acceso a base de datos, etc.

Una red LAN, seria la siguiente:



Una red WAN seria:



Los clientes y los servidores pueden estar conectados en una red local (LAN) o una red más amplia (WANs), como la que se implementa en una empresa.

El concepto de arquitectura web, ha evolucionado mucho más en los últimos años gracias a internet, ofreciendo la oportunidad de interactuar mediante las aplicaciones a través de los ordenadores u otros dispositivos móviles.

Gracias a internet, la arquitectura web ha adquirido más relevancia y se ha extendido, ya que gracias a ello puede ser utilizado por cualquier usuario dependiendo del lugar donde se encuentre, solo tiene que estar conectado a internet.

A la hora de crear una página web, se debe de tener en consideración el servidor donde estará alojada.

Una página web que se aloja en un servidor tiene una URL o localizador en inglés (Uniform Resource Locator) ,esta URL será el método de entrada en la página web por parte de los usuarios..

Según el proyecto, se debe de elegir un servicio de alojamiento, existen varios sitios gratuitos que a cambio ponen publicidad en la página, o también se podrán utilizar los alojamientos de pago.

Se debe tener en cuenta también el espacio que se va a ocupar, según sea mayor o menor la transferencia de datos entre el Cliente y el Servidor, estoy influirá bastante en la velocidad de descarga.

Otro punto a tener en cuenta es, como se accede al servidor, el más utilizado es el **FTP** en inglés (File Transfer Protocol). Se trata de un protocolo para la transferencia de archivos.

Además, debemos de tener presente que los alojamientos gratuitos ofrecen los dominios o dirección de la página web incluyendo su propia dirección, se intentara pedir un dominio propio, esto ayudara a que los usuarios recuerden mejor la página, además da personalidad a la página. Si no siempre quedara la opción de pago.

La URL de la página web está compuesto por:

Ventaja	Descripción
WWW	Estas siglas identifica que se va a conectarte a una página web
Nombre de la pagina	URL o nombre de la página web
Tipo	Extensión según el tipo de finalidad de la página. Tipos más comunes:.com. Paginas comerciales..Org. Páginas de organizaciones no gubernamentales..net. Páginas de internet y telecomunicaciones..gov. Paginas gubernamentales..edu. Paginas educativas.

Estructura:

WWW.”NOMBREDELAPAGINA”.TIPO.



Servidor de datos o archivo.

El servidor de archivo funciona como los demás servidores, sigue siendo una aplicación instalada en un ordenador y que servirá para almacenar y compartir archivos con otros usuarios de la red.

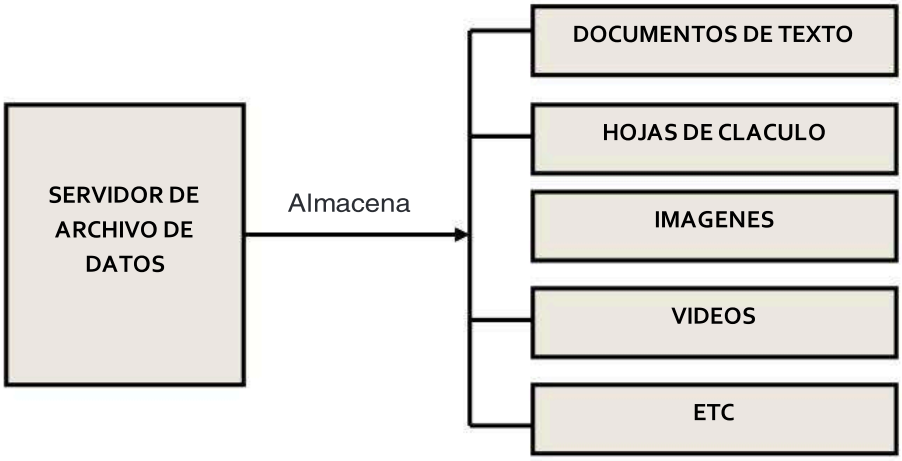
Así pues podemos definir el **servidor de datos** como el servidor que va a autorizar el acceso remoto a todos los archivos almacenados en él. La información que almacena en este servidor serán, documentos de texto, hojas de cálculo, imágenes, videos, etc.

Los servidores de bases de datos tienen su origen en la época de los 80 cuando surgió la necesidad de las grandes empresas de trabajar con gran cantidad y complejos volúmenes de datos.

Los servidores de datos pueden compartir información con muchos usuarios clientes, estos pueden ser usuarios por medio de un navegador web hasta servidores de aplicaciones, provee de gran seguridad y servicios independientemente de la plataforma que se utilice.

Para bases de datos que necesitan que se conectes gran cantidad de usuarios, es muy efectivo ponerlo en un servidor de datos.

Un servidor de datos puede ser cualquier ordenador que esté conectado en red y que tenga un software apropiado.



1.2. Enumeración de protocolos y tecnologías habituales

Importante

En primer lugar vamos a definir el concepto de protocolo, **un protocolo** desde el punto de vista informático, son una **agrupación de reglas y normas** que va a permitir que exista comunicación con una o más entidades, y se produzca la transmisión de información de unas a otras., en una red.

Desde el punto de vista del usuario, lo único que necesita saber, es que para que tú ordenador o cualquier otro dispositivo se comunique con otros ordenadores es necesaria la aplicación de los protocolos.

El protocolo más utilizado es:

Recuerda

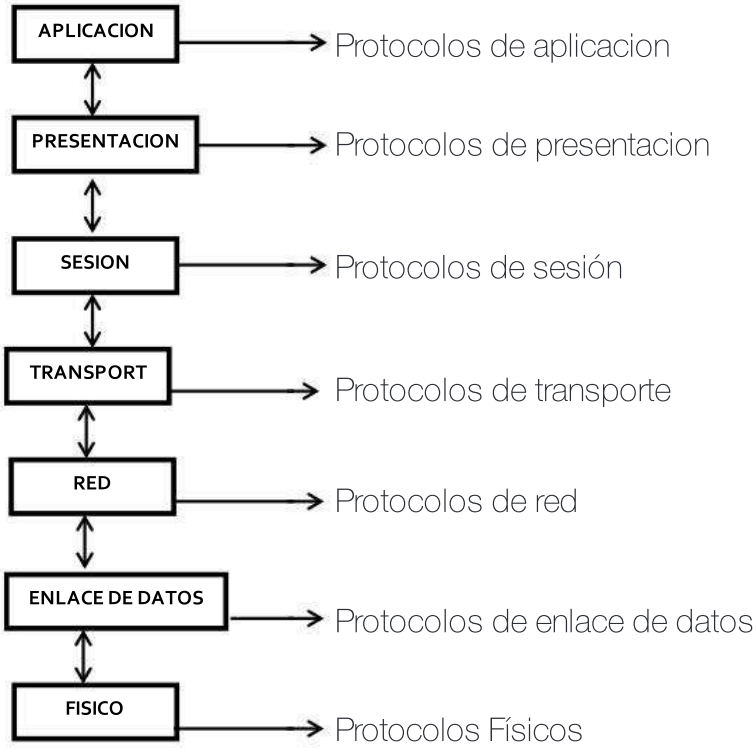
TCP/IP. Es el protocolo más básico de internet, suministra todos los servicios necesarios para el transporte de los datos. Cualquier protocolo de internet se basa en IP.

Los protocolos son un tipo de software que actúa en la red para poder realizar la comunicación de satos. Los ordenadores para comunicarse entre sí por medio de una red, deberán de utilizar el mismo protocolo.

Existen muchísimos protocolos de red, todos facilitan el acceso a una red, pero cada uno de ellos tienen funciones diferentes, ya que la función de un protocolo depende del modelo estándar de redes que es el modelo OSI.

OSI, Sus siglas significan interconexión de sistemas abiertos, según este modelo los protocolos se dividen en 7 capas, una cada una de ellas realiza una función determinada, pero todos tendrán el mismo objetivo, conseguir la transferencia de datos a través de una red.

El modelo OSI consiste en una pila de protocolos, que es un conjunto de protocolos que trabajan conjuntamente para conseguir transferir los datos a través de una de ordenadores.



Los protocolos se unen según sus funciones dentro de cada capa OSI, se clasifican en tres tipos:

- Protocolos de aplicación.
- Protocolos de transporte.
- Protocolos de red.

El conjunto de protocolos de aplicación, el de transporte y los protocolos de red, que son los que se van a ocupar de que se ejecuten los procesos de comunicación concretos en la red.

Los más utilizados y habituales son:

TCP/IP



Es el protocolo más utilizado por todos los ordenadores que se conectan a internet para comunicarse entre sí.

Este protocolo proporcionara la comunicación entre ordenadores, aunque estos ordenadores sean incompatibles en Hardware y Software, da igual el tipo de dispositivo o sistema operativo que utilice.

Al igual que en el modelo OSI, el protocolo TCP/IP, se divide en cuatro capas, con nombres similares a las de OSI. Todas estas capas tienen funciones específicas para para que se produzca la comunicación.



Las capas en que se divide el protocolo TCP/IP son:

- Aplicación.
- Transporte.
- Internet.
- Acceso de red.
- Física.



- **La capa de Aplicación.**

  Esta capa trata con protocolos de alto nivel, como los protocolos utilizados para realizar transferencias de archivos, enviar correos y conexiones remotas.

  Los protocolos que incluyen son: **FTP, HTTP, SMTP, TELNET Y DNS.**

- **La capa de transporte.**

  Los protocolos que contiene esta capa serán los responsables de la transferencia de un ordenador a otro.

  Estos son los protocolos **TCP y UDP**.

- **La capa de internet.**

  En esta capa es donde se realizan las funciones para localizar la ruta para enviar los paquetes de datos.

  Este protocolo es el **IP**, que lo que hará será determinar el paquete de datos y enviarlo.

- **La capa de acceso a red.**

  En esta capa se realiza la entrega del paquete de datos de la IP al ordenador destino, es el enlace físico real con los medios de red.

- **La capa física.**

  Corresponde al interfaz entre el dispositivo y el medio de transmisión.

Ahora ya tenemos claro cuáles son los protocolos más utilizados en una arquitectura web, son: **FTP, HTTPSMTP, TELNET, DNS, TCP, UDP e IP.**

<table>
<tr><td>Aplicación</td><td rowspan="3">TELNET</td><td rowspan="3">FTP</td><td rowspan="3">SMTP</td><td rowspan="3">SNMP</td><td rowspan="3">DNS</td><td rowspan="3">HTTP</td></tr>
<tr><td>Presentacion</td></tr>
<tr><td>Sesion</td></tr>
<tr><td>Transporte</td><td colspan="6">TCP UDP</td></tr>
<tr><td>Red</td><td colspan="6">IP</td></tr>
<tr><td>Enlace de datos</td><td colspan="6">PPP,ETHERNET,APRPANET</td></tr>
<tr><td>Datos</td><td colspan="6">Bits</td></tr>
</table>

Importante

Protocolo FTP:

Este tipo de protocolo se define también para la transferencia de archivos entre sistemas que están conectados en red.

La diferencia con **HTTP** es que es menos seguro, el intercambio de información se realiza sin ningún tipo de cifrado o encriptación, con lo que existe la posibilidad de que puedan acceder al servidor y apoderarse de los archivos que se estén transfiriendo.

El objetivo de este protocolo es la de permitir que equipos remotos puedan compartir archivos, permite que los sistemas de archivos de los equipos del cliente y del equipo del servidor , sean independientes .

En ingles File Transfer Protocol, coma ya su nombre nos indica, es un protocolo cuya función va a ser la de transferir archivos entre equipos.

Este protocolo es el que determinara como se va a enviar los datos a través de una red TCP/IP, desde un ordenador cliente se puede conectar a un servidor para enviar o descargar archivos.

Por defecto utiliza los puertos 20 y 21.

El puerto 20 es el utilizado para el flujo de datos entre el cliente y el servidor y el puerto 21 para el flujo de control, es decir, para enviar las órdenes del cliente al servidor

El FTP, en la mayoría de servidores, es la única manera de conectar con nuestro sitio, para asi, poder subir(cargar) y bajar (descargar) archivos.



Flujo de datos puerto20

Diagrama de flujo de datos Ftp



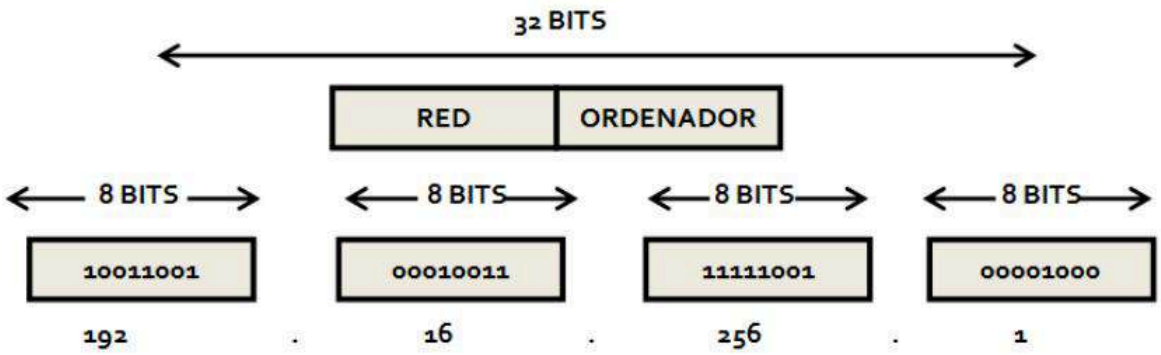
Protocolo IP.

En ingles Intetnet Protocol, este protocolo no es orientado a conexión, se utiliza tanto por el origen de la comunicación como por el destino, se realiza a través de una red de paquetes.

Es uno de los protocolos de Internet más importantes ya que permite el desarrollo y transporte paquetes de datos, aunque sin garantizar su entrega. En realidad, el protocolo IP procesa los paquetes de datos de manera independiente al definir su representación, ruta y envío.

Con este protocolo no se certifica que el envío o recepción se produzcan, no contrala los errores.

Son números que corresponderán con un equipo.



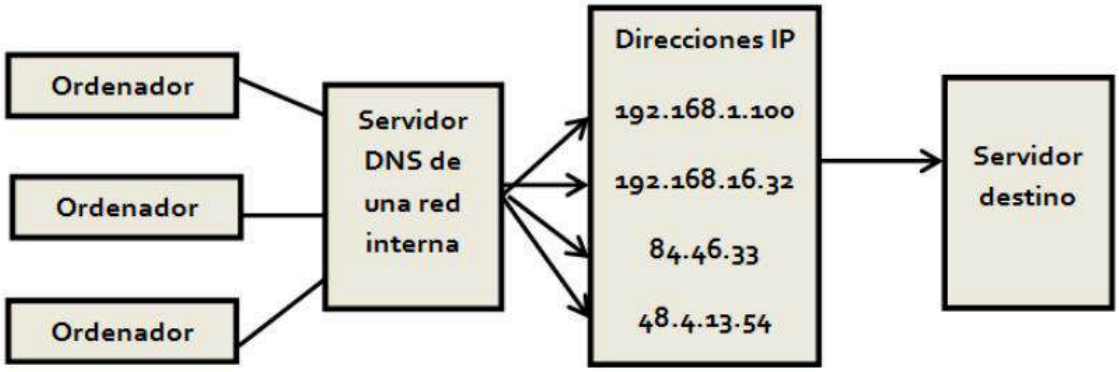
Importante

Protocolo DNS.

En ingles Domoin Name System, se utiliza para dar nombres de dominio, una de las funciones más importe de este protocolo es la de dar nombres a las direcciones IP (que son números), con lo que proporcionara más seguridad que las IP.

Se trata de una base de datos que almacena información sobre nombres de dominio de redes, como es internet.

Diagrama de DNS



Conjunto de equipos de la red interna

Importante

TELNET.

Es un protocolo cuya misión principal es permitir que el usuario de un ordenador pueda conectar con los recursos al mismo tiempo que otro.

Este protocolo se encargará de establecer una sesión de trabajo con un ordenador remoto, para poder establecer la sesión se necesitara una cuenta de usuario y una contraseña.

Telnet permite el acceso al servidor remoto

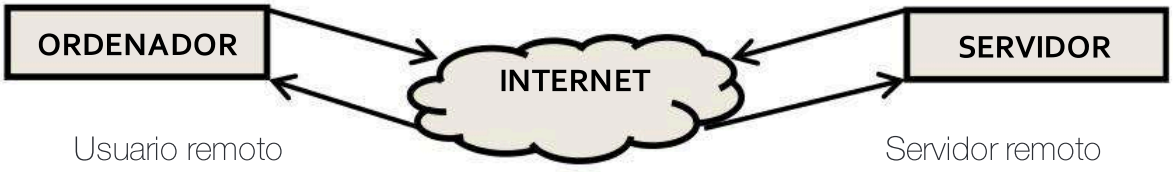


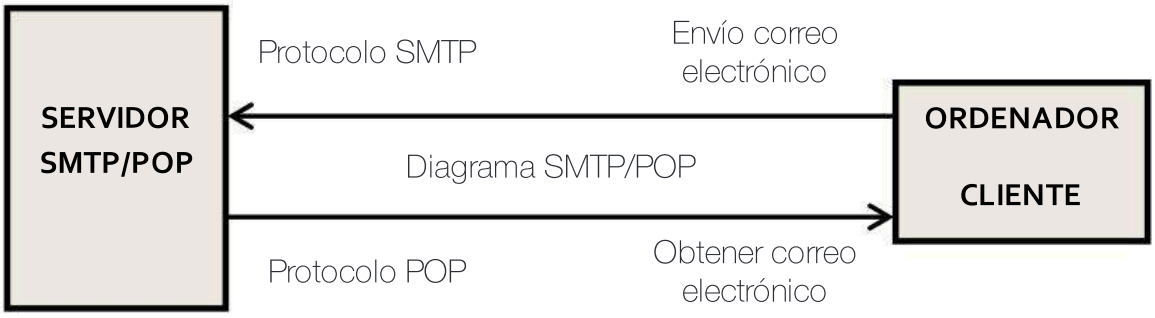
Diagrama Telnet

Importante

SMTP/POP.

Sus siglas significan protocolo simple de transferencia de correo. Este tipo de protocolo es el utilizado para la transferencia de correos electrónicos en internet, usa adicionalmente otro protocolo llamado **POP** que permitirá que los usuarios puedan guardar sus correos en un buzón en el servidor y descargarlo cuando el usuario lo desee.

SMTP solo interviene en el envío ya que este protocolo tiene limitaciones para mantener los mensajes almacenados en el servidor. Y **POP** interviene en la recepción de los correeros electrónicos.





Protocolo HTTP.

Es el protocolo por defecto que se utiliza en la arquitectura Web basado en Cliente/ Servidor

En ingles HyoerText Transfer Protocol, este es uno de los protocolos más utilizados en internet, este protocolo no guarda nada referente a conexiones.

Este protocolo va a permitir que se produzca la transferencia de archivos, especialmente en formato HTML entre un navegador o browser que será el cliente y un servidor que será el servidor Web.

Normalmente es el cliente el que inicia la comunicación HTTP pidiendo algún recurso al servidor.



Su función principal será la de transmitir documentos web sobre internet. Este protocolo va a ser el que va a transformar y dar significado a los mensajes intercambiados entre el cliente y el servidor.

HTTP, determina la construcción y el lenguaje que van a utilizar para realizar transacciones, siguiendo el diagrama petición- respuesta entre un cliente y un servidor.



TCP.

Es un protocolo de la capa de transporte, es el encargado de proporcionar servicio de transferencia de datos y métodos para trasladar los datos encapsulados hacia otro protocolo del nivel de aplicación, es un protocolo fiable y está orientado a conexión.



UDP.

Al contrario que el protocolo TCP este no está orientado a conexión, el flujo de los datos es unidireccional, los paquetes se envían de un ordenador a otro sin pedir conexión de una a la otra.

Otras tecnologías habituales son:



Las tecnologías más utilizadas para implementar un servidor de aplicación son:

Tecnología J2EE.

Es una estrategia comercial está teniendo mucho éxito para implementar servidores de aplicaciones, de hecho ya se puede distinguir como un tipo de servidor de aplicaciones llamado Java, para distinguir los servidores de aplicaciones que implementan soluciones con J2EE.

J2EE es una especificación que propone un estándar para la creación de servidores de aplicaciones, los servidores de aplicaciones que quieran representar un servidor de aplicación J2EE, tienen que pasar un test de compatibilidad, con lo que se garantizará la implementación correcta de las tecnologías Java.

La infraestructura de java es ideal para la obtención de servicios añadidos.

J2EE también tienen cosas negativas, ocupan bastante espacio en memoria ya que utiliza una gran cantidad de recursos, necesita una máquina virtual y es bastante lento en la ejecución.



Tecnología no basada en J2EE.

Todas las tecnologías nuevas que han salido para la realización de un servidor de aplicación, se basa en J2EE, pero antes de aparecer, ya existían tecnologías desarrolladas para implementar un servidor de aplicaciones.

Estas tecnologías están basadas generalmente en lenguajes propios de script, tiene una ventaja muy importante y es que su aprendizaje es bastante rápido y sencillo.

Dos, de estas tecnologías más utilizadas son: PHP y Codlfusion.

- **PHP.**

  Es una lenguaje muy potente que lo que va a hacer es permitir crear de forma fácil y sencilla aplicaciones web. Es de código abierto.

- **Coldfusion.**

  Es un lenguaje basado en script, esta era independiente de J2EE, pero como se usa en la actualidad las tecnologías J2EE, pues se ha creado una nueva versión que integra la tecnología J2EE con la integración de un contenedor Servlet.



Microsoft.

Como siempre Microsoft va por libre y crea sus propias tecnologías, si se va a implementar un servidor de aplicaciones, todo lo que se necesite para ello deberá de utilizar la plataforma Microsoft completa.

Las tecnologías utilizada por Microsoft para realizar aplicaciones web son:

- ASP.
- IIS (Internet information server).
- La tecnología de objetos distribuidos COM. Los nuevas tecnologías que han salido son ASP .NET.

Todas estas soluciones para realizar e implementar una aplicación web se basan todo en Microsoft, con lo que no utiliza estándares abiertos, excepto una excepción en SP.NET que incluye el soporte SOAP.

El desarrollo de una aplicación web se realiza en varias fases y en el transcurso de estas fases, se utilizaran lenguajes de programación muy variados.

Para crear una aplicación web vamos a utilizar algunas herramientas específicas.

Se necesitara herramientas para el diseño, herramientas para la maquetación y otras herramientas para la programación y depuración.

Herramienta de diseño	Herramienta de maquetación	Herramienta de programación y depuración

Para desarrollar una aplicación web, lo primero que necesitamos es un sistema operativo, es muy importante la elección de este, tenemos que ver que aplicaciones soporta y que costes conlleva.

Microsoft tiene aplicaciones que va a utilizar en el desarrollo, pero son bastantes costosas. Algunas aplicaciones de Microsoft son, ASP,.NET,Dreamweaver,Firework,etc, por otro lado existen aplicaciones que no son de Microsoft pero que se pueden instalar en Windows como PHP,GIMP,etc.

Otra cosa que debemos tener en cuenta, es que debemos de elegir un sistema operativo que aporte facilidades al desarrollador, GNU/LINUX es el mejor ya que Windows fue desarrollado para personas que no tienen muchos conocimientos informáticos, mientras que GNU/LINUX, sí.

- Fase de diseño.
- Maquetación.
- Depuración.
- Programación.

- Fase de diseño.

  El diseño va a residir en la creación de esbozos de cómo se quiere que la web sea, para ello se pueden utilizar herramientas graficas como por ejemplo, Photoshop, Gimp o Inkscape.

- Maquetación.

  La maquetación es un proceso que va a consistir en convertir esos esbozos en plantillas HTML, su correspondiente hoja de estilos y las imágenes que se vayan a utilizar.

- Programación.

  Estará la programación del cliente, que se basara en Java Script.

  Estará la programación del lado del servidor, aquí es donde se creara la aplicación Web, en el que los lenguajes de programación más utilizados son, PHP, ASP, .NET, PYTHON, PERL, etc.

- Depuración.

  Es donde se realizan pruebas unitarias, aserciones, etc.

El lenguaje utilizado.

- Lenguaje XML
- Lenguajes Estructurados.
- Lenguajes de presentación.



Para poder describir la información de documentos web a través del cliente al servidor se realiza mediante **el lenguaje HTML** en inglés (HiperText Markup language).

El cliente pedirá una página web al servidor, este inmediatamente enviara un archivo de texto con extensión HTML, el cual, el navegador del cliente se encargara de traducir.

- **Lenguaje XML (extensible Markup Language).**

  Este tipo de lenguaje es utilizado para crear estructuras de documentos.

  XML proviene del Estándar SGML (Standar Generalized Markup Language), que sirve para definir los lenguajes de marcado.

  XML puede guardar información en un formato diferente al del documento final que recibe el usuario, tiene una estructura sencilla y fácil de usar, es flexible, fiable y es gratuito.

La diferencia entre lenguaje **HTML y XML** es que uno representa la información y el otro describe la información.



- **Lenguajes estructurados.**

  - **Lenguaje HTML (HyperText Markup Language).**

    Este tipo de lenguaje es el utilizado para definir la estructura y el contenido de una página web. Es un lenguaje de hipertexto que está formado por un conjunto de etiquetas que van a marcar cuando comienza y acaba cada elemento, mediante estas es posible insertarse forma estructurada textos, imágenes, scripts, etc.

Que HTML esté estandarizada favorece que cualquier página web creada a con este lenguaje se pueda visualizar en cualquier navegador.

En los últimos años, se ha dejado un poco más de lado el desarrollo del estándar HTML para investigar otro lenguaje el XHTML (eXtensible HyperText Markup Language), es una versión más limpia de HTML, que estará preparada para trabajar con herramientas basadas en XML. Así apareció el nuevo Estándar HTML 5.



- **Lenguajes de presentación.**
  - **Hoja de Estilo CSS** (Cascading Style Sheets).

    Este tipo de lenguaje se ocupa de las características graficas que se les puede aplicar a los elementos de un documento HTML. Con este tipo de lenguaje se conseguirá realizar páginas web de una manera más exacta.

Los beneficios que reporta el empleo de CSS son: es más legible y reduce el peso de la página web, es más accesible, mejor mantenimiento y actualización del sitio, es más bursátil.

Habitualmente, un proyecto ambicioso o de gran envergadura se desarrolla usando lenguajes de programación, ya sean compilados como C,C++, semicompilados como.Net o Java, interpretados como Python. Se utilizan para desarrollar la funcionalidad y la interfaz del usuario.

La persona que se va a encargar de esta labor es el desarrollador web, quien se tendrá que ocupar de que el software funcione correctamente.

Los lenguajes de programación más usados en la realización del software necesario tanto en el Cliente como en el servidor, son :

- ASP.
- NET.
- PHP.
- JPS.
- HTML.

Aunque nos podemos encontrar a desarrolladores que utilizan:

- ASP.
- MACROMEDIA COLDFUSION.
- PERL.

El que se utilice un lenguaje u otro vendrá dado por las necesidades de la plataforma.

Desde los inicios de internet, aparecen diferentes demandas de los usuarios, a los cuales se les ha ido dando soluciones, con el avance de las nuevas tecnologías, se han ido desarrollando lenguajes de programación para webs dinámicas, que permitirán la interacción con el usuario y utilizar bases de datos.

Otra de las tecnologías mas utilizadas es AJAX.



AJAX.

Es una tecnología para el desarrollo web para poder crear páginas web, es capaz de crear aplicaciones interactivas que serán ejecutadas en el cliente, es decir, en el navegador web del usuario y va a ser el encargado de que exista una comunicación asíncrona con el servidor en segundo plano.

AJAX es una combinación de tres tecnologías ya existentes, que crea aplicaciones web interactivas combinado, el DOM que servirán para visualizar dinámicamente e interactuar con la información presentada. XML, que nos servirá para intercambiar y manipular datos y java script que servirá para realizar la unión de todas estas tecnologías.

AJAX es un patrón de diseño que va a proponer un nuevo modelo de diseño combinando estas tecnologías.

DOM es el Modelo de Objetos de Documento, es una forma de representar los elementos de un documento estructurado como objetos que tienen sus propios métodos y propiedades.

El responsable del DOM es el World Wide Web Consortium.



El DOM es una API para acceder, añadir y cambiar dinámicamente contenido estructurado en documentos con lenguajes como Java script. El DOM se ha diseñado para ser utilizado en cualquier lenguaje de programación.

En el DOM, los documentos tienen una estructura lógica que es muy parecida a un árbol.

El DOM es un modelo lógico que puede implementarse de cualquier manera que sea conveniente.

Una propiedad importante de los modelos de estructura del DOM es su estructura: si dos implementaciones cualesquiera del Modelo de Objetos del Documento se usan para crear una representación del mismo documento, ambas crearán el mismo modelo de estructura, con exactamente los mismos objetos y relaciones

El Modelo de Objetos del Documento consiste actualmente de dos partes, el Núcleo del DOM y el DOM HTML. El Núcleo del DOM representa la funcionalidad usada para los documentos XML, y también sirve de base para el DOM HTML.



CGI.

Esta fue una de las primeras tecnologías utilizadas para que el contenido de las paginas web fueran dinámicos.

Esta tecnología esta compuesta por un protocolo de comunicación que determina una interfaz que permite el intercambio de información entre el usuario y el servidor o servidores.

La entrada de datos, es por donde se envía los parámetros del cliente utilizando HTTP POST y PUT. En el método GET, se utiliza para enviar formularios, la información del usuario se envía dentro de la propia URL.

Este tipo de tecnología a ocupado grana importancia en el desarrollo de aplicaciones web , como sufre de ciertas limitaciones, han ido apareciendo nuevas tecnologías pero ninguna es capaz de sustituirla por completo, solamente son una evolución de las CGI.

1.3. Análisis de los modelos de programación estándares de facto

Desde los principios de internet las webs han sufrido muchos cambios evolutivos debido a la gran cantidad de formatos y tecnologías que existen y se han ido desarrollando.

Esto hacia que existieran incompatibilidades en la reproducción según el navegador web.

En la actualidad para solucionar este problema existe el World Wide Web, que se ocupa de crear unos estándares web que unifiquen la dirección del desarrollo web para conseguir un mismo acceso al contenido.



Los lenguajes de programación más utilizados en el desarrollo de aplicaciones web son principalmente:

- HTML.
- ASP.
- JAVASCRIPT.
- PHP.
- JSP.

Recuerda

Lenguaje HTML:

Se trata de un lenguaje estático que se utiliza para el desarrollo de sitios web. Este lenguaje fue desarrollado por w Word wide web, los archivos HTML tienen una extensión htm o html.

Este es un lenguaje de marcado, su función principal será la de dar formato al contenido de la página web, no se trata de un lenguaje en sí, sino que por medio de este lenguaje se podrá visualizar contenido generado por otros lenguajes de programación.

Se trata de un lenguaje estructurado, es muy sencillo de aprender y como inconveniente es que es un lenguaje estático y cada navegador lo puede interpretar de forma diferente.

<Etiqueta>, significara el comienzo de una etiqueta y si se pone </etiqueta> significara el final de la etiqueta.

Luego también está para definir un párrafo que sería y , que significa que todo lo que este dentro de estas etiquetas aparecerá en negrita.

La sintaxis de este tipo de lenguaje es el siguiente:

```
<html> ⟶ Esta etiqueta definirá el comienzo del documento
<head>
(Cabecera)
</head>
<body>
(Cuerpo)
</body>
</html>esta etiqueta significara el fin del documento
```

Ventajas	Desventajas
– Texto estructurado	– Es estático.
– Fácil aprendizaje.	– Interpretación diferente por los navegadores.
– Archivos reducidos.	– Acumulación de etiquetas basura.



Lenguaje JavaScript.

Se trata de un lenguaje de scripts, un script es un pequeño programa que se incluye en un documento HTML, que están basados en objetos.

Estos scripts contienen una serie de instrucciones que se ejecutan cuando se carga un documento, este tipo de acción se denomina evento. Hoy en día todos los navegadores interpretan el lenguaje JavaScript.

Es un lenguaje interpretado, es decir que para ser ejecutado no necesita un intérprete, será el navegador, quien se ocupara de compilarlo.

Existen dos tipos de scripts, los que se ejecutan durante la descarga de la página en el navegador y los que se ejecutan cada vez que un usuario produce un evento.

La sintaxis es la siguiente:

```
<script type= “text/JavaScript”>…..</script>
```

Ventajas	Desventajas
– Lenguaje seguro.	– Visible para cualquier usuario.
– Lenguaje interpretado	– El código debe descargarse en su totalidad.

Recuerda 

PHP.

Se trata de un lenguaje de script interpretado que se utiliza para la creación de páginas web dinámicas, se encuentran también incluidos en documentos HTML y son ejecutados por el servidor. Este tipo de lenguaje es interpretado por lo que no tendrá que ser compilado para poder ejecutarse.

Para poder funcionar se necesita instalar Apache o IIS con librerías PHP.

Es un lenguaje muy sencillo de aprender. Este lenguaje ha tomado la sintaxis de otros lenguajes de programación como **C, PERL y JAVA.**

Su sintaxis es la siguiente:

```
<?

$mensaje=”buenos días”;

Echo $mensaje;

¿>
```

Ventajas	Desventajas
– Fácil aprendizaje – Es libre – Capaz de conectar con distintos dispositivos manejadores de base de datos.	– Necesidad de instalar un servidor web. – Todo se realiza en el servidor.



ASP.

Es un lenguaje desarrollado por Microsoft cuya función principal va a ser la de crear páginas web dinámicas. Las páginas web que serán desarrolladas en este lenguaje tendrá la necesidad de instalar Internet Information Server (IIS).

ASP es un lenguaje compilado no necesita de nada para ejecutarse y se podrán insertar dentro de documentos HTML.

La sintaxis es la siguiente:

```
<%   %>
```

Ventajas	Desventajas
– Excelente comunicación con SQL. – Uso de VBScript. – Tolera Java Script.	– Código desorganizado. – Tecnología propietario. – Alojamiento es caro.



JSP.

Es un lenguaje empleado para realizar páginas web en Java, se trata de un lenguaje multiplataforma, ya que se ejecutan en varios tipos de servidores web.

Este lenguaje está compuesto por código HTML/XML y scripts con lenguaje JAVA, se crea de forma parecida a ASP o PHP, los archivos que se crean, serán con extensión .JSP.

Para su funcionamiento necesita el servidor Apache Tomcat, es donde están contenidos los servlets, que se utilizaran junto con Java Server Pages o JSP.

Ventajas	Desventajas
– Multiplataforma. – Código abierto. – Se escribe en Java	– Aprendizaje difícil.

1.4. Uso de componentes orientados a objetos como base de desarrollo de aplicaciones en el modelo de programación web

En los comienzos de los lenguajes de programación orientada a objetos, se realizaba de manera lineal, esto daba muchos problemas a la hora de afrontar cuando los programas cada vez se hacían más grandes por las necesidades nuevas y continuas de las empresas u organizaciones, porque se empezaron a descomponer el programa en pequeños trozos.

Este tipo de descomposición fue la que dio lugar a la programación orientada a objetos actuales.

Hoy en día se hace más necesario crear programas cada vez más grandes y complejos, lo que ha hecho que los desarrolladores web inventaran una nueva forma que les permitiera dividir los sistemas en niveles y abarcar reglas de negocio muy complicadas.

Para este tipo de casos es imposible mantener una estructura lineal.

Para la construcción de las aplicaciones que van a desarrollar todo el sistema Cliente/servidor, se realizaran mediante lenguajes de programación orientada a objetos.

El software se organiza mediante una colección de objetos que contienen una estructura y un comportamiento.

Los lenguajes de programación orientada a objetos van a ofrecen medios y herramientas para describir los objetos manipulados por un programa. Más que describir

cada objeto individualmente, estos lenguajes proveen una construcción (Clase) que describe a un conjunto de objetos que va a poseer las mismas propiedades.



Un sistema orientado a objetos funciona mediante la cooperación entre los objetos que se interrelacionan entre sí. La programación Orientada a Objetos es una metodología que se utiliza para crear estructuras de los programas en torno a los objetos.

Los objetos software coinciden con los elementos que forman parte del mundo real.

Definimos un objeto como una entidad que tiene un estado, un comportamiento y una identidad propios.

- Un estado propio.
- Un comportamiento propio.
- Una identidad propia.

Los objetos poseen características y acciones que podrá realizar por sí mismo o interactuando con otros objetos.

Un objeto es una entidad que se caracterizada por tener una serie atributos propios y cuyo comportamiento está supeditado por las acciones o funciones que puedan modificarlo, así como también las acciones que van a requerir de otros objetos.

Un objeto tiene identidad y constituye una unidad que oculta tanto datos como la descripción de su manipulación.

Puede ser definido como una encapsulación y una abstracción: una encapsulación de atributos y servicios, y una abstracción del mundo real.

Los objetos durante el desarrollo de software de una aplicación, no solo serán objetos que equivalen a objetos de la vida real, o que son tangibles, también pueden ser abstractos.

En general son entidades que juegan un rol bien definido en el dominio del problema.

Por ejemplo un libro, una persona, etc.

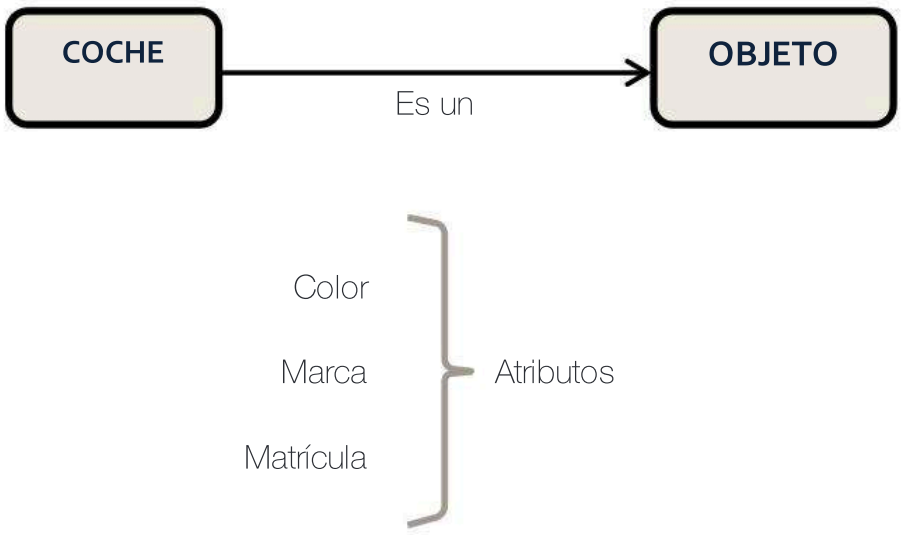
Las características de un objeto son:

- Un objeto se identifica por un nombre o un identificador único que lo diferencia de los demás.
- Un objeto posee estados.
- Un objeto tiene un conjunto de métodos
- Un objeto tiene un conjunto de atributos
- Un objeto tiene un tiempo de vida dentro del programa o sistema que lo crea y utiliza.
- Los objetos soportan encapsulamiento



El estado**.** El estado está compuesto de informaciones. Estas informaciones se les llama **atributos,** un objeto puede tener uno o varios atributos. Al objeto se le asigna un valor o una propiedad concretos.

Por ejemplo:



Importante

El comportamiento. Se refiere a las operaciones que se pueden hacer con el objeto, estas operaciones reciben el nombre de métodos.



Importante

Identidad .Es la propiedad que permite diferenciar un objeto de otro. El objeto es el valor de la propiedad que lo identifica.

Por ejemplo:

Coche. Coche es un objeto que tiene un valor propio. Su clase es vehículo, pero coche por si sola tiene su propia identificación.

La programación orientada a objetos tiene como soporte principal el objeto. Este tipo de programación hace el trabajo más natural y fácil.

La programación orientada a objetos utiliza estos para construir una solución. Un objeto es el concepto de algún hecho o entidad del mundo real, que contendrá atributos que son características de una entidad y los métodos.



Se puede definir la programación orientada a objetos como una propuesta tecnológica que tratara de solucionar problemas mediante el uso de objetos para el diseño de aplicaciones informáticas.

Un objeto representa una instancia de alguna clase y estas clases a su vez, pertenecen a una jerarquía de clases que se encuentran unidas mediante relaciones de herencia.

La programación orientada a objetos se basa en que utiliza objetos, cada objeto es una instancia de una clase y las clases se pueden relacionar con otras, en el que se puede dar el caso de relaciones de herencia.

Se crean objetos a partir de clases, por eso se dice que un objeto es una instancia de una clase, en el cual un miembro de una clase contendrá una serie de atributos, en vez de utilizar variables.

Los elementos más importantes de la programación orientada a objetos son:

- Relaciones entre clases.
- Clases.
- Objetos.

– **Objetos.**

La organización interna de un objeto está formada por dos componentes básicos:

· **Los atributos,** lo que van a hacer es realizar una descripción del estado de un objeto. Un objeto puede contener uno o varios atributos, estos se componen del nombre y el valor del atributo.

· **Métodos**, se refiere a las operaciones que se pueden realizar con un objeto. Realizan una descripción del comportamiento de un objeto. Cuando

llevamos a cabo un método, puede llegar a producir un cambio en el estado del objeto.

Un método está compuesto por el nombre y un cuerpo que va a ser el que va a realizar la operación o la acción, tendrá que estar asociado con el nombre del método.

Este cuerpo estaría formado por una agrupación de código que ejecutara la acción o la operación.

Objeto = Atributos + Métodos

Ejemplo:

Artículo	– Id_articulo int. – Nombre varchar (30). – Stock int.
	– Vender()

· **Clases.**

Las clases son uno de los elementos más importantes de la programación orientada a objetos.

Podemos definir una clase como una estructura por medio de la cual podemos representar un grupo de objetos del mundo real, los objetos pueden ser cosas, personas, animales, etc.

Cada uno de los objetos de una misma clase se diferencian por tener un estado específico y una serie de operaciones o métodos.

A partir de una clase se pueden determinar un número de objetos. Cada uno de estos objetos tendrá unas características propias, pero comparten los mismos métodos u operaciones.

Una clase se compone de:

› **Nombre:** contiene el nombre de la clase.

› **Atributos:** características de una clase.

› **Métodos:** operaciones de interacción de la clase en su entorno.

Los objetos ocupan mucho espacio en memoria por lo que para ocupar menos espacio se hace necesario instanciarse o destruirse algunos objetos.

- **Clases abstractas.**

  Son clases que pueden no existir en la vida real, pero que son muy útiles, esta clases no tienen instancias, pero las subclases de este tipo de clases abstractas si pueden tener instancias.

Por ejemplo, una clase abstracta puede ser un ser humano, solo con pensar en un ser humano aunque no lo veamos sabemos que un ser humano está compuesto por piernas, brazos, cabeza, ojos, etc., eso es la abstracción que con que solo digan ser humano ya se sabe de qué se compone sin tener que decirnos nada.



Relaciones entre clases.

Estas son muy importantes en la programación orientada a objetos, existen relaciones tanto entre clases como entre objetos.

Las relaciones entre clases son:

– Relaciones de generalización/especialización.

– Relaciones de agregación

– Relaciones de asociación.

Generalización/ especialización	Agregación	Generalización/ especialización

Recuerda

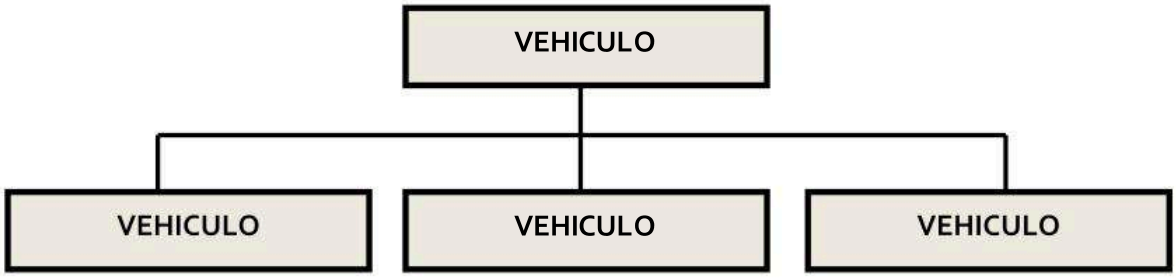
Relaciones de generalización /especialización.

Las clases poseen atributos comunes, estas clases que tienen atributos comunes se organizan en superclases.

Una superclase es una generalización de las subclases, de este modo una subclase es una clase que simboliza una especialización de la clase superior.

Una superclase es una generalización de la subclase y la subclase es una especialización de la superclase.

Un ejemplo: Tenemos una superclase VEHICULO y unas subclases que serían COCHE, MOTO, AUTOBUS.



Lo que es hacemos la siguiente pregunta, un vehículo puede ser tanto un coche, como una moto o como una autobús.

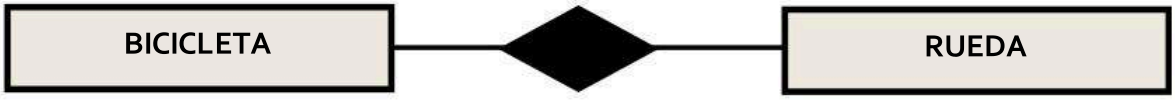
Importante

Relaciones de agregación.

Este tipo de relación va a representar a los objetos compuesto, que son los que se componen a su vez de otros objetos.

Por ejemplo.

Tenemos un objeto BICICLETA y un objeto RUEDA.



En el que las ruedas que forman una bicicleta y una bicicleta está formada por ruedas.

Importante

Relación de Asociación.

Es un tipo de relación que consiste en la conexión entre los objetos de las clases que están involucradas en una relación, existirá dependencia entre objetos de dos clases y tendrá cardinalidad, consistirá en decir cuántas instancias de cada clase serán participes en una asociación.

Una asociación es bidireccional y se representa mediante una línea que unen las dos clases y un nombre.

Por ejemplo:



Donde, de un libro existe un ejemplar y un ejemplar es de un libro.

Recuerda

La programación orientada a objetos se basa en cuatro propiedades:

- Abstracción:
- Encapsulación.
- Jerarquía:
- Modularidad:

- Abstracción:

  Esta propiedad permite aislar las características más importantes de un objeto de las demás características que son menos importantes.

- Encapsulación.

  Esta propiedad va a permitir ocultar el estado de un objeto, es decir, el contenido de la información de un objeto.

  Cada objeto se encuentra aislado del exterior, este aislamiento protegerá a los datos asociados de un objeto, de alguna modificación.

- Modularidad:

  Esta propiedad va a permitir dividir una aplicación en partes más pequeñas que recibe el nombre de módulos, cada uno de ellos tiene que ser lo más independiente posible de la aplicación y de las partes.

- Jerarquía:

  Esta propiedad es la que va a permitir la ordenación de las abstracciones. Las dos jerarquías que hemos visto son:

  · La generalización/especialización.

  · La agregación.

UD1
Lo más importante

- **Las aplicaciones web** son todas apellas aplicaciones que son utilizadas por usuarios, accediendo a ellas a través de un servidor web mediante un navegador.

  Se trata de un software elaborado en un lenguaje de programación que pueda ser soportado por los navegadores web.

- **Un navegador web** es un software que se utiliza para acceder a internet. Un navegador va a permitir que un usuario puede visitar paginas web y realizar acciones en ella, como por ejemplo iniciar sesión, ver el contenido multimedia, enlazar con otras paginas, imprimir o enviar o recibir correos, entre otras.

  Un navegador web puede recibir también el nombre de **browser,** los navegadores interpretan información de archivos y de sitios web para que estos puedan ser leídos.

  Estos permitirán que los documentos se visualicen documentos de una pagina web y para ello se necesitara algunos recursos que estarán incrustados.

- **La arquitectura web** es un conjunto de componentes, que siguiendo una serie de reglas y procesos, permitirán utilizar una gran variedad de servicios informáticos que serán utilizados por una organización o empresa para un mejor rendimiento.

- **La arquitectura monocapa:**

  Es la estructura mas básica, en este tipo de arquitectura encontramos, en el que todas las partes involucradas se encuentran en un solo ordenador.

– Arquitectura Cliente/Servidor de dos capas:

  Este tipo de arquitectura es la básica, está formado por un navegador en la parte del cliente y un servidor web en la parte del servidor, también se necesitara una red a la que conectarse.

  La arquitectura Cliente/Servidor, se divide en dos capas una la del cliente que implementa la interface y otra es donde se encuentra el sistema gestor de base de datos.

– Arquitectura Cliente/Servidor de tres capas:

  Este tipo de arquitectura se basa en el modelo Cliente/Servidor, pero además se le añade una capa más entre el cliente y el servidor, que será un servidor de aplicaciones, que es donde se encontrara la lógica de la aplicación.

– La arquitectura web básica, se compone de un cliente y un servidor. El modelo es el llamado Cliente/Servidor.

– El Cliente es una aplicación ejecutable que permitirá el acceso a internet y serán capaces de mostrar y solicitar documentos sobre una red. Suelen ser las computadoras o cualquier otro dispositivo móvil que sirva como soporte para la entrada y salida de datos. Se puede decir que el cliente es el consumidor de un servicio.

– Los Clientes Ligeros. Son las computadoras, trabajan en una arquitectura Cliente/servidor, son los ordenadores o computadores o cualquier otro dispositivo móvil en el que se pueda instalar la aplicación, que este en red que y tenga acceso a internet, ya que se tiene que tener acceso a un navegador web, que serán utilizados por los usuarios

– Servidores Web Es un tipo de software que se instala y se ejecuta constantemente en un ordenador, se mantiene a las espera de las peticiones del cliente. Hoy en día también se utiliza la definición de servidor web para referirse al ordenador en el cual funciona este software, su función principal será la de administrar los datos a otros ordenadores Cliente donde los utilizaran.

– Servidores de Aplicaciones. Los servidores de aplicación son los servidores web de nueva generación. Se denomina servidores de aplicaciones a un software que proporciona servicio de aplicación a los clientes. Este tipo de servidor se suele asociar con organizaciones o empresas que necesitan servidores de alto rendimiento, que sea de alto rendimiento viene definido por ejemplo, por una afluencia masiva de visitas, por un mayor número de aplicaciones, etc.

– Servidor de datos o archivo . Funciona como los demás servidores, sigue siendo una aplicación instalada en un ordenador y que servirá para almacenar y com-

partir archivos con otros usuarios de la red. Así pues podemos definir el servidor de datos como el servidor que va a autorizar el acceso remoto a todos los archivos almacenados en él. .

– Los protocolos más utilizados son:

  · TCP/IP. Es el protocolo más básico de internet, suministra todos los servicios necesarios para el transporte de los datos.

  · HTTP: Es el protocolo por defecto que se utiliza en la arquitectura Web basado en Cliente/Servidor. Su función principal será la de transmitir documentos web sobre internet.

  · FTP: Este tipo de protocolo se define también para la transferencia de archivos entre sistemas que están conectados en red. La diferencia con HTTP es que es menos seguro.

– Los estándares más utilizados son :

  · Lenguaje XML. Este tipo de lenguaje es utilizado para crear estructuras de documentos.XML proviene del Estándar SGML (Estándar Generalizad Markup Language) que sirve para definir los lenguajes de marcado.

  · Lenguaje HTML. Este tipo de lenguaje es el utilizado para definir la estructura y el contenido de una página web. Es un lenguaje de hipertexto que está formado por un conjunto de etiquetas que van a marcar cuando comienza y acaba cada elemento, mediante estas, es posible insertar formas estructurada textos, imágenes, scripts, etc.

  · Hoja de Estilo CSS .Este tipo de lenguaje se ocupa de las características graficas que se les puede aplicar a los elementos de un documento HTML. Con este tipo de lenguaje se conseguirá realizar páginas web de una manera más exacta.

– Definimos un objeto como una entidad que tiene un estado, un comportamiento y una identidad propios.

UD1
Autoevaluación

1. Un navegador web es:

    a. Es un hardware que se utiliza para acceder a un ordenador.
    b. Es un software que se utiliza para acceder a Internet.
    c. Es un hardware que se utiliza para acceder a Internet.
    d. Es un software para acceder a un ordenador.

2. Se define una arquitectura web como…

    a. Un sistema que da como resultado información a los usuarios, con independencia de donde este ubicada esa información.
    b. Es un sistema que no da como resultado información a los usuarios, con independencia de donde este ubicada esa información.
    c. Es una combinación de sistemas que colaboran entre si para dar como resultado a los usuarios información, con independencia de donde este ubicada la información.
    d. Es una combinación de sistemas que son incapaces de colaborar entre si para dar como resultado a los usuarios información, con independencia de donde este ubicada la información.

3. Un servidor web se ejecuta durante:

    a. Todo el día.
    b. 365 días al año.
    c. 12 horas al día.
    d. Todo el día los 365 días del año.

4. ¿Qué elementos forman parte de la arquitectura web?

    a. Cliente.
    b. Cliente y servidor.
    c. Servidor.
    d. Ninguno de los anteriores.

5. Ventajas de utilizar un cliente ligero:

    a. Información centralizada, menos coste de hardware, menos coste en administración.
    b. Información centralizada, menos coste de hardware, menos coste en administración, consumo energético.
    c. Información centralizada, menos coste de hardware, menos coste en administración, consumo energético, más seguro.
    d. Información centralizada, menos coste en administración, consumo energético, más seguro.

6. ¿A qué protocolo corresponde la función de transformar y dar significado a los mensajes intercambiados entre el Cliente y el Servidor?

    a. TCP/IP.
    b. FTP.
    c. HTTP.
    d. NEWS.

7. Los lenguajes estructurados son:

    a. XML, ASP, HTTP.
    b. HTML Y CSS.
    c. XML y HTTP
    d. FTP, XML y CSS.

8. Los lenguajes de programación más utilizados en el desarrollo de aplicaciones web son principalmente:

    a. HTML,JSP,ASP Y PHP.
    b. ASP, HTML, JAVASCRIPT, PHP y JSP.
    c. FTP, ASP, PHP, JSP y JAVASCRIPT.
    d. FTP, ASP, PHP, JSP y HTML.

9. Los lenguajes de programación más utilizados en el desarrollo de aplicaciones web son principalmente:

    a. ASP y PHP.
    b. ASP y JAVASCRIPT.
    c. PHP y JAVASCRIPT.
    d. ASP y HTML.

10. Un objeto es una…

    a. Un estado y una identidad propia.
    b. Un comportamiento.
    c. Una entidad.
    d. Una entidad que tiene un estado, un comportamiento y una identidad propia.

Área: informática y comunicaciones

UD2

Arquitectura multicapa (N- tier)

2.1. Análisis de la arquitectura multicapa

2.2. Distinción y estudio del modelo de tres capa en Web: presentación, aplicación y datos

2.3. Diseño de arquitecturas de aplicación basadas en el modelo multicapa

2.4. Análisis del concepto de lógica de negocio y significado de la capa lógica

2.1. Análisis de la arquitectura multicapa

Las comunicaciones entre ordenadores se rigen por el modelo Cliente –Servidor, este es un modelo que intenta proveer de:

- Usabilidad.
- Flexibilidad.
- Interoperabilidad.
- Escalabilidad.



El modelo cliente/servidor empezó a ser usado en los años 80, su funcionamiento es muy simple, se tiene un ordenador cliente y un ordenador servidor, el ordenador cliente es el que requiere de un servicio de un ordenador servidor y este realizará la función para lo que estará programado.



Debemos recordar que se puede dar el caso que este modelo este corriendo en un mismo ordenador.

La arquitectura cliente-servidor consiste básicamente en la realización de peticiones por parte del cliente a un programa que se encuentra en el servidor, que es el que da la respuesta. Aunque puede existir la posibilidad de que todo se encuentre en el solo ordenador, lo más conveniente es utilizar un sistema multiusuario distribuido a través de una red de ordenadores.



La red cliente - servidor es aquella en la que existe una red de comunicaciones en la que los clientes están conectados a un servidor, en el que se van a centralizar los diferentes recursos y aplicaciones con los que se cuenta, ya que el servidor los va a poner a disposición de los clientes cada vez que lo soliciten.

Todas las gestiones que se realizan se concentran en el servidor, el en servidor se encontraran todos los archivos que son de uso privado y de uso publico, los requerimientos de los clientes, todo tipo de archivos, tanto archivos que son de solo lectura o que se puedan modificar ,etc.



El cliente, es el proceso por el cual va a permitir que el usuario pueda realizar el pedido de ciertos requerimientos y se los pasa al servidor.

El cliente maneja todas las funciones relacionadas con la manipulación y despliegue de datos, por lo que están desarrolladas sobre plataformas que permiten construir interfaces graficas de usuario, además podrá acceder a todos los servicios distribuidos en cualquier parte de la red.

Recordemos que el cliente es un software que se encuentra instalado en un ordenador , mediante un navegador web. Todos los sistemas operativos suelen incorporar el navegador web en la instalación del sistema operativo.

Microsoft por ejemplo, lleva incorporado el navegador web Internet Explorer.

Funciones del cliente:

- Administrar la interfaz de usuario.
- Interactuar con el usuario.
- Procesar la lógica de la aplicación.
- Generar peticiones de bases de datos.
- Recibir los resultados del servidor.
- Formatear los resultados.



El servidor es el proceso encargado de atender a las muchas peticiones de los clientes, que necesitaran algún recurso administrado por el.

El servidor habitualmente es el que va a manejar todo lo relacionado con las reglas de negocio y los recursos de los datos.

Las funciones del servidor van a ser:

- Aceptar los requerimientos de bases de datos que realizan los clientes.
- Procesar los requerimientos de la bases de datos.
- Formatear los datos para transmitirlos a los clientes.
- Procesar la lógica de la aplicación y realizar validaciones a nivel de base de datos.

Las características de una arquitectura cliente-servidor son:

- **Servicios**, el servicio lo proporciona el servidor y el cliente es el que utiliza estos recursos.
- **Recursos compartidos**, los clientes pueden utilizar los mismos recursos de los servidores y a través de ellos comparten tanto los recursos lógicos como físicos.
- **Protocolos**, en el que los clientes son los que inician la comunicación y los servidores esperan esa comunicación.
- **Transparencia física de servidores y clientes**, en el que el cliente no tiene porque saber no se encuentra el recurso que necesita.
- **Independencia** de la plataforma que se emplee.
- **Encapsulamiento** de servicios.
- **Escalabilidad**, se podrán añadir mas clientes y se podrán ampliar la potencia de los servidores.
- **Integridad**, que los datos y los programas se encuentres centralizados en el servidor facilitara la integración y mantenimiento de los datos.

Ventajas del modelo cliente-servidor:

- **Centralizar el control**, los accesos, recursos y la integridad de los datos son controlados por el servidor. Esta centralización facilitara la tarea a la hora de actualizar datos o recursos.
- **Escalabilidad**, se podrá aumentar el número de clientes y de mejoras en los servidores, por separado. Cualquier elemento puede ser mejorado o aumentado en cualquier momento, se podrán añadir más ordenadores clientes a la red.
- **Fácil mantenimiento**, ya que las funciones y responsabilidades están repartidas entre varios ordenadores. Se podrá reemplazar, reparar o actualizar o trasladar un servidor sin que se vean afectados los clientes, esta independencia es la encapsulación.
- **Existen tecnologías que aseguraran las transacciones, la interfaz y la facilidad de uso.**

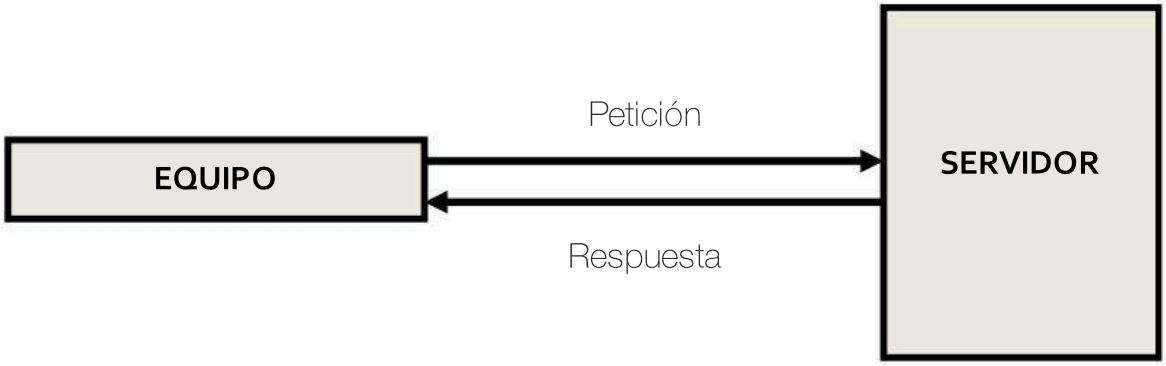
Inconvenientes del modelo cliente-servidor:

- **La congestión del tráfico de datos**, este ha sido siempre un problema común en el modelo cliente servidor. Que hayan multitud de peticiones de clientes simultáneas en un mismo servidor puede causar algunos problemas en el servidor. Cuanto mas nodos haya menor es el problema de concurrencia.

- **El cliente no dispone de los recursos que pueda existir en el servidor.**

La arquitectura Cliente/Servidor es una forma de dividir las tareas que cada uno de ellos realiza, esto dará como resultado una mayor eficiencia y facilidad a la hora de realizar actualizaciones y mantenimiento del sistema.

Básicamente consiste en que el Cliente realiza una petición y el Servidor le da la respuesta, este sistema es más eficiente en un sistema multiusuario distribuido a través de la red.



En esta arquitectura separa las funciones del Cliente y las funciones del Servidor. Se trata de una separación lógica, donde la aplicación no solo se ejecuta en un ordenador ni es solo un programa pueden ser varios.

Cuando se va a diseñar un sistema para su posterior implantación, se debe de tener en cuenta algunos aspectos tecnológicos que va a influir en que se realice con éxito o no un proyecto.

Existe una clasificación desde el punto de vista de su arquitectura tecnológica.



Tipos de arquitecturas:

- Arquitectura Monocapa.
- La arquitectura Cliente/Servidor genérica.
- La arquitectura de tres capas.

– **Arquitectura Monocapa.**

La característica principal en este tipo de arquitectura es que son sistemas monousuario, que serán utilizados por una sola persona, con lo que no existirá concurrencia al acceder a la información.

Son los llamados Desktop, ya que se basa en una aplicación que se instala en un ordenador y que está compuesta por un instalador y un programa ejecutable que va a permitir acceder a la misma.

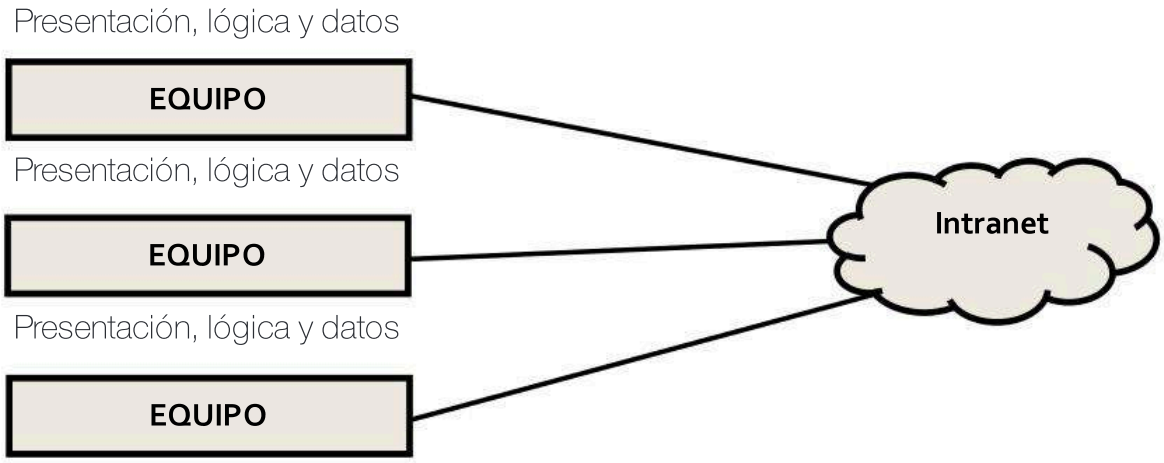
Es un elemento que es independiente y que no interactúa en ningún momento con otra aplicación.

En este caso la lógica de negocio está centralizada, en la que todas las reglas del negocio se encuentra en un solo sitio, en el código de la aplicación instalada en cada máquina.

Los datos de la aplicación se guardan en archivos dentro de la propia aplicación. Esto hará que se pueda recuperar la información para que vuelva a usarla la aplicación.

Estos datos se guardan en la misma maquina en la que se encuentre instalado el programa.

Este tipo de arquitectura se caracteriza por tener en un solo ordenador la capa de presentación, la capa de lógica de negocios y la de datos.



Todos los componentes, se encuentran en cada uno de los equipos, tanto la presentación, la lógica y los datos.

Ventajas	Desventajas
– Fácil construcción. – Facilidad para encontrar errores. – Acceso directo a los datos. – Fácil instalación.	– Fusión entre datos de distintos ordenadores es complicado. – Cuando se modifica un dato de la base de datos solo se registra en un solo ordenador que es que realiza la acción, por lo que habrá que juntar, las modificaciones de cada uno de los equipos. – Reunión de datos a mano. – Versiones distintas para cada sistema operativo por lo que se necesitara versiones distintas de la aplicación.

Recuerda

- La arquitectura Cliente/Servidor genérica.

  Se compone solo del Cliente y del Servidor, este tipo de arquitectura recibe el nombre de arquitectura de dos niveles o de dos capas. (Estudiada en la unidad didáctica anterior).

  Está compuesta por dos elementos:

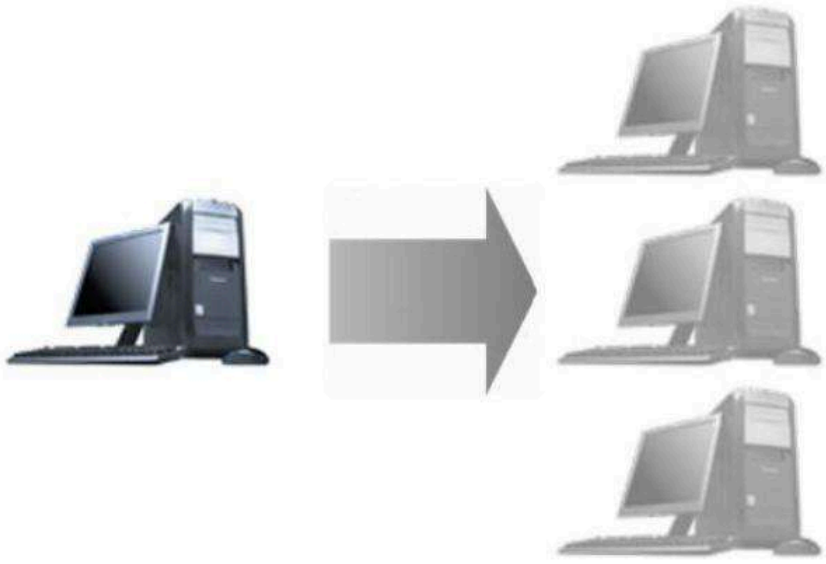
  - Aplicación Cliente:

    Que será una aplicación instalada en cada uno de los ordenadores de los usuarios, esta constituirá la primera capa de la arquitectura que incluirá la capa de presentación mediante una interfaz gráfica y la lógica de negocios.

  También tendrá un acceso a un servicio de datos para la gestión de los mismos.

  - Aplicación servidor:

    Por otra parte estará el servidor de base de datos que corresponderá a la capa de datos.



Modelo cliente-servidor

La base de datos, es un programa que va ejecutando en el servidor.

La aplicación cliente se conecta a la base de datos directamente, mediante un programa de base de datos.

Las bases de datos que se encuentran en el mercado, proveen de estos programas compatibles para los lenguajes de programación que son más utilizados.

Algunos de estos programas más conocidos son:

- **JDBC:** que es un driver para conectar bases de datos con aplicaciones realizadas en Java.
- **ODBC:** que es un driver más común para conectar con bases de datos.
- **ADO.NET:** que es un driver para conectar bases de datos con aplicaciones realizadas con .NET

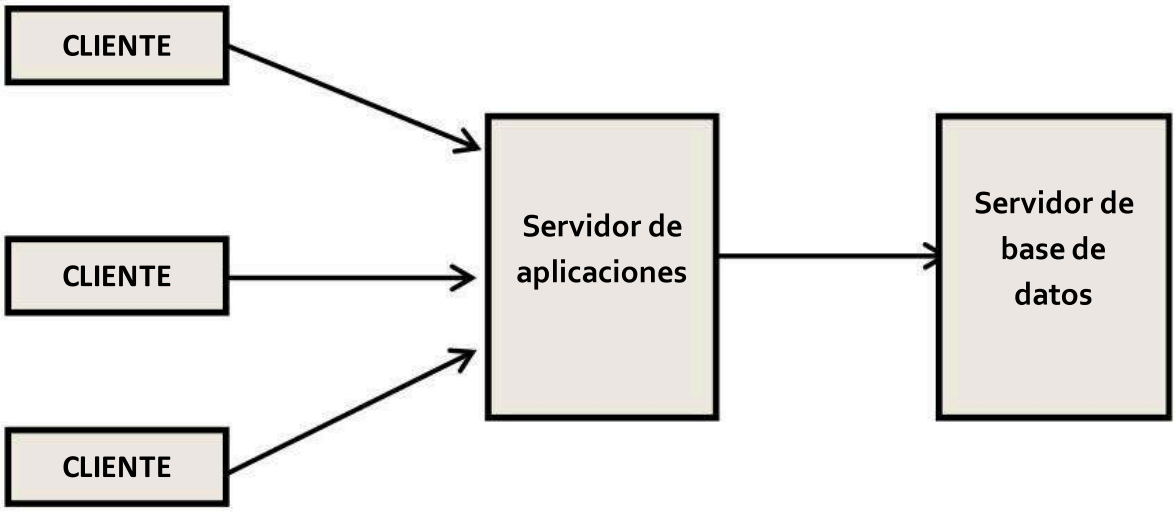
En este tipo de arquitectura, la lógica de negocio se encuentra repartida en dos partes:

- El código de la aplicación en el ordenador del cliente.
- La base de datos , que estará en el servidor , en el que se encontraran todos los datos con tablas, vista, procedimientos almacenados, disparadores y constantes

Ventajas	Inconvenientes
– Al incorporar la base de datos en el servidor, se gana concurrencia de consultas. – Carga simultanea de datos. – Gran volumen de datos e integridad de estos. – La mayor parte de las tareas se realizan en el servidor. – Independencia física de los usuarios.	– Problemas de acceso ,. – Todos los ordenadores cliente deben de encontrase en la misma red que el servidor, a no ser que se conecte a internet o una red LAN. – Instalación en el cliente de los drivers. – Aplicación cliente instalada en el sistema operativo. – Actualización de la aplicación. – Mayor cantidad de errores. – Problemas de seguridad.

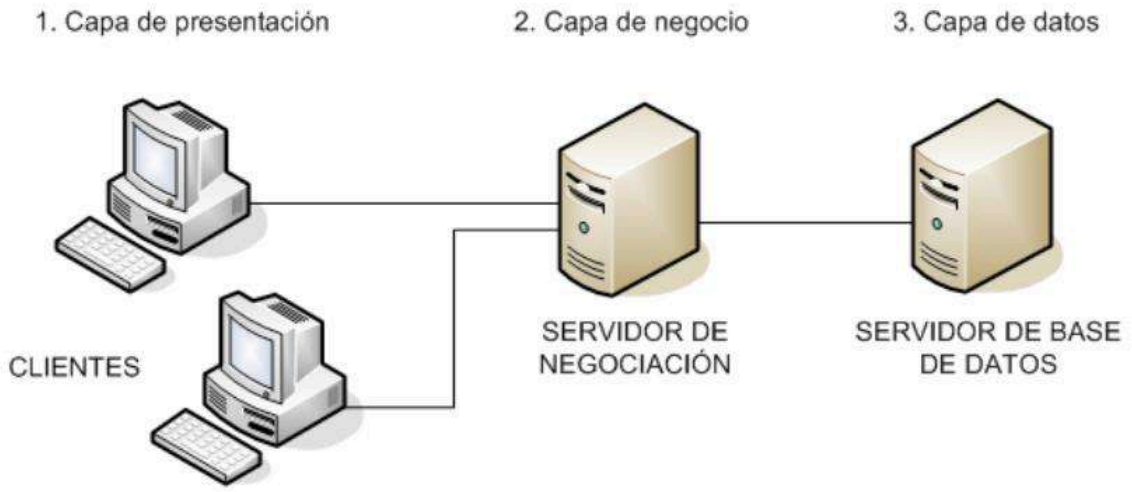
El problema que surge en este tipo de arquitectura es que si necesitara hacer cambios en la lógica de negocio, tendríamos que cambiar las aplicaciones cliente de los ordenadores cliente, y son 300, se hace más complicado, con lo que esta arquitectura se ha desarrollado para utilizar un servidor de aplicaciones que se encargara de toda la lógica de negocio, que da lugar a las arquitectura de tres capas.

La arquitectura de tres capas, se produce cuando disponemos además del Cliente y del servidor de base de datos, de un Servidor de aplicación, que su función será la de ocuparse de la lógica del negocio.



Se compone de tres elementos:

- El cliente, será un ordenador que tendrá un navegador al cual accederá a través de internet. esta corresponderá con la capa de presentación.
- Un servidor de aplicaciones, que es donde se ejecutara el código, este servidor corresponderá con la capa lógica de negocio.
- Un servidor de base de datos, de donde se almacenaran los datos. Corresponderá con la capa de acceso a datos.



Modelo tres capas.

Ventajas	Inconvenientes
– Reutilización de las capas. – Facilidad de estándares. – Limitar dependencias. – Pocos cambios en las capas a la hora modificar algo.	– Perdida de eficiencia. – Algunas veces no se logra realizar cambios sin tener que tocas todas las capas. – Dificultad a la hora de diseñar correctamente las capas.

Las diferencias entre la arquitectura de dos capas y la de tres capas son las siguientes:

Arquitectura de dos capas:

- Consiste en una capa de presentación y lógica de la aplicación, y otra capa de la base de datos.
- La base de datos se encuentra centralizada en el servidor.
- La base de datos es relativamente estática
- Requerirá un mantenimiento mínimo.

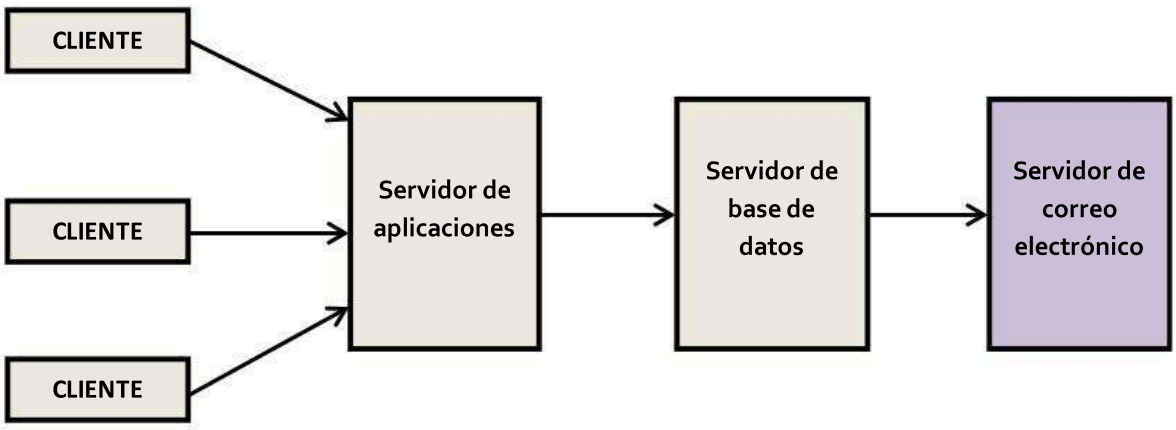
Mientras que en la arquitectura de tres capas:

- Consiste en una capa de presentación, otra de la lógica de aplicación y otra capa de la base de datos.
- Este soportara bastantes procesamientos de datos en la aplicación.
- Pueden ser sometidas a cambios sin que afecte la modificación de una capa a las demás.
- Si se quiere modificar la base de datos se podrá aislar.
- Se podrá separar el código del cliente, así se facilitara el mantenimiento.
- Es la mas adecuada para soportar tecnologías orientadas a objetos.

Las aplicaciones web suelen utilizar la arquitectura web de tres capas

Importante

La arquitectura n_capas, es un tipo de arquitectura que además de componerse del Cliente, Servidor de aplicación y Servidor de datos, pueden llevar otro o varios tipos más de servidores, como pueden ser, un servidor de chat, un servidor de correo electrónico, etc.



En los sistemas multicapas, el servidor de aplicación se puede separar en diferentes aplicaciones que pueden ser ejecutados por diferentes ordenadores, así se aumenta el grado de distribución del sistema.

Hoy en día las arquitecturas Cliente/Servidor son sistemas distribuidos, el procesamiento y ejecución de la información se distribuye en varios ordenadores en vez de estar en uno solo.

Existen dos tipos de sistemas distribuidos:

- Arquitectura Cliente/Servidor distribuido.
- Arquitectura de objeto distribuido.



- **Arquitectura Cliente/Servidor distribuido.** El cliente va a ser el consumidor de un servicio que prestara el servidor, el cual actuará como proveedor del servicio.

  Representa un modelo lógico en el que las aplicaciones del servidor y las del cliente son diferentes. Varios servidores aplicaciones se pueden ejecutarse en un solo ordenador.

  Cuando hablamos de servidores y clientes nos referimos a las aplicaciones lógicas, no a los ordenadores, estos serán el medio por el cual se ejecutan.

- **Arquitectura de objeto distribuido.** En este tipo de arquitectura, no existe diferencias entre el cliente y el servidor. Se puede ver como una agrupación de objetos que interactúan entre sí.

Para más información, consulta "Arquitectura de objetos distribuidos " en los anexos al final del libro.

En estos sistemas distribuidos, las aplicaciones utilizadas pueden estar repartidas por varios ordenadores en red, para que exista una comunicación e interacción entre estas aplicaciones, se necesitara un software llamado Middleware, este software permitirá la comunicación entre otras aplicaciones, software, redes, hardware y sistemas operativos.



El **middleware** actuara como conductor entre sistemas permitiendo que cualquier usuario pueda comunicarse con varias fuentes de información que se van a encontrar conectados por una red. Va a ser el intermediario entre el cliente y el servidor y se ejecutara en ambas partes.

La utilización de Middleware permite desarrollo de aplicaciones en arquitectura Cliente –servidor, independizando los servidores y los clientes, facilitando la interacción entre ellos y evitando el uso de tecnologías propietarias.

El middleware se estructura en tres niveles:

- Protocolo de transporte.
- NOS, Network Operating Sistem.
- Protocolo especifico del servicio.

Las características de un Middleware son:

- Simplifica los procesos de desarrollo de aplicaciones.
- Permite la interconectividad de los sistemas de información.
- Proporciona mayor control de negocio, ya que podrá contar con toda la información procedente de distintas plataformas que utilizan este mismo soporte.
- Facilitara el desarrollo de sistemas más complejos que utilizaran tecnologías y arquitecturas diferentes.

Los Middleware se dividen en categorías, según se ajuste a aplicaciones específicas.

Las ventajas al utilizar un sistema de distribución son:

Ventajas
– Sistema abierto, utilización de protocolos Estándar.
– Ejecución de varios procesos a la vez.
– Compartir recursos software o hardware.
– Capacidad para incrementar recursos, según necesidades de la empresa u organización.
– Tolerancia a fallos de hardware o software.

Pero, también tiene desventajas:

Desventajas
– Son más complejos.
– Menos seguridad en la integridad de los datos.
– Tiempo de respuesta variable.



Una aplicación web típica está formada por una capa de presentación, que corresponderá a todo lo referente a la interfaz del usuario, una capa lógica de negocio, que es donde se encuentra todo lo relacionado con el procesamiento de las reglas de negocio y una capa de datos, que es donde se encuentran almacenados todos los datos y el acceso a ello.

Las características propias que se suelen dar en una arquitectura que se basa en capas son las siguientes:

- Cada una de las capas se ocupara de unos u otros servicios, con lo que se descompondrá las funciones entre las capas, que interactuaran con las capas vecinas.
- Cada una de las capas pueden encontrase repartidas en varios ordenadores.
- Cada una de las capas ofrece abstracción y responsabilidad.
- Los elementos forman cada capa se comunican con los elementos de la otra mediante el uso de interfaces.
- Consta de una base de datos.

A la hora de diseñar un sistema Cliente/Servidor se deberá concretar la estructura lógica de la aplicación que se esté realizando.

La arquitectura multicapa también recibe el nombre de n_Tier en inglés, algunas veces llegamos a confundirnos con el término arquitectura n_layer. Son casos diferentes.



- El **n_Tier** es el número de capas y la distribución física de estas, ¿qué queremos decir con esto?, que es donde suceden los procesos y se encuentra el código.
- El **n_layer** se refiere a como se reparten las capas lógicas, es decir, como va a estar organizado el código.

Las arquitecturas de más de dos capas reciben el nombre de arquitectura multicapa.

En la actualidad la arquitectura más consolidada es la de tres capas, aunque ya se utilizan hasta 7 capas. Pero las más usadas en aplicaciones empresariales es la de tres capas.

La estructura lógica de este modelo se divide en tres:

- Capa de gestión de datos.
- Capa de aplicación o de negocio.
- La capa de Presentación.

La capa de Presentación:

En esta capa lo que se tratara es el diseño de la interfaz gráfica, es decir, como va a ver la información el usuario, y como va a interactuar este con el software.

Capa de aplicación o de negocio:

Está relacionada con el funcionamiento lógico de la aplicación.

Capa de gestión de datos:

Esta capa se encargara de todo lo relacionado con las operaciones sobre la base de datos..

2.2. Distinción y estudio del modelo de tres capa en Web: presentación, aplicación y datos



La arquitectura basada en capas tiene como objetivo el reparto de funciones y responsabilidades de forma jerárquica, así se conseguirá separar de forma segura las responsabilidades, de cada uno. Este tipo de arquitectura se da cuando se van a realizar aplicaciones web.

Para realizar el diseño de cualquier software, se debe de seguir un patrón arquitectónico.

La arquitectura de tres capas, se utiliza para determinar cómo se va a organizar el diseño de las capas, que van a estar distribuidas físicamente, lo que se quiere decir es que cada capa estará compuestas por una serie de elementos que pueden hacer referencia a elementos de capas que sean inferiores.

Este tipo patrón arquitectónico lo que hace es que sea más fácil de comprender y organizar el desarrollo de sistemas más complejos, disminuyendo las dependencias, de esta forma las capas más bajas ni son conscientes de los proceso de las capas superiores y no estarán involucrados en nada.

Para solucionar todos estos temas se desarrolló la arquitectura de tres niveles o capas. La aplicación se divide en tres capas lógicas que son totalmente distintas, en cada una de estas capas incluirá una interfaz.

La primera capa, se denomina capara de presentación y suele formarse por una interfaz gráfica de usuario.

La capa intermedia o capa de negocio, en la que va a residir la aplicación lógica de negocio. Básicamente la capa de negocio es el código al cual recurre la capa de presentación para obtener los datos que necesite.

Y la tercera capa, que es la capa de datos, que está formada por los datos que serán utilizados por la aplicación.

Que esté separada la lógica de negocio del cliente, proporciona una mayor flexibilidad al diseño de la aplicación, se podrá construir y abrirse multitud de interfaces de usuario sin que esto afecte a la lógica de la aplicación.



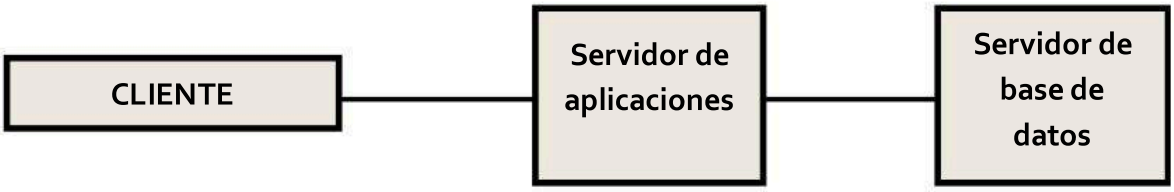
A la hora de realizar un sistema distribuido, es muy importante distinguir estas capas ya que lo que queramos conseguir es que cada capa vaya en un ordenador diferente.

Importante

La arquitectura de tres niveles contiene un nivel intermedio, la arquitectura está constituida por tres capas:

- Capa de presentación.
- Capa de aplicación lógica o de negocio.
- Capa de gestión de datos o de datos.

En este tipo de arquitectura la aplicación se comparte entre el Cliente, un servidor de aplicación y un servidor de base de datos.



En esta arquitectura existe una descentralización, es decir, cada uno de los servidores realizan una tarea determinada.

La arquitectura por capas es un procedimiento de Software propia de la programación orientada a objetos.

Si se sigue paso a paso este modelo facilitará que el proyecto se estructure de forma ordenada, esto se debe a que al ser dividida se puede manejar las capas de forma independiente pero de forma paralela.

Esto hará también que sea más fácil y cómodo actualizar o realizar modificaciones en la aplicación.

El procedimiento por capas sigue un Estándar de desarrollo que hará que la tarea no sea tan molesta y pesada.

Al utilizar este tipo de arquitectura de tres niveles se puede mantener la mayor parte del código en el nivel de aplicación o de negocio, con lo que se aísla de la interfaz del usuario y de la base de datos, con lo que se conseguirá hacer modificaciones en el código del nivel de aplicación sin que afecte a ninguna de las otras capas.

Este tipo de arquitectura es muy dado en el ámbito Web.

Al integrar un servidor de aplicaciones, la interfaz con el Cliente va a seguir siendo HTTP y el servidor de aplicaciones contendrá y tendrá acceso externo de un servidor Web..

Las principales capas que componen este modelo son:



Capa de Presentación:

La capa de presentación representa la interfaz del usuario. Se trata de cómo se presenta la aplicación ante el usuario. El objetivo de esta capa será la de ayudar al usuario a interaccionar con la aplicación.

La interfaz usa patrones ya previamente definidos para cada tipo de aplicación y tipo de necesidad del usuario.

La función principal de esta capa será que el programa tenga una interfaz sencilla y fácil de utilizar, ya que el usuario es el que se va a encargar de usar el sistema, además se dará cuenta de fallos o de mejoras que se puedan hacer, así ayudara al equipo de desarrollo.

La interfaz no debe tener más información que la que se necesite, la información debe de ser concreta, clara y concisa.

Esta capa nos va a proporcionar una interfaz que es necesaria para poder visualizar la información y poder reunir los datos.

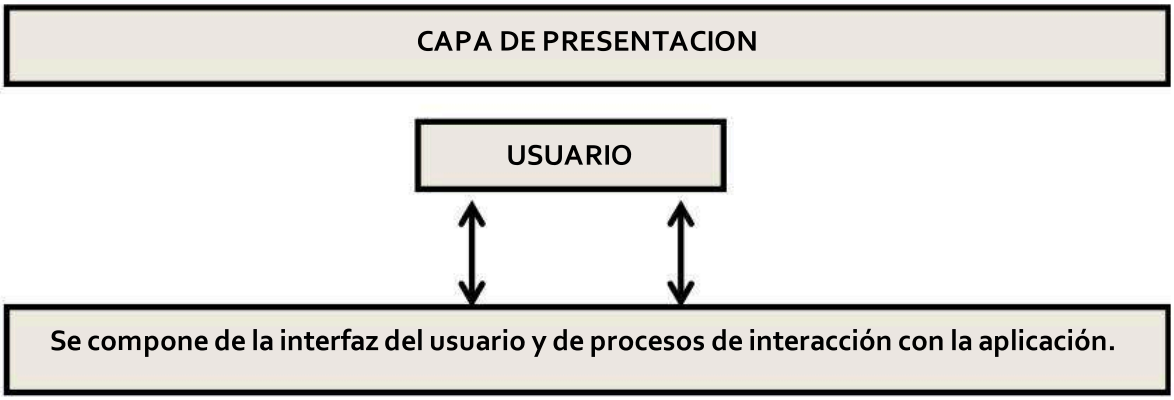
Los servicios que aporta la capa de presentación, son identificados en la interfaz del usuario, casi siempre es un programa que se encuentra en el ordenador del usuario final.

El cliente es el que facilita la presentación, normalmente se realiza por medio de Internet Explorer o Netscape u otro navegador web, que es el que va a permitir ver los datos remotos a través de una capa de presentación HTML.

Este tipo de modelo va a proporcionar que se separe adecuadamente la capa de presentación de la capa de aplicación y de la de base de datos u otros contenidos que se puedan encontrar en una página web.

Esto da la libertad necesaria a los desarrolladores de utilizar herramientas que le sean mas familiares para poder realizar modificaciones en la interfaz.

La capa de presentación es la responsable de la obtención de la información del usuario, enviar la información del usuario a la capa de negocios para que los datos sean procesados, recibir los resultados de la información ya procesados por la capa de negocios y se encargará además de la presentación de los resultados al usuario.



Funciones de la capa de presentación
– Obtención de la información del usuario. – Enviar la información del usuario a la capa de negocios para que los datos sean procesados. – Recibir los resultados de la información ya procesados por la capa de negocios. – Presentación de los resultados al usuario.



Capa de aplicación lógica o de negocios:

La capa de aplicación o de negocio, es la capa a de enlace entre el usuario y la base de datos.

Responden a las peticiones de los usuarios, se realiza por medio de reglas de negocio a los datos más importantes.

Con esto lo que se trata de conseguir es que no interaccionen directamente el usuario con la capa de datos o con la base de datos.

La capa de negocio hace de puente entre el usuario y la base de datos. Esta capa es la encargada de responder las peticiones de los usuarios.

Todo esto lo realiza aplicando procedimientos formales y reglas de negocio a los datos más importantes.

Cuando la información de la petición de un usuario reside en un servidor de base de datos, realizara la petición a la base de datos y garantizara que la petición se realice aplicando las reglas de negocio.

Una regla de negocio es una serie de normas de una empresa u organización, que servirá para alcanzar el objetivo de esta. Estas suelen estar definidas en la aplicación mediante un software.

Las reglas de negocio son unas condiciones o reglas estándares o reglas de una organización o empresa durante el flujo del proceso.

Con unas reglas de negocio se podrá diseñar un control de flujo de datos sencillo o de operaciones más complejas.

Después de procesar la información, vuelve al nivel de presentación para dar el resultado final al usuario.

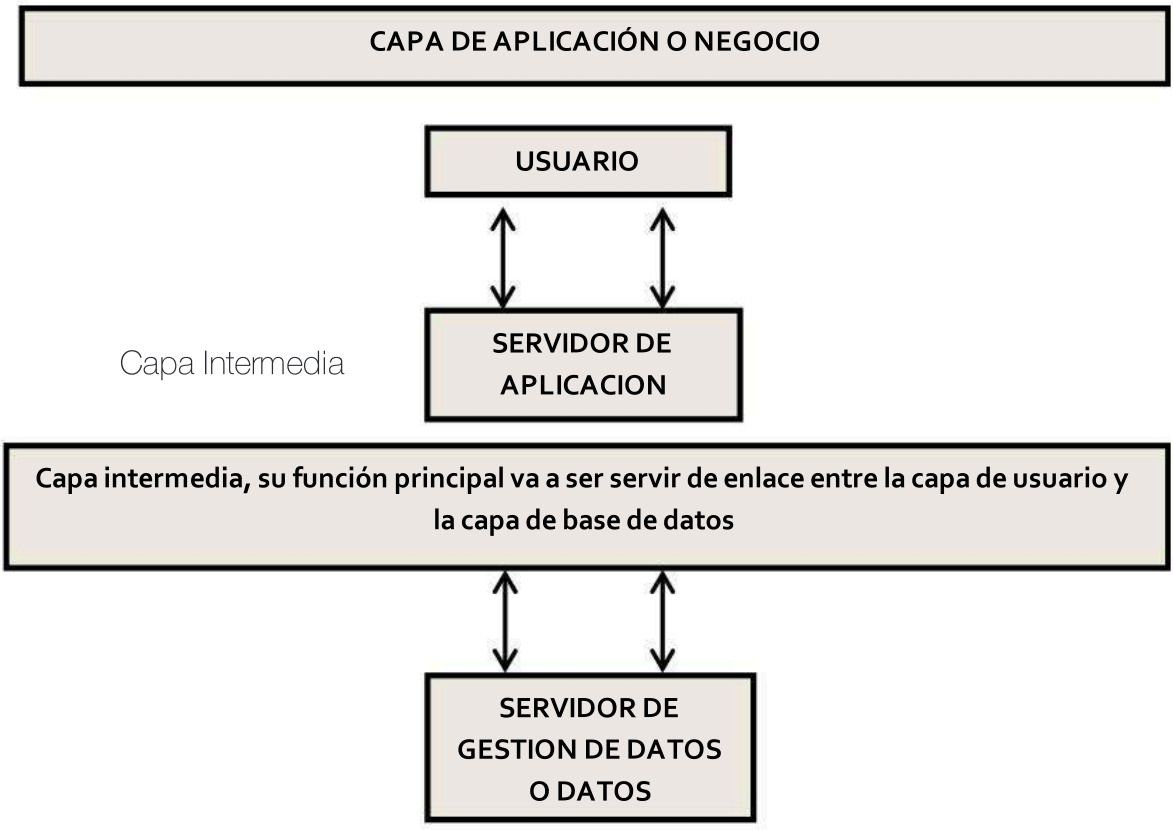
Esto hace que el usuario no interactúe directamente con la base de datos. Una tarea de negocio, es una operación para introducir por ejemplo una orden de compra o

imprimir un listado de artículos, las reglas de negocio son políticas que controlan el flujo de las tareas.

Las reglas de negocio son una parte que suele cambiar con frecuencia según las necesidades, por lo que lo mejor que se puede hacer es encapsularlos para estén separadas lógicamente de la lógica de la aplicación en sí.

La capa de negocio es responsable de recibir peticiones del usuario, interactuar con la base de datos para recabar los datos, como por ejemplo el procesamiento de una ordenes, la preparación de una factura, etc. Y además se ocupara de enviar el resultado a la capa de presentación.

Funciones de la capa de negocios.
– Recibir peticiones del usuario. – Interactuar con la base de datos para recabar los datos. – Enviar el resultado a la capa de presentación.





Capa de gestión de datos o de datos.

La capa de datos corresponderá al almacenamiento de los datos y al acceso a ellos. Esta capa es la responsable del tratamiento de gestión y de almacenamiento de datos.

Va a ser el encargado de suministrar al servidor de aplicación, los datos que le solicito. El servidor de base de datos permitirá la manipulación de grandes cantidades de datos.

Esta capa va a ser la encargada de almacenar, recuperar por medio de consultas, mantener y dar integridad a los datos.

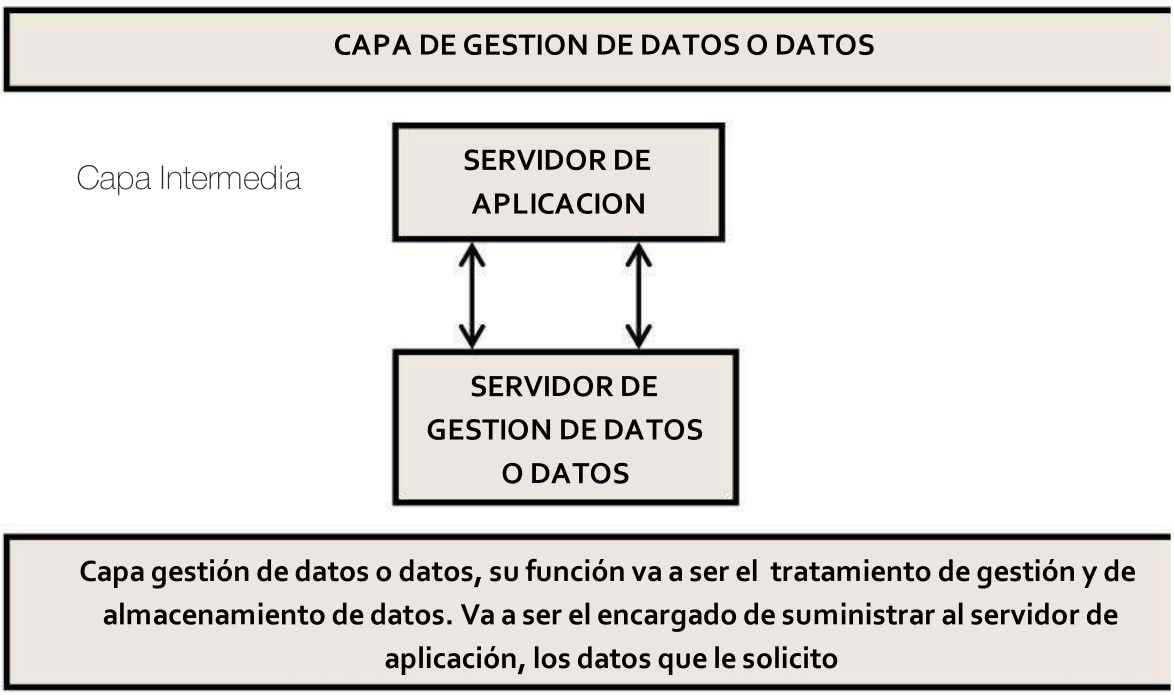
En esta capa se hayan los procesos de aplicación que van a permitir recibir las peticiones de las otras capas y dar respuesta con los datos solicitados. Este trabajo le corresponde a un servicio, un servicio es un proceso que se ejecuta en el servidor y que escuchan las solicitudes de datos del cliente.

El servicio de base de datos más usados en las aplicaciones distribuidas son SQL Server u Oracle. Existen una gran variedad de bases de datos, pero la más utilizadas son las bases de datos relacionales.

Funciones de la capa de base de datos.
– Acceso a datos.
– Tratamiento de gestión y de almacenamiento de datos.
– Enviar el resultado a la capa de aplicación.

Este tipo de servicio de base de datos tiene todo lo necesario para escuchar y responder a las peticiones de otros equipos Cliente, van a tramitar la concurrencia e incluso integran su propia seguridad. Una de las características importantes que tiene un sistema gestor de base de datos es que va a permitir crear reglas de negocio.

La mayoría de las aplicaciones utilizan una base de datos relacional para almacenar los datos de los datos. Las bases de datos relacionales, como SQL Server, dan mucha eficacia a la hora de administrar una gran cantidad de datos. Se podrán realizar acciones como, creación, modificación, actualización, consulta o borrado de datos.



2.3. Diseño de arquitecturas de aplicación basadas en el modelo multicapa

El diseño de aplicaciones más actuales se basan en la división de la aplicación en varias capas:

- Capa de usuario.
- Capa media de objeto de negocio.
- Capa de acceso a datos.

Las aplicaciones basadas en el modelo multicapa o de tres capas, deben de realizar al menos, cálculos y procesos de negocios, ejecutar las reglas de negocio, validar los datos que están relacionados al negocio, manipular los datos, ejecutar las reglas de datos relacional, interactuar con más aplicaciones e interactuar con otros usuarios.

Funciones de una aplicación basada en el modelo multicapa.
– Ejecutar las reglas de negocio. – Realizar cálculos y procesos de negocio. – Validar los datos. – Manipular los datos. – Ejecutar las reglas de datos relacionales. – Interactuar con más aplicaciones. – Interactuar con otros usuarios.

Estas acciones o servicios se reparten entre las tres capas de la siguiente forma:

- **Capa de presentación.**
  - Interactuar con otros usuarios.
  - Interactuar con aplicaciones externas.
- **Capa de negocios.**
  - Procesos de cálculos y otros tipos de procesos de negocios.
  - Ejecución de las reglas de negocio.
  - Validación de datos.
- **Capa de base de datos.**
  - Manipulación de datos.
  - Ejecución de las reglas relacionales.

Cada una de las capas se encargara de realizar unos procesamientos concretos:

Capa de Presentación	– Relacionarse con otros usuarios. – Relacionarse con aplicaciones externas.
Capa de Aplicación o de Negocio	– Realizar operaciones de cálculo y procesos. – Crear y ejecutar reglas de negocio. – Certificar los datos del servicio.
Capa de Gestión de datos o datos	– Manejar datos. – Ejecutar reglas de datos relacionales.



A este reparto de procesos entre las capas se le conoce como **Procesamiento Distribuido.**

Este tipo de reparto de procesos entre las capas es una distribución lógica no física.

Estas tres capas pueden encontrarse en un solo ordenador o repartir los procesos en varios ordenadores que estén conectados en red, sea cual fuere la organización física, la aplicación se va a desarrollar en tres capas.

Hemos hecho hincapié en que una aplicación distribuida es una aplicación que se divide en n_capas y la más común y popular es la de tres capas.

También hay que recalcar que existe una diferencia entre las capas lógicas, que será el software por partes, que tendrán una responsabilidad específica en cada una de la capas, y el nivel físico, que es el medio físico sonde se encontrará repartido cada uno de esos software específicos

Las características que debe de tener una aplicación basada en el modelo de tres capas son las siguientes:

Caracterísctica de las aplicaciones web
– Utilización de una base de datos.
– Transaccional, propiedades ACID.
– Escalables.
– Segura.
– Disponibilidad.
– Integración.
– Tipo de interfaz.
– Tener una arquitectura multicapa, de tres o más capas.
– Separación

Utilización de una base de datos.

Generalmente la utilización de una base de datos relacional.

Una base de datos relacional es un tipo de base de datos basado en el modelo relacional, este tipo es el modelo más utilizado para la creación de bases de datos que ya han sido planificados.

Este tipo de bases de datos permiten que los datos se relacionen mediante tablas, que es donde se encuentran almacenados los datos en una base de datos.



Transaccionales., propiedades ACID.

Toda aplicación que se base en el modelo multicapa debe de ser transaccional.

Con transaccional nos referimos a un conjunto de órdenes que se ejecutan como una sola, no pueden dividirse. Estas transacciones lo que hacen es dar integridad a los datos.

Una transacción es realizada una vez, se ejecutaran las ordenes unas después de otras hasta que finalicen.

Cuando una transacción no se ejecuta completamente, no se realizara, y deshará las ordenes que se hayan ejecutado ya, en el mismo orden.

Toda transacción debe de cumplir las cuatro propiedades ACID, que son:

- Atomicidad.
- Consistencia
- Aislamiento.
- Permanencia.

Atomicidad.

La atomicidad es una propiedad que lo que va a hacer es asegurar que una transacción se haya realizado con éxito o no.

Si ocurriera un fallo y no se pudiera realizar y se quedara a medias, no se realizara ninguna de las ordenes de la transacción y se deshará todas las ordenes que se hayan ejecutado ya.

Consistencia.

La consistencia es una propiedad que lo que consigue es dar integridad a la base de datos. Esta propiedad asegurara que si se realiza una transacción se finalizará, con lo que asegurara de que al realizar operaciones se puedan romper las reglas de integridad de la base de datos.

Una transacción podrá llevar a la base de datos de un estado valido a un estado no valido, cualquier dato que se agregue o modifique en la base de datos tiene que ser validado de acuerdo a las reglas ya definidas.



Aislamiento.

Esta propiedad lo que asegurará es que la ejecución de una transacción se realice concurrentemente, pero cada una es independiente de la otra.

Durabilidad.

Es la propiedad que asegurará después de confirmar una transacción ya se quede almacenada y da igual que haya cortes de suministro eléctrico o errores en el sistema.

Características de la arquitectura multicapa:

- Tener una arquitectura multicapa, de tres o más capas.
- Disponibilidad.
- Segura.
- Integración.
- Tipo de interfaz.

– Separación

– Escalables.



Escalables.

Es una característica en la que se desea que una aplicación web tenga flexibilidad para adaptarse si se sufre un crecimiento en el trabajo, teniendo así que soportar volúmenes de cargas más grandes sin tener que modificar el software, solamente añadir más estaciones de trabajo nuevas.

Disponibilidad.

Esta es una característica muy importante, ya que no pueden en ningún momento dejar de prestar los servicios que ofrece.

Segura.

Que sea segura, no permitirá acceder a todos los usuarios a todas las funcionalidades, algunos tendrán unos accesos y otros accesos diferentes.

Integración.

Sera preciso en algunas aplicaciones web integrar aplicaciones que usen diferentes tecnologías.

Tipo de interfaz.

Existirán dos tipos de interfaces, una que será para intranets y otra que se podrá utilizar tanto en Internet como en intranet.

– De entorno de ventanas. En intranet.

– De web. En internet y en intranet.

Separación clara de la interfaz gráfica y el modelo de aplicación web

Tener una arquitectura multicapa, de tres o más capas.

La arquitectura de una aplicación es una vista conceptual.

Todas las aplicaciones están formadas por el código de presentación, el código de procesamiento de datos y el código de almacenamiento de datos.

Importante

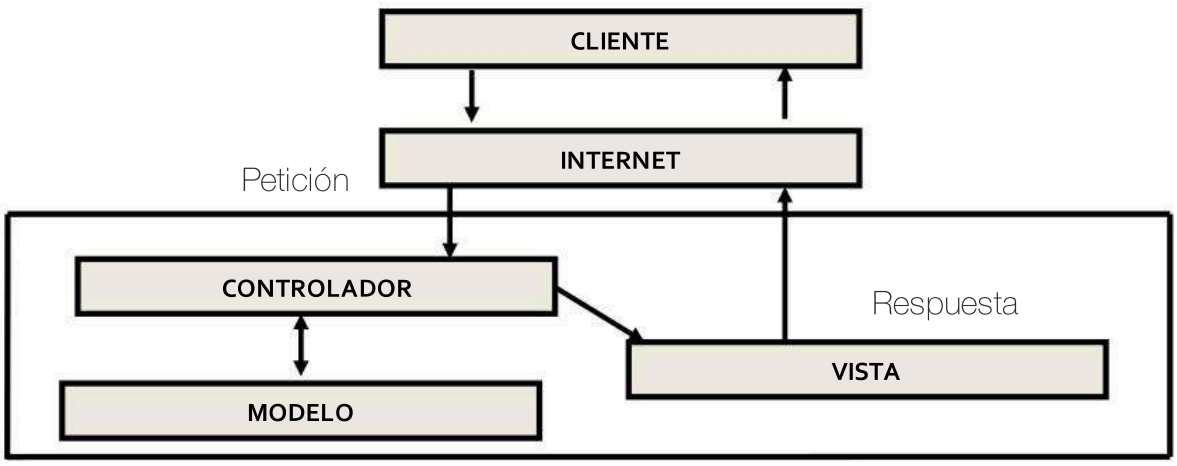
Para unificar las aplicaciones web se utiliza un patrón arquitectónico llamado, **modelo vista controlador MVC**

Este modelo fue creado para realizar una separación entre la interfaz grafica y el código de programación que hará que funcione la aplicación. Este tipo de patrón fue adquiriendo mucha importancia y este modelo se aplico a los lenguajes que están basados en objetos.

En el prototipo MVC, las entradas de usuarios, los modelos del mundo exterior y la retroalimentación visual están claramente separados y manejados por tres tipos de objetos, cada uno especializados en realizar unas funciones determinadas.

La función y objetivo principal del modelo MVC es la de dar soporte a los modelos según sus funciones y hacer una visión mental de la información que va a ser importante para los usuarios, así se permitirá realizar un modelo que facilite la consulta y manejo de la información.

El funcionamiento de este patrón es el siguiente:



Para desarrollar una aplicación basada en el modelo multicapa existe un patrón arquitectónico que se puede seguir para el diseño y construcción de la misma.

El más utilizado es el patrón ´modelo, vista, controlador´, llamado por sus siglas **MVC**.

El patrón **MVC**, consiste en distribuir una aplicación en tres partes bien diferenciadas unas de otras, de esta manera si se produce alguna modificación en alguna de ellas, no afecte en nada a las otras.



Las partes del modelo MVC son: Vistas, Modelo y controlador.

Partes del modelo MVC
– Vista
– Modelo
– Controlador

– **El Controlador:**

En esta parte se incluye todo lo que esté relacionado con la lógica de control de la aplicación, no tiene nada que ver con las características propias del negocio.

El controlador responde a las acciones de usuarios y solicita peticiones al modelo cuando se realiza alguna solicitud sobre algún tipo de información.

El controlador será el encargado de interpretar las instrucciones que realiza el usuario realizando acciones sobre el modelo, si se realiza algún cambio comenzara a actuar tanto si se produce una modificación en una vista o en el modelo.

El controlador hace de intermediario ente la *vista* y el *modelo*

– **El Modelo:**

En la parte del modelo es donde se implementa la lógica de negocio, es decir, se representa la información del sistema, es donde se realiza el tratamiento de todos los accesos a dicha información, como pueden ser las consultas o actualización implementando también los privilegios de acceso.

Se trata de un conjunto de clases que van a representar la información del mundo real que el sistema va a reflejar. Por ejemplo, la administración de una empresa tendrá un modelo que representara los empelados, los departamentos, los tipos de empleado, etc.

En esta parte no se tendrá en cuenta la forma en que se representa la información que se va a mostrar.

El modelo no tiene porque tener conocimientos de la existencia de las vistas y el controlador, en la práctica es difícil de aplicar ya que pueden existir interfaces que van a permitir a los módulos comunicarse entre si.

Esta parte envía a la vista la información que le solicita para que sea mostrada al usuario.

Las peticiones de acceso y manipulación de datos llegan al modelo a través del controlador.

- Vista:

En esta parte se encuentra el código encargado de presentar la información del modelo en el formato adecuado para que interactúen con el usuario. En una aplicación Web el formato será HTML o XML por ejemplo.

Las *vistas* serán las encargadas de representar los datos contenidos en el modelo al usuario, cada vista se asocia a un modelo pero a la vez pueden existir muchas vistas asociadas al mismo modelo.

La vista necesitara la información del *modelo* para realizar un despliegue. La vista cambia a través de notificaciones generadas por el modelo de la aplicación. Se tratara básicamente de la representación visual del modelo.

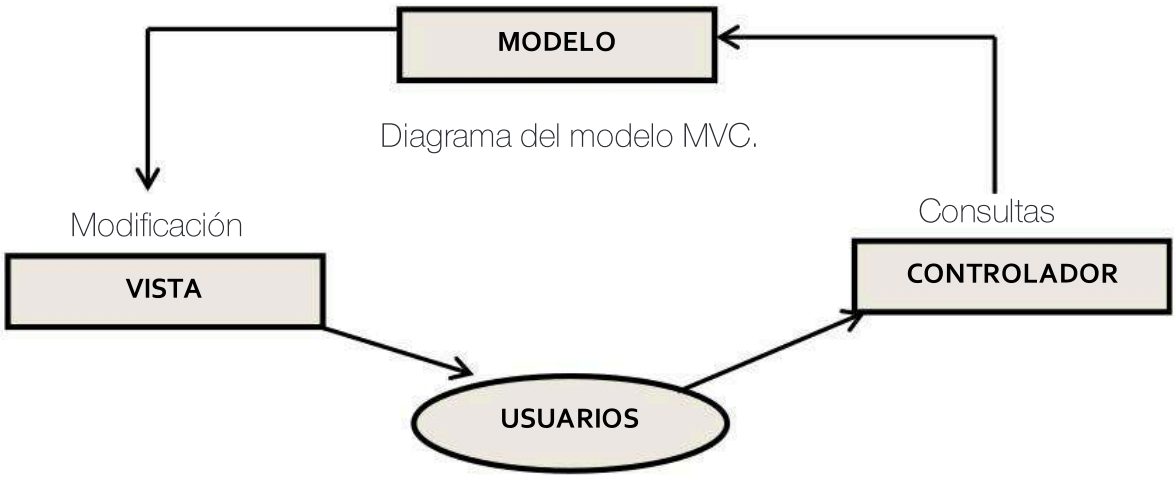


Diagrama del modelo MVC.

Para que este modelo funcione correctamente, estas tres partes tienen que interactuar entre sí.

El flujo de información se produce de la siguiente manera:

En primer lugar el usuario interactúa con la interfaz de usuario, por ejemplo pinchando en un enlace o en un botón.

El controlador recibe la petición que ha sido solicitada por el usuario, el controlador se encarga de procesarla utilizando si le hace falta los servicios del modelo, posiblemente se realizara una modificación que será la adecuada, solicitada por el usuario.

Una vez que se ha realizado el procesamiento de la información se entregan los datos a la vista y esta se encarga de formatearlos adecuadamente, la vista no interacciona con el modelo en ningún momento.

Las aplicaciones web utilizan este patrón, en el que el controlador recibe la petición de usuario en HTTP y las proveerá haciendo uso del modelo para la entrega de la información a la vista y esta se encarga de construir la respuesta en HTTP con las cabeceras idóneas.

El modelo MVC es un patrón de arquitectura de software que lo que va a hacer es separar los datos y la lógica de negocio de la interfaz del usuario y el modulo que se encargará de ejecutar las acciones y las comunicaciones.

Las ventajas que proporciona utilizar este patrón son:

- La implementación se realiza por módulos.
- Sus vistas siempre muestra información actualizada. El programador no tiene que estar preocupado en actualizar las vistas ya que esto se realizara directamente por medio del modelo de la aplicación.
- Cualquier modificación como por ejemplo aumentar datos contenidos o aumentar el numero de métodos, implicara solo la modificación en el modelo y las interfaces del modelo con las vistas, no afectara al mecanismo de comunicación y actualización entre modelos.
- Se modifican la representación de la información no su tratamiento
- Este modelo de diseño es un patrón bien elaborado ya que las aplicaciones que lo implementan presentado dos características muy importantes, son extensibles y su mantenimiento es mucho mejor que en otras aplicaciones realizadas con otro patrón.

Las desventajas que podemos encontrar son:

- Para desarrollar una aplicación utilizando el modelo MVC va a ser necesario utilizar mucho tiempo de desarrollo. Normalmente este patrón hace que el programador tenga que desarrollar un mayor numero de clases, que en otros entornos de desarrollo. Es una desventaja relativa, ya que luego para mantener la aplicación es mucho mas fácil, que en otra aplicación que no ha implementado este modelo.

- El MVC va a necesitar una estructura inicial sobre la que construir estas clases e interfaces para poder desarrollar , modificar y comunicar los módulos de la aplicación. La arquitectura inicial de de contener las acciones o eventos para proporcionar las notificaciones que va a generar el modelo, una clase Modelo, una clase Vista y una clase Controlador, estas se ocuparan de la comunicación y la actualización y serán totalmente transparentes para el desarrollo del a aplicación.

- El patrón MCV es un patrón orientado a objetos por lo que su implementación va a ser mas costosa en lenguajes que no siguen este modelo.

Una aplicación distribuida está formada por un conjunto de distintos componentes que se dividirán según sea;

- Del Lado del Cliente.

- Del Lado del servidor:



Lado del servidor:

Por el lado del servidor tendremos un programa que se ejecuta en uno o varios ordenadores conectados en red. Se encuentra esperando las peticiones del usuario cliente, por ejemplo un Servidor Web.

Lado del cliente :

Es el programa que utiliza el usuario de la aplicación para realizar las peticiones al servidor de aplicación por medio de una red. Por ejemplo, un navegador Web.

Protocolos comunicación Cliente/Servidor:

Es el lenguaje con el que se van a comunicar por ejemplo entre un navegador web y un servidor web, se realizara por medio del Lenguaje HTML, que definirá el formato de cómo se van a comunicar entre sí.

Los distintos componentes de una aplicación distribuida deberán de comunicarse entre ellos por medio de estándares y Frameworks de comunicación.

En la actualidad existen diferentes estándares para definir las características de un componente, como puede ser CORBA y J2EE..

CORBA	Es un Estándar muy conocido que lo que busca es la integración de aplicaciones, facilitar el desarrollo de estas en la lógica de negocio de una empresa u organización.
J2EE	Es un Estándar para el desarrollo de aplicaciones multicapa, hace más fácil y simple las aplicaciones, basándose en componentes modulares y estandarizados y gracias a este estándar las funciones de la aplicación se realizan de forma automática, sin necesitar una programación muy complicada.
DNA, ESB, XML Y .NET.	Estas tecnologías facilitaran a las empresas u organizaciones a trabajar en entornos más complejos, multiplataforma y con la capacidad de tener un conjunto de ordenadores separados físicamente y conectados entre sí, según el caso.

Si se quiere construir una arquitectura multicapa que sea independiente de los fabricantes, podremos utilizar tecnología que implementan interfaces estándares.

Recuerda 

Para construir la interfaz entre el servidor de aplicaciones y la base de datos, tenemos numerosas opciones que dependerán del lenguaje que se vaya a utilizar.

Algunos lenguajes utilizados son: Java, C, Visual BASIC, etc. Y el tipo de base de datos, que suele ser principalmente relacional, algunos de los tipos de bases de datos son, SQL Server, MySQL, Oracle, etc.

Esto se debe de plantear tras haber definido la base de datos o tras haber elegido el lenguaje de programación que se desea utilizar.

Para la creación interfaz entre el cliente y el servidor de aplicaciones, la opción más básica es el lenguaje HTTP.



Existen otras opciones que permiten mas interactividad entre el código en el cliente y el código en el servidor, como pueden ser los lenguajes script como VBScript, JavaScript, etc., estos se utilizaran para controlar los datos que se introducen y validarlos.

La comunicación entre el cliente y el servidor de aplicaciones se basa en preguntas y respuestas sincrónicas.

Lenguajes para la construcción de la interfaces entre servidor de aplicaciones
– Aplicaciones CGI. – ASP. – JAVA. – SERVLETS DE JAVA. – VISUAL BASIC. – PHP. – PERL – PYTHON – C

Lenguajes para la construcción de la interfaces entre cliente y servidor
– JAVASCRIPT – APPLETS JAVA – ACTIVE X. – VBSCRIPT – HTML

2.4. Análisis del concepto de lógica de negocio y significado de la capa lógica

Toda aplicación utiliza un código con el cual se podrán implementar las reglas de negocios, los procesos de cálculos o procesos de datos u otras actividades que tienen relación con los negocios.

Este código es el que va a formar parte de la capa de negocios, uno de los puntos más importantes a tener en cuenta del diseño lógico cliente/servidor, es mantener separadas totalmente la lógica de negocios de la capa de presentación y de la capa de datos.

Estas capas se pueden encontrar en un mismo ordenador, la división que se produce es una división a nivel lógico, al igual también podrán estar estas tres capas divididas en diferentes ordenadores.



Para separar la capa lógica de negocio, lo más utilizado es el uso de componentes orientados a objetos, en el que la lógica de negocio se encapsula en un conjunto de objetos o componentes que no tienen ningún tipo de código de servicios de datos o presentación.



Separando en capas lógicas la aplicación, se conseguirá flexibilidad a la hora de almacenar físicamente la capa de negocios.

Con esto se conseguirá ejecutar la lógica de negocio sobre un servidor de aplicaciones al cual accederán los clientes a un recurso centralizado.

Los servicios de negocios sirven como puente entre el usuario y los servicios de datos. Responderán a peticiones del usuario u otros servicios de negocios.

Esto se realiza aplicando procedimientos formales y reglas de negocio a los datos, cuando los datos que se necesitan se encuentran en un servidor de bases de datos garantizaran los servicios de datos que serán imprescindibles para cumplir las reglas de negocios.

Esto lo que hará es aislar al usuario para que no interactué de forma directa con la base de datos.

Una tarea de negocios se trata de una operación que será definida por los requerimientos de la aplicación, como por ejemplo imprimir una lista de artículos o realizar una orden de compra.



Las reglas de negocio son políticas que controlan el flujo de las tareas, tienden a cambiar con mas frecuencia que las tareas especificas de negocios a las que dan soporte, estas deberán de encapsularse en componentes que estén lógicamente separados de la lógica de la aplicación en si.

Los objetos de negocio son creados para representar un negocio, se puede crear un modelo con entidades y relaciones que se identifiquen con un negocio en concreto.

Un ejemplo muy básico de entidad es un cliente, un producto, un empleado, etc.

Cliente	Producto
– Cod_cliente int	– Cod_producto int
– Nombre varchar (30)	– Nombre varchar (30)
– Apellidos varchar(50)	– stocks varchar(50)

Todos estos objetos pertenecen también a objetos del mundo real, el objetivo de intentar representar objetos de la vida real va a ser la de crear una representación de esos mismos objetos y ponerlos dentro de una aplicación y hacer que estos objetos se relacionen como en la vida real.

Todas las lógicas de negocio no son iguales, existirán empresas que tengan la necesidad de tener una aplicación que pueda soportar procesos con intensivo flujo de datos y requerirán accesos mas rápidos, habrá otros que no lo requiera.

Concepto de lógica de negocio:

Se utiliza sobre todo en análisis y diseño orientados a objetos, es la parte del sistema que se encargara de los procesos orientados a un negocio, como puede ser la venta de un producto, controlar la contabilidad o realizar un inventario.

La lógica de negocio es conocida como un software que realiza una comunicación con un usuario por medio de una interfaz (capa de presentación), procesando los datos provenientes del usuario y la entrega del resultado de estos datos de nuevo a la interfaz del usuario es conocido como **lógica de negocio**.

Las aplicaciones distribuidas ofrecen multitud de soluciones para los sistemas que tenga gran concurrencia o necesidad de un código reutilizable.

Es muy importante donde se va a ubicar los componentes lógicos ya que influirá en el rendimiento, la reutilización del código o la facilidad en la programación.

En cualquier aplicación distribuida de deber de independizar completamente la capa de presentación de las reglas de negocio.



La capa de negocios será el responsable de:

- Recibir los datos del nivel o capa de presentación.
- Interactuar con los servicios de datos para ejecutar las operaciones de negocios para los que la aplicación fue diseñada.
- Enviar el resultado al nivel o capa de presentación.



La capa de presentación solo se encargará de la presentación y el manejo de los datos a través de uso de los objetos de la capa de negocio.

A la capa de negocio también se le llama capa intermedia, en una arquitectura de tres capas.

Cuando se crean la interfaz de usuario nunca hay que olvidar que las reglas de negocio no van a ir en esta interfaz, sino en las capas inferiores que es donde se contienen a los objetos.

La función de la capa de presentación solo va a ser la de manejar los datos y actualizarlos mediante los objetos de negocio.

No se puede olvidar esto porque si no se puede caer en el error de colocar las reglas de negocio dentro de la interfaz del usuario, y así no podrá volver a reutilizar el código y se complica la distribución y procesamiento de la aplicación.



La capa de negocio corresponde a la lógica de funcionamiento en cualquier aplicación distribuida.

En esta capa se localizan todos los elementos que podrán ser reutilizados, esta capa asila a la capa de presentación y a la capa del servidor de base de datos.

La función principal de esta capa va a ser la de reutilizar la lógica de negocio desde cualquier capa de presentación y conseguir actualiza y modificar la capa de datos de la base de datos.

La capa de negocio lo conforma el código para poner en funcionamiento las reglas de negocio , el procesamiento de datos y cálculos relativos al negocio.

La lógica del negocio se debe de mantener separada de la capa de presentación y la de gestión de datos, que se mantenga separar tendrá como objetivo ganar flexibilidad a la hora de almacenar físicamente la lógica de negocio.

Cuando está orientado a objetos, se deberá de capturar y guardar la lógica de negocios en un conjunto de componentes que no tienen presentación o código en del servidor de datos.

Los objetos de negocio, no realizan nada por sí mismos, sino que lo que hace es alojar un numero de atributos y vincularlos con otros. Los objetos de negocio servirán para reflejar el negocio.

Por ejemplo:

Empleado	Cliente	Producto	Venta
DNI	DNI	ID producto	DNI_empleado
Nombre	Nombre	Nombre	DNIR cliente
Apellidos	Apellidos	Descripción	ID_producto

Todos estos objetos pertenecen al mundo real, cuándo utilizamos un objeto de negocio, lo que intentamos es crear un objeto que lo represente dentro de la aplicación.

Lo que queremos conseguir con la utilización de estos objetos es que se relacionen entre si como si fuera en la vida real. Por ejemplo: Imprimir una lista de clientes.



La capa de negocios servirá para conectar al usuario con los servidores de datos. Responderán a las peticiones de los usuarios.

Las reglas de negocio son un conjunto de peticiones, reglas y normas de un negocio, estas forman parte de una aplicación informática. Hoy en día lo que se intenta es gestionar de forma organizada y centralizada las reglas de negocio y para ellos se pueden utilizar motores de regla de negocio.

Una de las características más importante de las reglas de negocio es que son **dinámicas**, esto quiere decir que, se pueden ajustar a los cambios y se pueden utilizar en todo tipo de aplicaciones.

Los motores de reglas de negocio, son unos componentes que a partir de un conjunto de reglas detectara que reglas deberán de utilizarse y cuál será el resultado de aplicarlas.

Un motor de reglas de negocio esta constituidos principalmente por tres elementos:

- Conjunto de reglas.
- Base de conocimientos.
- Procesador de reglas

El motor de reglas utilizara la base de conocimientos para poder decir que reglas se deberán de impulsar.



Según el diseño que se vaya a utilizar, existen dos formas de organizar la lógica de negocio:

- Por medio de un diseño procedimental.
- Por medio del diseño orientado a objetos.

En el diseño procedimental, se organiza el código en funciones y estructuras de datos simples, normalmente estas estructuras se pasan como parámetros a las funciones.

Las principales características que atare a los programadores a utilizarla son:

- Implementar funciones es fácil.
- No hace falta diseñar la aplicación.
- Es más fácil de aprender por personas con poca experiencia y
- conocimientos.

Este tipo de diseño funciona mejor si es para negocios con lógicas de negocio sencillas, pero si estas lógicas de negocio se complican puede llegar a crear grandes volúmenes de código. Esto hace que sea más difícil de entender y de mantener.



En el diseño orientado a objetos, se organiza en código en objetos que contienen dos características muy importantes que son los datos del objeto y su comportamiento.

Estos objetos interactúan entre sí para resolver la lógica de una aplicación, este tipo de diseño es más fácil de entender, manejar y de reutilizar, que el diseño procedimental.

El diseño orientado a objetos, tiene como finalidad realizar gestiones en desarrollo de aplicaciones con bastante complejidad, además de permitir que el código se reutilice para gestionar la lógica de negocio. Se necesitara crear objetos que describan correctamente el negocio o empresa.

Es muy importante para el programador diferenciar todos los objetos que va a necesitar una empresa, se debe de conocer todos los atributos y funciones o acciones que

va a tener cada objeto, esto hará que exista una abstracción para el desarrollo de una solución.

Ventajas
– Código reutilizable
– Escalable.
– Utilización de contenedores.
– Es simple.
– Orientación a objetos.

En conclusión la capa lógica de negocio será la que se encargue de la manipulación y del procesamiento de la información introducida, o solicitada de los datos por parte del usuario en la capa de presentación o por parte de la capa del servidor de base de datos, será el corazón de la aplicación.

Los datos se encapsulan, por lo que se conseguirá la integridad de los datos , para asegurar la existencia de duplicidades en los datos.

UD2
Lo más importante

- **El modelo cliente/servidor** empezó a ser usado en los años 80, su funcionamiento es muy simple, se tiene un ordenador cliente y un ordenador servidor, el ordenador cliente es el que requiere de un servicio de un ordenador servidor y este realizará la función para lo que estará programado.

- **La arquitectura Cliente/Servidor genérica**, se compone solo del Cliente y del Servidor, este tipo de arquitectura recibe el nombre de arquitectura de dos niveles o de dos capas.

- **La arquitectura de tres capas**, se produce cuando disponemos además del Cliente y del Servidor de aplicación, un servidor de base de datos, que su función será la de almacenar los datos de los servidores de aplicación

- **La arquitectura n_capas**, es un tipo de arquitectura que además de componerse del Cliente, Servidor de aplicación y Servidor de datos, pueden llevar otro o varios tipos más de servidores, como pueden ser, un servidor de chat, un servidor de correo electrónico, etc.

- Existen dos tipos de sistemas distribuidos:

    · **Arquitectura Cliente/Servidor**. El cliente va a ser el consumidor de un servicio que prestara el servidor, el cual actuará como proveedor del servicio.

    Representa un modelo lógico en el que las aplicaciones del servidor y las del cliente son diferentes. Varios servidores aplicaciones se pueden ejecutar en un solo ordenador.

- **Arquitectura de objeto distribuido**. En este tipo de arquitectura, no existe diferencias entre el cliente y el servidor. Se puede ver como una agrupación de objetos que interactúan entre sí.

– El **n_Tier** es el número de capas y la distribución física de estas, ¿qué queremos decir con esto?, que es donde suceden los procesos y se encuentra el código.

– El **n_layer** se refiere a como se reparten las capas lógicas, es decir, como va a estar organizado el código.

– La arquitectura de tres niveles contiene un nivel intermedio, la arquitectura está constituida por tres capas: **capa de presentación, capa de aplicación lógica o de negocio y capa de gestión de datos o de datos.**

  En este tipo de arquitectura la aplicación se comparte entre el Cliente, un servidor de aplicación y un servidor de base de datos

– Capa de Presentación:

  La capa de presentación representa la interfaz del usuario. Se trata de cómo se presenta la aplicación ante el usuario. El objetivo de esta capa será la de ayudar al usuario a interaccionar con la aplicación.

– Capa de aplicación lógica o de negocios:

  La capa de aplicación o de negocio, es la capa a de enlace entre el usuario y la base de datos.

  Responden a las peticiones de los usuarios, se realiza por medio de reglas de negocio a los datos más importantes.

– Capa de gestión de datos o de datos.

  La capa de datos corresponderá al almacenamiento de los datos y al acceso a ellos.

– Al reparto de procesos entre las capas se le conoce como Procesamiento Distribuido.

  Este tipo de reparto de procesos entre las capas es una distribución lógica no física.

- Una aplicación distribuida está formada por un conjunto de distintos componentes que se dividirán según sea del Lado del Cliente o según sean del Lado del servidor:

  · Lado del servidor:

    Por el lado del servidor tendremos un programa que se ejecuta en uno o varios ordenadores conectados en red. Se encuentra esperando las peticiones del usuario cliente, por ejemplo un Servidor Web..

  · Lado del cliente :

    Es el programa que utiliza el usuario de la aplicación para realizar las peticiones al servidor de aplicación por medio de una red. Por ejemplo, un navegador Web.

  · Protocolos comunicación Cliente/Servidor:

    Es el lenguaje con el que se van a comunicar por ejemplo entre un navegador web y un servidor web, se realizara por medio del Lenguaje HTML, que definirá el formato de cómo se van a comunicar entre sí.

- Concepto de lógica de negocio:

  Se utiliza sobre todo en análisis y diseño orientados a objetos, es la parte del sistema que se encargara de los procesos orientados a un negocio, como puede ser la venta de un producto, controlar la contabilidad o realizar un inventario.

- La lógica de negocio es conocida como un software que realiza una comunicación con un usuario por medio de una interfaz (capa de presentación), procesando los datos provenientes del usuario y la entrega del resultado de estos datos de nuevo a la interfaz del usuario es conocido como lógica de negocio.

- Los motores de reglas de negocio, son unos componentes que a partir de un conjunto de reglas detectara que reglas deberán de utilizarse y cuál será el resultado de aplicarlas.

- Las reglas de negocio son un conjunto de peticiones, reglas y normas de un negocio, estas forman parte de una aplicación informática. Hoy en día lo que se intenta es gestionar de forma organizada y centralizada las reglas de negocio y para ellos se pueden utilizar motores de regla de negocio.

UD2
Autoevaluación

1. La arquitectura Cliente/Servidor básica se compone de :

    a. Cliente y servidor.
    b. Cliente, servidor de aplicaciones y servidor de datos.
    c. Servidor de aplicación y servidor de datos.
    d. Ninguna de las anteriores es correcta.

2. Una arquitectura n_capas se compone de:

    a. Cliente, servidor de aplicaciones, servidor de datos.
    b. Cliente y servidor de aplicación.
    c. Cliente, servidor de aplicación, servidor de datos y otros servidores capaces de dar servicios.
    d. Servidor de aplicación, servidor de datos.

3. ¿Qué permitirá Middleware?

    a. Permitirán la comunicación entre las aplicaciones que están repartidas por una red de ordenadores.
    b. Nunca permitirán la comunicación entre las aplicaciones que están repartidas por una red de ordenadores.
    c. A veces permitirán la comunicación entre las aplicaciones que están repartidas por una red de ordenadores.
    d. Ninguna de las anteriores es correcta.

4. El n-Tier

    a. Es el número de capas lógicas.
    b. Es el número de capas lógicas y físicas.
    c. Es el número de capas y la distribución física de estas.
    d. Es el número de capas.

5. La estructura lógica del modelo multicapa se divide en:

    a. Capa de presentación y capa de aplicación o de negocio.
    b. Capa de aplicación o de negocio.
    c. Capa de presentación, capa de aplicación o de negocio y capa de gestión de datos.
    d. Capa de presentación y capa de gestión de datos.

6. ¿A que capa corresponderá la función de almacenamiento de datos?

    a. A la capa de gestión de datos.
    b. A la capa de presentación.
    c. A la capa de aplicación o de negocio.
    d. A la capa de presentación y a la de aplicación o de negocio.

7. ¿Qué patrón arquitectónico utiliza para unificar las aplicaciones web?

    a. VCA.
    b. MVC.
    c. NCV.
    d. CML.

8. Una aplicación distribuida esta formada por componentes que se dividen según…

    a. Sea del lado del servidor.
    b. Sea del lado del Cliente.
    c. Sea del lado del cliente o del lado del servidor.
    d. Las tres anteriores son correctas.

9. ¿Cuál será el objetivo de separar la capa lógica o de negocio?

    a. Ahorro económico.
    b. Ahorró energético.
    c. Perder flexibilidad.
    d. Ganar flexibilidad.

10. Los motores de negocio son:

    a. Un conjunto de peticiones, reglas y normas del negocio.
    b. Un conjunto de componentes que a partir de un conjunto de reglas detectara cuales deben de utilizarse.
    c. Un conjunto de reglas.
    d. Un conjunto de reglas y estándares.

Área: informática y comunicaciones

UD3

La capa de presentación

3.1. Descripción de la capa de presentación: El lenguaje de hipertexto

3.2. Descripción de la capa de presentación avanzada: El lenguaje de Scripting y lenguaje de hipertexto dinámico

3.3. Análisis de lenguajes orientados a la preparación de la capa de presentación y a la ejecución de solicitudes desde clientes ligeros Web. (JSP, SERVLETS, ASP, PHP)

3.1. Descripción de la capa de presentación: El lenguaje de hipertexto

La capa de presentación está compuesta por diversos elementos o componentes que va a implementar la interfaz de usuario de una aplicación web, esta interfaz, se ocupara de mostrar la información o datos y manipular las interacciones con el usuario.

Esta capa es la más cercana al usuario, y va a ser la capa que va a dar paso a la información desde las capas adyacentes, van a permitir la comunicación entre las aplicaciones en distintos sistemas informáticos. Esta capa forma parte del modelo OSI.



La función principal de la capa de presentación es la de ocuparse del formato, de cómo se va a visualizar la información.



La base de toda interfaz de usuario, es el lenguaje de marcado de Hipertextos HTML, en inglés (HiperText Markup Language), este es un lenguaje que proviene del XML.

La capa de presentación está compuesta por un grupo de componentes que son los que se encargaran de establecer la interfaz de la aplicación, visualizando información e interaccionando con el usuario.

El fundamento del interfaz de un usuario de una aplicación Web, es el Lenguaje HTML, es un lenguaje de marcado, este lenguaje es un derivado de XML, permitirá mostrar texto, imágenes , videos uy otras formas de interacción con el usuario, que se realizará por medio de un navegador web.

La utilización de HTML permitirá que se puedan mostrar videos, imágenes, audio, etc. Que interaccionan con el usuario mediante un navegador web, también llamado web browser.

Este navegador es el que se va a ocupar de descifrar el lenguaje HTML que se ha descargado desde un servidor web y visualizarlo en la pantalla de forma que el usuario le sea fácil, coherente y eficaz.

HTML posibilitara mostrar imágenes, textos, videos, etc., desde un navegador Web. El navegador Web es el que se ocupara de traducir y comprender este lenguaje y mostrarlo al usuario de forma lógica, funcional y atractiva.

Los componentes que conforman la capa de presentación se dividen en dos:

Componentes de presentación	Son los componentes de la interfaz que dejan al usuario interaccionar con la aplicación. Todos los componentes que vienen incluidos en el lenguaje HTML. Como por ejemplo: una etiqueta, un botón de acción, tablas, etc. También se pueden utilizar componentes en lenguaje Scripting.
Componentes de procesos	Son componentes que se utilizan para interacciones con el usuario, más complejas. Estos complementos se conseguirán utilizando un FrameWork de Java Script.

Descripción de la capa de presentación: El lenguaje de hipertexto. (Continuación).

A la hora de llevar a cabo la capa de presentación debemos de tener en cuenta los siguientes aspectos:

Decidir visualización de datos.
Validación de datos.
Decidir lógica de negocio.
Decidir comunicación.

– Habrá que decidir cómo se van a visualizar los datos. Se trata de dar formato a la capa de presentación, determinando como se van a visualizar los datos en la interfaz del usuario.

Por ejemplo: si se van a poner columnas o si se van a utilizar imágenes, etc.

– Dar autenticidad o validar los datos, al utilizar HTML o JavaScript ya protegerá la aplicación Web de entradas que no queramos.

– Decidir la lógica de negocio, en este caso se decidirá cómo será el procesamiento y la presentación de los datos.

– Habrá que decidir cómo se van a comunicar las diferentes capas, se tendrá que elegir como se va a comunicar la capa de presentación con la capa de negocio y con la capa de datos.

  Con la utilización de una vista controlador se puede hacer más sencillo este diseño.

  Otros aspectos a tener en cuenta en el diseño de la capa de presentación:

Utilizar modelos de diseño.
Separación de componentes.
Buen uso del diseño.
Necesidades del usuario.

– Utilizar modelos de diseño, esto hará más fácil el manejo y mantenimiento de esta capa.

– Se deberá de realizar una separación de los componentes de presentación y los componentes de proceso. Crear componentes para mostrar los datos y crear otros componentes para realizar operaciones más complejas del usuario.

– Tener en cuenta que se están utilizando buenos usos en el diseño del interfaz del usuario. Con esto nos referimos, a la utilización de logotipos de la empresa u organización, colores que se van a utiliza, las imágenes, la forma de leer la página Web, etc.

  Todo esto dependerá de la actividad de la empresa u organización.

– Tener en cuenta siempre las necesidades que puedan tener los usuarios, intentando que el diseño sea apropiado para él.

Cuando se desarrolla el diseño de la capa de presentación suelen aparecer problemas que son muy comunes a la hora de desarrollar esta capa, son:

Problemas de cache:

– Pueden existir datos que no sirvan para nada, que no se puedan reutilizar.

– Se podrán guardar datos que son importantes y que se encuentren sin encriptar.

– Que se haya realizado una selección incorrecta del almacenamiento para la cache.

– Seleccionar una cache incorrecta.

– Pensar que un dato se encuentra en la cache pero en realidad no se encuentra.

Composición de la interfaz web:

– No utilizar patrones de diseño y frameworks que puedan soportar vistas y páginas dinámicas en tiempo de ejecución.

– Uso de componentes que tienen dependencia de servicios o de clases auxiliares.

– Fallo al no tratar correctamente las acciones entre los componentes y fallos al no separar correctamente los módulos que forman la aplicación.

Tratamiento de errores:

– Fallo al eliminar recursos.

– Fallo al no tratar excepciones.

Entrada de datos:

– Fallo al implementar una entrada de datos que no sea sencilla.

– Fallos en el diseño de la accesibilidad.

Disposición de componentes en la pagina:

– Fallo al utilizar elementos que son inapropiados para una pagina web.

– Fallo al utilizar componentes y tecnología.

– Implementar una interfaz que no sea apropiada para el flujo de trabajo.

Navegación:

– Navegación frágil.

– Lógica duplicada a la hora de tratar acciones de navegación.

Tratamiento de peticiones de usuario:

– Mezclar lógica de proceso con lógica de presentación.

– Tratamiento de peticiones de usuario incorrecto.

– Bloquear la interfaz de usuario durante largos periodos de tiempo.

Experiencia de usuario:

– Falta de respuesta cuando ejecuta una acción un usuario.

– Interfaces de usuario muy complejas, ineficientes y no personalizadas.

Componentes de presentación:

– Crear componentes que no son necesarios.

– Fallos para mantener el patrón vista controlador.

Componentes de proceso:

– Implementar componentes que son innecesarios, patrones de diseño incorrectos y mezclar la lógica de presentación con la lógica de la interfaz de usuario.

Validación:

– Error al validar los datos de entrada.

– No identificar las reglas de negocio.

A continuación vamos a presentar los diferentes tipos de lenguajes utilizados, pero deberemos diferenciar los lenguajes del lado del Servidor y los lenguajes del lado del cliente.

El lenguaje HTML se basa en el metalenguaje SGML, HTML es el formato de los documentos de la World Wide Web, esta organización es la que desarrollo los estándares para el desarrollo de la web y es la que concreta las especificaciones que tienen que ver con el lenguaje HTML.

HTML fue creado como un lenguaje para el intercambio de documentos. HTML solucionó el problema que se tenía utilizando SGML, ya que era bastante complejo, mediante HTML se consiguió resumir un poco y con solo un reducido número de etiquetas estructurales y semánticas se pueden realizar documentos sencillos.

HTML se hizo muy popular, y desde que comenzó a utilizarse se han ido creando nuevos elementos para que HTML pudiera usarse como un estándar y se adaptó para que tuviera más funciones web,

Como es la utilización de elementos multimedia o la utilización de elementos con animaciones , que hacen que las páginas web sean más llamativas para el usuario.

Los documentos HTML son archivos de texto, que se pueden crear mediante editores de texto, pero también existen programas concretos para editar un documento HTML, los editores más conocidos son FrontPage de Microsoft, Netscape y Dreamweaver, todos estos programas tienen como objetivo crear páginas web en HTML.



HTML se utiliza como un lenguaje de presentación, los archivos en HTML se leen mediante un navegador web como puede ser Mozilla o Internet Explorer, la presentación de la página puede variar según el navegador que se vaya a utilizar, HTML solo se ocupa de definir la estructura y como va a ser el contenido, pero no se ocupara del formato de la página web ni de la apariencia.

La WWW, tiene una presentación atractiva para el usuario. Todos los documentos WWW tiene un mismo aspecto y una única interfaz, esto hará que sea más fácil el manejo para cualquier persona.

Esto se hace posible porque el lenguaje HTML permitirá establecer enlaces a diferentes documentos y es independiente de la plataforma que se vaya a utilizar.

Un documento HTML contiene toda la información necesaria sobre cómo se va a visualizar la información y cómo van a interaccionar con el usuario, y es mas tarde el navegador que vayamos a utilizar el que se asegurara que el documento tenga un aspecto coherente, independientemente del tipo de ordenador en el que se esté realizando la consulta.

Los archivos generados con HTML tienen la extensión htm o html, es un lenguaje muy simple que se basa en un lenguaje de marcas cuya misión va a ser la de enlazar documentos o partes de documentos.



Existen dos herramientas para la creación de un documento HTML, una va a ser un editor HTML, que se utiliza para crear documentos HTML y otro es un navegador, para poder visualizar estos documentos.

Para crear un documento html se necesita
– Un procesador de textos o un editor de documentos HTML. – Un navegador web

Editores

Los documentos HTML se pueden crear con un editor de texto normal, como puede ser Word , o el bloc de notas de Windows, este archivo creado con un editor de texto será interpretado por el navegador , siempre y cuando el archivo o documento este guardado con extensión htm o html.

Estos crean texto sin formato.

El texto que utilizamos para crear el documento tiene dos partes, uno será el contenido de la información y el conjunto de las etiquetas HTML, mediante estas etiquetas se podrá determinar el estilo o tipo de letra que queremos que tenga nuestro documento.

– Contenido de la información. – Etiquetas HTML.

Se podrá escribir párrafos sin etiquetas de formato, pero a la hora de visualizarlo por medio de un navegador web, este no hace caso de los espacios ni tabulaciones, con lo que se corre el riesgo que la información no se visualice correctamente.

En la actualidad existen muchos editores que simplifica la tarea, y no se tendrá que trabajar ya con texto plano y etiquetas en código HTML, sino que esos programas de edición van a dar la posibilidad de que mientras se crea se insertará los comandos HTML de forma automática.

Por ejemplo:

- FRONTPAGE.
- DREAMWEAVER.
- NOTEPAD.
- HOMESITE.
- ULTRAEDIT.
- EDITPLUS

Los editores van a dejar introducir elementos multimedia, imágenes, audio, video, JavaScript, etc. Además de incorporar una serie de plantillas con modelos de páginas, orientados según el tipo de página que se quiera hacer y también tienen estilos predeterminados para dar uniformidad a todo un sitio web.

Los programas utilizados en la actualidad no solo se van a ocupar del formato de la WWW, sino también de estructura, mostrar la localización de un enlace, crear tablas de contenido web, índices, etc.

En la actualidad se han creado una variedad de sistemas para gestionar Hipertexto y muchos de ellos son gratuitos, aunque los de pago son los que suelen ofrecer muchas más prestaciones.

La utilización de estos editores de páginas web tienen desventajas frente a los editores de texto sin formato, y esto se debe a que va a depender del software propietario y puede presentar incompatibilidades entre los navegadores, también produce gran cantidad de etiquetas que son repetitivas. Estas etiquetas no afectan a como se va a ver la página.

También se pueden combinar las dos cosas un editor de HTML y un editor de testo sin formato.

Existen tres tipos de editores HTML:

- Los que visualizan mientras se crean.
- Los que necesitan un navegador.
- Los mixtos.
    - Los que se van creando y muestran en pantalla de forma inmediata lo que se está creando.

- Los que van a necesitar un navegador para mostrar lo que se está creando
- Y los mixtos, que se divide en dos pantallas, una visualizando el código HTML y otra visualizando el resultado final.



Las etiquetas utilizadas en el lenguaje HTML son las mismas que se utilizan en el lenguaje SGML, se trata de marcas o etiquetas que van enmarcadas por los signos <>, todos los documentos HTML comienzan con la etiqueta <html> y terminan con la etiqueta </html>.

Dentro de estas etiquetas existen dos zonas, el encabezamiento, que se va a identificar con la etiqueta <HEAD> y para finalizarla </HEAD>, este encabezamiento va a servir para una serie de valores en todos el documento. Y el cuerpo del documento, que se representa con la etiqueta <BODY> y para finalizar se pone </BODY>, esta etiqueta se encargara de mostrar la información del documento.

Dentro del encabezamiento, la etiqueta se encuentra la etiqueta <TITLE> que corresponderá con el título.

En el cuerpo se pueden establecer el tipo de fuente y establecer la estructura más básica del documento.

Dentro del cuerpo se va a poner el contenido de la página, como pueden ser tablas, formularios, imágenes, texto, etc., la información se puede estructurar en párrafos, enlaces a alguna parte del mismo documento o a distintos documentos.

También se podrá establecer el tipo de letra y el estilo, como pueden ser los de final de línea que sería
 y el final de párrafo que sería <p>, para que el texto aparezca en negrita se utilizara <b>, si es cursiva >i>, si el texto va aparecer centrado seria>center>, etc.

Los elementos más característicos del lenguaje HTML son los anclajes que se utilizara para realizar los enlaces, ya sea de texto o gráfico.

Para poner un enlace, el texto debe de aparecer subrayado, para que el usuario pueda pulsarlo y le dirija a donde está la URL.

La etiqueta de anclaje debe de incluir el parámetro href="URL" y el texto para pulsar, si se van a insertar imágenes se utilizara la etiquetas ="URL".

Existen una gran cantidad de etiquetas cada una realizara una función determinada.

Estructura de un documento HTML:

Cualquier documento HTML va a comenzar con la etiqueta <html> y va a terminar siempre con la etiqueta </html>.

Dentro del documento debemos de diferenciar dos partes:

Partes principales de un documento HTML
– Encabezamiento. – Cuerpo.



– **El encabezamiento**

El encabezamiento va a ser utilizado para definir valores para todo el documento, el encabezamiento comienza con la etiqueta <HEAD> y va a finalizar con la etiqueta <HEAD/.

Dentro del encabezamiento nos encontraremos con el elemento <TITLE> que se utiliza para especificar el título del documento HTML.

– **El cuerpo.**

El cuerpo es donde va a estar toda la información o contenido del documento. El cuerpo comienza con la etiqueta <BODY> y finaliza con la etiqueta </BODY>.

Existen muchos más elementos dentro del encabezamiento, pero la estructura básica de un documento HTML, va a ser este.

```
<HTML>

<HEAD>

 <TITLE>

 </HEAD>

 <BODY>

  Aquí se coloraran diversas etiquetas que van a ser posible la visualización de la página.</BODY>

</HTML>
```

La estructura básica de un documento HTML es la siguiente:

<HTML> Etiqueta comienzo de documento

<HEAD> Etiqueta comienzo del encabezamiento

<TITLE> Etiqueta de título de la pagina

</HEAD> Etiqueta final del encabezamiento

<BODY> Etiqueta comienzo del cuerpo

Aquí se coloraran diversas etiquetas que van a ser posible la visualización de la página.

</BODY> Etiqueta finalización del cuerpo

</HTML> Etiqueta finalización del documento HTML.

El texto puede contener cabeceras, que se representa mediante la etiqueta <h1>,<h2>, etc., hasta seis cada número representa el tamaño del texto, el más grande seria el 1.

Existen otras etiquetas como <p> que servirá para distinguir párrafos, la etiqueta <center y que se utilizara para centrar el texto o todo lo que haya entre esta etiqueta, una serie de etiquetas más., para la apariencia del documento.

A continuación vamos a ver los tipos de etiquetas existentes y que se encuentran introducidas en el cuerpo del documento, o sea, <BODY>

Tipos de etiquetas
– Etiquetas para definir estilos y efectos básicos. – Tablas. – Listas. – Saltos de línea. – Estilo y tipo de letra. – Texto



Etiquetas para definir estilos y efectos básicos.

– **Título.**

Mediante la etiqueta de títulos se puede organizar el documento en varios niveles según la importancia. HTML tiene seis niveles de cabeceras que llegan desde el número 1 al número 6.

Es para poner el título más grande o más pequeño según la importancia de la información.

Se representa de la siguiente forma:

Código	Visualización
<h1> cabecera de tipo 1	Será el título más grande, se visualizara como título principal o título más importante del documento.

A partir de aquí hasta seis, ira disminuyendo la importancia del título.



Texto

Las etiquetas que actuaran sobre el texto son:

- , esta etiqueta representara a los párrafos.
- <pre>…. </pre>, con esta etiqueta se identificara si un texto ha sido formateado antes.

Estilo y tipo de letra.

Mediante algunas etiquetas se puede determinar el estilo y el tipo de letra que queremos que tenga el documento final.

- <u>…</u>,
- <i>…>/i>,
- ….,

- Si se pone …., es para poner la letra en negrita.
- Si se pone <i>…>/i>, es para poner la letra cursiva.
- Si se pone <u>…</u>,es para subrayas texto.

Saltos de línea.

HTML por sí solo no reconoce los saltos de línea, por lo que necesitaremos utilizar la etiqueta
, que se ocupara de desplazar el texto a la línea siguiente, también la etiqueta lo desplaza pero con una línea más de separación

Existen dos comandos para realizar el salto de línea.

-
-

- , esta etiqueta es para fijar los párrafos, si se pone tal cual, el párrafo se alineara a la izquierda.
- ….. , que alineara el párrafo a la derecha.
- … , el párrafo se alineara en el centro.



Listas.

Se pueden crear listas, existen tres tipos de listas, las numeradas, las no numeradas y las listas ya definidas

- Listas no numeradas
- Lista numeradas
- Lista definidas

- **Listas no numeradas**, mediante la etiqueta <ul> introducción de elementos de la lista con la etiqueta <li> y cierre de la etiqueta .

- **Lista numeradas**, mediante la etiqueta <oll> introducción del listado con la etiqueta <li> y cierre dela etiqueta <ol>.

- **Lista definidas**, mediante la etiqueta <dl> seguido de <dt> más el primer término a definir, luego se pone la etiqueta de definición del primer término y cierre de la etiqueta >/dl>.

Tablas.

Las principales etiquetas para las tablas son, <table>…</table>, <table border>… </table>,<tr>….</Tr>, <TD>…</TD>,<TH>…</TH>. y <caption>…</caption>

Etiqueta	Misión de la etiqueta
<table>…</table>	Es la tabla en sí, que va a contener los datos
<table border>… </table>	Etiqueta para tablas con bordes.
<tr>….</Tr>	Establece una fila dentro de una tabla
<TD>…</TD>	Define una celda dentro de una tabla
<TH>…</TH>	Centra una cabecera de una tabla.
<caption>…</caption>	Fija un título en la parte superior de la tabla.



Enlaces.

Las etiquetas que se utilizan para determinar enlaces son:

- …, esta etiqueta va a definir el destino de un documento.
- …., con esta etiqueta se determina el destino en el mismo documento.
- …, esta etiqueta determinara el vínculo a otro archivo.
- < a href =URL#nombre_vinculo">…, esta etiqueta determina un vínculo a un destino en otro documento.

…,

….,

…,

…

3.2. Descripción de la capa de presentación avanzada: El lenguaje de Scripting y lenguaje de hipertexto dinámico

Para realizar la capa de presentación más avanzada se utilizan lenguajes de hipertexto dinámico, es decir, HTML dinámico y lenguajes de scripting.

- HTML dinámico o DHTML.
- Lenguajes de scripting.



Lenguaje HTML dinámico.

También llamado DHTML, este tipo de lenguaje abarca un conjunto de técnicas para obtener dos objetivos claramente definidos, proporcionar control absoluto al diseñador de páginas HTML y dejar de lado el carácter estático de este tipo de documentos, que hasta el momento , un documento HTML que se había mostrado ya no se podía modificar.

El HTML dinámico va en contra del HTML, que como ya hemos analizado en el apartado anterior, es una norma diseñada para ser completamente independiente de la plataforma utilizada, ya que en sí, su propio diseño es independiente..

Tanto el diseño de páginas web es tan importante como la independencia de la plataforma utilizada.

Existen dos variedades de DHTML, una son las hojas de estilo y otra el uso de los objetos.

Muchos de los elementos de HTML varían según el navegador, existen componentes que son comunes en ambos., en HTML y EN DHTML que son, las hojas de estilos y la utilización y colocación de objetos.

- Control de la colocación de objetos y diseño.
- Las hojas de estilo en cascada.



Las hojas de estilo en cascada.

Es te concepto es fácil de entender, cuando utilizamos por ejemplo un documento Word tiene una serie de plantillas que van a determinar el estilo que va a tener las partes de un documento, pues estas plantillas son hojas de estilo.

Estas hojas de estilo definen como se va a presentar las partes del documento, ese es el objetivo que tienen también las hojas de estilo en cascada.

Las hojas de estilo en cascada es la forma establecida por WWW, por lo que aparecerán en los navegadores, como Netscape o Microsoft Internet Explorer.



Control de la colocación de objetos y diseño.

Una de las características más importante de HTML dinámico es el control por parte de los desarrolladores, sobre el diseño de los documentos y colocación de los objetos en la página.

Tanto Netscape con internet Explorer se dedican a innovar en esta materia y comparten algunas cuestiones, como las extensiones de las hojas de estilo en cascada para colocar objetos, cada uno de los navegadores tienen una forma diferente de colocar estos objetos.



El modelo dinámico de objetos es el núcleo de HTML dinámico, el modelo de objetos forma un mecanismo, mediante el cual se pueden realizar scripts con página, cambios de programación de estilos y atributos de componentes de la página, reemplazar elementos y reaccionar ante las acciones del usuario.

El modelo dinámico de objetos va a permitir al desarrollador trabajar con todos los objetos que se encuentren en la página web.

Para crear estos scripts y programas se pueden utilizar cualquier lenguaje de script que sea soportado por los navegadores, como por ejemplo JavaScript y VBScript.

Para realizar estos scripts se necesita conocimientos de programación en cualquier lenguaje script que este al lado del cliente.



Los lenguajes del lado del Cliente, corresponderán a los lenguajes que no tendrán que transformar los datos para que sean entendibles por el navegador.

Son lenguajes que entienden perfectamente el navegador Web, ya que se realizaran en HTML, y no tendrán que ser pre tratados. El lenguaje del lado del cliente es independiente del servidor, esto hará que la página pueda albergarse en cualquier sitio sin necesidad de pago.

Como el lenguaje HTML por sí solo no puede controlar los elementos dinámicos de una web, con esto nos referimos a los elementos con efectos, se incrustan scripts o pequeños programas en el código HTML, de esta manera cuando al navegador le llega, es capaz de interpretar y dar formato al contenido de la página, también proporcionara los scripts a un programa que interpreta estos y estará instalado en el navegador, recibe el nombre de **plug-in**.

Estos plug-in también pueden encontrarse en el servidor, que ya lo convertirá en HTML sin tener que instalarlo en el navegador.



Un script se trata de un programa que suele estar dentro del contenido de HTML. Cuando se ejecuta, se cargará todas las instrucciones de dicho programa, a esta acción se le denomina **evento**.

Los Scripts aportaran:

Que el documento sea dinámico al producirse la carga
Controlar eventos.
Validación de datos.

En un documento HTML se utiliza dos tipos de script, el que se ejecuta cuando se descarga una pagina por medio de un navegador y los que se ejecutan en un navegador medio de un evento, por ejemplo, pulsar un botón.

Importante

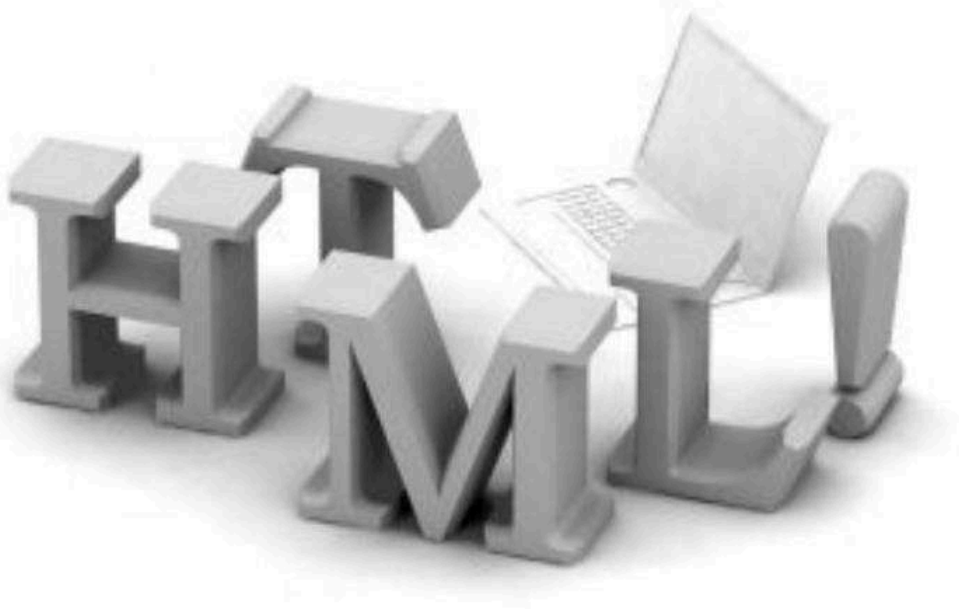
Los lenguajes de programación del lado del cliente más utilizados son HTML, JAVA y JavaScript (estos se incluyen en el código HMTL), aunque encontramos muchos más lenguajes.

Lenguajes del lado del Cliente:

HTML	Javascript	Applets de java
Visual basic script	Flash	CSS

Tipos de lenguaje del lado del cliente:

HTML:



Logotipo de HTML.



En ingles HiperText Markup Langage, es un lenguaje de marcado para elaborar páginas Web, se utiliza para determinar el contenido de una página web, como puede ser el texto, imágenes, videos, etc.

Este lenguaje consiste en etiquetas que tiene el siguiente formato:

<" ">, por ejemplo si se pone <P> significa párrafo y cuando se acabe este se cerrara con </P>.

Servirá para determinar el estilo de la página Web.

Es un lenguaje interpretado, con lo que no requerirá compilación. (Ya explicado en la pregunta anterior).

Ventajas:

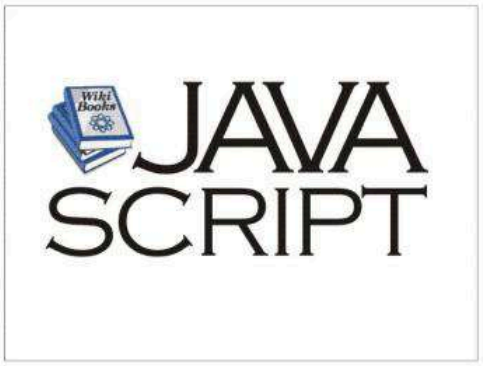
- Es sencillo.
- Forma estructurada y y amena.
- No es necesario tener muchos conocimientos.
- Rápido desarrollo.
- Lo aceptan todos los navegadores.
- Son archivos pequeños.

Desventajas:

- Es un lenguaje estático.
- La interpretación de cada navegador puede ser diferente.
- Limitación en el diseño siendo más lento.

Importante

JavaScript:



Logotipo JavaScript.

Es un lenguaje de programación orientado a objetos, es un lenguaje interpretado, con lo que no necesitara la ayuda de compiladores.

En estos momentos la mayoría de los navegadores solo interpretan las últimas versiones de código JavaScript., pero es el más utilizado como lenguaje de programación del lado del cliente.

Java Script es un lenguaje basado en objetos y diseñado para el desarrollo de aplicaciones cliente /servidor a través de internet.

En una aplicación cliente, las sentencias JavaScript se encuentran embebidas en el documento HTML, pueden reconocer y responder a eventos que han sido generados por el usuario, como por ejemplo un clic con el ratón, rellenar información por medio de formularios, etc.

El ejemplo más utilizado, es la construcción de un scripts por medio de JavaScript para verificar información que se ha introducido por un usuario, por ejemplo que una contraseña y un nombre de usuario sea correcto, esto se realizara sin que haya que transmitir datos por la red.

Un documento HTML con JavaScript embebido será capaz de verificar que sea correcta o no la información aportada por el usuario, y avisa al usuario de si no es correcto.

También se podrán ejecutar archivos de audio, applets o comunicación mediante plug_ins para dar respuesta a la apertura o cierre de una página.

JavaScript es un lenguaje que imita a JAVA, este soporta la mayoría de expresiones y sintaxis de Java, JavaScript trabaja en base a un simple sistema de tiempo de ejecución, mediante el cual permitirá el uso de tipos de datos como enteros, booleanos y string. Java script soporta objetos basados en instancias no en clases.

Java script es un lenguaje pequeño y dinámico, estos ofrecen herramientas de programación , que son muy fáciles de utilizar y proveen una interfaz más especializada, solo necesitara un mínimo requerimiento para poder crear objetos.

Características de JAVASCRPT:

Características
– Es interpretado en el cliente.
– Está basada en objetos.
– El código se encuentra embebido en HTML.
– Utiliza variables y tipos de datos que no tendrán que ser declarados.
– Realizan referencias a objetos que se ejecutan al momento.

Java Script puede ser embebido en el documento HTML de las siguientes formas:

– Por medio de funciones y sentencias usando un elemento llamado <script>.

– Usando atributos manejadores en HTML.

Un script que se encuentra embebido en HTML utilizara el siguiente formato:

```
<script>
  Sentencias;
   .....
  </script>
```

Java Script diferencia las mayúsculas y las minúsculas. Un ejemplo de script es el siguiente:

```
<HTML>
<HEAD>
</HEAD>
<BODY>
<SCRIPT>
    Document. Write("hola");
</SCRIPT>
</BODY>
</HTML>
```

Con Java Script podremos crear efectos en las páginas y determinara las interacciones con el usuario.

El navegador del cliente será el que traducirá las instrucciones de JavaScript y se ocupara de ejecutarlas.

Recuerda 

Con Java Script se podrá realizar efectos para poder crear contenidos más dinámicos y también se podrá dar movimiento a los componentes de la página.

Se podrán crear páginas con programas como por ejemplo una agenda o un calendario.

Ventajas:

- Es más seguro y fiable.
- Capacidades limitadas, con lo que tendrán mayor seguridad.
- Se ejecuta en el cliente.

Desventajas:

– Código accesible a cualquier usuario.

– Sus recursos no son tan extensos.

– Sus opciones 3d son limitadas

– Sistemas que no sean tan complejos puede aparecer errores.

– Los usuarios pueden desactivar JavaScript en su navegador



Applets de java:

Un applet es una programa que se ha escrito en Java y forman parte de los componentes de una agina web.

Estos han sido creados y usados para dar funcionalidad a las páginas web que no les ha sido suficiente únicamente usando HTML, la idea de la creación de applets es que deben de ser bastantes pequeños y proporcionar una función específica y definida.

El uso de applets en Java es adecuado ya que el código es totalmente independiente de los sistemas operativos donde se esté corriendo el navegador.

El código de applets es ejecutado por el navegador, usando lo que se llama Máquina virtual de Java.

Funciones que pueden realizar los applets.

– Mostrar secuencias de imágenes y agregar efectos especiales.

– Mostar imágenes con sonido y agregar efectos sonoros.

– Permitirán la introducción de grafico interactivos, realizando alguna acción cuando pinchamos por ejemplo con el ratón.

– Permite realizar cálculos.

– Permitirá agregar efectos y animaciones a los textos.

– Crear juegos sencillos.

– Crear diagramas y gráficas.

Se trata de pequeños programas que están construidos y compilados en JAVA que se pasan y se ejecutan en el navegador Web.



Los applets son más difíciles de programar que los Scripts de Java Script.

Una de las cualidades más importante de su utilización es que son más independientes del navegador y del sistema operativo. Éstos también son mucho más lentos de procesar.

Ventajas:

– Independiente del navegador.

– Gran cantidad de contenidos interactivos.

– Aumenta las funciones disponibles en una aplicación y mejora las características disponibles en una página web

Desventajas:

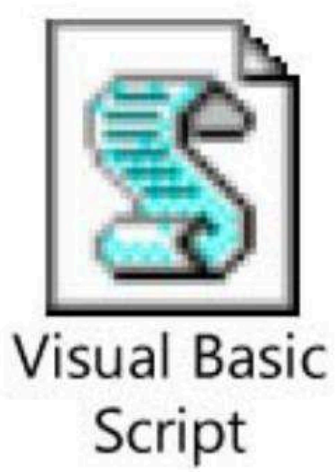
– Procesamiento lento.

– Poco espacio en la página donde se ejecutan

– Errores en la carga

Importante

Visual Basic Script:



Logotipo Visual Basic Script.

Es una versión del lenguaje de programación Visual Basic, especialmente preparada para conectar con ASP.

El conjunto de los objetos de ASP y VBScript permite programar aplicaciones web

VBScript no tiene todas las funcionalidades de Visual Basic. VBScript es utilizado en páginas HTML para que sean ejecutadas por medio de un navegador.

Esta es sin duda la gran virtud de ASP: no importa qué navegador tiene el cliente, ya que sólo se le envía HTML estándar.

Por ahora, el único lenguaje de comandos compatible con todos los navegadores modernos es Java Script, aunque existen algunas incompatibilidades entre versiones.

Los tipos de comandos, en VBScript son:

– Constantes

– Variables

– Operadores

– Instrucciones

– Funciones

Sintaxis y formato de VBScript embebido en HTML es:

```
<HTML>
< HEAD>
</HEAD>
< BODY>
<% llamada a la rutina %>
< /BODY>
< /HTML>
```



Solo es compatible con Internet Explorer. Está basado en el lenguaje de programación Visual Basic, es una versión más reducida. Es muy similar a JavaScript y también se utiliza para crear páginas web.

Ventajas:

– El código fácil de depurar.

– No es necesario disponer de un compilador.

– Fácil de mantener y rápido.

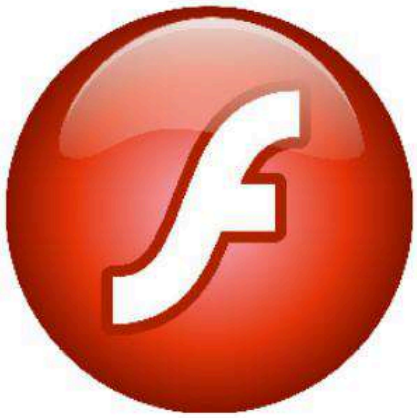
Desventajas:

– El código es visible por lo que el usuario tiene acceso al código y puede modificarlo, con lo que puede provocar errores.

– Ocupa bastante espacio en disco.

Importante

Flash:



Logotipo Flash.

Se trata de un Programa multimedia desarrollado por Macromedia adobe flash que utiliza imágenes, sonidos, vídeo y audio bidireccional para crear proyectos multimedia. Flash es el entorno desarrollador y Flash Player es el programa (la máquina virtual) utilizado para poder ejecutar archivos generados con flash.

Se puede utilizar para crear simples animaciones o crear animaciones más complejas.

Los archivos de Flash suelen tener la extensión swf y aparecen frecuentemente en páginas webs en forma de animaciones y aplicaciones.

Características:

- Los archivos Flash suelen abrir más rápidamente
- Se pueden crear proyectos más complejos.
- Flash está muy difundido. Casi todos los ordenadores tienen algún plugin para visualizar Flash.
- Es barata.
- Flash es un de código libre

No se trata de un lenguaje en sí, se trata de un programa que tiene como función la creación de componentes con efectos especiales en páginas web.

Se utiliza para hacer páginas dinámicas. Para visualizar archivos flash el navegador debe de tener instalado un plug-in.

Ventajas:

– Se pueden crear animaciones de textos, imágenes, etc.

– Se pueden realizar páginas muy atractivas al usuario.

– Se pueden introducir sonidos o imágenes en movimiento.

Desventajas:

– Para ver las páginas en Flash hace falta instalar plug-in flash en el ordenador.

– Se ven en algunos navegadores y en otros no.

– Paginas mucho más pesadas.

– Las páginas realizadas en Flash no aparecen en los navegadores.

– Dificultad de crear paginas dinámicas de interacción con el usuario.



CSS:

Las hojas de estilo se desarrollaron para solucionar los defectos de HTML con respecto a la presentación y al diseño de las páginas. HTML tiene varias etiquetas para modificar la presentación y definir los estilos del texto.

Al utilizar hojas de estilo, cuando se necesite cambiar la apariencia de un sitio que tiene varias páginas web solo se tendrá que editar las definiciones de la hoja de estilo en un solo lugar para cambiar la apariencia del sitio completo.

Reciben este nombre porque se pueden definir múltiples hojas y los estilos pueden aplicarse a todas las páginas.

Las hojas de estilo pueden utilizarse para:

– Lograr una apariencia uniforme de todo el sitio al activar una sola definición de estilo en cada página.

– Cambiar un aspecto en todo el sitio Web con tan sólo editar unas pocas líneas,

– Hacer que los códigos HTML sean más fáciles de leer ya que los estilos se definen por separado.

– Permitir que las páginas se carguen más rápido ya que hay menos cantidad de HTML en cada página.

– Posicionar los elementos de la página de una manera más uniforme.



Son mucho más eficaces a la hora de elaborar una página web que utilizando HTML, se pueden realizar muchas más cosas, como por ejemplo definir un estilo propio.

Ventajas:

– Accesibilidad, así como ofrecer la posibilidad de personalizar una página.

– Flexibilidad en la forma.

– Uniformidad, cuando se modifica algo se modifica en toda la página.

Desventajas:

– Colapso Márgenes.

– La falta de variables.

– Compatibilidad con diferentes exploradores.

– Limitación de control vertical.

3.3. Análisis de lenguajes orientados a la preparación de la capa de presentación y a la ejecución de solicitudes desde clientes ligeros Web. (JSP, SERVLETS, ASP, PHP)



Los lenguajes del lado del servidor corresponderán a los lenguajes admitidos, procesados y representados por el servidor y que serán enviados al cliente de forma que los entienda.

Son los lenguajes que se procesan en el lado del servidor y que generan la página antes de enviarla al cliente.

El lenguaje del lado del Servidor, es también independiente del cliente con lo que dará mayor flexibilidad a la hora de utilizar cualquier navegador.

Lenguajes del lado del Servidor:

CGI
PERL
ASP
JSP
PHP
DHTML
XML

Los lenguajes más utilizados son:

- JSP.
- SERVLETS.
- ASP.
- PHP.

Para más información, consulta "Lenguajes menos usados" en los anexos al final del libro.



JSP y Servlets:

Java Server pages (JSP),es na tecnología utilizada para crear páginas web dinámicas basada en el lenguaje de hipertexto HTML o XML, utiliza el lenguaje de programación JAVA.

JAVA es un lenguaje de programación que deriva de C y C++, las aplicaciones Java son compiladas y pueden ejecutarse en cualquier ordenador, que realizara la función de máquina virtual Java.

Es un lenguaje concurrente o paralelo, orientado a objetos y que se basa en clases.



Para desarrollar y dejar correr JSP se requiere de la ayuda de un servidor web compatible con contenedores servlets como por ejemplo Apache Tomcat, que es el más utilizado.

JSP puede crear clases que pueden manipular la lógica de negocio y acceder a los datos. Con esto lo que se conseguirá es hacer una separación en niveles de la aplicación web, JSP se ocupara de generar el documento HTML.

JSP puede procesar aplicaciones en múltiples plataformas u ordenadores sin que a esta le produzca ningún cambio. Se podrá desarrollar una aplicación en un ordenador y ejecutarla en otro distinto.

Los JSP y los servlets se desarrollan a través de máquinas virtuales de Java, cada servlet o JSP se ejecuta en su propio entrono, no se empiezan a ejecutar hasta que le llega una petición, responden a la petición y se mantienen a la espera de la siguiente, esto hará que sea más eficiente a la hora de conectar con una base de datos, por ejemplo.

Los JSP servirán para crear servlets, se llamara a los servlets una vez y JSP traducirá el código del servlet. El servlet se ejecutara y dará como salida la respuesta en código HTML.

Ventajas:

– Fuerte capacidad de manipulación de gráficos.

– Cargas de Apis.

– Código abierto.

Desventajas:

– Baja el desempeño de la red.

– Aprendizaje lento



PHP:

HyperText Preprocessor (PHP), es un lenguaje muy conocido popularmente por ser un lenguaje de código abierto, esto quiere decir que es libre de uso y gratuito, y para el desarrollo de páginas web dinámicas, se incrusta en el código HTML, siguiendo una serie de reglas.

Suele utilizarse también para crear el contenido dinámico de una página y las interacciones con el usuario.

PHP se ejecuta en el servidor por eso se dice que es un lenguaje del lado del servidor.

Las páginas web PHP están compuestas o desarrolladas con código PHP que estarán contenidos en el código HTML. La estructura de PHP es una estructura compuesta de etiquetas que se ponen al comienzo y al final.<"PHP"> </ >. Los servidores serán los que interpreten este código.

Una de las ventajas de utilizar PHP es, que es muy simple para una persona inexperta o con pocos conocimientos, pero a la vez es un lenguaje muy especializado para utilizar los más expertos.

PHP es capaz de procesar la información por medio de informes, crear páginas con contenido dinámico, crear imágenes a partir de datos por ejemplo:

Una empresa que consta de un departamento de contabilidad con diez empleados, y otro departamento con administrativos de 25 empleados, pues con PHP puedes crear una imagen parecida a esta.

Es compatible con los estándares de datos para el desarrollo web, como puede ser XHTML o ficheros de XML, y además será capaz de interactuar con otros servidores utilizando protocolos.

Ventajas:

– Fácil manejo.

– Fácil aprendizaje.

– Gran variedad de funciones predefinidas en el sistema.

– Gratuito.

– Código libre.

Desventajas:

– Errores en él no se gestiona adecuadamente.

– La seguridad depende de los desarrolladores



ASP:

Active Server Pages (ASP), es un tipo de tecnología que fue creada por Microsoft para realizar paginas dinámicas del lado del servidor, es decir que cuando el usuario solicita un dato, ASP mediante un Script que contendrá unas instrucciones de programación, se ejecutara para enviar al navegador web la respuesta en código HTML.

ASP se desarrolla en la página web y utiliza el lenguaje de programación Visual Basic Script, pero pueden utilizar otros tipos de lenguajes como Perl, Script, etc.

Con ASP se podrán desarrollar un montón de aplicaciones distintas. Nos dejara acceder a todos los recursos de que disponga el servidor.

Una de las mejores ventajas que proporciona ASP es la seguridad ya que el código se encuentra en el servidor y los usuarios no tienen acceso a él

En la actualidad ya se ha creado la nueva versión de ASP, ASP.NET, contiene algunas mejoras respecto a la rapidez de funcionamiento, tienen diferentes sintaxis que ASP, así que se manejaran de forma diferente.

Ventajas:

– Permite un rápido desarrollo

– Totalmente escrito

– Soporta varios lenguajes.

– Realizar multitud de aplicaciones diferentes

– Seguridad del código

Desventajas:

– Es costoso.

– Es difícil de manejar para principiantes.

UD3
Lo más importante

- La función principal de la capa de presentación es la de ocuparse del formato, cómo se va a visualizar la información.

- La base de toda interfaz de usuario, es el lenguaje de marcado de Hipertextos HTML, en inglés (HiperText Markup Language), este es un lenguaje que proviene del XML.

- **La capa de presentación** está compuesta por un grupo de componentes que son los que se encargaran de estableces la interfaz de la aplicación, visualizando información e interaccionando con el usuario.

  El fundamento del interfaz de un usuario de una aplicación Web, es el Lenguaje HTML, es un lenguaje de marcado, este lenguaje es un derivado de XML.

- **HTML** posibilitara mostrar imágenes, textos, videos, etc., desde un navegador Web. El navegador Web es el que se ocupa de traducir y comprender este lenguaje y mostrarlo al usuario de forma lógica, funcional y atractiva.

- **Los lenguajes del lado del Cliente**, corresponderán a los lenguajes que no tendrán que transformar los datos para que sean entendibles por el navegador. Son lenguajes que entienden perfectamente el navegador Web, ya que se realizaran en HTML, y no tendrán que ser pre tratados. El lenguaje del lado del cliente es independiente del servidor, esto hará que la página pueda albergarse en cualquier sitio sin necesidad de pago.

– **Un script** se trata de un programa que suele estar dentro del contenido de HTML. Cuando se ejecuta, se cargará todas las instrucciones de dicho programa, a esta acción se le denomina **evento**.

– Los lenguajes de programación del lado del cliente más utilizados son HTML, JAVA y JavaScript, (estos se incluyen en el código HMTL), aunque encontramos muchos más lenguajes.

  · **HTML**:

    En ingles HiperText Markup Langage, es un lenguaje de marcado para elaborar páginas Web, se utiliza para determinar el contenido de una página web, como puede ser el texto, imágenes, videos, etc.

    Es un lenguaje interpretado, con lo que no requerirá compilación.

  · **JavaScript**:

    Es un lenguaje de programación orientado a objetos, es un lenguaje interpretado, con lo que no necesitara la ayuda de compiladores. En estos momentos la mayoría de los navegadores solo interpretan las últimas versiones de código JavaScript., pero es el más utilizado como lenguaje de programación del lado del cliente.

    Con JavaScript se podrá realizar efectos para poder crear contenidos más dinámicos y también se podrá dar movimiento a los componentes de la página

  · **Applets de java:**

    Se trata de pequeños programas que están construidos y compilados en JAVA que se pasan y se ejecutan en el navegador Web. Los applets son más difíciles de programar que los Scripts de Java Script. Una de las cualidades más importante de su utilización es que son más independientes del navegador y del sistema operativo. Estos también son mucho más lentos de procesar.

  · **Visual Basic Script:**

    Solo es compatible con Internet Explorer. Está basado en el lenguaje de programación Visual Basic, es una versión más reducida. Es muy similar a JavaScript y también se utiliza para crear páginas web.

- Flash:

  No se trata de un lenguaje en sí, se trata de un programa que tiene como función la creación de componentes con efectos especiales en páginas web. Se utiliza para hacer páginas dinámicas. Para visualizar archivos flash el navegador debe de tener instalado un plug-in.

- CSS:

  Son mucho más eficaces a la hora de elaborar una página web que utilizando HTML, se pueden realizar muchas más cosas, como por ejemplo definir un estilo propio.

– **Los lenguajes del lado del servidor** corresponderán a los lenguajes admitidos, procesados y representados por el servidor y que serán enviados al cliente de forma que los entienda. Son los lenguajes que se procesan en el lado del servidor y que generan la página antes de enviarla al cliente

– Los lenguajes más utilizados son, JSP, SERVLETS, ASP Y PHP.

- JSP y Servlets:

  Java Server pages (JSP),es na tecnología utilizada para crear páginas web dinámicas basada en el lenguaje de hipertexto HTML o XML, utiliza el lenguaje de programación JAVA.

- PHP:

  HyperText Preprocessor (PHP), es un lenguaje muy conocido popularmente por ser un lenguaje de código abierto, esto quiere decir que es libre de uso y gratuito, y para el desarrollo de páginas web dinámicas, se incrusta en el código HTML, siguiendo una serie de reglas.

- ASP:

  Active Server Pages (ASP), es un tipo de tecnología que fue creada por Microsoft para realizar paginas dinámicas del lado del servidor.

UD3
Autoevaluación

1. La base de toda interfaz de usuario es:

    a. Lenguaje ASP.
    b. Lenguaje de marcado HTML.
    c. Lenguaje XML.
    d. Lenguaje C++.

2. Los componentes que forman la capa de presentación se dividen en:

    a. Componentes de aplicación.
    b. Componentes de aplicación y componentes de desarrollo.
    c. Componentes de presentación y componentes de procesos.
    d. Componentes de procesos y componentes de desarrollo.

3. Los lenguajes del lado del cliente corresponderán a:

    a. Los lenguajes que tendrán que transformar los datos para que lo entiendan el navegador web
    b. Los lenguajes que tendrán que transformar los datos para que lo entiendan el servidor de datos.
    c. Los lenguajes que no tendrán que transformar los datos para que lo entiendan el navegador web
    d. Los lenguajes que tendrán que transformar los datos para que lo entiendan el servidor de datos.

4. Un Script es…

    a. Un pequeño programa que suele estar fuera del contenido HTML.
    b. Un pequeño programa que suele estar dentro del contenido HTML.
    c. Un pequeño programa que suele estar fuera y fuera del contenido HTML.
    d. No es un programa.

5. ¿Qué es Java Script?

    a. Un lenguaje interpretado.
    b. Un lenguaje orientado a objetos.
    c. Un lenguaje no compilado.
    d. A y b son correctas.

6. Con Java Script se podrá crear:

    a. Efectos especiales para crear contenidos estáticos.
    b. Efectos especiales para crear contenidos dinámicos..
    c. Efectos especiales para crear contenidos dinámicos y dar movimiento a los componentes de la página web.
    d. Efectos especiales para crear contenidos estáticos y dar movimiento a los componentes de la página web.

7. Visual Basic Script solo es compatible con:

    a. Internet Explorer.
    b. Mozilla Firefox.
    c. Opera.
    d. Google Chrome.

8. ¿Qué significa CSS?

    a. Hoja de estilo.
    b. Cascada.
    c. Hoja de estilo en cascada.
    d. Ninguna de las anteriores.

9. ¿Cuáles son los lenguajes más utilizados del lado del servidor?

    a. CGI, PERL, ASP.
    b. PHP,DHTML,XML
    c. ASP, PHP, PERL.
    d. ASP, PHP, JSP y Servlets.

10. PHP es un lenguaje de:

    a. Uso libre y gratuito.
    b. Uso libre y de pago.
    c. Uso libre de pago o gratuito.
    d. Uso libre.

Área: informática y comunicaciones

UD4

Diseño de base de datos relacionales

4.1. Definición de bases de datos relacionales.

4.2. Diseño de base de datos en varios niveles

4.3. Análisis de los distintos tipos de relaciones y su implementación en la base de datos

4.4. Descripción del lenguaje de acceso a base de datos

4.5. Descripción de correlaciones entre el modelo relacional y el modelo orientado a objetos

4.6. Nociones sobre el almacenamiento de objetos en las bases de datos relacionales

4.1. Definición de bases de datos relacionales.



Los sistemas de gestión de bases de datos, llamada también por sus siglas SGBD organizan y estructuran los datos de forma que puedan ser recuperados y manejados por usuarios y programas de aplicación.

Las estructuras de datos y el acceso proporcionados por SGBD, reciben el nombre de **modelo de datos.**

Existe, un tipo de lenguaje estándar para trabajar con base de datos denominado SQL, este lenguaje se utiliza para bases de datos relacionales y utiliza el modelo de datos relacional.

Evolución de los modelos de datos primitivos:

Sobre los años setenta empezaron a surgir tecnologías centradas en la gestión de bases de datos, los primero gestores de bases de datos lo que hacían era almacenar los datos permanentemente en un sistema informático, por ejemplo los registros de contabilidad, estos se almacenaban en archivos individuales.

Un sistema llamado **sistema de gestión de archivos**, se proporcionaba como parte del sistema operativo que llevara el ordenador, lo que hacía era recordar los nombres de los archivos y su ubicación. No tenía ningún modelo, daba igual que fuera lo que llevara dentro de información de igual que fuera texto o que contuviera números.

Cuando se fue evolucionando y se empezaron a encontrar problemas para poder mantener grandes sistemas de archivo se empezaron a desarrollar de forma que pudieran conseguir que los datos estuvieran estructurados para poder acceder a ellos automáticamente sin tener que pasar por programas que gestionaban esos datos.

Se distinguen tres tipos de estructuras de base de datos más utilizadas desde entonces, aunque existen más:

Importante

Jerárquica

Es la primera que se utilizó, se basa en implantar jerarquías o niveles entre los diferentes campos de un registro, se basa en que los campos de mayor nivel serán los más universales, y tiene una estructura de árbol.

Para poder acceder a los datos de un campo de un nivel determinado, para localizarlo se tendrá que partir desde el primer nivel.

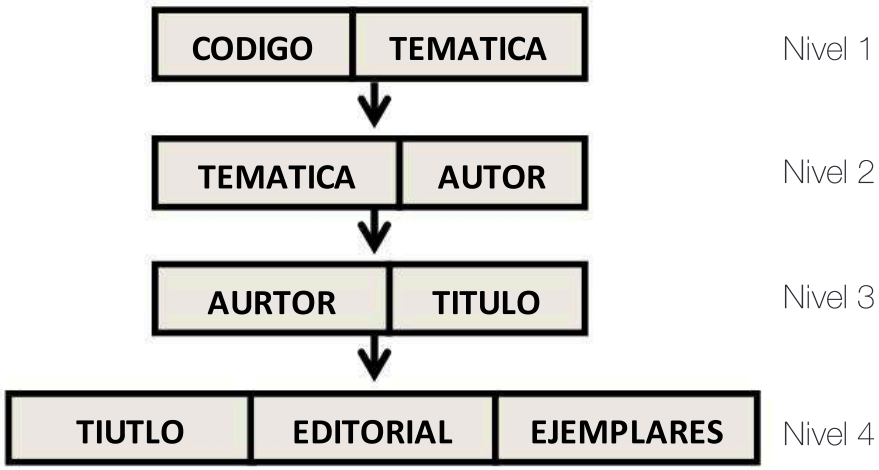
Vamos a utilizar como ejemplo un biblioteca, es el más habitual para explicar estas estructuras.

Podríamos situar como nivel superior el código de la temática y el título de la temática, y que el título de la temática sea el campo que identifique a esa temática.

En segundo nivel, colocaríamos las tablas con los autores que tienen relación con la temática, y en un tercer nivel pondríamos una tabla de los autores con su nombre y el título de su libro, así hasta el último nivel en el que utilizaríamos tablas en el que se incluirían el título del libro, editorial, número de ejemplares, etc.

Todo esto hacia que esta estructura fuera realmente lenta.

Ejemplo



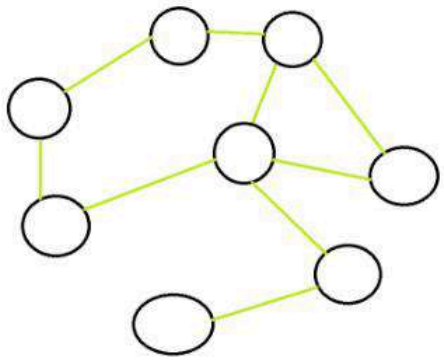
Importante

Bases de datos en red

En este tipo de estructura desaparece la jerarquía, en esta existe más de una conexión entre los diferentes niveles, con lo que se conseguirá recorrer los distintos caminos sin necesidad de empezar por el raíz, esto hace que la búsqueda de información sea flexible.

En el ejemplo de la biblioteca se podrán por ejemplo hacer una lista de los títulos de los libros, a partir de un título sin tener que acceder cada vez que lo pida a los autores.

La desventaja que produce este tipo de estructura es que utiliza mucha memoria, al tener que ir almacenando las posiciones por medio de apuntadores.



Estructura de bases de datos en red

Importante

Bases de datos relacionales.

En problema que se suele presentar a la hora de realizar una base de datos que está formada por varias tablas está en que hayan datos repetidos, es decir, campos que se repiten en diferentes tablas, es lo que se llama redundancia, lo cual va a hacer más difícil su gestión y su manipulación.



Para resolver este problema es necesario una fusión entre las diferentes tablas y que se controle la repetición de los datos.

Así es como aparecieron los SGDB relacionales que permitirán resolver los problemas de redundancia, en este tipo de estructura se elimina totalmente las jerarquías entre los campos, pudiendo utilizarlos como clave de acceso.

Para más información, consulta "Tipos de estructuras de base de datos" en los anexos al final del libro.



Una **Base de Datos Relacional**, es una base de datos que utiliza el modelo de estructura relacional, este modelo es el más utilizado en la actualidad para poner en funcionamiento unas bases de datos ya planificadas.

Permiten establecer relaciones entre los datos, que se guardaran en tablas, y relacionar los datos de las tablas, de ahí proviene su nombre modelo relacional.

Sus bases la propulsaron Edgar Frank Codd.

Una base de datos relacional es la que cumple con el modelo relacional, el modelo relacional es el más utilizado actualmente para implementar bases de datos que ya han sido planificadas y van a permitir establecer interconexiones, que reciben el nombre de relaciones.

Entre los datos que se encuentran guardados en tablas, y a través de las interconexiones podremos relacionar los datos de dos tablas o más.

El modelo relacional del que se basa una base de datos relacional, está compuesto por datos que se encuentran estructurados a nivel lógico en tablas formadas por filas y columnas, a nivel físico puede tener una estructura completamente diferente.



El modelo relacional es una alternativa para organizar y representar información que se va a almacenar en una base de datos.

Se basa en el modelo teórico matemático y nos proporciona elementos que son básicos para el desarrollo del modelo, incorpora una serie de operadores para su manipulación, que se caracterizan por no ser ambiguos.

El modelo relacional es muy formal por lo que hace que sea muy sencilla su representación y gestión por medio de herramientas informáticas.

Es un referente para el desarrollo de la mayoría de los sistemas de gestión de base de datos comerciales, también será el modelo de referencia a seguir para la elaboración del esquema lógico de una base de datos, después de realizar el análisis de los requerimientos y de diseñar el esquema conceptual, de todo esto hablaremos más adelante.



El modelo relacional tiene sus bases en la teoría matemática de la relación. La información se representa mediante tablas o también llamadas relaciones, donde cada fila de la tabla es interpretada como una relación ordenada de valores.

Por ejemplo:

Id_cliente	Nombre	Apellidos
1235	Manuel	Gómez Guzmán
1532	Juan	Peña Gutiérrez
785	Elena	Benítez Rey

En este ejemplo se representa un grupo de clientes de una determinada empresa, y la información sobre estos clientes.



Podemos definir una relación como un conjunto de **n-filas**, donde una **fila** se define a su vez como un grupo de valores atómicos, con esto quiere lo que se quiere decir es que van a ser indivisibles.

A las filas en el modelo relacional se les llama tuplas.

En el ejemplo anterior, diríamos que tenemos una relación con tres tuplas , y en cada una de las tuplas sus valores están relacionaos solo por el simple hecho de que están describiendo a un mismo cliente.



Cada una de las relaciones o tablas, está formado a su vez de un grupo de **columnas**, cada una de estas columnas recibe el nombre de **atributo**.

Cada atributo tienen un nombre que no puede ser igual a otro atributo de una misma tabla o relación, dentro de cada atributo se guarda un valor, al valor se le denomina **dominio**.

En el ejemplo anterior los atributos serian:

Id_cliente	Nombre	Apellidos

Cada uno de los atributos contendrá un dominio, el dominio en el ejemplo anterior serian:

1235	Manuel	Gómez Guzmán
1532	Juan	Peña Gutiérrez
785	Elena	Benítez Rey

En el que cada dominio representa un dato que en el caso ejemplo seria el id_cliente que será un entero, y nombre y apellidos serán una cadena de caracteres.



El esquema de una relación o tabla es una descripción de su estructura interna, es decir, es una descripción de los atributos que la forman.

La sintaxis seria la siguiente:

```
RELACION (ATRIBUTO1,ATRIBUTO2…..ATRIBUTO N)
```

El esquema de la relación del ejemplo de cliente seria el siguiente:

```
Clientes (id_cliente,nombre,apellidos)
```

Se puede decir que el esquema de una relación es una parte de la relación que no varía, es decir, el tipo de información que contiene una relación siempre va a ser la misma, siguiendo el ejemplo anterior de clientes, los atributos que forman la relación cliente van a ser siempre los mismos, id_cliente, nombre y apellidos.

Mientras, que la información que se encuentra almacenada en la relación si puede cambiarse, en una empresa pueden desaparecer trabajadores, clientes, departamentos, etc.



El conjunto de tuplas o filas que forman una relación es la parte que puede variar de una relación o tabla.

Cada tupla de una relación se representa de la siguiente forma:

```
(valor1,valor2…..valorn)
```

Valor corresponde al valor de la tupla para el atributo1 y el valor de la tupla valor1 para era el atributo n.

Siguiendo el ejemplo una tupla seria:

1235	Manuel	Gómez Guzmán



Partiendo del esquema de la relación o tabla será posible determinar su grado.

El **grado** es el número de atributos de los que está formada.

Así, en el ejemplo que estamos siguiendo de cliente, el grado de la relación es 3.

Una relación consiste en un conjunto de tuplas, pero eso no implica que las tuplas estén ordenadas.

La misma relación se podrá representar de dos maneras.

Id_cliente	Nombre	Apellidos
1235	Manuel	Gómez Guzmán
1532	Juan	Peña Gutiérrez

O de esta otra forma:

Id_cliente	Nombre	Apellidos
1532	Juan	Peña Gutiérrez
1235	Manuel	Gómez Guzmán

Hasta este momento hemos visto el componente principal en él se basa todo modelo relacional, que es la relación.

Que una relación está formada por tuplas y que cada una de estas tuplas contiene información sobre un objeto concreto del mundo real, estos proporcionan valores a los atributos de una relación.



Algunas veces se puede dar el caso que no se conozcan los valores de algún atributo de un determinado objeto.

En los casos en el que el valor de un atributo para una determinada tupla se desconozca o no tenga valor, el modelo relacional va a permitir el uso de un valor especial, llamado **valor nulo o null.**

Ejemplo:

Id_cliente	Nombre	Apellidos
1235	Manuel	Gómez Guzmán
1532	Juan	Peña Gutiérrez
785	Elena	NULL O NULO



El uso del valor nulo debe de ser evitado en la medida de lo posible ya que puede producir errores a la hora de manipular o acceder a la información.

Para garantizar la consistencia y facilidad de manipulación de la información representada , existen unas reglas que constituyen el modelo relación, a estas reglas se le conocen como restricciones de integridad.

Existen varios tipos de restricciones:

- De dominio.
- De clave.
- De entidad.
- De referencia.



Restricción de Dominio.

Los dominios o valores de los atributos de una relación deben de ser atómicos. Esta restricción obliga a que los valores de las tuplas de una relación correspondientes a los atributos de la relación deben de ser atómicos, es decir esos valores son indivisibles no se pueden descomponer en valores más pequeños.

Esta restricción garantizará que las relaciones tengan un formato regular, para que se pueda manipular de forma sencilla a través de un procedimiento o un algoritmo.

Por ejemplo:

Id_cliente	Nombre	Apellidos	Localidad
1235	Manuel	Gómez Guzmán	Málaga Ronda
1532	Juan	Peña Gutiérrez	Sevilla
785	Elena	Benítez Rey	Córdoba

En este caso, el ejemplo que hemos estado utilizando le hemos añadido otro atributo más que será la localidad.

Pues en este atributo Localidad, en la primera tupla, el valor de Localidad presenta un doble valor, esto no se podría poner ya que incumple la restricción de integridad, con lo que la tabla quedaría de la siguiente manera:

Id_cliente	Nombre	Apellidos	Localidad
1235	Manuel	Gómez Guzmán	Málaga
1235	Manuel	Gómez Guzmán	Ronda
1532	Juan	Peña Gutiérrez	Sevilla
785	Elena	Benítez Rey	Córdoba



Restricción de Clave.

En una relación no puede existir ninguna tupla repetida.

No admite que existan elementos repetidos en su contenido. Que la extensión de una relación no puede incluir tuplas repetidas, esto implica que todas las tuplas que contiene una relación se diferencien entre sí por el valor de al menos un atributo.

Esto nos lleva a concepto de **clave primaria** de una relación. Cualquier subconjunto de atributos de una relación que nos permitirá diferenciar de forma exclusiva una relación.

Toda relación cuenta con una o varias claves primarias, **la clave primaria** son todos los atributos de una relación que identifican por completo a la relación.

En el ejemplo que estamos siguiendo, pueden ser claves primarias los siguientes conjuntos de atributos.

Pueden ser clave primaria:

- (id_cliente, nombre, apellidos)
- (id_cliente, nombre)
- (id_cliente)
- (id_cliente, apellidos)

En este caso no puede existir dos tuplas en la relación clientes en la que coincidan los valores de Id_cliente, nombre y apellidos, ni los valores id_cliente y nombre, ni los valores de Id_cliente, ni los valores de Id_cliente y apellidos.

Por lo tanto el conjunto de atributos que no constituyen una clave primaria de la relación seria:

- (nombre, apellidos)
- (apellidos)
- (nombre)

En este caso puede existir un cliente que de la casualidad que tenga el mismo nombre y apellidos, por lo que no podría ser clave primaria, tampoco podría ser apellidos, porque pueden existir dos clientes o más que tengan los mismos apellidos y tampoco podrá ser el nombre, porque pueden existir dos o más clientes con el mismo nombre.



También existen las **claves candidatas**, llamamos clave candidata de una relación al conjunto de atributos que pueden llegar a ser clave.

En el caso del ejemplo que estamos, siguiendo al conjunto de atributos que forman clave primaria como (id_cliente, nombre, apellidos) si eliminamos el id_cliente, no sería la clave mínima que hace falta para identificar de forma única la relación.



El modelo relacional recomienda seleccionar una sola de las posibles claves candidatas como clave primaria de la relación.

La clave primaria de una relación se representa en el esquema lógico, subrayando los nombres de los atributos que forman parte de la clave primaria.

Ejemplo:

(id_cliente, nombre, apellidos)



Restricción de integridad Entidad.

Ninguna tupla de una relación puede tomar valores nulos en los atributos que forman parte de su clave primaria.

Esta restricción es clara, ya que la clave primaria es la que nos va a permitir distinguir a las tuplas entre sí, los valores que corresponden a la clave deben de ser conocidos en cada tupla para poder identificarla.

Ejemplo:

Id_cliente	Nombre	Apellidos	Localidad
1235	Manuel	Gómez Guzmán	
1235	Manuel	Gómez Guzmán	
1532	Juan	Peña Gutiérrez	Sevilla
785	Elena	Benítez Rey	Córdoba

Si vemos en el ejemplo anterior que la clave primaria de clientes era (id_cliente, nombre, apellidos, localidad), si son desconocidos la localidad, es imposible distinguir las dos tuplas que tienen vacía la localidad, por lo tanto esta relación es incorrecta, ya que viola las reglas de integridad de la entidad.



Restricciones de integridad referencial.

Si una tupla de una relación1 hace referencia a otra relación2, debe referirse a una tupla que exista en la relación2. Este tipo de restricción permitirá garantizar la consistencia en el caso de relaciones que mantengan una vinculación.

Vamos a poner un ejemplo, tenemos una relación 1 de clientes:

Id_cliente	Nombre	Apellidos
1235	Manuel	Gómez Guzmán
1532	Juan	Peña Gutiérrez
785	Elena	Benítez Rey

Y una relación 2 de artículos:

Código	Nombre	Id_cliente
15	Pilas	1235
23	Televisor 22"	785

La relación lo que pretende representar es el código de un artículo, que será la clave primaria de esa relación, su nombre y el id del cliente que ha comprado el producto.

Para evitar problemas de redundancia, en lugar de representar los datos de cada empleado, se incluye una referencia a la tupla que le corresponde a la relación de clientes.

Esta referencia se realiza por medio del valor de la clave primaria de la tupla, que en este caso será el id_cliente.

El atributo id_cliente es un ejemplo de clave foránea. Los atributos que son clave primaria en la relación clientes, hace referencia a a relación artículos.



La **clave foránea** es la clave primaria de una relación, que se propaga para ser una clave referenciada en otra relación.

Podemos resumir la estructura relacional se compone de:

- Dominios.
- Atributos.
- Relaciones.
- Claves.
- Esquema de la base de datos.



El modelo relacional es el que se selecciona habitualmente para realizar la elaboración del esquema lógico de la base de datos. Una base de datos desde el punto de vista relacional se encuentra formada por un conjunto de relaciones.

Según esto, el esquema lógico de una base de datos consistirá en la unión de todos los esquemas de todas las relaciones que componen la base de tatos junto con todas las restricciones de integridad que van a afectar a estas relaciones.

Además para identificar las claves foráneas en las relaciones que las contengan se representaran gráficamente junto con el esquema.



Los objetivos más importantes del modelo relacional son:

- **Independencia física de los datos**. El modo en que se almacenan los datos no debe de influir en la manipulación lógica.
- **Independencia lógica de los datos**. Los cambios que se vayan a realizar en los objetos de la base de datos, no deben de afectar a los programas y usuarios que acceden y hacen uso de ella.
- **Flexibilidad** al ahora de presentar a los usuarios los datos de la forma más adecuada posible a la aplicación que utilicen.
- **Uniformidad** en la presentación de estructuras lógicas de los datos, que serán las tablas o relaciones, esto facilitara la manipulación de la base de datos por parte de los usuarios.
- **Sencillez**, el lenguaje de usuario es sencillo, y hace que este modelo sea fácil de utilizar y comprender por el usuario.

Una característica muy importante del modelo relacional es que su estructura lógica es muy sencilla.

El modelo relacional, se ocupa de tres aspectos de los datos:

- Estructura de los datos.
- Integridad de los datos.
- Manejo de los datos

Las principales ventajas de utilizar el modelo relacional son:

- Trabajan sobre las tablas.
- Se pueden realizar consultas complejas en las que utilizan varias tablas
- Son fáciles de manejar.

Bases fundamentales de las bases de datos relacionales son:

- Todos los registros de la tabla deben de tener el mismo número de campos, aunque alguno de los campos quede vacío, deben de contener una longitud predeterminada.
- Cada campo se representa con un nombre que se define previamente.
- La base de datos estará formada por muchas tablas, uno por lo menos por cada tipo de registro.
- Dentro de una tabla no puede contener dos campos con el mismo nombre
- No puede haber dos registros en una misma tabla que sean idénticos.
- Los registros pueden estar en el orden que sea.
- El contenido de un campo puede estar delimitado por un rango de posibles valores.
- Se pueden crear tablas a partir de tablas ya creadas, relacionando los campos de las distintas tablas.

Para que un sistema sea considerado un sistema de gestión de bases de datos relacional, este sistema deberá de utilizar sus capacidades relacionales para gestionar la base de datos.

Para llevar a cabo la administración de una base de datos que está basada en el modelo relacional, se utiliza un sistema gestor de bases de datos relacionales, o también llamado **SGBD**.

4.2. Diseño de base de datos en varios niveles

Una base de datos sirve para almacenar información que va a ser utilizada en un sistema de información determinado.

Las necesidades y los requisitos de los usuarios del sistema de información se deben de tener en cuenta para poder tomar decisiones adecuadamente con anterioridad.

La parte principal de la estructura de una base de datos es el modelo de datos, en el aparatado anterior definimos lo que era el modelo de datos, pero vamos a recordar que era, el modelo de datos es un conjunto de herramientas conceptuales para hacer una descripción de las relaciones, la semántica y las condiciones de consistencia de los datos.

Los diferentes modelos de datos que se han propuesta hasta ahora se clasifican en tres grandes grupos:

- Modelos lógicos basados en objetos.
- Modelos lógicos basados en registros.
- Modelos físicos.



Modelos lógicos basados en objetos.

Los modelos lógicos basados en objetos se utilizan para describir datos en el nivel lógico y de vistas.

Se caracterizan por proporcionar capacidad estructural flexible y que permitirán que las condiciones de los datos sean específicas. Existen modelos muy diferentes.

Los más conocidos son:

- Modelo de entidad-relación.
- Modelo orientado a objetos.



El modelo entidad-relación es el más aceptado para realizar el diseño de una base de datos y se da mucho en la práctica.

El modelo orientado a objetos incluye muchos conceptos del modelo entidad-relación pero además de representar los datos también representa código que es ejecutable. Este último está ganando adeptos.

Modelo entidad-relación.

El modelo de datos entidad-relación se basa en la precepción del mundo real por medio de un conjunto de objetos básicos que serán las entidades y las relaciones existentes entre estos objetos.

Una entidad es un objeto del mundo real que se distingue de otros objetos.

Las entidades se describen en una base de datos mediante un conjunto de atributos., una relación es una asociación entre varias entidades.

El conjunto de todas las entidades del mismo tipo y el conjunto de todas las relaciones del mismo tipo se llaman conjunto de entidades y conjunto de relaciones.

Además de las entidades y de las relaciones, el modelo entidad-relación representa también algunas reglas o condiciones que deben cumplir los contenidos de la base de datos.



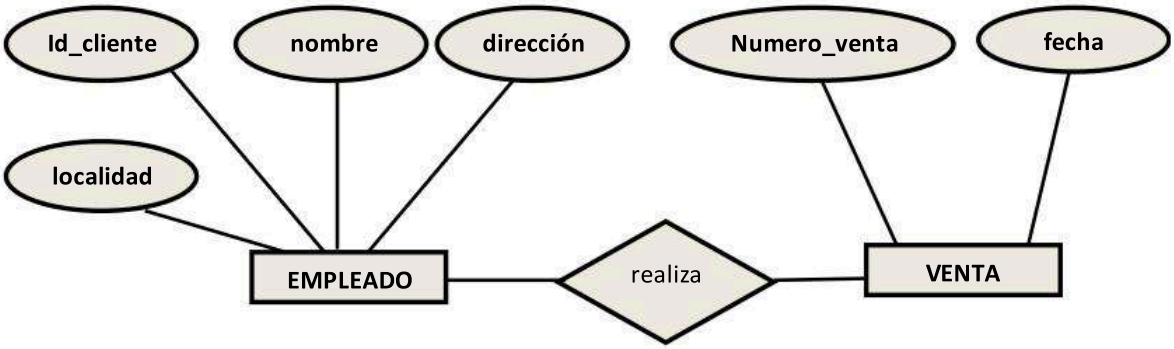
Una regla muy importante es la correspondencia de cardinalidades, que significa el número de entidades con las que otra entidad se puede asociar a través de un conjunto de relaciones.

Las estructuras lógicas de una base de datos se representan gráficamente mediante el llamado diagrama entidad-relación,.

Que se compone de los siguientes elementos:

- **Rectángulos**, que representan a las entidades.
- **Elipses**, que representan a los atributos.
- **Rombos**, que representan las relaciones existentes entre las entidades.
- **Líneas**, que unen los atributos con las entidades y las entidades con las relaciones.

Ejemplo:



Importante

Modelo orientado a objetos.

Como en el modelo entidad-relación, el modelo orientado a objetos se basa en un conjunto de objetos. Un objeto está compuesto por una serie de valores que se alma-cenaran en variables de ejemplares dentro de ese objeto.

Un objeto también va a contener un pequeño código que operan en el objeto a este código se le llama método.

Los objetos que tienen el mismo tipo de valores y los mismos métodos recibe el nombre de clases. Una clase es utiliza para representar una entidad y los métodos que la componen.

La única forma de que un objeto pueda acceder a los datos de otro es mediante la llamada de un **método**. , a esta acción se le llama paso de mensaje al otro objeto.

Solo será visible en la interfaz la llamada a los métodos pero no las variables de ejemplares ni el código de estos métodos.

Como resultado se obtiene dos niveles de abstracción de datos.

Al contrario que el modelo entidad- relación, cada objeto tiene su propia identidad, independientemente de los valores que contenta, así, dos objetos pueden tener los mismos valores pero ser diferentes.

La distinción de los objetos se mantiene en el nivel físico.



Modelos lógicos basados en registros.

Los modelos lógicos basados en registros se utilizan para describir datos en los niveles lógicos y de vistas.

La diferencia con los modelos de datos basados en objetos, es que estos se utilizan tanto para especificar la estructura lógica completa de la base de datos como para dar una descripción de alto nivel de la implementación.

Los modelos basados en registros reciben este nombre porque la base de datos se estructura en **registros** de formato fijo de los diferentes tipos.

En cada tipo de registro se define un número fijo de atributos y cada atributo tienen una longitud fija.

El uso de registros de longitud fija hace más fácil la implementación en el nivel físico de la base de datos, esto se diferencia de los modelos basados en objetos, ya que dicha estructura es más rica conduce a registros de longitud variable en el nivel físico.

Los tres modelos basados en registros más utilizados son:

- Modelo relacional.
- Modelo de red.
- Modelo jerárquico

Recuerda 

Modelo relacional

El modelo relacional es el que usa una colección de tablas para representar tanto los datos como las relaciones entre estos datos.

Cada tabla está formada por columnas y cada columna tiene un único nombre.

Es el ya estudiado en la pregunta anterior.

Modelo en red.

Los datos en el modelo de red se representan mediante colecciones de registros y las relaciones entre los datos se representan mediante enlaces. Los registros en la base de datos se organizan como colecciones de grafos.

Modelo jerárquico.

El modelo jerárquico se parece mucho al modelo de red, en que los datos y las relaciones entre ellos se representan mediante registros y enlaces, este modelo se diferencia del de red es que los registros se organizan como colecciones en forma de árbol, en lugar de utilizar grafos.

Diferencias entre los modelos.

El modelo relacional se diferencia del modelo de red y jerárquico en que no usa punteros o enlaces. El modelo relacional relaciona registros mediante los valores que ellos contienen. La no utilización de punteros o enlaces hace que este modelo sea más fácil y se defina mediante un fundamento matemático formal.



Modelo de datos físico.

El modelo de datos físico se utiliza para describir datos en un nivel más bajo. Existen pocos modelos de datos físicos.

Podemos nombrar dos de los más conocidos.

- Modelo de unificación.
- Modelo de memoria por marcos.

En este caso estudiaremos el modelo relacional que es el modelo más usado para la creación de una base de datos.



Se denomina concepto de **diseño de base de datos** a la tarea de tomar decisiones que son fundamentales para poder desarrollar las bases de datos relacionales como por ejemplo puede ser decidir las relaciones , que atributos deben de tener las relaciones, que claves primarias o claves secundarias que se deben de definir.

Una base de datos se utiliza fundamentalmente para almacenar información, para crear, modificar, consultar o eliminar datos.

El diseño de la base de datos va a radicar en fijar una estructura de los datos que se va a tener en la base de datos.

En el caso del modelo relacional la estructura se compondrá de esquemas de relación con atributos, claves primarias, claves foráneas, etc.

El diseño reside en conseguir una representación concreta de los datos a partir del mundo real.

El diseño de una base de datos no es una tarea sencilla, la complejidad de la información y los numeroso requisitos de los sistemas de información hace que sea aún más complicado, por eso cuando se diseña una base de datos es mejor optar por aplicar la estrategia de dividir.

Interesa descomponer el proceso de diseño en varias etapas, en cada una de las etapas se obtendrá un resultado intermedio que servirá de punto de partida a la siguiente etapa, y en la última etapa se obtiene el resultado final deseado.

De este modo, no hace falta resolver toda la problemática que plantea el diseño de golpe. Así lo que conseguiremos es dividir el problema y simplificar el proceso de diseño.

Etapas de diseño de una base de datos:



- **Etapa del diseño conceptual:**

  En esta etapa conseguiremos configurar la estructura de la información que contendrá la base de datos, independientemente de la tecnología que haya que emplear.

  En esta etapa todavía no se sabrá que tipo de base de datos se utilizara, podrá ser relacional o jerárquica, etc.

  Tampoco se tendrá en cuenta todavía que sistema de gestión de base de datos ni el lenguaje que se va a utilizar para implementar la base de datos.

  El resultado de esta etapa suele ser el modelo entidad-relación, ya que es uno de los más usados, este es un modelo de alto nivel.

La etapa de diseño conceptual permite centrarse solo en cómo se va a estructurar la información, sin preocuparse de resolver cuestiones tecnológicas.

– Etapa de diseño lógico.

  Esta etapa comienza con el resultado de la etapa conceptual y se tomara la decisión de que modelo de base de datos se prefiera, en esta etapa es importante tener en cuenta la tecnología que se va a utilizar, y ser lo más cercano y preciso para ajustarse al modelo se sistemas de gestión de base de datos en el que se vaya a implementar la base de datos.

  El modelo más usado hemos dicho que es el modelo relacional, con los cual de esta etapa conseguiremos obtener un conjunto de relaciones, con sus atributos, claves primarias y claves foráneas.

Esta etapa comienza cuando ya se ha resulto en gran medida la problemática de la estructura de la información, gracias a la etapa conceptual.

En esta etapa se produce el cambio o la transformación del modelo entidad- relación en un modelo relacional.

– Etapa del diseño físico:

  En esta etapa se convierte la estructura obtenida del lógico. Nos vamos a referir a los aspectos físicos para la implementación de las relaciones, la selección del tamaño de las memorias o de las páginas, etc.

  El objetivo principal de esta etapa es conseguir un mayor rendimiento de la base de datos y reflexionar sobre los volúmenes que se espera tener que almacenar.

  En esta etapa se completa aspectos de la implementación física que dependerá del sistema gestor de base de datos.

Los aspectos de la implementación se refiere a elegir el tamaño de la memoria, como se van a implementar las relaciones, etc.

– Diseño Conceptual:

  El diseño conceptual se puede desarrollar para cualquier tipo de base de datos, da igual que sea relacional, jerárquica o en red.

  El modelo de entidad relación es el modelo de datos más utilizado en estos momentos ya que es un modelo sencillo y simple. Su semántica es muy tolerante.

Es una herramienta muy útil para los desarrolladores para conseguir reflejar por medio de un modelo conceptual los requerimientos del mundo real.

El modelo entidad relación es fácil de aprender y de utilizar en la mayoría de las aplicaciones., además, existen herramientas que pueden ayudar al diseño como son las herramientas CASE, estas herramientas que pueden ayudar al diseño del modelo Entidad/relación.

El origen del modelo entidad/relación se encuentra en los estudios que se efectuaron por Peter Chen en los 70, se han ido haciendo algunas variaciones de este modelo.

Recuerda 

Las herramientas **CASE**, son herramientas informáticas que son utilizadas para el desarrollo de software, con la utilización de estas herramientas se podrá reducir el costo tanto a nivel de tiempo como a nivel económico. Estas herramientas ayudan al desarrollo del diseño de una aplicación.

Uno de los editores CASE más utilizada es:

El editor UML.

Los objetivos de estas herramientas son:

- Mejorar el desarrollo y mantenimiento de la aplicación.
- Reducir los costes de mantenimiento y desarrollo de la aplicación.
- Poder gestionar todas las fases de desarrollo de la aplicación con una misma herramienta.
- Facilitar el uso de diferentes tecnologías.
- Ayudar a la reutilización de la aplicación.

Según el modelo de datos se debe de tener en cuenta tres aspectos que son importantes:

- La estructura.
- La manipulación.
- La Integridad.

El modelo entidad/ relación se utiliza para hacer reflejar los aspectos de la estructura de los datos , de su integridad , pero no se ocupa de la manipulación.



Los elementos que forman el modelo entidad-relación son:

- Entidad.
- Atributos.
- Relaciones.

Se denomina **entidad** a la representación de un objeto en el mundo real, que se puede distinguir del resto.

Un ejemplo de entidad puede ser un alumno, un empleado, un artículo, etc. Aunque además puede ser algo que no es tangible como por ejemplo una asignatura, un pedido, etc.

Existen dos tipos de entidades:

- Entidad fuerte.
- Entidad débil.



Entidad fuerte.

Es una entidad normal, en la que por medio de uno o varios atributos que la componen identifican de forma única a una entidad.

Un ejemplo de entidad es el siguiente:





Una entidad se representa gráficamente con un rectángulo y con el nombre de la entidad en mayúsculas.

Entidad débil

Una entidad débil es la que por medio de los atributos que la componen no identifican una entidad de forma única, solo la identifican de forma parcial.

Para que esta entidad se convierta en una entidad fuerte debe relacionarse con otra entidad que va a ser la que la va a ayudar a su identificación completa. Una entidad débil se representa con un rectángulo doble.

Un ejemplo de entidad débil es el siguiente:





Las entidades débiles se representan mediante un rectángulo dentro de otro y con el nombre de la entidad en mayúsculas.

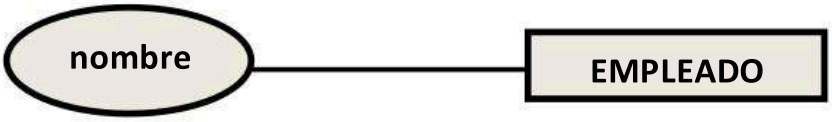
- **Una entidad se compone de atributos**, un atributo son las característica o propiedades que pueden definir a la entidad. Por ejemplo una entidad empleado se le puede asignar atributos como, el nombre, el apellido, la dirección y el teléfono.

Una entidad se puede utilizar tanto para representar un objeto individual como para representar un conjunto de objetos que tienen las mismas características o atributos.

Por ejemplo, se puede representar un solo empleado de una empresa o al conjunto de todos los empleados de la empresa.

Importante

Los atributos se representan físicamente mediante una elipse con su nombre en minúsculas unidos mediante un guion al rectángulo de la entidad a la que pertenezca.



Cuando existen muchos atributos de una entidad , se pondrán los mas útiles y necesarios y los que no sean tan útiles se guardaran a parte del diagrama, para que no sea muy complicado visualizarlo.

Todos los atributos de una entidad coge un valor o un conjunto de valores que no pueden ser descompuestos, un atributo solo tiene un valor para cada entidad.

Por ejemplo, el atributo sueldo puede coger un valor de un rango de valores pero únicamente coge un valor para cada empleado, lo que se quiere decir con esto es que, ningún empleado puede tener más de una valor para el atributo sueldo.

Importante

Para que una entidad sea distinguible del resto será necesario tener un atributo o varios que la identifiquen, estos atributos recibirán el nombre de clave primaria de la entidad.

Ejemplo:

Municipio			
DNI	nombre	apellidos	sueldo

Importante

La entidad empleado contiene una clave que sería el DNI, ya que todos los empleados tendrán uno diferente, ningún empleado puede tener el mismo dni, por lo que el dni ,es un atributo que identifica completamente al empleado.

A esta se le llama **clave primaria** y se representa físicamente subrayando el atributo que funcionara de clave primaria.

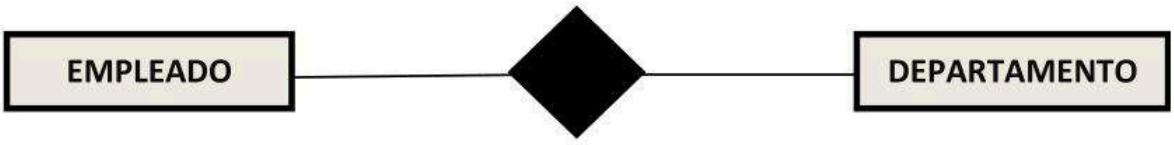
DNI

Una entidad puede tener más de una clave, siempre que identifique ese atributo de forma única a la entidad, al atributo que puede ser clave primaria pero que no se elige como tal, se le denomina **clave secundaria o clave candidata**.

Una **relación**, se define como la unión entre entidades. Se representa físicamente por medio de un rombo. Junto al rombo se le coloca el nombre de la relación., se debe de poner con letras mayúsculas.

Por ejemplo, tenemos una entidad empleado y una entidad departamento, en la que un empleado va a pertenecer a un departamento.

En la siguiente lección, nos adentraremos en todos los tipos de interrelaciones que existen.



Pertenece

– **Diseño Lógico.**

En el diseño lógico se transformará el resultado que nos dará el diseño conceptual que se ha realizado por medio del modelo entidad relación.

Se comenzará transformando las entidades del modelo entidad relación. Cada entidad se transforma en una relación del modelo relacional. Los atributos de la entidad serán los atributos que corresponderán a la relación.

Así será la transformación:

Empleado			
DNI	nombre	apellidos	sueldo

Entidad diseño lógico



EMPLEADO (DNI, nombre, apellidos, sueldo)

Una vez que se han transformado todas las entidades, se tendrán que transformar todas las relaciones en las que participan las entidades.



– **Diseño físico**

  En esta etapa se parte del resultado del diseño lógico, aquí se describirá la metodología del diseño físico para bases de datos relacionales.

En esta etapa se dan una serie de instrucciones para escoger las estructuras de almacenamiento de las relaciones base, decidir si se van a crear índices y cuando desordenar el esquema lógico e incluir redundancias.

El objetivo principal de esta etapa va a ser una descripción de cómo se va a desarrollar la base de datos en memoria secundaria.

Aquí se incluirán las estructuras de almacenamiento y los métodos de acceso que se utilizaran para conseguir un mayor rendimiento y eficacia a los datos.

Mediante el diseño físico se implementara en la base de datos en memoria secundaria, esta etapa traducirá e implementara las relaciones, entidades y atributos que resultan del diseño lógico.

4.3. Análisis de los distintos tipos de relaciones y su implementación en la base de datos

En el tema anterior hicimos una pequeña mención de las relaciones que se producen entre entidades, definimos una relación o interrelación a una la unión entre distintas entidades.

Se representa físicamente con un rombo que une las distintas entidades.



En una relación se pueden unir dos o más entidades, al número de entidades que integran una relación se le llama **grado de la relación**.

En una relación pueden participar desde una entidad consigo misma, dos entidades, tres entidades hasta n entidades.

Las relaciones que se producen entre solo dos entidades se les denominan relación binaria, a las relaciones en la que intervienen tres entidades recibe el nombre de ternaria y así sucesivamente.

Todas las relaciones que unan más de tres entidades se les denomina relación n-aria.

También existe la de grado uno, en la que solo participa una entidad.

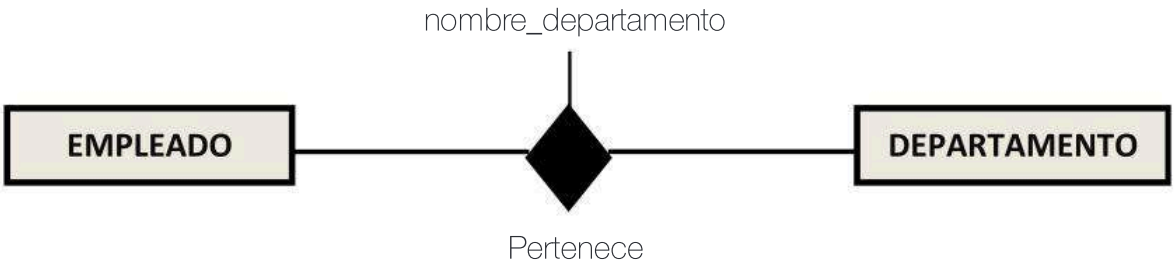
Por ejemplo un empleado que a la vez es jefe, a este tipo de grado de relación se le denomina relación recursiva.

Tipos degrado de relación:

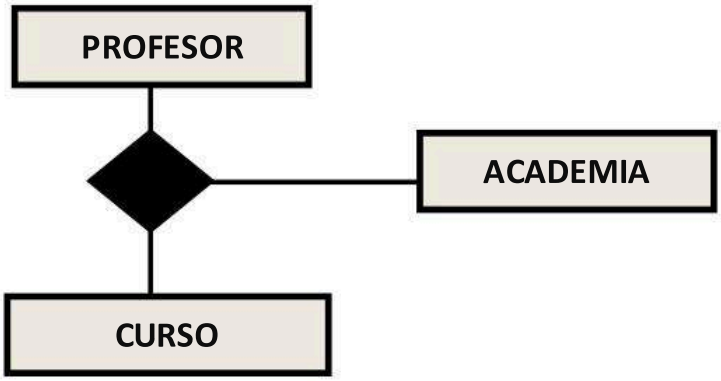
- **Grado 1.** Relación entidad consigo misma, es llamada recursiva.
- **Grado 2**. Relación de dos entidades, recibe el nombre de binaria.
- **Grado 3**. Relación de tres entidades, recibe el nombre de ternaria.
- **Grada n-arias**. Relación de más de tres entidades.

En muchas ocasiones una relación puede llevar un atributo

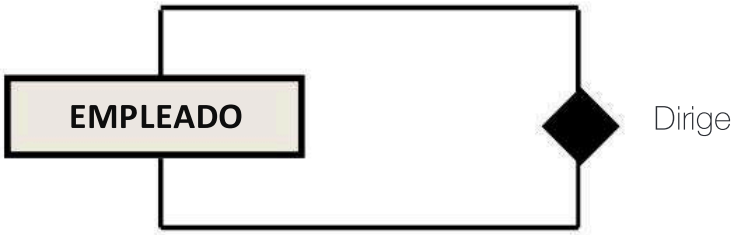
Ejemplo de relación entre dos entidades



Ejemplo de relación entre tres entidades.

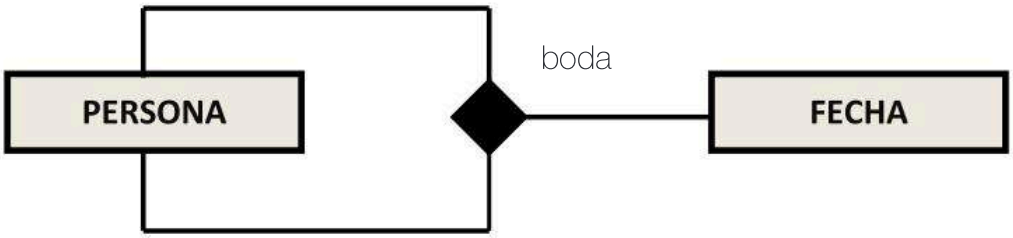


Ejemplo de relación recursiva.



Ahora bien, una relación recursiva también puede ser tanto binaria como n-aria.

Ejemplo de relación recursiva ternaria.



Después de saber qué tipo de relaciones existen, vamos a hablar de la cardinalidad o conectividad de una relación.

Importante

La cardinalidad es el tipo de correspondencia que se establecerá entre las entidades de la relación. Se refleja el número de ocurrencias de una entidad con las ocurrencias de la otra entidad con la que existe una asociación.

Se define la carnalidad como el grado de participación de las entidades en una relación.

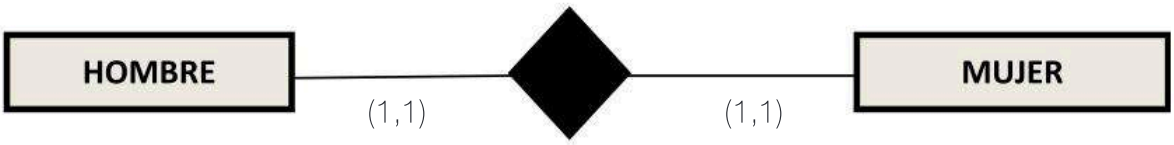
Para saber cuál va a ser la cardinalidad se realizan las siguientes preguntas: ¿Cuántos elementos de una entidad participarán en la relación con un elemento concreto de la segunda entidad? y ¿cuántos elementos de la segunda entidad participarán en la relación con un elemento concreto de la primera entidad? La respuesta será 1 o muchos.

Importante

Existen tres tipos de cardinallidad entre dos entidades:

- **Relación uno a uno se representara así: (1:1)**, se representa poniendo un 1 a cada lado de la relación. A cada elemento de la primera entidad le corresponde solo uno de la segunda entidad y a la inversa.

Ejemplo:



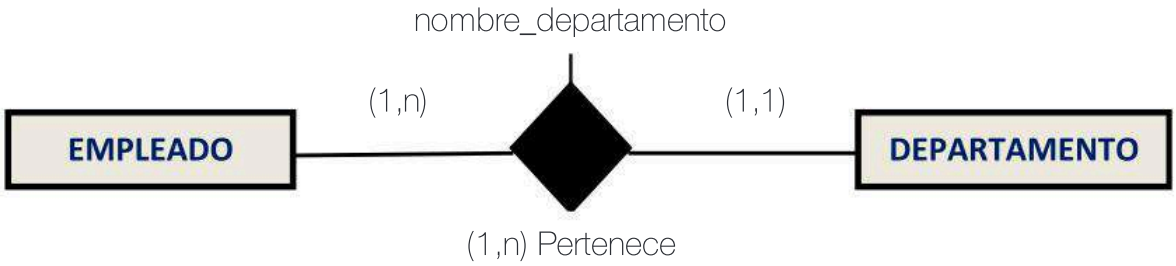
(1,1) Matrimonio

En este ejemplo, la relación matrimonio tiene una cardinalidad 1:1, significa que un hombre está casado con una sola mujer y una mujer esta casada con un solo hombre.

Importante

- **Relación uno a varios, se representara así (1: n)**, se representa poniendo un 1 en el lado de una entidad y n en el lado de la otra entidad relacionada. A cada elemento de la primera entidad le corresponde uno o mas elementos de la segunda entidad, y a cada elemento de la segunda entidad le corresponde uno solo de la primera entidad.

Ejemplo:

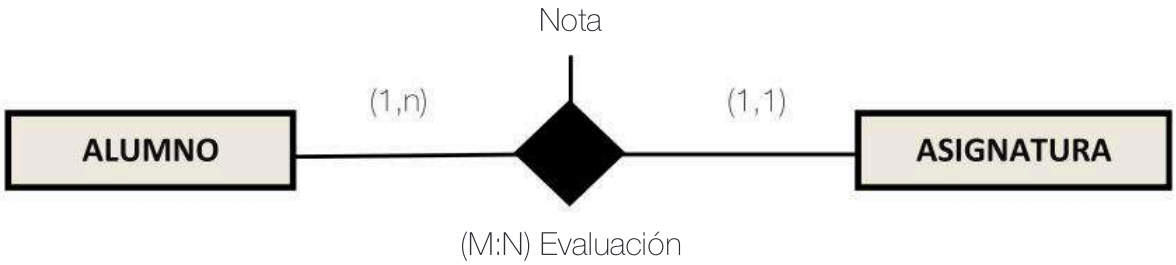


Importante

La relación Pertenece es una relación 1 : n, en la que un empleado pertenece a un solo departamento y un departamento puede tener uno o más empleados.

- **Relación varios a varios, se representa así: (M: n)**, se representa poniendo M y N correspondiente a cada entidad relacionada. A cada elemento de la primera entidad le corresponde uno o más elementos de la segunda entidad y a cada elemento de la segunda entidad le corresponde uno o mas elementos de la primera entidad.

Ejemplo:



Importante

La relación Evaluación es una relación M: n, en la que un alumno puede evaluarse de varias asignaturas y una misma asignatura puede ser evaluada a muchos alumnos.

En este tipo de relación se da mucho que aparezcan atributos, mientras que en las relaciones 1:1 y 1: n no va a ser necesario que los tengan.

Existen dos tipos de entidades:

- Entidad fuerte.
- Entidad débil.

Importante

- **Entidad fuerte**, que son las entidades que por medio de sus atributos clave se puede identificar del resto.
- **Entidad débil**, que son las entidades en las que por medio de sus atributos no la identifican del todo, sino de forma parcial.

  Esta entidad está obligada a intervenir en una relación que la ayudara a identificarla.

Una entidad débil se representa con un rectángulo doble, y la relación que va a servir para darle identidad se va a representar con una línea doble.

Ejemplo



En este ejemplo, los empleados fijos pueden ser uno o más de un empleado, y fijo puede va a ser uno, en este ejemplo nos ha valido la relación PUEDE_SER para identificar la situación de un empleado.

Las entidades débiles siempre debe de realizar una relación que permita identificarla, siempre va a ser una relación binaria y su cardinalidad siempre va a ser 1: N, la entidad débil siempre tendrá el lado N.



Especialización/generalización.

Suele ocurrir que haya casos en las existan ocurrencias de una entidad que tienen características específicas que nos interesa modelizar.



La especialización consiste en la unión de dos o más entidades de bajo nivel para dar como resultado entidades de más alto nivel. La generalización se realiza para agrupar entidades parecidas de bajo nivel e intentar quitar las diferencias e igualarlas.

Tratará de buscar todos los atributos que sean iguales de varias entidades que darán como resultado una entidad que englobara todos estos atributos iguales, con lo que quedara un nivel más alto de las entidades originales.

A esta entidad resultante de alto nivel se le denomina entidad **superclase**.

Debemos de señalar las diferencias entre :

- Entidad superclase.
- Entidad subclase.

Superclase y subclase.

Importante

- **Una entidad superclase** es la que es capaz de reunir los atributos y relaciones que son comunes en diversas entidades.
- **Entidad Subclase**. Esta consiste en reunir todos los atributos y características que le son propias de su especialidad.

Toda ocurrencia de una entidad subclase tienen que ser una ocurrencia de su entidad superclase.

Un ejemplo para comprender esto sería así; un empleado de una empresa puede ser un técnico, una administrativo o un directivo.

La representación de la especialización /generalización es con una flecha que parte de las entidades subclase hasta llegar a la entidad superclase.

La relación entre una superclase y una subclase siempre tendrá una cardinalidad uno uno.



La superclase seria empleado y las subclases de empleados serian técnico, administrativo y directivo, ya que un empleado puede ser cualquiera de los oficios de una empresa, un empleado esta está especializado en algo.

Importante

En la especialización/generalización los atributos de la entidad superclase se propaga hacia las entidades subclase, ha esto se le denomina **Herencia de propiedades**.

Para realizar el diseño de una generalización/especialización, se pueden realizar uno de los siguientes procesos:

- Que se identifique primero la entidad superclase y después identifique las cualidades específicas que van a hacer necesarias las entidades subclase. . en este caso se dice que se ha llevado a cabo un proceso de especialización.
- Que se modelice primero las entidades subclase después identifique las características que son comunes e identificar la superclase. A este proceso se le llama generalización.

La especialización/generalización puede ser de dos tipos:

- Exclusiva
- Solapada

- **Exclusiva.**

  En la que una misma ocurrencia no puede aparecer en dos entidades subclase diferentes. Se representa con la letra D.

- **Solapada.**

  En este caso una misma ocurrencia puede aparecer en dos entidades subclase diferentes. Se representa con la letra S.

A su vez también tanto exclusiva como solapada puede ser:

- **Total**, significa que Torda ocurrencia de una entidad superclase debe pertenecer a alguna de las entidades subclase. Se representa con la letra T.
- **Parcial**, En este caso es al contrario que la exclusiva total, Torda ocurrencia de una entidad superclase no tiene por qué pertenecer a alguna de las entidades subclase. se representa con la letra P.

- Exclusiva total.
- Exclusiva parcial.
- Solapada total.
- Solapada parcial

- **Exclusiva total.**

  La cardinalidad seria (0,1) o (0,1).

  Donde por ejemplo, un empleado puede ser un técnico o un administrativo, pero solo una de las dos.

- **Exclusiva parcial.**

  La cardinalidad seria (0,1) o (0,1).

  Donde por ejemplo, un empleado puede ser un administrativo o técnico y además otra categoría no incluida.

- **Solapada total.**

  La cardinalidad seria (0,1) o (1,1).donde por ejemplo, un empleado puede ser un administrativo y técnico pero no otra categoría que no esté incluida.

- **Solapada parcial.**

  La cardinalidad seria (0,1) o (1,1).

  Donde por ejemplo, un empleado puede ser administrativo y técnico y además otra categoría no incluida.



Ahora tocara, transformar el modelo entidad-relación al modelo relacional.



Ya sabemos que los elementos que conforman el modelo E-R son las entidades y las relaciones, las entidades cuando se transforman al diseño lógico da relaciones y las relaciones cuando se transforman pueden dar lugar a claves foráneas o pueden dar una nueva relación.

Para la transformación de las relaciones hay que tener en cuenta la cardinalidad.

Las relaciones binarias 1:1, N: 1 y 1: N dan lugar a claves foráneas o claves ajenas.

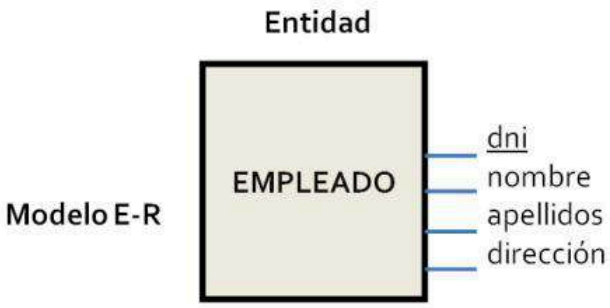
La clave foránea aparece cuando en una relación entre dos entidades, una de las claves primaria de una entidad pasa a ser referencia en la otra entidad, se propagará la clave de una relación a otra.

Las relaciones M: N y todas las relaciones n-arias se transformarán en nuevas relaciones.

En primer lugar vamos a ver como se transforma una entidad del modelo entidad-relación, cada entidad se transforma en una relación del modelo relacional.

Los atributos que forman la entidad ahora pasaran a ser atributos de la relación, teniendo en cuenta que la clave primaria de la entidad será la clave primaria de la relación.

Ejemplo:



Transformación de la entidad al modelo lógico relacional.

EMPLEADO(dni, nombre, aplellidos,dirección)

Importante

Ahora transformaremos las relaciones binarias.

Para transformar una relación binaria hay que tener en cuenta la cardinalidad.

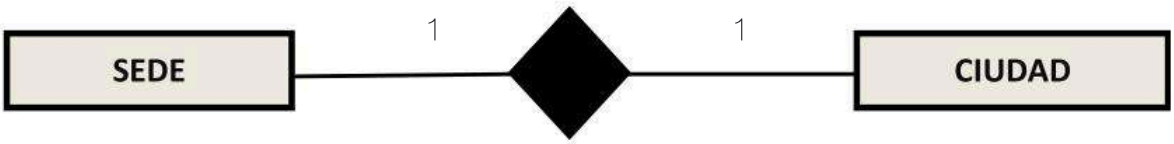
Relación 1:1,

Cada una de las entidades que interviene se transforma automáticamente en relaciones con sus correspondientes atributos.

En este caso solo se añadirá a cualquiera de las dos relaciones una clave ajena que hará referencia a la otra.

Ejemplo: Se estipula concretamente que queremos una entidad sede que se encuentre solo en una ciudad, y que no haya ninguna ciudad con más de una sede.

Diseño Conceptual E-R.



(1:1) ubicación

Diseño Lógico, modelo relacional

SEDE (cod_sede, nombre, cp, teléfono, dirección, cod_ciudad) donde {cod_ciudad} referencia CIUDAD (cod_ciudad, nombre);

También puede ser al contrario

SEDE (cod_sede, nombre, cp, teléfono, dirección, cod_ciudad);

CIUDAD (cod_ciudad, nombre, cod_sede) donde {cod_sede} referencia SEDE

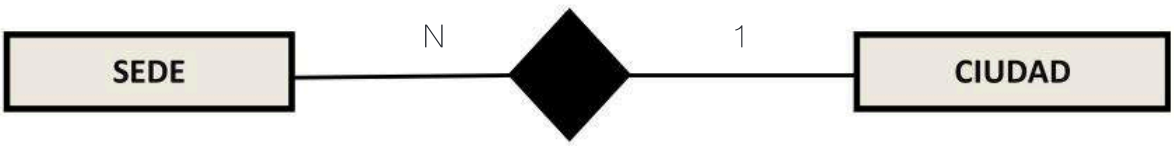
Importante

Relación 1: N y N: 1,

Cada una de las entidades que interviene se transforma automáticamente en relaciones con sus correspondientes atributos en estos casos se añadirá la clave ajena a la entidad del lado N.

Ejemplo: Tomando como punto de partida que una sede puede estar solo en una ciudad, y que en una ciudad puede haber una o más sedes.

Diseño Conceptual E-R.



(1:N) ubicación

Diseño Lógico ,modelo relacional

CIUDAD (cod_ciudad, nombre);

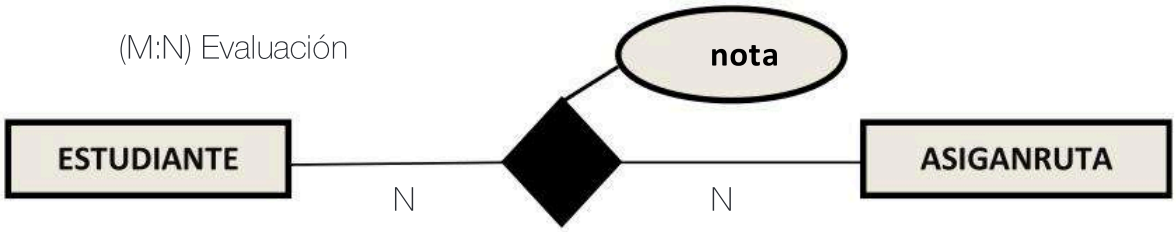
SEDE (cod_sede, nombre, cp, teléfono, dirección, cod_ciudad) donde {cod_ciudad} referencia CIUDAD



Relación M: N, en este caso la relación se transforma en otra relación. La clave primaria de la relación será los atributos clave primaria de las dos entidades relacionadas

Ejemplo: Tomando como punto de partida que una sede puede estar en una o muchas ciudades y que en una ciudad pueda haber una o muchas sedes y además queremos poner un atributo provincia a la relación UBICACIÓN con la provincia donde se ubicara esa ciudad.

Diseño Conceptual.



Diseño Lógico

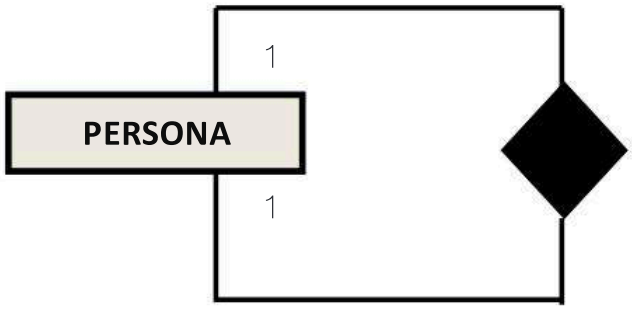
ESTUDIANTE (dni, nombre, cp, teléfono, dirección, cod_asigantura);

ASIGNATURA (cod_asignatura, nombre);

EVALUACION(dni, cod_asigantura, nota) donde {cod_asignatura} referencia a ASIGANTURA y {dni} referencia ESTUDIANTE;

Importante

También existen **relaciones recursivas**, una relación recursiva es cuando una entidad está relacionada más de una vez. Este tipo de relación puede ser también binaria como n-arias.



Importante

Ahora seguimos con la transformación de relaciones ternarias.

Las transformación de las relaciones ternarias es muy parecida a la transformación M:N. en esta relación no es posible representarla mediante claves foráneas, es necesario crear otra relación.

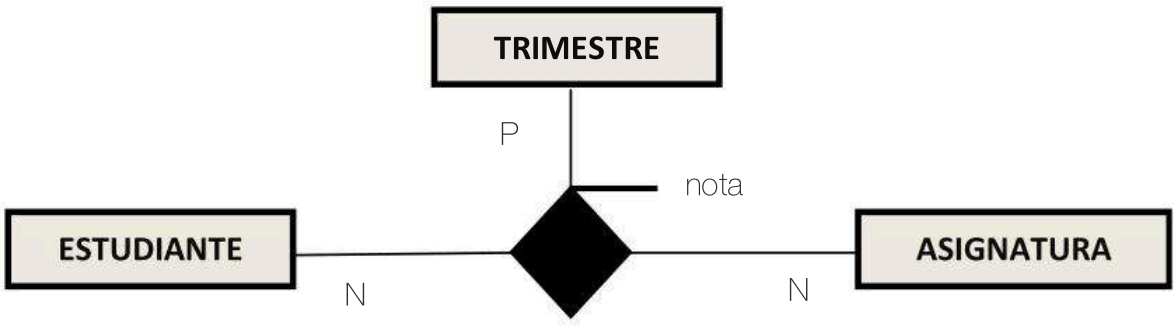
Es necesario que contenga las claves primarias de las tres entidades relacionadas junto con los atributos.

En resumen, todas las relaciones en que intervienen tres entidades van a dar lugar a una relación nueva.

- **Cardinalidad M:N:P**

  Se transforma en una relación, que contendrá todos los atributos clave primaria de cada una de las entidades involucradas en la relación.

Ejemplo:



(M: N: P) evaluación

La transformación es la siguiente:

ESTUDIANTE (dni, nombre, cp, teléfono, dirección);

ASIGNATURA (cod_asignatura, nombre);

TRIMESTRE (cod_trimestre, año);

EVALUACION_TRIMESTRAL (dni, cod_asigantura, cod_trimestre ,nota) donde {dni} referencia ESTUDIANTE

{cod_asignatura} referencia a ASIGANTURA,

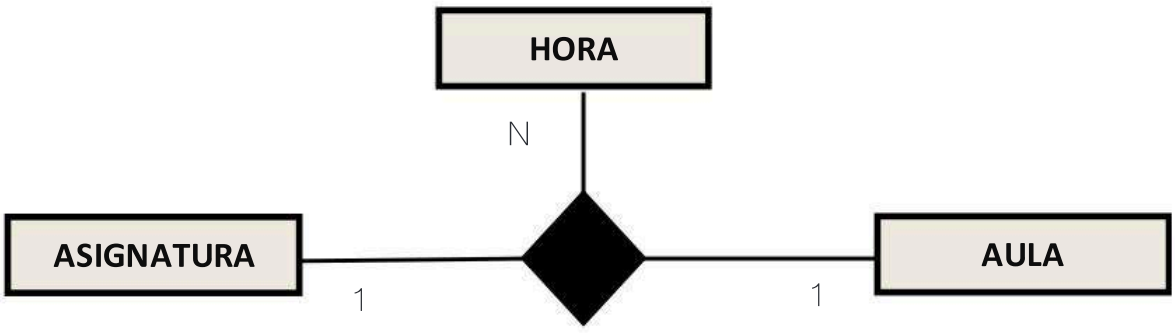
{cod_trimestre} referencia a TRIMESTRE;

Importante

Cardinalidad N: 1:1.

Cuando la cardinalidad es N: 1:1, la relación que se transforma va a estar compuesta por la clave primaria de la entidad con el lado N y los atributos que forman clave primaria de cualquiera de dos entidades que tienen el 1, se podrá escoger la que venga mejor para el diseñador, que va a ser el que va a decidir.

Ejemplo:



(N: 1:1) Duración

La transformación se puede realizar de dos formas esta es la primera, en la que se pasa es la siguiente:

ASIGNATURA (cod_asignatura, nombre);

AULA (cod_aula, nombre);

HORA (cod_hora);

DURACION_CLASE (cod_asigantura, cod_hora,cod_aula) donde {cod_asignatura} referencia a ASIGANTURA,

{cod_hora} referencia a HORA,

{cod_aula} referencia a AULA;

O de la siguiente forma:

ASIGNATURA (cod_asignatura, nombre);

AULA(cod_aula, nombre);

HORA (cod_hora);

DURACION_CLASE (cod_asigantura, cod_hora,cod_aula) donde {cod_asignatura} referencia a ASIGANTURA,

{cod_hora} referencia a HORA,

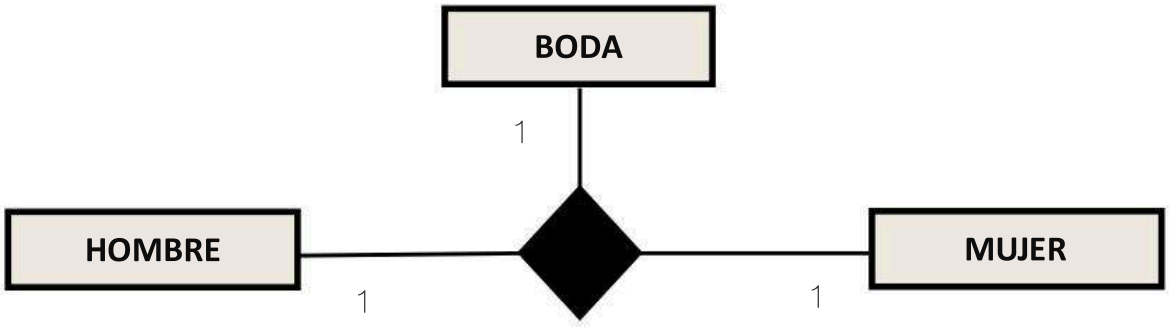
{cod_aula} referencia a AULA;

Importante

Cardinalidad 1:1:1

Surgirá una relación que estará compuesta por los atributos clave primaria de dos de las entidades de las tres que están involucradas en la relación.

Ejemplo:



(1:1:1) Matrimonio

La transformación se puede realizar de dos formas esta es la primera, en la que se pasa es la siguiente:

HOMBRE (dni_hombre, nombre, apellidos);

MUJER (dni_mujer, nombre, apellidos);

BODA (n_boda, fecha);

MATRIMONIO (dni_hombre, dni_mujer,n_boda) donde {dni_hombre} referencia a HOMBRE,

{dni_mujer} referencia a MUJER,

{n_boda} referencia a BODA;

O de la siguiente forma:

HOMBRE (dni_hombre, nombre, apellidos);

MUJER (dni_mujer, nombre, apellidos);

BODA (n_boda, fecha);

MATRIMONIO (dni_hombre, dni_mujer,n_boda) donde {dni_hombre} referencia a HOMBRE,

{dni_mujer} referencia a MUJER,

{n_boda} referencia a BODA;



Ahora seguiremos con la transformación de relaciones n_arias.

La transformación de las relaciones n_arias es muy parecida a las ternarias.

En todos los casos de cardinalidad dará como resultado una relación que va a estar formado por todos los atributos que son claves primarias de las n entidades involucradas en la relación.

Se dan los siguientes casos.

- Si todas las entidades están relacionados con cardinalidad MUCHOS la clave primaria de la relación serán los atributos clave primaria de cada una de las entidades involucradas en la relación.

- Si una o más de una de las entidades están relacionadas con cardinalidad UNO, la clave primaria de la relación estará formada por las claves primarias de todas las entidades involucradas en la relación menos una de ellas, con la condición de que la entidad que no se haya incluido debe de tener una cardinalidad UNO.



Ahora transformaremos la generalización/especialización.

Todas las entidades superclase y subclase que forman parte de una especialización/generalización se transforma en una relación;

La relación de la entidad superclase va a estar formada por clave primaria de la entidad superclase y contendrá todos los atributos comunes.

Las relaciones de las entidades subclase estarán formadas por la clave primaria de la clave primaria de la entidad superclase y los atributos específicos de la subclase.

Ejemplo:



Pasarlo del modelo conceptual al modelo lógico del modelo relacional se realizaría de la siguiente manera:

EMPLEADO (dni, nombre, dirección, cp, provincia);

DIRECTIVO (dni, nombre, dirección, teléfono);

Donde {dni} referencia EMPLEADO

ADMINISTRADOR (dni, nombre, apellidos, cp, provincia);

Donde {dni} referencia Empleado

TECNICO (dni, nombre, apellidos, dirección, cp, provincia);

Donde{dni} referencia Empleado



Resumen de las transformaciones del modelo entidad- relación al modelo relacional:

Componente del modelo entidad-relación	Transformación al modelo relacional
– Entidad	Se transforma en una relación.
– Relación 1:1	se propaga clave primaria, para convertirse clave foránea.
– Relación 1:N oN:1	se propaga clave primaria, para convertirse clave foránea.
– Relación M:N	
– Relación n_aria	Da como resultado una nueva relación
– Relación recursiva o reflexiva	Da como resultado una nueva relación.
– Especialización/generalización	Igual que en las relaciones no recursivas – Propagar clave para cardinalidad 1:1, N:1 y 1:N. – Da como resultado una relación la cardinalidad M:N y n_aria Se transforma en una relación para la entidad superclase Y una relación para cada una de las entidades subclase.

El modelo relacional se basa en las doce reglas de Codd que es el creador de este modelo, las reglas de Codd son las siguientes:

– **Regla de la información.**

  Toda la información que reside en una base de datos relacional se representa en el nivel lógico mediante tablas.

– **Regla de acceso garantizado:**

  Para todos y cada uno de los datos de una base de datos relacional se asegurara de que son accesibles a nivel lógico utilizando la combinación de nombre de tabla o relación, valor de clave primaria y nombre de columna.

- **Regla de tratamiento sistemático de valores nulos.**

  Se debe de establecer valores nulos para representar información que se desconoce.

- **Regla de catálogo dinámico en línea basado en el modelo relacional.**

  La descripción de la base de datos se representa a nivel lógico de la misma forma que los datos normales, de modo que los usuarios que tengan acceso podrán aplicar el lenguaje relacional de consulta, igual que se haría con datos normales.

- **Regla del sublenguaje de datos completo.**

  Un sistema relacional debe de soportar varios lenguajes y varios modos de uso del terminal, como por ejemplo puede ser rellenar un formulario.

  Debe de existir un lenguaje cuyas sentencias se expresen mediante una sintaxis ya definida, como puede ser cadena de caracteres y que soporte:

  - Definición de datos.
  - Definición de vistas.
  - Manipulación de datos.
  - Restricciones de integridad.
  - Restricciones de transacciones.

Además deberá de tener interfaces para realizar consultas. Un lenguaje que cumple con todos los requisitos es SQL.

- **Regla de actualización de vistas.**

  Todas las vistas se pueden actualizar por el sistema.

- **Regla de inserción, actualización y borrado de alto nivel.**

  La capacidad de manejar una relación base o derivada como uno solo se aplica no solo a la consulta sino también la inserción, modificación y borrado de datos.

- **Regla de independencia física de datos.**

  Los programas de aplicación y actividades del terminal permanecerán sin cambios a nivel lógico, cualesquiera que sean los cambios tanto en la representación del almacenamiento como en los métodos de acceso.

- **Regla de independencia lógica de datos.**

  Los programas de aplicación y actividades del terminal no sufrirá cambios a nivel lógico si se realizan modificaciones a las tablas base que son los que conservan la información.

- **Regla de independencia de integridad.**

  Las restricciones de integridad que son específicas para una determinada base de datos relacional deben ser definidas en el sublenguaje de datos relacional, y almacenados en el catálogo, no en los programas de aplicación.

  El objetivo de una base de datos almacenar los datos, las relaciones y evitar las restricciones se codifiquen en los programas.

Como parte de las restricciones relacionadas al modelo relacional están:

- La integridad de entidad:
- Toda tabla tiene que tener una clave primaria.
- La integridad de dominio
- Toda columna de una tabla contendrá valores de un determinado conjunto de valores.
- La integridad referencial
- Toda clave foránea no nula debe existir en la relación donde es clave primaria.

- **Regla de independencia de distribución.**

  Una base de datos es independiente de la distribución.

  Las ordenes y programas son los mismos y se ejecutan igual en una base de datos centralizada o distribuida.

  Son fácilmente distribuibles.

Esta regla será responsable de los tres tipos de transparencia de distribución:

- Transparencia de localización.
- El usuario tiene la impresión de estar trabajando sobre una base de datos local.
- Transparencia de fragmentación.
- El usuario no se dé cuenta si la relación con la que está trabajando está fragmentada.
- Transparencia de replicación.
- El usuario no se da cuenta de que cabe la posibilidad de que existan copias de una misma relación en sitios diferentes.

- **Regla de la no subversión.**

  Un sistema relacional tiene un lenguaje de nivel bajo, este nivel no puede saltarse las reglas de integridad uy las restricciones que se realizan con lenguajes relacionales de más alto nivel.

  Algunos problemas no se pueden solucionar con un lenguaje de alto nivel.

  Normalmente se utiliza el lenguaje SQL para solucionar estos problemas.

Resumen de las 12 reglas.

Doce reglas de CODD

- Regla de la información.
- Regla de acceso garantizado.
- Regla de tratamiento sistemático de valores nulos.
- Regla de catálogo dinámico en línea basado en el modelo relacional.
- Regla del sublenguaje de datos completo.
- Regla de actualización de vistas.
- Regla de inserción, actualización y borrado de alto nivel.

- Regla de independencia física de datos.
- Regla de independencia lógica de datos.
- Regla de independencia de integridad.
- Regla de la no subversión.
- Regla de independencia de distribución.



Después de obtener las relaciones tras el paso del modelo entidad –relaciona al modelo relacional , se tiene que realizar un proceso llamado **normalización**.

La normalización se aplica para:

- Evitar redundancias.
- Evitar problemas de actualización de los datos en las tablas.
- Proteger la integridad de los datos.



La normalización consiste en aplicar una serie de reglas. Es un proceso por el cual se transforman los datos más complejos en un conjunto de estructuras de datos más pequeñas.

Son una serie de reglas, mediante las cuales ayudan a los diseñadores de bases de datos a desarrollar un esquema para que sea más fácil resolver problemas de lógica.

La normalización facilita el proceso de diseño., además el uso de la normalización hace que la base de datos consuma menos espacio, existirán menos repetición de datos. El proceso de normalización se divide en tres fases.

La normalización tiene varios grados o niveles, son:

- Primera forma normal o 1FN.
- Segunda forma normal o 2FN.
- Tercera forma normal o 3FN.



Primera forma normal o 1fn.

La primera forma normal establece que las columnas repetidas deben de eliminarse y situarlas en tablas separadas.

Además de identificar las columnas de datos que son clave primaria.

Ejemplo de tabla no normalizada de ninguna forma.

Id_ pedido	Fecha_ pedido	Id_ cliente	Nombre_ cliente	Estado_ pedido	Cod_ articulo	Nombre_ articulo	Cantidad	precio
6501	2/15/14	65	Repuestos s.l.	Realizado	112	Rueda	8	85
6501	2/15/14	65	Repuestos s.l.	Realizado	112	Frenos	10	45
6501	2/15/14	65	Repuestos s.l.	Realizado	112	aceite	14	100
6502	3/13/14	100	Alirepuestos s.l.	Proceso	325	Aceite plus	4	5.20
6503	4/11/14	96	aerorepuestos	No procesado	112	Funda	2	45
6503	4/11/14	96	aerorepuestos	No procesado	210	filtros	2	20

En este ejemplo tenemos una tabla sin normalizar, en la que nos encontramos una tabla de venta de un producto.

Al fijarnos en la tabla, podemos damos cuenta de que existen grupos de datos de algunas columnas que son iguales. La primera forma normal prohíbe la repetición de conjuntos de datos, por lo que tendremos que convertirla a esta forma normal.

En primer lugar lo que tenemos que hacer es eliminar estos datos repetidos, se relaza mediante la creación de una tabla que tendrá la clave primaria de la tabla base y de la columna donde estén los datos repetidos.

Quedaría de la siguiente forma, obtendríamos dos tablas a las que llamaremos PEDIDO y ARTICULOS:

PEDIDO

Id_pedido	Fecha_pedido	Id_cliente	nombre_cliente	Estado_pedido
6501	2/15/14	65	Repuestos s.l.	Realizado
6502	3/13/14	100	Alirepuestos s.l.	Proceso
6503	4/11/14	96	aerorepuestos	No procesado

Id_pedido	Cod_articulo	Nombre_articulo	Cantidad	precio
6501	112	Rueda	8	85
6501	112	Frenos	10	45
6501	112	aceite	14	100
6502	325	Aceite plus	4	5.20
6503	112	Funda	2	45
6503	210	filtros	2	20

Ahora nos toca aplicar la segunda forma normal.

En la que se determinara cuáles de las columnas no clave, se eliminara esas columnas y se creara una segunda tabla con esas columnas y las columnas que contenga la clave primaria de la que dependen.

La tabla pedidos está ya en segunda forma normal, pero la tabla artículos no, se realizara de la siguiente manera:

ARTÍCULOS

Id_pedido	Cod_articulo	cantidad
6501	112	8
6501	112	10
6501	112	14
6502	325	4
6503	112	2
6503	210	2

ARTÍCULOS_PEDIDO

Cod_articulo	Nombre_articulo	precio
112	Rueda	85
112	Frenos	45
112	aceite	100
325	Aceite plus	5.20
112	Funda	45

Ahora nos toca aplicar la tercera forma normal.

En la que se deben de determinar las columnas que dependen de otra columna clave primaria, se eliminan esas columnas y se crea otra tabla con esas columnas y con la columna que no es clave de la cual son dependientes.

Quedará de la siguiente manera:

PEDIDO

Id_pedido	fecha	Id_cliente
6501	2/15/14	65
6502	3/13/14	100
6503	4/11/14	96

CLIENTES

Id_cliente	Nombre_cliente	estado
65	Repuestos s.l.	Realizado
100	Alirepuestos s.l.	Proceso
96	aerorepuestos	No procesado

Podemos resumir que la normalización es una técnica utilizada para crear relaciones lógicas entre tablas de una base de datos, ayuda a prevenir que se produzcan errores lógicos en la manipulación de los datos, también facilitaran la agregación de nuevas columnas sin tener que romper el esquema actual ni las relaciones.

4.4. Descripción del lenguaje de acceso a base de datos

Los lenguajes se pueden dividir en dos grandes grupos:

- **Lenguajes de consulta procedimentales.**

  Es un tipo de lenguaje en el que el usuario facilita unas instrucciones al sistema para que realice las operaciones en la base de datos para obtener un resultado final.

- **Lenguajes de consulta no procedimentales.**

  En este tipo de lenguaje el usuario realiza una explicación de la información que desea sin llevar a cabo ningún procedimiento concreto para obtener esa información.



El lenguaje utilizado para acceder a las bases de datos es el llamado SQL y se trata de un lenguaje no procedimental.

El lenguaje SQL o lenguaje de consulta estructurada, llamado por sus siglas en inglés Structured Query Language.



Se trata de un lenguaje declarativo, esto quiere decir que se basa en el desarrollo de programas especificando o declarando una serie de condiciones, restricciones, cálculos o trasformaciones que van a descubrir un problema y aplican una solución.

La solución se obtiene mediantes controles internos. No van a existir asignaciones destructivas y las variables utilizadas tienen transparencia referencial.

Este lenguaje declarativo de acceso a base de datos relacionales, van a permitir concretar diversos tipos de operaciones que van a operar con los datos de una base de datos.

Una de las características más importante se encuentra en el manejo del algebra y cálculo relacional, por medio de los cuales permitirá realizar consultas con el fin de recuperar de forma sencilla, toda la información de interés de una base de datos, también podrá realizar cambios sobre ella.

Es un lenguaje informático de 4 **generación**.

SQL es una herramienta que servirá para organizar, gestionar y recuperar datos que se encuentran almacenados en una base de datos relacional.

Su aprendizaje no solo va a servir para esta aplicación, sino, para todas las aplicaciones existentes que soporten este tipo de lenguaje, ya que es un lenguaje estándar, por haberse consolidado por el instituto ANSI y por la institución ISO.

En una base de datos relacional que constara de varias tablas, se pueden realizar tres operaciones fundamentales que permitirán la creación de nuevas tablas a partir de la ya existentes

Las operaciones fundamentales del algebra relacional en el que se basa el lenguaje SQL son:

- Selección.
- Proyección.
- Concatenación o unión.

- **La selección:**

  La selección va a consistir en conseguir una nueva tabla formada por algunas filas (Registros) a partir de otra tabla que ya existe.

La selección consiste en usar unos criterios que permitirá decidir que registros de la tabla queremos que aparezcan en la tabla nueva.

El usuario podrá seleccionar los datos que le interesen de las tablas y visualizarlos en pantalla.

Por ejemplo: queremos que aparezca de una tabla ya existente de alumnos, todos los alumnos que tengan de nota un notable, pues en la nueva tabla aparecerán todos los alumnos con la nota de notable.

Tabla de Alumnos ya existente

DNI	NOMBRE	APELLIDOS	NOTA
45789685 V	José María	Martínez Jurado	Suficiente
58962566 B	Aurelia	Gómez García	Notable
78596632 Z	Fernando	Gutiérrez González	Bien
78774578 C	Ana	Fernández Martin	Notable

SELECCION

Tabla nueva con los alumnos solo que tienen notable

DNI	NOMBRE	APELLIDOS	NOTA
58962566 B	Aurelia	Gómez García	Notable
78774578 C	Ana	Fernández Martin	Notable



La proyección:

La proyección radica en obtener una nueva tabla formada por algunos campos (columnas) que han sido seleccionadas de otra tabla ya existente.

Puede pasar el caso, de que al elegir un campo, el resultado produzca registros que son idénticos, en la nueva tabla solo aparecerá un registro de los repetidos. Habitualmente se pueden realizar la selección y la proyección en el mismo momento.

Ejemplo:

Queremos hacer una proyección en la que aparezca los campos TEMATICA y AUTOR.

Tabla de Libros ya existente

COD	TEMATICA	TITULO_LIBRO	AUTOR	EJEMPLARES
5	Informática	Windows XP	Agustín Romano	4
5	Informática	SQL Server 2000	Agustín Romano	1
2	Ciencia	Estudio de las neuronas	Agustín Romano	3
3	Arte	Arte Románico	Agustín Romano	1

PROYECCION

Tabla nueva con el autor y la temática, aunque la temática se repite por el autor, no aparecerá en la nueva tabla.

TEMATICA	AUTOR
Informática	Agustín Romano
Ciencia	Agustín Romano
Arte	Agustín Romano



La concatenación o unión:

La concatenación radica en obtener una tabla nueva a partir de la unión de dos tablas ya existentes.

Por lo general la unión de registros se produce cuando en las dos tablas se encuentra el contenido de un determinado campo, cuando este se produce, se crea un nuevo registro en la tabla nueva añadiendo los campos a la segunda tabla.

Por ejemplo:

En primer lugar vamos a realizar una selección de una Tabla ya existente, en la que vamos a seleccionar las materias que sean de ciencia y de informática, el resultado lo vamos a concatenar o unir con una tabla nueva que va a contener los campos autor, nacionalidad y profesión.

A partir de aquí obtendríamos una nueva tabla que tendrá más campos pero con el mismo número de registros que la tabla existente.

Tabla existente

AUTOR	NACIONALIDAD	PROFESION
Agustín Romano	Española	Profesor
Roberto Guzmán	Argentina	Profesor
William Howard	Estadounidense	Cátedra



Concatenación

Tabla resultante de la unión de ambas.

Cod	Tematica	Titulo_libro	Autor	Ejemplares	Nacionalidad	Profesion
5	Informática	Windows XP	Agustín Romano	4	Española	Profesor
5	Informática	SQL Server 2000	Agustín Romano	1	Española	Profesor
2	Ciencia	Estudio de las neuronas	Agustín Romano	3	Española	Profesor
7	informática	Iniciación a Internet	Roberto Guzmán	2	Estadounidense	Catedra

El campo Autor no se repite, aunque aparezcan en las dos tablas que se unen o concatenan.

Existen distintos lenguajes para base de datos, según las operaciones que se vayan a realizar en la base de datos.

Las diversas operaciones que se pueden realizar en una base de datos, son muchas, pero dependiendo de las operaciones que vamos a realizar utilizaremos un lenguaje u otro.

Según las operaciones que se van a realizar, estos se pueden dividir en:

Lenguajes de control de datos DCL, lenguajes de definición de datos DDL y lenguaje de manipulación de datos DML

Control de datos DCL	Definición de datos DDL	Manipulación de datos DML

– **Lenguaje de control de datos o DCL o de acceso:**

  Este lenguaje permitirá al administrador de la base de datos a controlar el acceso a los datos contenidos en la base de datos.

  En cualquier sistema es muy importante la seguridad y se hace más necesario cuando es un sistema multiusuario, con lo que será necesario establecer una acreditación y administración de usuarios, administrar privilegios y funciones, administrar contraseñas de usuarios y establecer los límites de los recurso de la base de datos.

  Las operaciones básicas son: **GRANT Y REVOKE**.

En el próximo tema nos centraremos en el control de acceso a los datos.

– **Lenguaje de manipulación de datos o DDL:**

  Este lenguaje es que se encargara de la transformación de la estructura de los objetos de la base de datos. Las DDL son las instrucciones para modificar, borrar o determinar las tablas en las que se almacenan los datos. Las operaciones básicas DDL son: **CREATE, ALTER, DROP, TRUNCATE.**

– **Los lenguajes de manipulación de datos o DML:**

  Este lenguaje lo utilizan los usuarios para formular consultas y también podrá realizar actualizaciones de la base de datos, no utilizan ningún tipo de lenguaje de programación, si no especificar la consulta de una manera ordenada.

Es un lenguaje suministrado por el sistema gestor de base de datos y que accede a que los usuarios de la misma pueda realizar consultas y manipulación de los datos de forma siempre ordenada.

Las DML son: **SELECT, INSERT.DELETE Y UPDATE.**

4.5. Descripción de correlaciones entre el modelo relacional y el modelo orientado a objetos

Los modelos que se basan en objetos se utilizan para hacer una descripción de los datos. Existen diferentes modelos distintos, pero los que son más conocidos y utilizados son:

- El modelo entidad – relación.
- Modelo orientado a objetos.



El modelo entidad - relación :

Este tipo de modelo está basado en una colección de objetos que se identifican en el mundo real. Una entidad es un objeto de la vida real.

Una entidad es un objeto de la vida real que tienen características que lo distinguirán de otros objetos.

Las entidades se describen en la base de datos mediante una serie de atributos, que serán las cualidades y características que tiene una entidad y la diferenciará de las demás.

Una relación es la unión entre entidades.

Una condición de gran importancia en el modelo E-R es la cardinalidad, que es número de entidades con las que otra entidad se puede unir a través de relaciones.

Se representan mediante un diagrama E-R, que está compuesto de:

- **Las entidades**, que se representan con un rectángulo.
- **Los atributos**, que se representan con elipses.
- **Relaciones entre entidades** se representa por un **rombo**.
- Los atributos de las entidades y las relaciones que se producen entre ellas se representa con una línea.



Modelo orientado a objetos.

Al igual que el modelo E-R, se basa en objetos. En este caso el objeto tiene un valor que se encuentra almacenado en una variable dentro de un objeto.

Un objeto contiene código que realiza operaciones en el objeto, este código recibe el nombre de métodos. Los objetos y los métodos se asocian juntos en clases.

La manera de que un objeto pueda acceder a los datos de otro, es por medio de una llamada al método del objeto. La parte que es visible del objeto se define por medio de la llamada a los métodos, las variables y el código de este n o es visible, con lo que se consigue una abstracción de los datos.

Al contrario que el método E-R, cada uno de los objetos tiene una identidad única, puede existir dos objetos que contengan los mismos valores pero ser objetos diferentes.

Diferencias:

– En el modelo E-R se definen los objetos con tablas que representan a una entidad, que contendrá las características de la identidad y un identificador único de esa entidad.

  En el modelo basado en objetos, se almacenan los objetos

  Que crearan una entidad del mundo real, con una identificación única pero que se podrán unir para trabajar con otros objetos que sean iguales o distintos.

– Otra cosa en la que se diferencian uno del otro es que en el modelo E-R utilizan clave primaria, que será como se identifica de manera única una entidad.

  En el caso del modelo basado en objetos. Se define el OID (Object identity) que le dará al objeto una identidad única.

  Este no puede ser cambiado ni introducido por un usuario, como es el caso del modelo E-R. Si desaparece un objeto, el mismo sistema elimina ese OID y no vuelve a dárselo a ningún otro objeto que sea nuevo.

– En el modelo E-R se usa un lenguaje estándar de consultas SQL, el sistema gestor determina la forma de encontrar un dato, este estándar dejara que las aplicaciones lo utilicen sin importar en que lenguaje de programación se hayan escrito.

  En el modelo basado en objetos, se utilizan diferentes sistemas para recuperar datos pero al no tener un lenguaje específico para consultas como SQL, no se controla el acceso y se pueden producir errores.

– Las bases de datos basadas en objetos dejan almacenar archivos multimedia ya que el objeto puede ser cualquier cosa, mientras que en el modelo E-R, no se permite realizar ese almacenamiento, se realiza mediante el guardado de la dirección del archivo.

4.6. Nociones sobre el almacenamiento de objetos en las bases de datos relacionales



Para llevar a cabo una base de datos, necesitaremos un sistema gestor de base de datos, también llamado SGDB, este sistema se define como una colección de datos relacionados entre sí, estructurados y organizados, y un conjunto de programas que acceden y gestionan esos datos, la Colección de datos se llama **BASE DE DATOS**.

El objetivo más importante de un sistema gestor de base de datos es proporcionar la máxima eficiencia y seguridad a la hora de manipular y almacenar los datos de las bases de datos.

Estos sistemas están diseñados para poder gestionar gran cantidad de bloques de datos, entre los servicios que presta el sistema gestor de base de datos esta la definición de estructuras para el almacenamiento, crear mecanismos para la gestión de los datos.

Una base de datos es una gran almacén de datos, este se crea una sola vez, a los datos se pueden acceder de forma simultanea varios usuarios a la vez, en la base de datos se almacenaran también la descripción de los datos, es lo que se llama metadatos en el diccionario de datos, el diccionario de datos es un componente de un SGDB, más tarde explicaremos los componentes que forma un SGDB.

Todo SGDB debe de prestar los siguientes servicios:

- Creación y definición de base de datos.
- Manipulación de los datos.
- Acceso controlado a los datos.
- Mantener integridad y consistencia
- Acceso compartido a la base de datos.
- Mecanismos de respaldo y recuperación.

- **Creación y definición de la base de datos.**

  Concretando los aspectos de la estructura, el tipo de datos, las restricciones y las relaciones que se van a producir entre ellos, esto se hace por medio de los lenguajes de definición de datos.

- **Manipulación de los datos.**

  Las acciones que realizará sobre los datos serán, consultar, insertar y modificar datos utilizando el lenguaje de definición de datos.

- **Acceso controlado a los datos.**

  Se realiza mediante mecanismo de seguridad de acceso a los usuarios.

- **Mantener la integridad y consistencia de los datos.**

  Se utilizan mecanismos para evitar que los datos de una base de datos sean utilizados para realizar cambios que no están autorizados.

- **Acceso compartido a la base de datos.**

  Se realizan controles para la interacción de usuarios concurrentes.

- **Mecanismos de respaldo y recuperación.**

  Estos mecanismos se utilizaran para poder volver a establecer la información en caso de que existan fallos en el sistema.

La arquitectura de un sistema gestor de base de datos (SGDB).se compone de:

- Usuarios.
- Nivel físico o interno.
- Nivel conceptual.
- Nivel externo o de visión.



- **Usuarios.**

  Un usuario es toda persona que tenga cualquier tipo de contacto con el sistema gestor de base de datos. Desde la persona que lo diseño, la persona que lo elaboro, la persona que lo termino hasta las personas que la usan.

  Se dividen en tres:

  - Programador de aplicaciones.
  - Administrador de base de datos.
  - Usuario final.



  · **Programador de aplicaciones.**

    Los programadores deberán de tener conocimientos del lenguaje de manipulación de datos ya que por medio de este lenguaje se interactúa con el sistema por medio de llamadas en test lenguaje, estas interacciones están incorporadas en un programa que estará escrito en cualquier lenguaje de programación por ejemplo, pascal, etc.

    Esta persona será la encargada de escribir el programa de aplicación que van a utilizar la base de datos.

- Administrador de la base de datos.

  Que será la persona o personas encargadas del control general del sistema gestor de la base de datos.

- Usuario final

  Que serán cualquier persona que accede a la base de datos desde un terminal.

  Existen dos tipos de usuarios finales:

  › Los usuarios que utilizan un lenguaje de consulta que será proporcionado como parte integral del sistema gestor de base de datos.

  › También pueden ser los usuarios finales que recurren a un programa de aplicación escrito por el usuario programador de aplicaciones, que acepta órdenes desde el terminal del usuario y a su vez formule solicitudes al sistema gestor de base de datos en nombre del usuario final.

– Nivel interno o físico.

  El nivel físico es el más cercano al almacenamiento físico, nos referimos a como se almacena en un ordenador. Determina la estructura física que va a tener la base de datos mediante un esquema interno.

Este esquema se especifica con un modelo físico y determina los detalles de cómo se van a almacenar físicamente los datos: los archivos que van a contener la información, como se van a organizar, los métodos que se van a utilizar para acceder a los registros, los tipos de registros, la longitud, los campos, etc.

– Nivel conceptual.

  Determina la estructura de toda la base de datos para determinados usuarios, mediante un esquema conceptual. Este esquema describe las entidades, los atributos, las relaciones y las restricciones.

  Representa la información que va a contar la base de datos. Este diseño ya lo vimos en el apartado anterior.

– Nivel externo o de visión.

  Es el más cercano a los usuarios, es decir, es donde se describen varios esquemas externos o vistas de usuarios.

Cada esquema describirá la parte de la base de datos que interesa a una serie de usuarios, en este nivel se representa la visión individual de un usuario o de varios usuarios.

Los componentes de un sistema gestor de base de datos son:

- Lenguajes del sistema gestor de base de datos.
- Diccionario de datos.
- Seguridad e integridad de los datos
- Administrador de la base de datos.



Lenguajes del sistema gestor de base de datos.

Los lenguajes utilizados por el sistema gestor de base de datos son:

- Lenguaje de manipulación de datos (DML)
- Lenguaje de definición de datos (DDL).

Estos lenguajes ya los hemos mencionado en el aparatado anterior.



Diccionario de datos.

El diccionario de datos es un sitio donde se va a colocar información referente a los datos que forman la base de datos.

Es como una guía en la que se describe la base de datos y los objetos que la forman.

El diccionario de datos proporciona:

- La estructura lógica y física de la base de datos.
- El espacio utilizado por los datos.
- Información sobre las restricciones.
- Las definiciones y descripciones de las tablas, vistas, índices, disparadores.
- Los valores que toman por defecto las columnas de las tablas.
- Accesos a los objetos.
- Privilegios de los usuarios.

Lo que caracteriza a un diccionario de datos es:

- Que este se encuentre integrado dentro del sistema gestor de base de datos.
- La manera de gestionar la información es eficiente la conexión entre el esquema interno y externo se realizan en tiempo de ejecución.
- Refleja la información que ha tendido cambios en la descripción de la base de datos.
- Esta almacenado en algún medio de almacenamiento al cual tenga un acceso directo para recuperar los datos fácilmente.



Seguridad e integridad de datos.

Un sistema gestor de base de datos ofrecen mecanismo para proteger los datos de accesos que no estén autorizados, mecanismos para implantar restricciones de integridad en la base de datos este tipo de restricciones evitará que se produzcan daños en los datos, proporcionara mecanismos para que se realicen copias de seguridad y restauración de la base de datos.

Utilizará mecanismo para que si la base de datos sufriera algún daño se pudiera llevar a un estado anterior en la que estaba bien.

Asegura el acceso concurrente.



Administrador de la base de datos.

Existen varias categorías para los usuarios, según los privilegios y permisos que tengan sobre los objetos de la base de datos.

- **Usuario administardor**.es la persona encargada de administrar la base de datos, estos tienen el nivel más alto de privilegios.
- Los usuarios que pueden realizar acciones de creación de sus propios objetos y tendrán permiso de acceso a los objetos si se les ha concedido los privilegios.
- Los usuarios que solamente pueden utilizar aquellos objetos para los que se les ha concedido permiso de acceso.

El administrador de la base de datos es el encargado de establecer los privilegios a los usuarios y también de crear los usuarios que se conectaran a la base de datos, siempre se intentara que sea solo una persona.

Funciones del administrador de la base de datos.

- Instalar el sistema gestor de base de datos.
- Crear la base de datos.
- Crear y mantener el esquema de la base de datos.
- Crear y mantener las cuentas de usuario.
- Arrancar, parar y cargar la base de datos.
- Establecer estándares de uso, políticas de acceso y protocolos de trabajo para los usuarios.
- Suministrar información necesaria sobre la base de datos a los equipos de análisis y programación de aplicaciones.
- Ejecutar tareas de explotación.
- Colaborar en almacenamiento ocupados en discos duros por el sistema gestor de base de datos.

Para una base de datos concreta solo existe un esquema interno y uno conceptual, pero puede existir varios esquemas externos definidos para uno o varios usuarios.

Con la entrada de la arquitectura de tres niveles se da el concepto de independencia de los datos.

Existen dos tipos de independencia de los datos:

- Independencia lógica.
- Independencia física.



- **Independencia lógica.**

  Es la capacidad de poder realizar modificaciones en el esquema conceptual sin que afecte a los esquemas externos ni a los programas de aplicación.

- **Independencia física.**

  Es la capacidad de modificar el esquema interno sin tener que realizar ninguna modificación ni en el esquema conceptual ni en los esquemas externos. Esta es más fácil de conseguir que la lógica ya que se refiere a separar las aplicaciones y las estructuras fiscas de almacenamiento.

Antes de nada vamos a explicar cuáles son los componentes de una base de datos.



Toda base de datos está constituida por uno o varios bloques de información que reciben el nombre de TABLAS, que tendrán características en común

Cada tabla está formada por REGISTROS. Un registro es la unidad básica de información de una tabla.

Cada registro estará formado por uno o varios elementos que reciben el nombre de CAMPOS. Un campo es una información que interesa guardarla en el registro.



Toda base de datos está constituida por uno o varios bloques de información que reciben el nombre de TABLAS, que tendrán características en común

Una TABLA es un conjunto de información del mismo tipo

El ejemplo más común utilizado para explicarlo es el de una biblioteca, una tabla está formada por la información relacionada con los libros y otra tabla que estará formada con información relevante de los lectores.

El sistema de gestión de bases de datos relacionales más utilizado es SQL Server, este permitirá conexiones con aplicaciones realizadas en una diversidad de lenguajes como C, C++, Visual, Java, PHP, etc.

Fue desarrollada por Microsoft y su misión va a ser de almacenar y recuperar datos que solicitan otras aplicaciones, tanto en un mismo equipo como en equipos diferentes que estén en red, incluyendo Internet, existen muchas ediciones diferentes de SQL, están desarrolladas tanto para aplicaciones sencillas como para aplicaciones más complejas como pueden ser los orientados a Internet, que necesitan soportar grandes cantidades de concurrencia de usuarios.

Una **Base de Datos Relacional**, permiten establecer relaciones entre los datos, que se guardaran en Tablas que es la forma en que se almacenan los datos y relacionar los datos de las tablas



Cada tabla está formada por REGISTROS. Un registro es la unidad básica de información de una tabla.

Por ejemplo, en la tabla de alumnos un registro correspondería a la información de cada alumno en concreto, su nombre, apellidos, dni, y dirección.

Recuerda

Cada registro estará formado por uno o varios elementos que reciben el nombre de CAMPOS. Un campo es una información que interesa guardarla en el registro.

En el ejemplo que hemos estado utilizando, un campo correspondería con el nombre del alumno, otro con el dni, otro con los apellidos y otro con la dirección.

Debido a la aparición de los programas usuarios se podrá realizar la gestión de tablas de una base de datos, sin tener que hacer programas adicionales que sean capaces de procesar esos datos, así facilitaran las operaciones de creación, actualización, consulta y creación de informes de los datos.

Desde el punto procedimental la forma de almacenar las tablas, registros y campos es distinta, dependerá de las características de la información que se quiera tratar.

Así es como se almacena los datos en la base de datos.

Ejemplo:

Campos				Registros
DNI	NOMBRE	APELLIDOS	DIRECCION	
45252235v	Ana	López Benítez	Calle álamos	
74855231c	Javier	Martínez Gómez	Calle Juan de Austria	
89564522b	Domingo	García Ferrer	Avenida de Andalucía	
77788455z	Francisco	Fernández González	Doctor Marañón	

UD4
Lo más importante

- Las estructuras de datos y el acceso proporcionados por SGDB, reciben el nombre de **modelo de datos**

- Se distinguen tres tipos de estructuras de base de datos:

  · **Jerárquica**

    Para poder acceder a los datos de un campo de un nivel determinado, para localizarlo se tendrá que partir desde el primer nivel.

  · **Bases de datos en red**

    En este tipo de estructura desaparece la jerarquía esto hace que la búsqueda de información sea flexible.

  · **Bases de datos relacionales.**

    Una **Base de Datos Relacional,** es una base de datos que utiliza el modelo de estructura relacional. Permiten establecer relaciones entre los datos, que se guardaran en tablas, y relacionar los datos de las tablas

- Se denomina concepto de **diseño de base de datos** a la tarea de tomar decisiones que son fundamentales para poder desarrollar las bases de datos relacionales.

– Etapas de diseño de una base de datos:

  · **Etapa del diseño conceptual:**

    En esta etapa conseguiremos configurar la estructura de la información que contendrá la base de datos.

  · **Etapa de diseño lógico.**

    Esta etapa comienza con el resultado de la etapa conceptual y se tomara la decisión de qué modelo de base de datos se prefiera.

  · **Etapa del diseño físico.**

    En esta etapa se convierte la estructura obtenida del lógico. Nos vamos a referir a los aspectos físicos para la implementación de las relaciones.

– Se denomina **entidad** a la representación de un objeto en el mundo real, que se puede distinguir del resto. Las entidades se representan físicamente en el modelo relacional por medio de un rectángulo .

– Un **atributo** son las características o propiedades que pueden definir a la entidad. Los atributos se representan físicamente mediante una elipse con su nombre en minúsculas unidos mediante un guion al rectángulo de la entidad a la que pertenezca

– Para que una entidad sea distinguible del resto será necesario tener un atributo o varios que la identifiquen, estos atributos recibirán el nombre de **clave** de la entidad.

– Al **atributo** que puede ser calve primaria pero que no se elige como tal, se le denomina clave secundaria o clave candidata.

– Una **relación**, se define como la unión entre entidades. Se representa físicamente por medio de un rombo. Junto al rombo se le coloca el nombre de la relación., se debe de poner con letras mayúsculas.

– Existen tres tipos de relaciones entre dos entidades:.

  · **Relación uno a uno se representara asi: (1:1)**, se representa poniendo un 1 a cada lado de la relación

  · **Relación uno a varios, se representara así (1:n)**, se representa poniendo un 1 en el lado de una entidad y n en el lado de la otra entidad relacionada

- **Relación varios a varios, se representa así: (M: n)**, se representa poniendo M y N correspondiente a cada entidad relacionada.

– Toda base de datos está constituida por uno o varios bloques de información que reciben el nombre de **TABLAS**, que tendrán características en común

– Cada tabla está formada por **REGISTROS**. Un registro es la unidad básica de información de una tabla.

– Cada registro estará formado por uno o varios elementos que reciben el nombre de **CAMPOS**. Un campo es una información que interesa guardarla en el registro.

– Las operaciones fundamentales son la **selección, la proyección y la concatenación o unión.**

- La **selección**. La selección consiste en usar unos criterios que permitirá decidir que registros de la tabla queremos que aparezcan en la tabla nueva.

- La **proyección**:

- La proyección radica en obtener una nueva tabla formada por algunos campos(columnas) que han sido seleccionadas de otra tabla ya existente.

- **La concatenación o unión:**

  La concatenación radica en obtener una tabla nueva a partir de la unión de dos tablas ya existentes.

– Tipos de lenguajes: **Lenguajes de control de datos DCL, lenguajes de definición de datos DDL y lenguaje de manipulación de datos DML**

- **Lenguaje de control de datos o DCL o de acceso.** Este lenguaje permitirá al administrador de la base de datos a controlar el acceso a los datos contenidos en la base de datos.

  Las operaciones básicas son: **GRANT Y REVOKE.**

- **Lenguaje de manipulación de datos o DDL**. Este lenguaje es que se encargara de la transformación de la estructura de los objetos de la base de datos. Las operaciones básicas DDL son: **CREATE, ALTER, DROP, TRUNCATE**.

- **Los lenguajes de manipulación de datos o DML**. Este lenguaje lo utilizan los usuarios para formular consultas y también podrá realizar actualizaciones de la base de datosLas DML son: **SELECT, INSERT.DELETE Y UPDATE.**

- El modelo entidad - relación . Este tipo de modelo está basado en una colección de objetos que se identifican en el mundo real. Una entidad es un objeto de la vida real.

- Modelo orientado a objetos. Al igual que el modelo E-R, se basa en objetos. En este caso el objeto tiene un valor que se encuentra almacenado en una variable dentro de un objeto.

– Toda base de datos está constituida por uno o varios bloques de información que reciben el nombre de TABLAS, que tendrán características en común

– Una TABLA es un conjunto de información del mismo tipo.

– Cada registro estará formado por uno o varios elementos que reciben el nombre de CAMPOS. Un campo es una información que interesa guardarla en el registro. En el ejemplo que hemos estado

– Cada tabla está formada por REGISTROS. Un registro es la unidad básica de información de una tabla,

UD4
Autoevaluación

1. ¿Qué nombre recibe las estructuras y datos proporcionados por SGBD?

    a. Jerárquica.
    b. Modelo de datos.
    c. Base de datos en red.
    d. Sistema de gestión de base de datos.

2. ¿En cuántas etapas se divide el diseño de una base de datos?

    a. En dos, etapa de diseño conceptual y etapa diseño lógica.
    b. En dos, etapa diseño conceptual y etapa diseño físico.
    c. En tres, etapa conceptual, etapa física y etapa de diseño abstracto.
    d. En tres, etapa conceptual, etapa lógica y etapa diseño físico.

3. ¿Qué es una entidad?

    a. Son las características o propiedades que pueden definir un atributo.
    b. Es la representación de un objeto en el mundo real, que se puede distinguir del resto.
    c. Es la unión entre atributos.
    d. Ninguna de las anteriores es correcta.

4. Un atributo se representa con un…

    a. Rombo.
    b. Elipse.
    c. Línea.
    d. Cuadrado.

5. Para que una entidad sea distinguible del resto será necesario tener un atributo o varios que la identifiquen, estos atributos recibirán el nombre de…

    a. Clave ajena.
    b. Clave foránea.
    c. Clave ajena o foránea.
    d. Clave primaria.

6. ¿Qué es una relación?

    a. La unión entre distintas entidades.
    b. La unión entre dos entidades.
    c. La unión entre tres entidades.
    d. La unión entre una, dos, tres hasta n-arias.

7. ¿Qué significa la cardinalidad de una relación?

    a. Es el tipo de correspondencia o cardinalidad que se establecerá entre las entidades de la relación. Se refleja el número de ocurrencias de una entidad con las ocurrencias de la otra entidad con la que existe una asociación.
    b. Es el tipo de correspondencia o cardinalidad que se establecerá entre los atributos de la relación. Se refleja el número de ocurrencias de un atributo con las ocurrencias del otro atributo con la que existe una relación.
    c. Es el tipo de correspondencia o cardinalidad que se establecerá entre las entidades de la relación. No se refleja el número de ocurrencias de una entidad con las ocurrencias de la otra entidad con la que existe una relación.
    d. Es el tipo de correspondencia o cardinalidad que no se establecerá entre las entidades de la relación. No se refleja el número de ocurrencias de una entidad con las ocurrencias de la otra entidad con la que existe una relación.

8. ¿Qué tres operaciones fundamentales permitirá la creación de nuevas tablas a partir de las ya existentes?

    a. Las operaciones fundamentales son el diseño lógico, el diseño conceptual y el diseño físico.
    b. Las operaciones fundamentales son la selección, la proyección y la concatenación o unión.
    c. Las operaciones fundamentales son la lógica, la proyección y la concatenación o unión.
    d. Las operaciones fundamentales son la selección, la lógica y la concatenación o unión.

9. Cuáles son los tipos de lenguajes utilizados según las operaciones que realizan?

    a. Lenguajes de control de datos DCL.
    b. Lenguajes de definición de datos DDL.
    c. Lenguaje de manipulación de datos DML.
    d. A, b y c son correctas.

10. SQL es un lenguaje…

    a. Es un lenguaje no declarativo.
    b. Es un lenguaje declarativo.
    c. Es un lenguaje declarativo y no procedimental.
    d. Es un lenguaje declarativo y procedimental.

Área: informática y comunicaciones

UD5

Acceso a bases de datos relacionales: capa de acceso a datos

5.1. Análisis del API de acceso a la base de datos

5.2. Nivel controlador

5.3. Interfaz de acceso a la base de datos (driver)

5.4. Análisis del nivel de aplicación

- 5.4.1. Establecimiento de la conexión con una base de datos.
- 5.4.2. Operar sobre la base de datos. Sentencias del lenguaje de acceso a base de datos. Objetos que permiten ejecutar una consulta. Objetos que permiten manipular el resultado de una consulta.
- 5.4.3. Integración de los tipos de datos propios del lenguaje de acceso de datos en el lenguaje de programación de la aplicación
- 5.4.4. Procedimientos almacenados
- 5.4.5. Transacciones distribuidas

5.1. Análisis del API de acceso a la base de datos

En la actualidad el desarrollo de aplicaciones va orientado a la utilización de procesos y elementos multimedia , con lo que ha cambiado mucho la manera de integrar una base de datos tradicional, en el que solo se utilizaban elementos de texto y numéricos.

Con la utilización de procesos multimedia se introducen muchos más elementos sonoros y audiovisuales donde se comparte con los elementos tradicionales.

Hoy en día es posible el desarrollo de bases de datos multimedia, al igual que todo en la informática, se han estandarizado los elementos multimedia, referentes a los tipos de formatos de archivos tanto de audio como de video y se han perfeccionado los métodos para comprender este tipo de datos.

Hasta no hace mucho, se utilizaban bases de datos no muy complejas, que eran desarrolladas para gestionar pequeñas empresas, se creaban con algunos programas como pueden ser:

- Access.
- FileMaker.

También solían ser productos comerciales, que se vendían a las empresas para que pudieran gestionar su propia base de datos.

En la actualidad esos programas se han visto en la necesidad de hacerlo compatible con la web, es la mejor manera de poder acceder, buscar y consultar datos en línea mediante el protocolo HTTP.

Los nuevas bases de datos para usar mediante web realizan gestiones por medio del navegador web.

Gracias a nuevas herramientas que han ido desarrollando, una persona puede tener su propia base de datos en su web.

Así es como poco a poco www, se ha convertido en una interfaz para acceder a datos que podrán ser utilizados por cualquier usuario.

Recuerda 

Los documentos web HTML, aparate de incluir su código de hipertexto, también se encuentran el embebidos una serie de scripts, otros elementos llamados APIs y controladores de conectividad de base de datos, que ayudaran al intercambio de información.

Algunos de estos elementos de conectividad son :OLE en ingles Open Data Connecting, CGI en ingles Common Gateway Interface, JDBC en ingles Java Data Base Connetivity,SQL Link,etc.

Estos, son los que hacen posible que en la web haya elementos dinámicos y que en sí, un documento de hipertexto sea dinámico.

Estos elementos deben de ser diseñados en la interfaz de aplicación para el acceso a datos y en la interfaz de objetos de acceso a los datos.

Las bases de datos se pueden crear con programas y con sistemas gestores de bases de datos con:

- Microsoft Access.
- Oracle.
- SQL Server.
- MySQL.
- Y todos tipos de programas de sistemas de gestión de base de datos.

Se realiza utilizando de intermediario un conjunto de herramientas para acceder a la base de datos como son CGI, DAO, ODBC, JDBC, etc.

Todo esto hace posible que los datos sean accesibles vía web y utilizadas por cualquier usuario en línea y da igual donde se encuentre.

Las tecnologías más utilizadas para realizar aplicaciones web son:

- CGI.
- ASP.
- .NET.
- JSP.
- PHP



CGI o interfaz de pasarela común.

Es la especificación del protocolo HTTP, que va a ser la que va a permitir al servidor web establecer comunicación con programas o scripts externos.

Es una de las soluciones que mas se han utilizado para crear interfaces web/sistema gestor de base de datos, este tipo de de programación destaca con por sencillez ya que es fácil de netender , además, es un lenguaje de programación independiente ya que los CGI se pueden construir con varios lenguajes .

También es un estándar que se usa en los servidores web y funcionan bajo una arquitectura independiente, ya que ha sido creado para trabajar en cualquier arquitectura web.

Como la aplicación CGI se encuentra funcionando de forma independiente, no pone en peligro en ningún momento al servidor.

Una desventaja que presenta, es que el servidor tiene que cargar el programa CGI y conectar y desconectar con la base de datos cada vez que se le requiere, a parte, tampoco existe un registro del estado del servidor sino que todo se debe de realizar manualmente.

Los programas CGI corren en el servidor web o servidor de aplicaciones web, estos pueden implementarse utilizando diferentes lenguajes de programación como por ejemplo, C, C++, COBOL, Perl, etc.

Los pasos que se ejecutan al iniciar por parte de un usuario la recuperación de un documento dinámico HTML por medio de CGI , es la siguiente:

– El usuario rellena una lista de campos de un formulario HTML, y pulsa algún botón de opción que servirá para enviar esta información.

  Antes de producirse él envío, el navegador web decide cual va a ser el método HTTP para hacer el envío, identifica los campos del formulario, construye y codifica el conjunto de datos.

– El navegador realiza la solicitud HTTP al servidor de aplicaciones, enviando los datos del formulario para que sea procesado por el programa especificado.

– El servidor de aplicaciones recibe la solicitud y se laza un nuevo proceso CGI que recibe la información necesaria para que se ejecute.

– El programa CGI se ejecuta y procesa la información, devolviendo el resultado al servidor de aplicaciones.

– El servidor recibe el resulta del procesamiento de los datos CGI y realiza la preparación de la respuesta HTTP que será enviada al cliente web.

– En último lugar, es el navegador web el que muestra el resultado al usuario.



ASP.

ASP es una tecnología creada por Microsoft para desarrollar sus propias aplicaciones y servidores web. Las primeras que aparecieron se basaban en el servidor ISS (internet información server) este es un lenguaje de script ASP y los objetos distribuidos COM (Componente Object Model).

El lenguaje ASP da acceso a datos basándose en los objetos ADO (ActiveX Data Objects) y ODBC.

Mediante la interfaz ODBC, ASP podrá trabajar sobre cualquier sistema gestor de base de datos, que de un controlador o driver.

Los objetos ADO ofrecen métodos para encapsular el acceso a datos para su utilización en páginas ASP.



.NET.

Esta es la última versión de aplicación desarrollada por Microsoft, esta incluye ASP+, C#, y deja de lado las versiones en los que introducía programas de Java. Se basan en estándares propietarios, además .NET incluye soporte SOAP.

JSP.

El acceso a una base de datos por medio de JSP, al igual que pasa con los servlets, se basa en la tecnología JDBC de java. Para ellos se precisa un driver o controlador que va a ser el que proporcione el acceso a la base de datos, por ejemplo de MySQL.

JSP es un lenguaje de código abierto que permite la creación de manera sencilla de aplicaciones web. La J2EE, es la tecnología más utilizada, ya que incluso hasta se puede crear un servidor de aplicación Java.

PHP

Esta tecnología ofrece interfaces propias de acceso a muy diversas fuentes de datos, como son las bases de datos, servidores de directorio, texto XML, etc.



La API, es el interfaz de programación de aplicaciones, en inglés Aplication Programming Interface, son un grupo de funciones y procedimientos que contiene una biblioteca, que podrá ser usado por otro software como capa de abstracción.

Como capa de abstracción, nos referimos a ocultar detalles de implementación de algunas funciones generalmente entre los niveles inferiores y los superiores de un software.

La función principal de una API es la de suministrar un grupo de funciones que son de uso general, por ejemplo, dibujar un icono. Con una API al programador le será más fácil su trabajo

En la informática podemos encontrar un gran número de lenguajes y estándares, pero la gran mayoría no pueden comunicarse entre sí, SQL, en inglés, Structured Query Languaje, se ha convertido en un método estándar de acceso a base de datos más utilizado.

Las APIs o interfaz de programación de aplicaciones muestran el proceso de acceso y manipulación de los datos de una base de datos.

Las interfaces de programación para el acceso de datos se pueden dividir en los siguientes niveles:

- Aplicación.
- Interfaz de objetos de acceso a datos.
- Interfaz de programación de aplicaciones, o API.
- Base de datos.



Aplicación.

Esta interfaz corresponde a la interfaz de cada uno de los programas clientes.

Interfaz de objetos de acceso a datos.

Esta interfaz se encuentra en medio de las aplicaciones y las APIs que van a ser necesarias para poder acceder a los datos de la base de datos.

Las tecnologías utilizadas en esta interfaz son:

- DAO.
- ADO.
- RDO.
- RDS.
- MIDAS.

- DAO.(Data Access Object)

  Se trata de un componente que va a ser el encargado de suministrar una interfaz entre la aplicación web y la base de datos.

- ADO.(ActiveX Data Objects)

  Se trata de un componente que va a servir para acceder a una base de datos, es utilizado por los programadores para poder acceder y modificar datos que se encuentran almacenados en un sistema gestor de base de datos.es fácil de usar para poder incluir accesos de bases de datos s páginas web.

- RDO. (Remote Data Object)

  Se trata de un componente incorporado en las versiones de Visual Basic, mediante este componente permite a las aplicaciones acceder a datos ODBC saltándose el acceso de este. Es muy similar al DAO, pero se encuentra mucho más cerca de la capa ODBC y la base de datos.se utiliza sobre todo para acceder a datos ODBC de SQL Server u Oracle.

- RDS.(Radio Data System)

  Se trata de un protocolo de comunicación para la transferencia de datos digitales.

- MIDAS. (Middle-tier Distributed Application).

  Se trata de un componente que se utiliza para desarrollar aplicaciones distribuidas de forma simplificada, hace que las aplicaciones distribuidas sea más manejable para las empresas. Centraliza las reglas de negocio y el acceso a los datos de un servidor de aplicaciones web.



Interfaz de programación de aplicaciones API.

Esta interfaz es la encargada de establecer comunicación con la base de datos, para poder acceder y manipular los datos.

La función que tiene las APIs, es la de ser la interfaz entre las aplicaciones y las bases de datos, unas veces lo hará a través del cliente y otras a través del servidor de base de datos.Para poder acceder a los datos de la base de datos y manipularlos, se debe de instalar algunas APIs que van a ser imprescindibles para realizar la conectividad de los datos externos.

Algunos APIs que forman esta interfaz son:

- OLE DB
- ODCB
- JDBC
- ISAPI
- CGI
- SQL LINKS



OLE DB.(Object Linking and Embedding for Databases)

Se trata de una tecnología que es utilizada para tener acceso información de diferentes datos de bases de datos

ODBC (Open Data Conectivity).

Este componente va a ofrecer una interfaz común para tener acceso a base de datos SQL. ODBC está basado en SQL que es un lenguaje estándar de acceso a los datos de una base de datos.

Es muy utilizado por Windows, permite la conexión desde diversos lenguajes de programación.

ODBC se clasifica en tres categorías:

- ODBC, para realizar consultas y modificaciones.
- ODBC para crear tablas en una base de datos.
- OCBD que los integra el sistema gestor de base de datos, que permitirá la utilización de ciertas herramientas.



CGI (Common Gateway Interface).

Es la más utilizada para interfaces de tipo web con sistemas gestores de bases de datos.

Es una programación muy sencilla, estos ser pueden crear en varios lenguajes diferentes.

Es considerado un estándar para uso de servidores web y puede trabajar con una arquitectura independiente, puede trabajar en cualquier tipo de arquitectura web, al trabajar independientemente, no produce ningún daño en el servidor.

La única desventaja es que es lento, por las veces que tiene que conectar y desconectar de la base de datos.



ISAPI (Internet Server Application Programming Interface)

Esta interfaz es mucho más rápida que la CGI, esta se encuentra integrada en el servidor Microsoft Internet Información, IIS.

Como en el caso de las CGI, se habilita el acceso a un usuario remoto para que ejecutar un programa, buscar e intercambiar información de una base de datos o de otros que se entren en el servidor.

Se cargaran cuando se inicia el servidor web, se almacenan en la memoria por eso se ejecutan más rápidamente. Acepta realizar procesos de solitud y respuesta HTTP, por lo que se podrán utilizar para autentificar, acceder o cerrar sesiones.



JDBC (Java Data Base Connectivity).

Este es un estándar de conectividad entre el lenguaje Java y una gran variedad de sistemas gestores de bases de datos, se pueden utilizar tanto en el lado del cliente como en el del servidor de la base de datos o del lado de la aplicación.

Cuando se encuentre trabajando del lado de cliente, tendrá que hacer uso de la tecnología ODBC para acceder a los datos.

Existen cuatro tipos de controladores JDBC:

- Puente JDBC-OBCD.
- Java parcialmente nativo.
- JDBC de Java puro.
- JDBC nativo.

Estos serán estudiados más adelante.



SQL LINKS.

Son un tipo de controladores que se encargaran de establecer la comunicación entre la aplicación web y los servidores de bases de datos, la conexión es directa por lo que tarda muy poco, permite la conexión de diverso tipos de bases de datos.

SQL

Lenguaje estándar más utilizado

Microsoft SQL Server es un sistema de administración y análisis de datos relacionales, es compatible con HTTP Y XML, tiene funciones para dar mayor rendimiento y optimizar la base de datos con lo que facilitara las tareas más habituales.

Implementa facilidad Web de comercio electrónico, contiene XML enriquecido y soporte estándar de Internet, con lo que permitirá un acceso seguro y rápido a datos a través de la Web.

SQL, es un sistema gestor de base de datos que incluye capacidades de análisis integradas, herramientas de depuración y realizara servicios de transformación de datos y duplicidades.

Podemos resumir que SQL Server es un sistema cliente servidor para la administración de base de datos relacionales que esta creado para procesar transacciones en línea.

SQL es una herramienta muy importante para cualquier programador debido a que para introducir datos en páginas HTML, deberá tener algo de conocimientos. También la mayor parte de aplicaciones de gestión que se utilizan hoy en día en el desarrollo de sitios web, tienen que acceder a alguna de las bases de datos utilizando sentencias de SQL.

Los fabricantes ofrecen APIs nativas para poder acceder a sus bases de datos, pero que se utilicen APIs nativas, hace que lo programadores tenga que adaptar sus programas a las características de un servidor determinado.



Para solucionar este problema se creó **ODBC**, que quiere decir conectividad abierta de base de datos, en ingles Open DataBase Connectivity, es un estándar de acceso a bases de datos que se creó para facilitar el acceso a los datos desde cualquier aplicación, sin importar que sistema de gestión de datos se utilice.

Lo que realiza ODBC es que se cree una capa intermedia que recibe el nombre de nivel de interfaz de Cliente SQL, que estará entre la aplicación y el sistema gestor de base de datos.

La función principal de esta capa es la de traducir las consultas de datos de la aplicación en comandos que el sistema gestor de base de datos entienda.

Este software funcionara de dos modos, con un software que se utilice en el cliente, en el cual el driver es el que analiza la conexión y las llamadas. Estas serán traducidas por el API de ODBC hacia el sistema gestor de base de datos.

El otro modo para conectarse a la base de datos es creando un DSN dentro del ODBC el que se determinada la ruta y detalles de la conexión.



Más tarde apareció **JDBC**, en ingles JAVA Database Connectivity, que es un derivado que está inspirado en **ODBC**.

La necesidad de **JDBC**, es porque **ODBC** (capa intermedia entre aplicación y base de datos) es un interfaz escrito en C o C++, que es un lenguaje no portable, y con la incorporación de compiladores hará que sea más fácil que este nivel intermedio se desarrolle en JAVA.

ODBC se debe de instalar manualmente mientras que **JDBC** no, son más seguros y más portables.

Para más información, consulta "Conexión JDBC " en los anexos al final del libro.

Gracias a **JDBC**, un programa Java podrá aceptar a cualquier base de datos sin tener que modificar la aplicación., para conseguir esto es necesario instalar un driver que cumpla JDBC.

Un driver **JDBC** creará una capa intermedia que servirá para traducir las llamadas entre la aplicación y el sistema gestor de base de datos.

La API de JDBC va a ofrecer a los programadores de JAVA el modo para conectar con bases de datos, se creara un cliente que conectara con la base de datos, y que podrá ejecutar comandos SQL y dar el resultado de estas instrucciones.

La API de JDBC aceptara acceder a diferentes servidores de bases de datos, lo único que tiene que hacer es cambiar el driver JDBC por el que ofrezca el servidor al que se quiere acceder.

5.2. Nivel controlador



Si recordamos en temas anteriores hemos visto el patrón MVC, modelo vista controlador, que si recordamos lo que hacía era intentar de forma lógica separa la interfaz gráfica del código de la aplicación para su funcionamiento.

El nivel controlador es el que ocupará de centralizar las peticiones que recibe la aplicación, el controlador se encargara de interpretar el tipo de operación que hay que realizar mediante la solicitud, esto se llevara a cabo analizando algun valor que se envía en la petición.

El proceso de la petición consiste en que cuando el controlador determine qué operación se va a realizar, va a ejecutar las acciones que sean necesarias por medio de diferentes métodos.

Dependiendo de la acción que se vaya a realizar, el nivel Modelo necesitara datos enviados por el cliente por medio de la petición, estos se los proporcionara el controlador.

También si se realiza una búsqueda, la información resultante será entregada directamente al controlador.

Dicho esto, y habiendo hecho un recordatorio, seguimos hablando de cómo se va a acceder de la aplicación a la base de datos.

La aplicación interacciona con una base de datos relacional empleando instrucciones SQL. El envío de estas instrucciones al sistema gestor de base de datos, se puede realizar de dos maneras:

- Llamada interactiva.
- Llamada a través de un programa de aplicación



Por medio de una llamada interactiva.

Se usa un programa cuya función es recibir las instrucciones SQL, transmitirlas al sistema gestor de base de datos y mostrar los resultados al usuario.

Normalmente el sistema gestor de base de datos ofrece un programa de tipo textual con el que se podrá realizar dichas funciones.

Por medio de un programa de aplicación.

En este caso las instrucciones de SQL se llaman a través de un programa de aplicación, estas instrucciones SQL se ejecutan durante la ejecución de dicha aplicación, esta utilizara los resultados para producir la visualización de los datos al usuario.

En este caso el usuario no utiliza de forma directa las instrucciones SQL y ni siquiera saber que está utilizando una aplicación que accede directamente a una base de datos, lo único que visualiza el usuario es la interfaz que ofrece la aplicación.

Existen dos sistemas para escribir aplicaciones de este tipo:

- Usar APIS que gestione la comunicación con el sistemas gestor de base.
- Usar SQL embebido.

- **Usar APIS que gestione la comunicación con el sistemas gestor de base de datos.**

  En este caso se usaran APIs de comunicación que transmita las instrucciones SQL y que además permita manipular los resultados.

  Las APIs de este tipo son JDBC y ODBC.

  Los sistemas gestores de bases de datos también ofrecen su biblioteca de la que es propiedad.

Pero si utilizamos los estándares como JDBC o ODBC, las aplicaciones funcionaran con cualquier sistema gestor de base de datos que admita la interfaz solicitada por la biblioteca.

- **Usar SQL embebido.**

  En este caso, el código SQL se encuentra embebido en el código de un lenguaje huésped. Normalmente el código resultante se convierte primero por un pre-procesador y después se compila por el compilador del lenguaje huésped.

Las tecnologías más utilizadas y de más importancia para la conectividad a una base de datos son ADO y JDBC.



ADO.

Existen varios niveles o interfaces diferentes para conseguir el acceso a una base de datos a través de una aplicación.

ADO se utiliza en dos interfaces muy importantes para el acceso a base de datos, el ADO es más fácil de usar que las APIs. ADO es la interfaz de objetos de datos para OLE DB y RDO que será la interfaz para el objeto ODBC.

ADO es muy fácil de usar, lo que hace es encapsular la API de OLE DB. Esta utiliza diversos lenguajes de programación, puede acceder a cualquier recurso OLE DB.

El modelo ADO, define una jerarquía de objetos que pueden ser programables y que servirán para acceder a información que se encuentra almacenada en una base de datos.

ADO facilitara el diseño de webs en la que se pueda acceder repetidas veces a una misma base de datos, realizan una búsqueda.



JDBC.

El acceso **JDBC** soporta los modelos de acceso de datos en dos y tres capas.

– En el **modelo de dos capas**, la conexión se realiza directamente entre la capa intermedia que será aplicación Java que se estará ejecutando en el navegador y la base de datos.



Esta es la configuración cliente servidor básica. El driver JDBC estará en el cliente, y desde ahí se tendrá que acceder al servidor que esta la base de datos que podrá estar en cualquier ordenador y acceder a ella mediante una red

La aplicación cliente envía instrucciones en SQL a la base de datos y este le envía los resultados de vuelta a la aplicación.

– **En el modelo de tres capas**, las peticiones del cliente son enviadas a una capa intermedia entre el cliente y el servidor, que se encargará de transformar y enviar esas instrucciones SQL a la base de datos y también se encargará de devolver el resultado de la base de datos.

En este tipo de modelo, el usuario no tiene ningún contacto directo con la base de datos.

El nivel intermedio lo que permitirá será, que las operaciones sobre la base de datos se controlen siempre, los drivers no tienen que alojarse en el ordenador del cliente, con lo que tendrá que instalar nada, el usuario podrá utilizar una API de alto nivel mucho más sencillo y fácil, ya que de ello se ocupara la capa intermedia, en las que se realizaran peticiones a bajo nivel y dará ventajas respecto al rendimiento.

5.3. Interfaz de acceso a la base de datos (driver)



Java provee tres componentes: un gestor de drivers JDBC, la suite de testeo de drivers JDBC y el puente JDBC-ODBC.

El gestor de drivers, es la base de JDBC, su función va a ser la de conectar las aplicaciones Java al driver JDBC correspondiente y después liberarlo.

La suite de testeo lo que va a proporcionar va a ser seguridad a los drivers que se ejecutan.

El puente JDB-ODBC servirá para dejar que los drivers ODBC puedan utilizarse como driver JDBC.

Importante

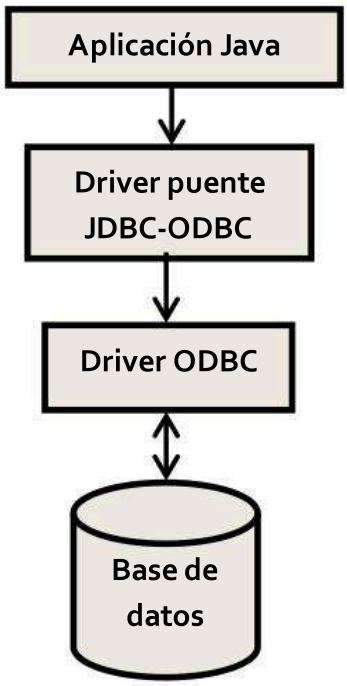
Existen varios tipos o niveles de driver JDBC:

- Puente JDBC-ODBC
- Java parcialmente Nativo (Java/Binario).
- Java nativo –NET
- Java nativo JDBC –Independiente.

- **Puente JDBC-ODEBC.**

  Este tipo de driver, establece un puente entre JDBC y ODBC, transforma operaciones JDBC en llamadas a la API ODBC, Estas llamadas son enviadas a la base de datos mediante el driver ODBC más adecuado.

  La estructura seria la siguiente:



Las ventajas de utilizar este tipo de driver, es que se puede trabajar con una gran cantidad de drivers ODCB, es una ventaja porque casi todos los programadores acceden a las bases de datos en entornos diferentes de Java por medio ODBC.

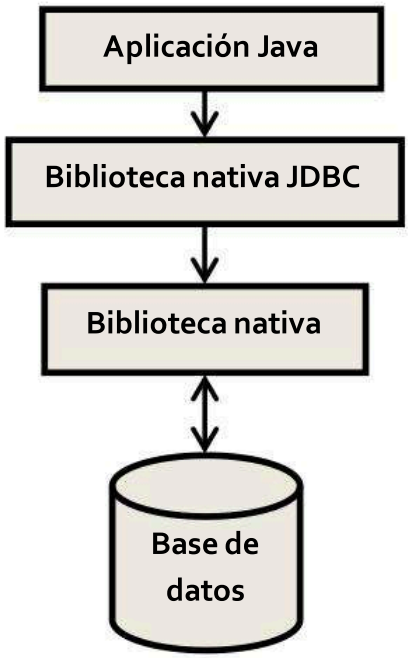
Las desventajas que tiene son, que los driver ODBC sus llamadas se convierten en llamadas a una librería que no tiene el fabricante del sistema de base de datos, con lo que produce mucha lentitud, por lo que se hace mejor utilizar un driver de JDBC y un driver de ODBC, con lo que ya no será necesaria la configuración cliente que se necesita en ODBC.

VENTAJAS	DESVENTAJAS
Gran cantidad de driver ODCB, facilita el acceso en entornos diferentes	Llamadas a llamadas, produce lentitud

- Java parcialmente Nativo (Java/Binario).

Este tipo de driver utiliza la interfaz de métodos nativos de Java para convertir solicitudes de API JDBC en llamadas a bases de datos como SQL Server, Oracle, Informix, etc.

La estructura seria la siguiente:



Estos dan mayor rendimiento que el driver puente JDBC-ODBC, este tipo de driver tiene el mismo problema que el puente, la interfaz de conectividad nativa tiene que estar instalada en el cliente, con lo que hace que este tipo de driver no se pueda utilizar para internet, a parte no muy es portable.

VENTAJAS	DESVENTAJAS
Mayor rendimiento.	No es muy portable.

- Java nativo –NET

  En este tipo de driver las solicitudes de la base de datos JDBC, estarán traducidas por un protocolo de red que es independiente de la base de datos y que serán dirigidas al servidor de capa intermedia.

  Este servidor será el que reciba las solicitudes y las enviara a la base de datos utilizando un driver tipo 1 o tipo 2.

  Consiste en tres capas:

  - Capa cliente JDBC y driver.
  - Capa intermedia.
  - Base de datos.

Para trabajar con JDBC es necesario utilizar controladores que permitan acceder a bases de datos distintas., el nivel de abstracción de JDBC es más alto que el de ODBC.

Pero por otro lado ODBC sigue siendo la API más popular para acceder a bases de datos, por eso diseñaron un puente que permitiera utilizar la API de JDBC junto con controladores ODBC.

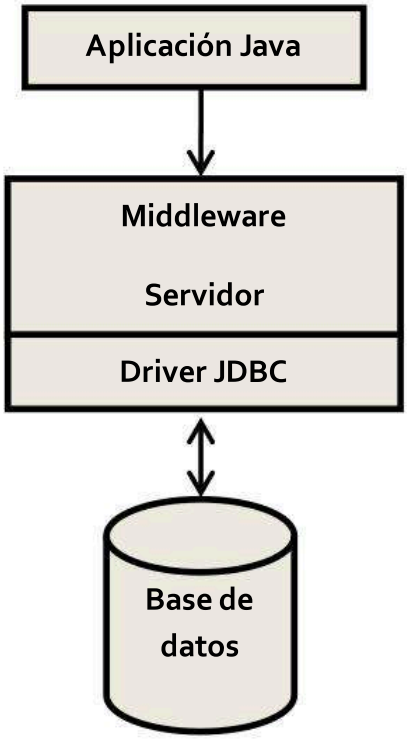
El driver se ejecuta en el cliente, tendrá la lógica necesaria para enviar comandos SQL al servidor JDBC a través de la red y recibir respuestas y manejar la conexión.

Este nivel o tipo de driver son los que mejor funcionan en redes que se basan en internet, con lo que no se tendrá que instalar bibliotecas nativas en los clientes.

VENTAJAS	DESVENTAJAS
Mejor funcionamiento.	Código especifico capa intermedia, lentitud.

La mayor parte de la aplicación web de base de datos que se basan en tres capas, dan seguridad, firewall y proxis, este tipo de driver lo ofrece.

Su estructura es la siguiente:



- Java nativo JDBC –Independiente.

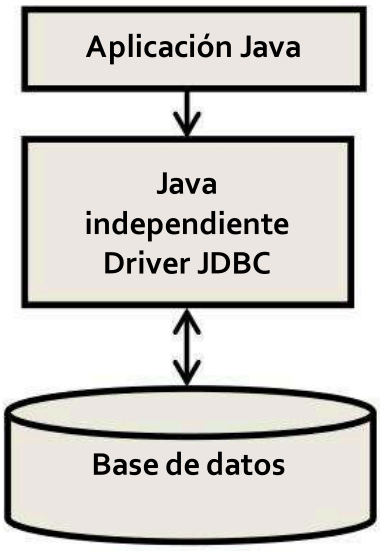
  En este tipo de driver la comunicación se realiza directamente con el servidor de base de datos utilizando el protocolo nativo del servidor.

  Se pueden escribir solo en JAVA, tienen independencia de plataforma y elimina la configuración del cliente.

  Este driver es específico de un determinado fabricante de base de datos, Si la quieres cambiar a un producto de otro fabricante, no se podrá utilizar el mismo driver. Estos driver no utilizan ODBC o APIs nativas, traducen JDBC directamente a protocolo nativo.

VENTAJAS	DESVENTAJAS
Mayor rendimiento.	No es muy portable.

No hay que utilizar ningún software especial en el cliente y no tendrá que traducir las solicitudes ODBC pero con este tipo de driver se necesitara un driver distinto para cada base de datos.



5.4. Análisis del nivel de aplicación

Importante

Una base de datos se trata de un conjuntos de informaciones que podrán estar relacionados entre si, etsas informaciones serán almacenadas de forma estructurada o agrupada dentro de la base de datos y esta a su vez será almacenada en un disco duro.

Se permitirá el acceso a la base de datos a través de aplicaciones o programas que facilitaran el manejo y manipulación de la información.

La utilización de las bases de datos es muy importante para cualquier empresa o institución, en ella se podrá almacenar grandes cantidades de información de forma rápida.

A través de un sistema gestor de base de datos se podrá acceder a gran cantidad de datos, y se podrán realizar operaciones y funciones que serán de gran ayuda para una empresa o institución.

Con la utilización de un sistema gestor de base de datos lo que se intenta conseguir es llevar a cabo un sistema que sea eficiente y eficaz, que permita el acceso a los datos los cuales siempre deberán de estar disponibles, se podrá adaptar si el contenido

va aumentando o modificando y sobre todo servirá para mantener la integridad de las informaciones o datos.

Un sistema gestor de base de datos proveerá de uan interfaz por la que los usuarios podrán introducir datos, recuperarlos o eliminarlos.

Además este proveerá que el acceso pueda ser limitada, para que no existan usuarios que puedan hacer peligrar la integridad de los datos. También proveerá seguridad y la utilización de estándares.

Un sistema gestor de base de datos se usara para obtener:

- Una interfaz única.
- Recuperar introducir, modificar, buscar o eliminar datos.
- Estándares.
- Seguridad.
- Integridad.
- Disponibilidad.
- Adaptación.
- Funciones.

Todo sistema gestor de base de datos proveerá de varios lenguajes que servirán para la creación y consulta de los datos que formaran la estructura de la base de datos. En las próximas lecciones veremos más a fondo estos lenguajes.

5.4.1. Establecimiento de la conexión con una base de datos.



Para que no tengamos problemas utilizaremos los estándares para conectar las bases de datos que serán o JDBC u ODBC.

La conexión se realizara de alguna de las dos formas o por medio de SQL embebido u hospedado en la programación de la aplicación o por medio de la interfaz ODBC.

Al realizar la conexión a la base de datos por medio de una aplicación, nos conectaremos a un sistema gestor de bases de datos, y allí por medio de este podremos realizar las operaciones de consulta y modificación de datos.



Loas elementos o componentes que forman una base de datos son:

- El Hardware, que serán todos los dispositivos de almacenamiento que se utilizaran.
- Un software, que será el sistema gestor de base de datos.
- Los datos, que están almacenados de acuerdo a la estructura externa y que serán procesados para convertirse en información.

Los tipos de usuarios de una base de datos son:

- Usuario final: es la persona que va a utilizar los datos, pero esta persona ya ve los datos convertidos en información.
- Desarrollador de aplicaciones: que será la persona encargada de desarrollar los sistemas o aplicaciones que interactúan con la base de datos.
- Administrador de la base de datos: que será la persona que asegurara la integridad, consistencia, redundancia y seguridad de los datos.

Luego tenemos a parte las personas que tienen acceso al sistema gestor de base de datos, que se clasifican de la siguiente manera:

- Usuarios ingenuos.
- Usuarios sofisticados.
- Programadores de aplicación.
- Usuarios especializados.



- **Usuarios ingenuos.**

  Estos usuarios son los que interactúan con el sistema gestor de base de datos pero por medio de aplicaciones que serán permanentes.

- **Usuarios sofisticados.**

  Estos usuarios son aquellos que pueden acceder a la información por medio de lenguajes de consulta SQL.

- **Programadores de aplicación.**

  Estas son las personas que tienen un completo dominio sobre el lenguaje de definición de datos DML, del lenguaje SQL, que serán capaces a través de este lenguaje de generar nuevos módulos u objetos capaces de manejar nuevos datos en el sistema.

- **Usuarios especializados.**

  Estos son los usuarios que no crean módulos nuevos para el manejo de datos, sino que utiliza aplicaciones avanzadas para reconocer imágenes, audio y demás.

Los sistemas gestores de base de datos son un tipo de software muy concreto, que algo que aporta es una interfaz entre la base de datos y las aplicaciones que la utilizan. Se compone de un lenguaje de definición de datos, de un lenguaje de manipulación de datos y de un lenguaje de consulta.

El motor de un sistema gestor de base de datos es el que recibe los requerimientos lógicos de entrada y salida y los convierte en operaciones de lectura y escritura.

En todo tipo de sistemas es muy necesaria la seguridad y más cuando es un sistema multiusuario. Como mínimo crear una autentificación y administración de usuarios, administrar privilegios, funciones, administrar contraseñas de usuarios y establecer recursos.

La persona que se ocupa de estas funciones es el administrador de la base de datos.

El sistema gestor de base de datos más utilizado es SQL server, existen muchas versión de SQL Server, una de las ultimas es la versión 2012.



Para acceder a la base de datos, cualquier usuario debe de hacerlo por medio de un nombre de usuario, que SQL identificará que esa la persona que está autorizada para usar esa cuenta.

Cada usuario debe de tener acceso a SQL Server mediante una cuenta de inicio de sesión con la que podrá conectarse. Esta cuenta de sesión se le asigna una cuenta de usuario SQL que servirá para controlar las actividades que se realizan sobre la base de datos, esto servirá para aprobar que permisos se le va a conceder al usuario.

Hay que dejar claro que un solo inicio de sesión se le da a cada cuenta de usuario creado en cada base de datos.

Si en la base de datos no se encuentra ninguna cuenta de usuario, el usuario no podrá acceder a la base de datos, aunque si pueda conectarse a una instancia de SQL server.

Durante la instalación de SQL, pedirá que se seleccione un modo de autentificación, hay dos modos posibles por medio de autentificación de Windows y el modo mixto

El modo autentificación de Windows se crea en Microsoft Windows y no en SQL Server.

El modo mixto se puede disponer tanto de la autentificación de Windows como la de SQL Server.

La autentificación de Windows esta disponible siempre y no se puede deshabilitar.



Así que el inicio de sesión se crea en Microsoft Windows, después se le concede a este inicio de sesión permiso para que se pueda conectar a una instancia de SQL, este acceso se concede al inicio de sesión en SQL Server.

La autentificación de Windows es el modo de autentificación predeterminado, es mucho más seguro que la autentificación de SQL.Por lo tanto, los nombres de usuarios de Microsoft Windows se pueden usar como nombres de inicio de sesión de SQL Server.

Los nombres de inicio de sesión, usuarios, funciones y contraseñas SQL pueden contener de 1 a 128 caracteres y se pueden incluir letras, símbolos y números. No incluyen el símbolo (\), no puede ser NULL o ser una cadena vacía.

Para conceder acceso de inicio de sesión en SQL Server a un usuario o a un grupo de usuarios, cuando hemos abierto la aplicación SQL, habrá que pulsar con el ratón en *grupo de servidores* y luego haz clic en un *servidor*, a continuación la opción *seguridad*, haz clic con el botón secundario del ratón en la opción *Inicios de sesión*, se abrirá una nueva ventana, en la que en *nombre* se escribe la cuenta de Microsoft Windows que permitirá el acceso a SQL Server.

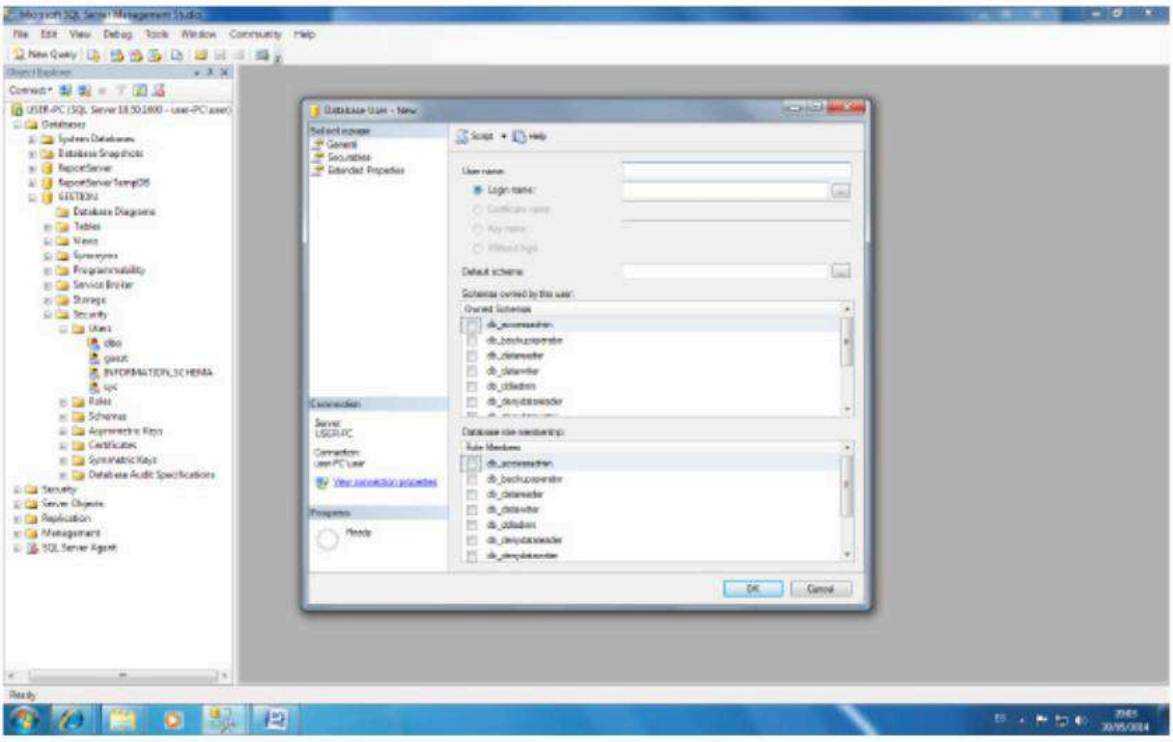
En autentificación hacer clic en *Autentificación de Windows*, en base de datos poner la predeterminada y en idioma se elige el idioma en el que se quiere que aparezca los mensajes al usuario. Así se haría desde el administrador corporativo.



La cuenta de inicio de sesión de SQL permite conectarse con un nombre de usuario de sesión y una contraseña especifica.

Los inicios de sesión de SQL permiten conseguir que sean compatibles con aplicaciones que tengan datos importados de otro proveedor de base de datos, están diseñados para aplicaciones en el que puedan trabajar usuarios.

Para poder agregar una cuenta de inicio de sesión de SQL Server usando el administrador corporativo, pincharemos para extender la opción de un *grupo de servidores* y luego seleccionamos un servidor, a continuación se expande seguridad y haciendo clic con el botón secundario del ratón en inicios de sesión, pincharemos en *Nuevo inicio de sesión* y en vez de elegir la autentificación de Windows elegiremos la autentificación SQL, será opcional poner contraseña, en base de datos se elegirá a la que se va a conectar la cuenta de inicio de sesión, en idioma se elige el idioma en el que quiere que se muestres los mensajes al usuario.



Agregar cuenta con administrador corporativo

Para conceder a una cuenta de SQL Server acceso a una base de datos con el administrador corporativo, se hace clic en *grupo de servidores* y al expandirse pincharemos en un *servidor*.

A continuación, se hace clic con el ratón en *base de datos* al expandirse debemos de expandir también la base de datos a la que se quiere obtener el acceso.

Luego, pincha con el botón derecho del ratón y haz clic en *nuevo usuario de base de datos*, se abrirá una nueva ventana, en nombre de inicio de sesión, elegir inicio de sesión de Microsoft SQL Server, el nombre de usuario es opcional. También opcionalmente se puede seleccionar miembros de función de base de datos, la que sale predeterminada es public.

Las funciones de bases de datos se encuentran en cada base de datos, no pueden incluir más de una base de datos. Un usuario puede tener activa más de una función de base de datos a la vez, las funciones de SQL pueden incluir grupos y usuarios de Windows y otras funciones de SQL, siempre que estos existan en la base de datos que se esté utilizando.

Un usuario puede pertenecer a más de una función de la base de datos, con las funciones se consigue un modelo escalable, que dará un nivel adecuado de seguridad dentro de la base de datos.

Los inicios de sesión también se pueden crear por medio del asistente para la creación de nombres de inicio de sesión que ofrece SQL Server.

Todo usuario de la base de datos debe de tener permisos adecuados según las funciones que vaya a realizar sobre esta. La administración de los permisos abarca la concesión o no de los derechos de un usuario para trabajar con datos o algunos elementos de la base de datos.

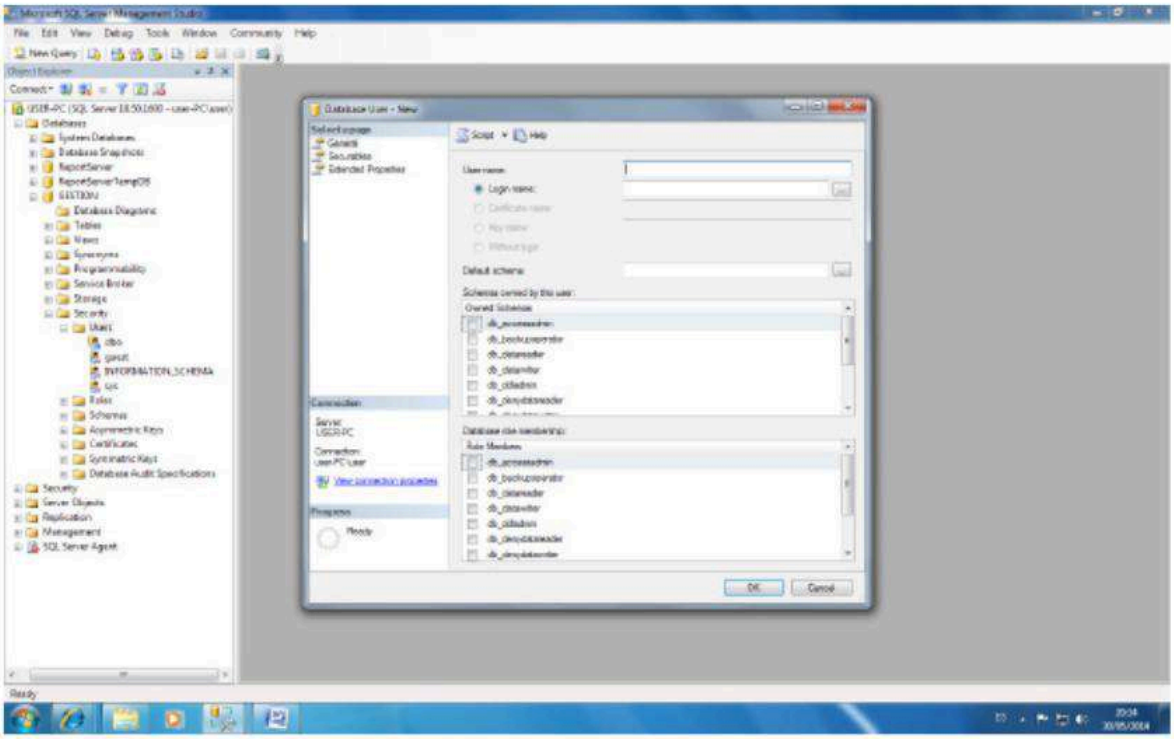
Importante

El administrador de la base de datos es el encargado de dar los permisos a los usuarios.

Un usuario debe de tener permisos adecuados para realizar cualquier actividad que implique realizar modificaciones en la definición de una base de datos o el acceso a su contenido.

La administración de permisos incluye dar o quitar derechos de usuario para trabajar con datos y ejecutar procedimientos, para crear una base de datos o algún elemento de ella, estos serán los llamados permisos de instrucción, y para utilizar los permisos para poder conceder o no las funciones predefinidas.

Todos los privilegios sobre la base de datos los tiene su propietario, pero no es el único que va a acceder a la base de datos, por eso, SQL ofrece sentencias para autorizarlas o no autorizarlas para otros usuarios.



Dar permisos por medio del administrador corporativo



La sentencia para conceder un permiso es **GRANT**.esta sentencia da permisos de seguridad permitiendo a un usuario de una base de datos trabajar con cualquier dato de esa base de datos o ejecutar instrucciones Transact SQL concretas.

Su sintaxis es la siguiente:

- GRANT ALL privilegios
- ON objeto
- TO usuarios
- [WITH GRANT OPTION]

Los privilegios pueden ser:

- **ALL PRIVILEGES**, Concede todos los privilegios sobre un objeto concreto.
- **USAGE**, utilización de un objeto determinado.
- **SELECT**, privilegio para realizar consultas.
- **INSERT, UPDATE, DELETE** [(columnas)], privilegios de inserción, modificación y borrado de todas o una columna o columnas determinada.
- **REFERENCES** [(columna)] privilegio para referenciar a un objeto en restricciones de integridad, se puede decir la columna o las columnas a las que se quiera referenciar.

Un objeto puede ser:

- Un dominio **DOMAIN**, que quiere decir el valor de una fila y columna determinadas.
- Una tabla **TABLE**.
- Una vista **VIEW**.

Usuarios pueden ser:

- **PUBLIC**, que son para todos los usuarios.
- Nombres identificador de usuario que queramos autorizar.

La opción **WITH GRANT OPTION**, lo que va a hacer es que un usuario al cual autoricemos, pueda a autorizar a otros usuarios a acceder a un objeto con los mismos privilegios con los que se les a autorizado a él.



La sentencia para revocar permisos es **REVOKE**. Esta sentencia se utiliza para revocar o denegar un permiso que se le ha dado a uno o varios usuarios.

Su sintaxis es la siguiente:

- REVOKE [GRANT OPTION FOR] privilegios
- ON objeto
- FROM [RESTRICT|CASCADE]

- Los privilegios, los objetos y los usuarios van a ser igual que en la sentencia **GRANT**.
- La opción **GRANT OPTION FOR**, es utilizada cuando se quiera eliminar el derecho a autorizar.
- La opción **RESTRCT**, permite revocar a un usuario si este ha autorizado a otros.
- La opción **CASCADE**, lo que es que cuando un usuario que queramos desautorizar haya autorizado a muchos usuarios, pues utilizando esta opción se conseguirá desautorizar a todos de una vez.

5.4.2. Operar sobre la base de datos. Sentencias del lenguaje de acceso a base de datos. Objetos que permiten ejecutar una consulta. Objetos que permiten manipular el resultado de una consulta.

ANSI propuso una arquitectura de tres niveles para los sistemas gestores de bases de datos, que lo que se quería conseguir era separar los programas de la aplicación, de la base de datos física.

En Esta arquitectura el esquema de una base de datos determina tres niveles de abstracción:

- El nivel Conceptual.
- El nivel externo o de visión.
- Nivel interno o físico



- **Nivel interno o físico**. Este nivel describe la estructura física de almacenamiento Emplea un modelo físico de datos y los únicos datos que existen se encuentran en este nivel.
- **El nivel externo o de visión**, este es el que está más cerca del usuario se especifica varios esquemas externos o vistas de usuario. Cada esquema externo explica la parte de la base de datos que interesa a un usuario o a un grupo de ellos y las que no las oculta.
- **El nivel Conceptual**, aquí se describirá toda la estructura de una base de datos mediante un esquema conceptual, en este esquema se describe las entidades, los atributos, las relaciones, operaciones de los usuarios y restricciones.

SQL Server es el sistema gestor de base de datos relacionales más utilizado, se basa en el modelo relacional, el modelo relacional fue desarrollado por Codd, propuso un modelo que se basa en la teoría matemática de las relaciones, cuyo objetivo principal va a ser el de mantener independencia de la estructura lógica de la forma de almacenamiento y otras características físicas.

Las funciones más importantes del modelo relacional son:

- **Crear independencia física de los datos**, el modo en que se van a almacenar los datos no tiene que afectar en la manipulación lógica de estos.
- **Crear independencia lógica de los datos**. Los cambios que se puedan producir de los objetos en la base de datos no deben de afectar a los programas y usuarios que acceden a la base de datos.
- **Dar flexibilidad** para aportar a los usuarios los datos de forma apropiada.
- **Homogeneidad** en la presentación de las estructuras lógica de los datos, que serán tablas, esto facilita el manejo de la base de datos por los usuarios.
- **Dar sencillez** para que al usuario le parezca sencillo la utilización del lenguaje, este modelo tienen que ser fácil de comprender y de utilizar.

Operar sobre la base de datos. Sentencias del lenguaje de acceso a base de datos. Objetos que permiten ejecutar una consulta. Objetos que permiten manipular el resultado de una consulta (Continuación).



En la lección anterior vimos cómo era un esquema E-R, ahora cuando ya vamos a utilizar el sistema gestor de base de datos debemos de transformar el modelo E-R a esquema relacional.

Esto consiste en:

- Transformar cada en entidad en una tabla.
- Transformar los atributos de una entidad en columnas dentro de una tabla
- El identificar único de una entidad se convierte en clave primaria.
- Todas las relaciones N: M se transformara en una tabla que contendrá la clave primaria de la unión de los atributos clave de las entidades asociadas.

Operar sobre la base de datos.

Las operaciones que se pueden llevar a cabo sobre una base de datos, se basan en la algebra relacional, esto consiste en la aplicación de un grupo de operadores con los cuales se podrán realizar operaciones de inserción, borrado, modificación y consulta de tablas.

– Las operaciones pueden ser:

- Unarias.
- Binarias.
- Derivadas.

– Las operaciones unarias son:

- Selección.
- Proyección.

  - **La operación de selección**, a partir de una tabla existente obtendremos otra tabla nueva de filas y columnas, por medio de una condición.

  - **La operación de proyección**, a partir de una tabla existente obtenemos una tabla nueva con las columnas de la primera tabla a la que le hemos realizado la proyección.

– Las binarias son :

- Unión.
- Diferencia.
- Producto cartesiano.

  - **La operación de unión**, lo que hace es a partir de dos tablas ya existentes, dará como resultado una tabla nueva con todas las filas de las dos tablas.

  - **La diferencia,** lo que hace es que a partir de dos tablas existentes dará como resultado una tabla nueva con las filas que están en una pero no están en la otra.

  - **Producto Cartesiano**, lo que hace es que a partir de dos tablas existentes, dará como resultado una tabla nueva con la suma de las columnas de una y

con la suma de las columnas de otra, en la tabla resultante no pueden haber columnas con el mismo nombre.

– Derivadas son:

  – Intersección.
  – Join.
  – Cociente.

  · **La operación de intersección**, lo que hace es que a partir de dos tablas ya existentes, dará como resultado una tabla nueva con las filas de las dos tablas y las columnas de una de ellas.

  · **La operación join**, a partir de dos tablas existentes, se combinan por medio de una condición para dar como resultado una tabla nueva con las filas que cumplan la condición determinada.

  · **Cociente**, se realiza entre dos tablas ya existentes que tienen diferentes columnas que cumplan las siguientes condiciones, que la primera tabla tiene que contener columnas de la segunda tabla y el número de columnas de la primera tiene que ser mayor que la de la segunda, la segunda tabla debe de tener como mínimo una fila.

Sentencias del lenguaje de acceso a base de datos.

SQL es lenguaje de consulta y administración de bases de datos, se ha convertido en un estándar para la administración de base de datos.

El lenguaje que utiliza la base de datos de SQL server se llama TRANSACT-SQL, se usa como un conjunto del lenguaje estándar definidos por ANSI e ISO, incluyen las siguientes clases de sentencias:

– **Lenguaje de consulta de datos DQL** en inglés, Data Query Langage.

  Este lenguaje se utiliza para consultar datos de bases de datos. Es la sentencia **SELECT**.

– **Lenguaje de definición de datos DDl**, en inglés, Data Definiyion Language.

  Este lenguaje se utiliza para crear, modificar o borrar objetos de la base de datos, los objetos que se pueden borrar van a ser tablas, columnas, vistas y secuencias. Las instrucciones son **CREATE**, **ALTER** y **DROP**.

- **Lenguaje de modificación de datos DML**, en ingles Data Modificación Language.

  Este lenguaje se utiliza para pregunta y manipulación de datos en tablas ya existentes.

  Los comandos son. **INSERT, UPDATE y DELETE**.

- **Lenguaje de control de transacciones TCL**, en ingles Transaccion control Language.

  Este lenguaje se utilizan para asegurar o restaurar transacciones de base de datos, las transacciones es realizar una o más sentencias SQL que están relacionadas entre si.

  Las sentencias son **COMMIT y ROLLBACK**.

- **Lenguaje de control de datos DCL**, en ingles Data Control Language.

  Este lenguaje se utiliza para controlar el acceso a la base de datos Las sentencias son **GRANT y REVOKE**.

- **Lenguaje de control de cursores CCL**, en inglés, Cursor Control Lengauge.

  Este lenguaje se utiliza para trabar sobre cada una de las filas de una tabla que dará como resultado varios registros. Las sentencias son **DECLARE CURSOR,FETCH INTO y UPDATE WHERE CURRENT.**

Lenguajes	Función	Sentencia
DQL	Consulta de datos	SELECT
DDL	Definición de datos	CREATE,ALTER y DROP
DML	Modificación de datos	INSERT,UPDATE y DELETE
TCL	Control de transacciones	COMMIT y ROLLBACK
DCL	Control de datos	GRANT y REVOKE.
CCL	Control de cursores	DECLARE CURSOR, FETCH INTO y UPDATE WHERE CURRENT.

Objetos que permiten ejecutar una consulta.

Dentro del lenguaje Transact-SQL nos encontramos el DQL que el lenguaje de consulta de datos, que permitirá obtener datos de tablas y especificar la forma en que quieren que se presente.

La instrucción o sentencia principal es **SELECT**.

Los objetos con los que se puede permitir ejecutar una consulta o manipular los datos de esos objetos puede ser: tablas, columnas, vistas, procedimientos almacenados, bases de datos e índices. Serán todos los objetos que componen una base de datos.

- Tablas.
- Columnas.
- Filas.
- Vistas.
- Procedimientos almacenados.
- Bases de datos.
- Índices.

- **Tablas.**

  Ya sabemos que las bases de datos relacionales está formado por uno o varios bloques que van a contener los datos, que son las llamadas tablas, en el diseño conceptual recibían también el nombre de relación.

- **Columnas.**

  Una tabla se define mediante un conjunto de columnas también llamados campos, estos columnas contendrá los atributos de las tablas. Cada columna representa un campo dentro de un registro.

- **Filas.**

  Las tabaleas a parte de contener columnas también contienen una serie de fialas. Cada fila representa a un registro único.

- **Vistas.**

  Las vistas son otro tipo de objetos de la base de datos relacionales, se trata de una tabla virtual cuyo contenido va a ser el resultado de una consulta. Una vista está formada de los mismos elementos que componen una tabla normal, columnas y filas de datos.

- **Procedimientos almacenados.**

  Es un objeto de la base de datos, servirá para guardar sentencias de SQL, como puede ser una instrucción de inserción, de actualización o borrado de datos.

  En vez de crear estas instrucciones una y otra vez, se crea un procedimiento que guardará una serie de instrucciones y se ejecutará cuando sea llamado.

- **Bases de datos.**

  También son objetos, porque podemos realizar transacciones en la que utilicemos datos de una o varias bases de datos diferentes.

- **Índices.**

  Es un objeto de la base de datos que se usa para encontrar uno o mas registros en una tabla de forma rápida. Un índice es la clave primaria o claves candidatas de la columna de una tabla.

De estos objetos los que permiten ejecutar una consulta son tabla, columnas y filas.





La sentencia por excelencia de consulta de datos es la sentencia **SELECT**.

La sentencia **SELECT** lo que va a hacer es obtener filas o registros de una o más tablas, también se podrá utilizar junto a otro sentencia o para realizar una subconsulta dentro de una consulta SQL.

Operar sobre la base de datos. Sentencias del lenguaje de acceso a base de datos. Objetos que permiten ejecutar una consulta. Objetos que permiten manipular el resultado de una consulta.

La forma más básica de la sentencia SELECT es:

```
SELECT  lista_de_atributos
FROM    lista_de _tablas
WHERE   condición_de_búsqueda
```

Todas las consultas deben de tener la cláusula **SELECT**, que especifica las columnas que deben de aparecer en el resultado.

La cláusula **FROM** que especifica el producto cartesiano de tablas. Se especifica la tabla o tablas a las que se le va a realizar la consulta.

La cláusula **WHERE** especifica las condiciones de la selección para las tablas indicadas en la clausula FROM.

La Parte **WHERE** es opcional, pero la parte **SELECT** y **FROM** son obligatorias.

Ejemplo:

Nos piden seleccionar los nombres el nombre del empleado con DNI=´748503399J´

Tabla CLIENTES

dni_cliente	nombre	apellidos
5689364v	Rafael	Gómez Muñoz
27800699H	Alberto	González Contreras
74850399J	Irene	Marín Benítez



Las operaciones de SQL como en este caso SELECT, reciben el nombre de **sentencias**, estas sentencias están formadas por **cláusulas** que en este caso son FROM y WHERE.

Este tipo de SQL se le llama SQL interactivo.

En la actualidad muchos son los casos en los que se quiere acceder a una base de datos desde una aplicación que se ha desarrollado en cualquier lenguaje de programación, este tipo de sistema ofrece mucha más potencia fuera del entorno de las bases de datos.

Para ello se necesitara sentencias especiales que permiten distinguir las instrucciones SQL de las del lenguaje de programación., que se realizara por medio de un precompilador.

A este tipo de SQL que se encuentra embebido en una aplicación se le llama SQL huésped.

Se puede decir que aunque SQL es un lenguaje estándar para base de datos relacionales y bases de datos relacionales comerciales, no ha sido capaz de completar el relejo de toda la teoría de Codd.

La sintaxis de la sentencia SELECT completa es la siguiente

```
SELECT [DISTINCT | ALL ] lista selección
INTO [nueva_tabla]
FROM tabla_origen
[WHERE condición_busqueda]
GROUP BY expresión_de_agrupamiento]
[HAVING condición_de_busqueda]
[ORDER BY expresión_order_by [ASC |DESC]
[UNION | UNION ALL | MINUS* (subconsulta)]
[cláusula order_by]
```

La sentencia SELECT es una sentencia compleja e iremos explorando las cláusulas de la sentencia.



La sentencia SELECT determina las columnas que va a devolver la consulta.

Las sentencias y las clausulas se pondrán en mayúsculas, los corchetes quiere decir que lo que hay entre ellos es opcional, se podría utilizar o no.



ALL .Significa que se van a devolver todas las filas.

DISTINC significa que las filas que sean duplicadas solo se devolverán una vez.

Top n, indica que solo se van a utilizar n filas del conjunto de resultado de la consulta, si se especifica PERCENT, significa que va a ser un numero entero entre 0 y 100.

WITH TIEA, solo se utilizará si existe una cláusula **ORDER BY**. Dara como resultado las filas del conjunto de resultados de la consulta con el mismo valor en las columnas ORDER BY.

Después, se coloca la lista de selección que incluirán las columnas que queremos seleccionar para realizar la consulta. Se separan por comas.



El * significa que se va a devolver todas las columnas de todas las tablas y vistas de la cláusula **FROM**.

El **nombre_columna** se refiere al nombre de la columna que se va a devolver. Primero se sitúan los nombres de las tablas y luego los nombres de las columnas.

Alias_columna es el nombre alternativo que se le da para sustituir al nombre de la columna en los resultados de la consulta. Esta cláusula se utiliza en las clausulas ORDER BY, y no se puede utilizar ni en la cláusula WHERE, GROUP BY o HAVING.



INTO, esta cláusula crea una nueva tabla con las filas resultantes de la consulta.

Sintaxis:

```
[INTO nueva_tabla]
```

El argumento nueva_tabla, determina el nombre de la nueva tabla que se va a crear por medio de la consulta, que está compuesta de las columnas y filas de la lista de selección de la cláusula **WHERE**.

El usuario que efectúa una instrucción **SELECT** con la cláusula **INTO** debe tener permiso **CREATE TABLE** en la base de datos.

FROM, esta cláusula determina las tablas de donde se van a obtener las filas resultantes de la consulta.

Su sintaxis es la siguiente:

```
[FROM { < tabla_origen >} [,...n]]
```

Se pondría FROM y el nombre de la tabla que queremos consultar.

WHERE, esta cláusula determina una condición de búsqueda para restringir las filas que se va a devolver.

Su sintaxis es la siguiente:

```
[WHERE <condición_busqueda> | < nombre_columna >]
```

El argumento condición de búsqueda restringe o limita las filas que se van a devolver en la consulta, no existe un límite de condiciones que se pueden incluir en una búsqueda. Admite los operadores AND y OR. AND Significan “y” y OR o.



GROUP BY, determina como se van a agrupar el conjunto de datos resultante. Se utiliza en funciones agregadas en la cláusula **SELECT** <lista_de_seleccion>, agrupa los datos resultantes de cada grupo de la lista de selección.

Su sintaxis es la siguiente:

```
[GROUP BY [ALL] expresión_group_by [,…n]]
```

El argumento ALL, significa que incluirá todos los campos y conjuntos de resultados, incluso aquellas filas que no cumplan la condición de búsqueda de la cláusula WHERE., cuando existe una columna que no cumpla la condición de búsqueda también se devolverá con el valor NULL.

El argumento expresión_group_by determina como se va a realizar el agrupamiento de columnas. **HAVING**, determina una condición para la búsqueda de un grupo, normalmente se utiliza con la cláusula **GROPU BY**, cuando no se utiliza **GROUP BY, HAVING** actuara como la cláusula **WHERE**.

Su sintaxis es la siguiente:

```
[HAVING <condición_busqueda >]
```

El argumento condición_busqueda determinara las condiciones de búsqueda de los datos que se debe de cumplir.

Cuando se utiliza **HAVING** con **GROUP BY ALL**, la cláusula **HAVING** reemplaza a **ALL**. Con la cláusula **HAVING** no se puede utilizar los tipos de datos text, image y ntest.

ORDER BY, esta cláusula determina el orden del conjunto de datos resultante. La cláusula ORDER BY no se puede utilizar con vistas, funciones, tablas originadas ni subconsultas, salvo que se le ponga TOP.

Su sintaxis es la siguiente.

```
[ORDER BY {expresión_order_by [ASC | DESC ]} [,…n]]
```

El argumento expresion_order_by determina la columna o columnas por la que se quiere realizar la ordenación. , pueden incluirse elementos que no aparezcan en la lista de selección. Con la cláusula ORDER BY no se puede utilizar los tipos de datos text, image y ntext.

El argumento ASC significa que los valores de la columna especificada para ordenar, se haga de manera ascendente, desde el valor más bajo al más alto, mientras que el argumento DESC significa que los valores de la columna especificada para ordenar, se haga de manera descendente, desde el valor más alto al más bajo.

Los valores NULL serán los más bajos, no tiene límite para el número de elementos que se pueden ordenar en una cláusula ORDER BY.

Recuerda 

UNION, este operador lo que hace es mezclar los resultados de dos o más consultas en un solo conjunto de resultados, que contienen filas que pertenecen a las consultas que se relacionan en una unión.

Para poder realizar la combinación de los resultados de dos consultas con UNION, el número y el orden de las columnas debe de ser igual en todas las consultas y los tipos de datos deben de ser semejantes.

La sintaxis es la siguiente:

```
{ <consulta > | ( < expresión_consulta >) }

        UNION [ALL]

        < consulta | ( < expresión_consulta >)

                [UNION [ALL] consulta | ( < expresión_consulta >) [ …n]]
```

El argumento <consulta> | (< exopresion_consulta > , especifica y devuelve los datos de la primera y de la segunda consulta respectivamente. No es necesario que las columnas que van a formar parte de la **UNION** sean iguales, pero deben ser semejantes para que se entiendan una a la otra.

El argumento **UNION** determina que se van a mezclar varios conjuntos de resultados para ser devuelto como un solo conjunto de resultados.

Cuando se pone **ALL**, se devuelven todas las filas de la consulta incluyendo los duplicados, si no se pone desaparecerán las filas duplicadas.



El orden de la clausulas en la sentencia SELECT es muy importante, cualquiera de las cláusulas opcionales se pueden suprimir, pero si no se hacen deben de llevar un orden.

5.4.3. Integración de los tipos de datos propios del lenguaje de acceso de datos en el lenguaje de programación de la aplicación

Hoy en día se da mucho el acceder a la base de datos por medio de una aplicación hecha en un lenguaje de programación cualquiera.



Para utilizar SQL desde cualquier lenguaje de programación, se utiliza el SQL hospedado o embebido, para trabajar con este SQL hospedado será necesario utilizar un pre-compilador para separar las sentencias del lenguaje de programación, de las del el lenguaje de base de datos SQL.

Para poder realizar este tipo de aplicaciones se utilizan las llamadas rutinas SQL/CLI.

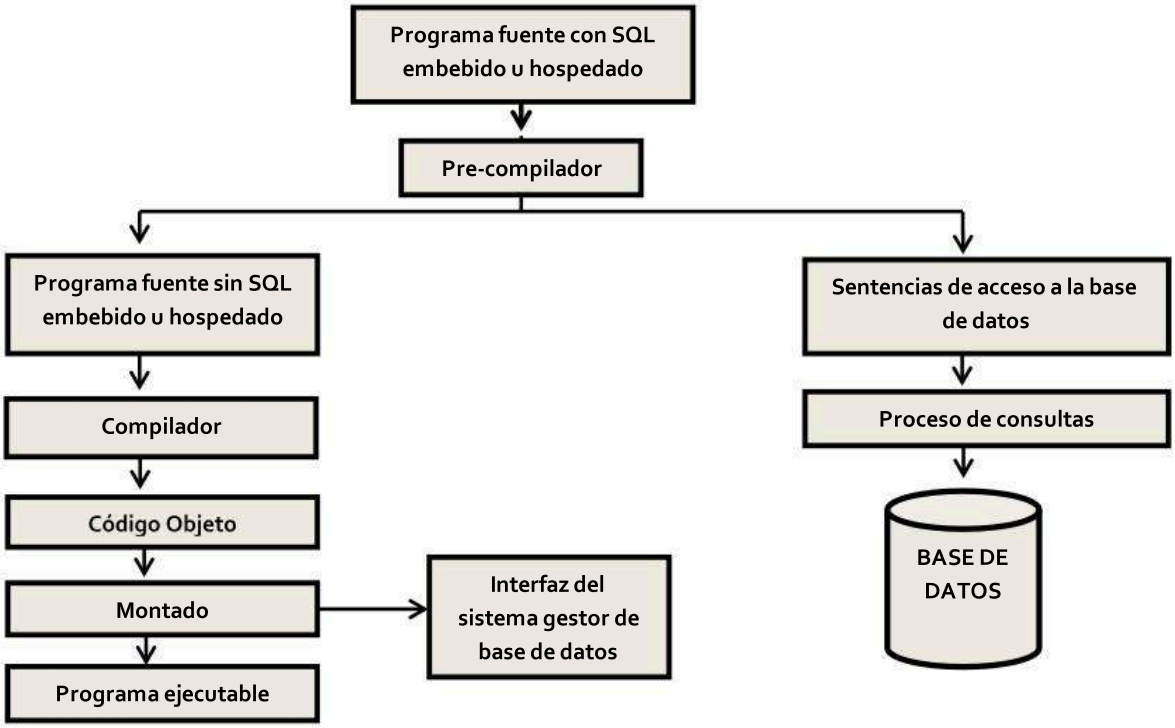
Para crear una base de datos relacional vamos a tener que utilizar SQL y además si queremos utilizar una aplicación con lenguajes de programación en Java, Cobol,c,Pascal,etc. , se puede utilizar el lenguaje SQL embebido en el lenguaje de programación que se vaya a utilizar.

De este modo se pueden utilizar las sentencias del lenguaje SQL dentro de nuestra aplicación.

Se necesitara de un pre-compilador, que es una herramienta que va a separar las sentencias SQL de las sentencias de programación.

Cuando en el programa fuente se localiza una llamada con una sentencia de acceso a la base de datos, se debe de insertar una llamada a la interfaz del sistema gestor de base de datos.

El programa fuente resultante de la pre-compilación ya se encontrar únicamente en el lenguaje de programación, estará preparado para ser compilado, montado y ejecutado.



Importante

Las SQL/CLI, van a dejar que aplicaciones desarrolladas en un cierto lenguaje de programación, puedan incluir sentencias SQL mediante llamadas a librerías.

Estas sentencias se interpretan en tiempo de ejecución del programa, la diferencia con el SQL embebido u hospedado, es que este no necesitara un pre-compilador.

La interfaz ODCB define una librería o API de funciones que permitirá a las aplicaciones acceder al sistema gestor de base de datos utilizando SQL.

Las rutinas SQL/CLI están basadas en las características de la interfaz ODBC. Estas son simplemente rutinas que llaman al sistema gestor de base de datos para que interprete las sentencias SQL que la aplicación pide.

Para que no tengamos problemas utilizaremos los estándares para conectar las bases de datos que serán o JDBC u ODBC..

La conexión se realizara de alguna de las dos formas o por medio de SQL embebido u hospedado en la programación de la aplicación o por medio de la interfaz ODBC.

El lenguaje SQL se compone de varios elementos que serán utilizados en la mayor parte de las sentencias.



Identificadores:

Son los nombres de los objetos, cualquier elemento de SQL puede servir de identificador como son las tablas, las columnas, índices, procedimientos, las vistas, bases de datos e índices.

El identificador de un objeto en la base de datos es su nombre, las restricciones son una exclusión ya que no tiene por qué tener identificadores. El identificador de un objeto se crea cuando se crea el objeto, a partir de ahí, el identificador se utiliza para hacer referencia al objeto.

Existen dos tipos de identificadores:

- Identificadores delimitados.
- Identificadores normales.

– **Identificadores normale**s, sigue las reglas de formato de los identificadores, este tipo de identificador no tiene que estar delimitados por [] cuando se usan instrucciones Transact-SQL.

  Ejemplo: CREATE TABLE DEPARTAMENTO. Esta instrucción es para crear una tabla que la hemos denominado DEPARTAMENTO. Es una instrucción Transact en el que hemos utilizado el identificador normal, sin tener que estar delimitado.

– **Identificadores delimitados**, que se incluyen entre corchetes [].

  Ejemplo: SELECT * FROM [DEPARTAMENTO]

  Es una instrucción de consulta, lo que quiere decir con esta instrucción es que queremos visualizar todos los atributos de la tabla DEPARTAMENTO. El * se utiliza para que nos enseñe todos sus campos.

  En esta ocasión hemos utilizado un identificador delimitado, con [Departamento], que lo que hace es delimitar la consulta a solo DEPARTAMENTO.

Los identificadores normales y delimitados deberán abarcar desde 1 a 128 caracteres. En los identificadores normales el primer carácter debe de ser una letra de la "a" a la "Z", puede ser un signo "_","@" o numérico.

El identificador no puede ser una palabra reservada de Transact SQL.

El nombre completo de un identificador se divide en de cuatro identificadores: el nombre del servidor, el nombre de la base de datos, el nombre del propietario y el nombre del objeto.

Su estructura o formato seria el siguiente:

servidor. [nombre_base_de_datos].[nombre_propietario].nombre del objeto

Servidor indicara el servidor local, el nombre de la base de datos, indica el nombre de la base de datos que se esté utilizando y nombre de propietario que corresponderá al nombre de usuario de la base de datos que ha iniciado la sesión.

Los nombres de las cuatro partes de un identificador se suelen utilizar en consultas distribuidas o las llamadas a procedimientos remotos.



Tipos de datos:

Todos los objetos que contienen datos tienen asociado un tipo de datos en Transact-SQL, el tipo de dato lo que va a hacer es precisar las clase de datos del objeto, que pueden ser, binario, carácter, entero, decimal, etc.

Los objetos de una base de datos que tienen tipos de datos son las columnas de las tablas y vistas, los parámetros de procedimientos almacenados, las funciones de Transact-SQL y las variables.

SQL proporciona un conjunto de tipos de datos del sistema que define todos los tipos de datos que pueden utilizarse con SQL .

Cuando se establece un tipo de datos a un objeto se fijan tres atributos del objeto:

– **Tipo de datos que contiene el objeto:** que puede ser carácter, entero o binario.

- **La longitud** que ocupa el valor que se va a almacenar, que se determina en Bytes, es número de bytes que serán necesarios para almacenar el valor del objeto.

- **Escala** que será el número para tipo de datos numéricos.

Los tipos de datos Transact SQL son:

<table>
<tr><th>Tipos de datos</th><th>Numerico exactos</th></tr>
<tr><td rowspan="4">Numericos</td><td>
– Entero:
· Bigint: números enteros entre -2 y 2 -1.
· Int: números enteros entre -2 y 2 -1.
· Smallint: números enteros entre 215 y 215 -1.
· Tinyint: números enteros entre 0 y 255.
– Bit:
· Bit: datos con valor o 1, corresponde a false y a true.
– Decimal y numeric:
· Decimal: datos de escala numérica y precisión comprendidos entre -1038 +1 y 1038 -1.
· Numeric: equivale a decimal.
– Money y smallmoney
· Money. Valores de moneda entre -263 y 263 -1.
· Smallmoney: valores de monedas entre -214.748 y +214.748.
</td></tr>
<tr><td>Numerico aproximados</td></tr>
<tr><td>
– Numéricos:
· Float: números con precisión de coma flotante entre -1,79E +308 y 1,79E + 308.
· Real:números con precisión de coma flotante entre -3,40E +38 y 3,40E+38
– Datetime y smalldatetime:
· Datetime: datos de fecha y hora entre 1 de enero de 1753 y 31 de diciembre de 9999.
· Smalldatatime: datos de fecha y hora entre el 1 de enero de 1900 y el 6 de junio de 2079.
</td></tr>
</table>

Tipos de datos	Cadena de caracteres
Cadena de binarias y de caracteres	– **Char, varchar y text:** · **Char**: datos de caracteres no UNICODE de longitud fija con una longitud máxima de hasta 8.000 caracteres. · **Varchar**: datos no UNICODE de longitud variable con una máximo de 8.000 caracteres. · **Text**: datos no UNICODE de longitud variable con longitud máxima de 231 -1 caracteres. – **Cadenas de caracteres Unicode:** · **Nchar**. Datos Unicode de longitud variable hasta 4.000 caracteres. · **Nvarchar**: datos Unicode de longitud variable máximo 4.000 caracteres · **Ntext**: datos Unicode de longitud variable máximo de 230 -1 caracteres. · **Cadenas binarias:** · **Binary**: datos binarios de longitud fija máxima de 8.000 bytes. · **Varbinary**: datos binarios de longitud variable con un máximo de 8.000 bytes. · **Image**: datos binarios de longitud variable como máximo de 231 -1 bytes
Otros tipos de datos	– **Cursor**: referencia a cursor. – **Sql_variant:** Es un tipo de dato que almacena valores de varios tipos. – **Table**: es un tipo de dato especial que se emplea para almacenar resultados de un proceso anterior. – **Tiemstamp**: es un número único para una clase de datos que se actualiza cada vez que actualiza una fila. – **Uniqueidentifier**: es un identificador global.

El lenguaje Transact SQL utiliza formatos para fechas y números. Los formatos que utiliza SQL para los diferentes tipos de datos son:

Tipos de datos	Descripcion
Datatime y smalldatetime	Los datos de fecha y hora se ponen entre comillas simples (´).tipos de formato: – Formato alfabético, por ejemplo: ´17 de diciembre de 2001´. – Formato numérico, por ejemplo: ´17/12/2001´. – Formato de cadena sin separación, por ejemplo: ´17122001´. – Formato de hora, se pone entre comillas, por ejemplo. ´12:30:20:1´.
Char y varchar	El formato se representa como char(n) y varchar(n), n corresponde a la longitud de la cadena. Los datos char y varchar almacenan datos que están formados por caracteres en minúscula o mayúsculas, numéricos y caracteres especiales. Ejemplo: char (10) o varchar (10).
Decimal o numeric	El formato es el siguiente, decimal (p,s) o numeric(p,s). P corresponde al número de dígitos que puede contener un objeto. Y si corresponde al número de dígitos que pueden ponerse después de la coma decimal.
Float y real	No se utilizan para asignar una valor numérico exacto, son tipos de datos numéricos aproximados, pero no almacena el valor exacto.
Money u samll-money	Almacena datos de moneda, no es necesario poner el valor entre comillas simples, pero sí que tiene que ir delante el símbolo de la moneda. Tienen una limitación de cuatros espacios decimales si se necesitaran más hay que utilizar el tipo de dato decimal. Por ejemplo €589,45



Constantes

Una constante también llamada literal, representa el valor específico de un dato. El formato de las constantes variara según el tipo de valor que representa. Clasificación de constantes:

- **Constante de Cadena de caracteres:**

  Las constantes de cadena de caracteres van entre comillas (´ ´) e incluyen caracteres alfanuméricos tanto mayúsculas o minúsculas y números de 0 a 9 y caracteres especiales como (¡), @ y #. Una cadena vacía se representa como dos comillas simples con una separación entre ellas,(´ ´).

  Por ejemplo:

  ```
  ´Adolfo´´García´
  ```

- **Constante Cadena Unicode:** el formato es semejante al de las cadenas de caracteres, pero delante se coloca una N que tiene que ser mayúscula.

  Por ejemplo:

  ```
  N´Adolfo´
  ```

- **Constante de Cadenas binarias:** tienen el prefijo Ox y son cadenas de números hexadecimales. No se ponen entre comillas.

  Por ejemplo:

  ```
  0x12ef
  ```

- **Constante bit:** se representan con los números 0 o 1, no se ponen entre comillas.

- **Constante datetime:** se representan por valores de fecha en formato cadenas de caracteres específicos. Se incluyen entre comillas.

  Ejemplo:

  ```
  De fecha ´ 12/17/2001´, de hora ´05:25 PM´
  ```

- **Contantes integer:** se representan como una cadena de numérica que no se incluye entre comillas y no contiene decimales.

  Por ejemplo:

  ```
  5478
  ```

- **Constantes decimal:** se representan por una cadena numérica no se incuyen entre comillas y tienen una coma decimal.

  Por ejemplo:

  ```
  5487.1478
  ```

- **Constantes float y real:** se representan con signos científicos.

  Por ejemplo:

  ```
  108.9E5, 0.9E -2
  ```

- **Constante money:** se representan con una cadena numérica con coma decimal y el símbolo de la moneda, no se ponen entre comillas simples.

  Por ejemplo:

  ```
  € 25
  ```



Operadores.

Cuando queremos realizar una consulta de una tabla de una base de datos, puede ser necesario utilizar un operador para conseguir una información determinada.

Los llamados operadores lo que nos va a dejar realizar son operaciones de comparación, concatenación y dar valores. Para poder cambiar los datos por medio de operadores se necesitara de permiso.

Los operadores son utilizados en SQL para cambiar datos ya sea de manera temporal o indefinida, se puede utilizar también para realizar una búsqueda de filas o columnas que van a cumplir una condición concreta.

Clasificación de operadores:

Operadores de comparación: Comparan un valor con otro, el resultado del operador será TRUE o FALSE.

Los operadores de comparación son:

=	significa igual a.
>	Significa mayor que.
<	Significa menor que.
>=	significa mayor que o igual que.
<=	Significa menor que igual que.
<>	Significa distinto a.
¡=	Significa no es igual a
¡<	Significa no es menor que.
¡>	Significa no es mayor que.

Importante

Operadores lógicos: prueban si una condición es cierta o no. Devuelven el tipo de datos booleano True o False.



Los operadores lógicos son:

Predicados	Descripción
BETWEEN	Se utilizara para expresar una condición en la que se quiere encontrar un valor que este entre unos límites concretos.
IN / NOT IN.	Se utilizara para comprobar si un valor coincide con los elementos de una lista, para ver si no coincide se utilizara NOT IN.
LIKE	Se utilizara para comprobar si una columna de tipo carácter cumple alguna de condición determinada.
IS NULL / IS NOT NULL	Se utilizara para comprobar que un valor es nulo y para averiguar si no los es IS NOT NULL
ANY/SOME y ALL	Se utilizara para comprobar si una columna cumple todas sus filas que sería con ALL, o algunas de sus filas que sería con ANY/SOME
EXIST	Se utilizara para comprobar si una subconsulta produce fila de resultados, si queremos comprobar que una subconsulta no produce ninguna fila de resultados utilizaríamos NOT EXISTS.



Operadores aritméticos: realizaran operaciones aritméticas.

Son:

+	seria la suma.
–	sería la resta.
*	seria la multiplicación.
/	seria la división
%	que devolverá el resto entero de una división.



Operadores unarios: se realiza en un único operando. Son:

+	El valor numérico es positivo.
–	el valor numérico es negativo.

Operadores bit a bit: transforma temporalmente valores numéricos normales en enteros y realizan operaciones aritméticas bit a bit. (0 y 1).

&	Operador bit a bit AND con dos operandos.
|	Operador bit a bit OR con dos operandos.

Operadores de concatenación: es la combinación de dos cadenas en una.

El operador de concatenación permite concatenar dos cadenas con el signo +.

Operadores de asignación: asina un valor a una variable. El operador es.

=	que lo que hace es asignar un valor a una variable.

Funciones de agregación.

SQL nos ofrece las siguientes funciones de agregación para efectuará varias operaciones sobre los datos de una base de datos, se suelen utilizar más en subconsultas:

FUNCIÓN	DESCRIPCIÓN
COUNT	Dara como resultado el número total de filas seleccionadas.
SUM	Sumará los valores de una columna
MIN	Nos dará como resultado el valor mínimo de una columna
MAX	Nos dará como resultado un valor máximo de una columna
AVG	Nos dará como resultado el cálculo del valor medio de una columna.

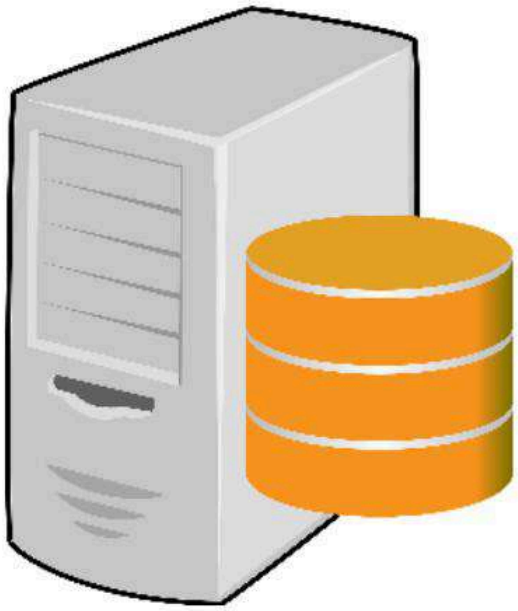
5.4.4. Procedimientos almacenados



Un procedimiento almacenado está formado por una serie de instrucciones SQL, que servirán para definir un determinado proceso, este podrá ser ejecutado directamente en el servidor de Base de datos, permitiendo la ejecución de una o varias acciones.

Acepta parámetros de entrada y devuelve un valor o conjunto de resultados. Pueden tanto recibir como devolver información, a este es a lo que nos referimos con que un procedimiento almacenado emplea parámetros de entrada y salida.

Un procedimiento almacenado es un conjunto de sentencias SQL y de instrucciones que se encuentran guardados en la base de datos, la ventaja de utilizar procedimientos almacenados es que se ejecutan directamente en el motor de la base de datos, que se encuentra habitualmente en otro servidor aparte.



Un procedimiento almacenado puede contener parámetros de entrada como de salida, estos serán pasados por referencia, así también se podrá devolver el valor de retorno.

Al ser objetos de una base de datos se hayan sometidos a los esquemas de seguridad que determinara el administrador del sistema.

Existen varios tipos de procedimientos almacenados

- Procedimientos almacenados del sistema.
- Procedimientos almacenados definidos por el usuario.
- Procedimientos almacenados extendidos.
- Procedimientos almacenados remotos.



Procedimientos almacenados en el sistema.

Son procedimientos almacenados en la base de datos Master y se identificaran con el prefijo sp-.

Realizan muchas tareas diversas para sustentar las funciones SQL, soportara llamadas de aplicaciones externas, procedimientos para la administración de la base de datos y funciones de administración de seguridad.

Estos procedimientos almacenados proporcionan un método muy efectivo de recuperar datos de tablas del sistema.

Los procedimientos almacenados del sistema pueden ejecutarse en cualquier base de datos.



Procedimientos almacenados definidos por el usuario.

Se dividen en dos:

- Procedimientos almacenados locales.
- Procedimientos almacenados temporales.



- **Procedimientos almacenados locales.**

  Serán los procedimientos almacenados que se crean por usuarios individuales en una base de datos.

  Estos están diseñados para completar tareas en la base de datos donde reside, se utilizan también para crear un procedimiento con código de procedimientos almacenados en el sistema, para hacer esto, se copia el código del procedimiento almacenado del sistema en un procedimiento almacenado como un procedimiento local.

- **Procedimientos almacenados temporales.**

  Son muy parecidos al local, pero solo existirá hasta que se cierre la conexión que lo creo.

  Puede ser local o global o local. Cuando es global se pone ## y el nombre del procedimiento y si es local # y el nombre del procedimiento.

Procedimientos almacenados extendidos.

Este tipo de procedimiento usa un programa externo, compilado como una DLL, que expande las capacidades de un procedimiento almacenado.

Casi siempre se identifican con el prefijo XP-.

Se ejecutan de forma parecida a los procedimientos almacenados normales.

Procedimientos almacenados remotos.

Este tipo de procedimientos son una característica anterior a SQL. Las consultas distribuidas admiten esta función.

Para el usuario de una base de datos, la velocidad y la eficiencia son muy importantes. Los procedimientos almacenados son una gran herramienta para obtener datos de forma óptima.

Un procedimiento almacenado es una consulta que se almacena en la base de datos, en lugar de almacenarse en el código de programación de la aplicación.

Los procedimientos almacenados, son un objeto más dentro de una Base de datos, son reutilizables, lo que queremos decir con esto es que, un cliente puede llamar a un procedimiento para que se ejecute en el servidor tantas veces como sea necesario.

Los procedimientos almacenados permitirán que se aproveche todos los recursos del servidor, se ahorra tiempo creando consultas complejas y permitiendo relacionarse con otros objetos de la base de datos.

Se ejecutan de forma instantánea por lo que el uso de procedimientos almacenados mejorara bastante la potencia y eficacia de SQL.

Los procedimientos almacenados son muy efectivos y útiles en todo tipo de arquitectura cliente/servidor, donde existe un servidor muy potente y capacitado para ejecutar procesos, consultas y actualizaciones complejas en las base de datos.



Los procedimientos se utilizan también para obtener por medio de instrucciones condicionales y cálculos más complejos, información de varias tablas. Otro uso de un procedimiento es la validación de datos que integran la estructura del banco de datos, a estos procesos almacenados se les denomina disparadores o tiggers.

La sintaxis para crear unos procedimientos almacenados es:

```
CREATE PROCEDURE nombre_procedimiento [ @ parametro1 <tipo>, …]
AS
[sentencias_del_procedimiento];
```

Después de crear un procedimiento almacenado se pueden realizar las siguientes tareas:

– Determinar agrupamiento de procedimientos almacenados.

– Definir parámetros de entrada y salida, sus tipos de datos y sus valores por defecto.

Los procedimientos almacenados pueden hacer referencia a tablas, vistas, funciones definidas por el usuario y otros procedimientos almacenados.

Si se crea una tabla local temporal por medio de un procedimiento almacenado, la tabla solo existirá hasta que finalice la ejecución del mismo.

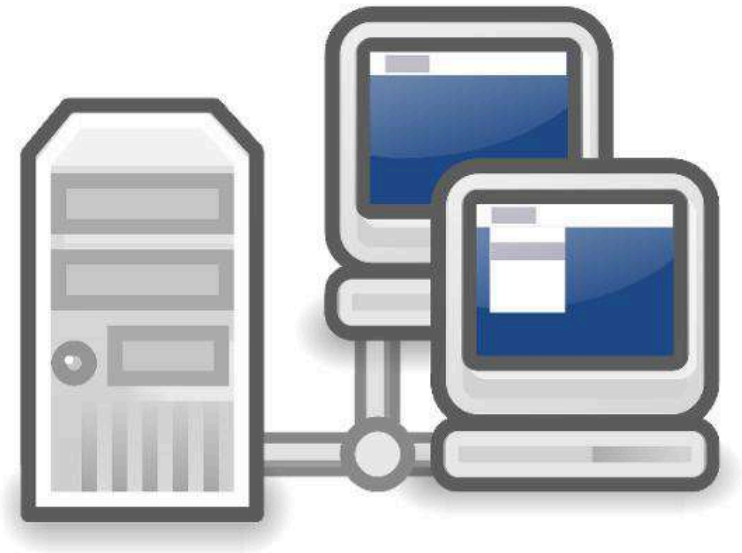
Importante

Esta sentencia, no puede ser combinada con otras sentencias SQL en un solo proceso por lotes. Puede incluir dentro de esta sentencia cualquier instrucción SQL, con la excepción de que no podrá contener las instrucciones de creación de objetos:

- CREATE DEFAULT.
- CREATE PROCEDURE
- CREATE RULE
- CREATE TRIGGER.
- CREATE VIEW

Para poder crear un usuario procedimientos almacenados, debe de tener permiso para poder crearlo. El tamaño máximo de un procedimiento almacenado es de 128 megabytes.

Cuando se definen parámetros de entrada y salida, estos siempre irán con @ seguido del nombre del parámetro o variable, y después deberá de tener una indicación del tipo de dato y de su tamaño.



Los parámetros de salida para diferenciarlos de los de entrada incluyen la palabra OUTPUT.

Se usara códigos de retorno para mostrar la información.

Ejemplo:

```
USE gestión

CREATE PROCEDURE inserta_cliente @ nombre varchar (50);

    @ primer_apellido varchar (100);

            @ segundo_apellido varchar (100);

    @ dni varchar (20);

AS

INSERT INTO CLIENTES

Nombre, primer_apellido, segundo_apellido, dni) VALUES

(@nombre. @ primer_apellido,@segundo_apellido,@dni);
```

En este ejemplo se va a crear un procedimiento llamado insertar_cliente, en el cual vamos a hacer un procedimiento que al ser llamado insertara un cliente nuevo en la tabla clientes. Por medio de las variables o parámetros introducidos de tipo varchar



Para ejecutar un procedimiento almacenado se utiliza la siguiente sentencia:

```
USE gestión

EXEC inserta_cliente 108749504V´, ´Antonio', 'González Muñoz´
```

Si se quiere comprobar si se ha efectuado y ejecutado correctamente el procedimiento, podemos realizar una consulta con la sentencia SELECT.

Ejemplo:

```
SELECT * FROM clientes
```

Para modificar un procedimiento almacenado se emplea la sentencia ALTER.

La sintaxis seria la siguiente:

```
ALTER PROCEDURE <nombre_procedimiento> [ “parametro1 <tipo>,..]
AS
IF (@parametro1 comparacion)
        BEGIN
        END
ELSE
        BEGIN
        [sentencias_del_procedimiento]
                VALUES(@parametro1,@parametro2…)
        END
```

```
USE gestión
ALTER PROCEDURE inserta_cliente @ nombre varchar (50);
                                @ primer_apellido varchar (100);
                                @ segundo_apellido varchar (100);
                                @ dni varchar (20);
AS
IF (@dni=´)
    BEGING
    END
ELSE
        BEGIN
            INSERT INTO CLIENTES
Nombre, primer_apellido, segundo_apellido, dni) VALUES
(@nombre. @ primer_apellido,@segundo_apellido,@dni);
            END
```

En este ejemplo, si el dni es igual a vacío no se realiza ninguna acción y no se añadirá, y si no está vacío se insertará un cliente en la tabla clientes.

Los **BEGIN** y **END** son el principio y final del **IF**, y **RETURN** que lo que va a hacer es que salga del procedimiento.

Para eliminar un procedimiento almacenado se realizara con la sentencia **DROP**.La sintaxis para eliminar un procedimiento seria:

```
DROP PROCEDURE nombre_del_procedimiento;
```

Beneficios o ventajas de la utilización de procedimientos almacenados:

- Disminuyen la ejecución de tareas repetitivas.
- Reducción del tráfico en la red.
- Las instrucciones se procesan más rápido que si se ejecutan de forma interactiva.
- Fija consistencia ya que ejecuta tareas de la misma forma.
- Permite el desarrollo de aplicaciones.
- Permiten forzar las reglas de negocio que son bastante complejas.
- Retención de errores antes de que ellos puedan acceder a la base de datos.
- Provee seguridad.

5.4.5. Transacciones distribuidas

Gracias a los avances tecnológicos de hoy en día se han producido cambios a la hora de ver los sistemas de información.

La demanda por parte del usuario de que exista una mayor eficiencia, mayor número de servicios, mayor flexibilidad y mayor rendimiento, hace que se creen soluciones integrando tecnologías nuevas.

La evolución de los sistemas de bases de datos ha sido impulsada por los usuarios para satisfacer las necesidades de capacidad que debe de tener una base de datos, surge con la necesidad de integrar información proveniente de diferentes fuentes.

Esta evolución da como fruto los sistemas de bases de datos distribuidas.

Una base de datos distribuida es un conjunto de elementos que se procesan automáticamente, estas interconectadas por medio de una red de comunicación y colaboran entre ellos para realizar una serie de tareas.

Importante

Una base de datos distribuid aes un conjunto de bases de datos relacionales, que se encuentran distribuidas entre diferentes ordenadores que están interconectados por una red de comunicación

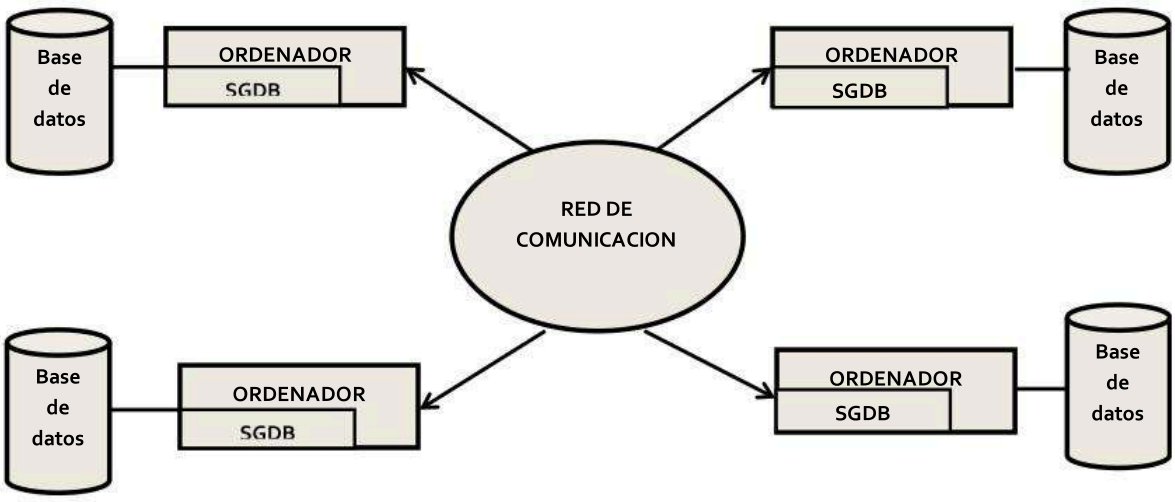
y un sistema gestor de base de datos que es el encargado del manejo de la base de datos, facilita mecanismos de acceso que son transparentes por el usuario.

En conclusión, la organización es la siguiente de un sistemas de base de datos distribuida es el conjunto de sitios o nodos los cuales tienen una base de datos local, un sistema gestor de base de datos y facilidad de comunicación por medio de una red.

Estos sitios pueden estar situados geográficamente donde sea y estarán conectados por medio de una red de comunicación WAN, si están los nodos geográficamente en la misma ubicación se utilizara la red de comunicación LAN, en este caso, sería una bases de datos centralizada.

Todo esto se lleva a cabo por medio de las transacciones de los usuarios.

En este esquema se representa como seria la organización de un sistema de base de datos distribuido.



Las bases de datos centralizadas son las que se encuentran en un solo lugar físico, es la base de datos que solo esta almacenada en un solo ordenador, en donde los usuarios trabajan en ordenadores que solo muestran resultados, son todos aquellos que se ejecutan en un solo ordenador sin interacción con ningún otro ordenador.

Se caracteriza por:

- Almacenamiento local central.
- No tiene elementos de procesamiento ni mecanismo de interconexión.
- Los componentes que la forman son los datos, un sistema gestor de base de datos y dispositivos de almacenamiento.
- No tienen muchos problemas de seguridad, por la poca concurrencia.

Ventajas:

- Se evita redundancia.
- Se evita inconsistencia.
- Se pueden aplicar restricciones de seguridad.
- Conserva la integridad de los datos.
- Mayor rendimiento respecto al procesamiento.
- Cumple las normas establecidas.

Desventajas:

- No son proporcionales precio/rendimiento.
- Las cargas de trabajo no se pueden repartir.

Mientras lo que caracteriza a los sistemas de bases de datos distribuidas son:

- Proporciona comunicaciones avanzadas, dando acceso a ordenadores remotos.
- Permite la transferencia de consultas y datos entre ordenadores.
- Compartición de recursos.
- Las interfaces son públicas.
- Se pueden crear a partir de hardware y software heterogéneos.
- Soporta mucha concurrencia.
- Son muy escalables.
- Tolerante a fallos.
- Son totalmente transparentes para el usuario.

Desventajas:

- Complejidad.
- Coste.
- Se debe considerar que se tendrá una mayor posibilidad de error.
- Mayor tiempo de procesamiento que las bases de datos centralizadas.



Una transacción de base de datos, es una unidad de trabajo realizada por una o varias sentencias SQL que están relacionadas.

Se califica transacción a una serie de operaciones que se realizan en una sola unidad lógica de trabajo.

Toda unidad lógica debe de tener cuatro cualidades que se denominan ACID, son, la atomicidad, coherencia, aislamiento y durabilidad.

Que son:

- Atomicidad.
- Coherencia o consistencia.
- Aislamiento.
- Durabilidad.

- **Atomicidad:** Una transacción debe de ser una unidad atómica, o se ejecuta todo o no se ejecuta nada.

- **Coherencia o consistencia:** cuando se finaliza, una transacción debe de dejar todos los datos en un estado de coherencia, la correcta ejecución de una transacción leva a la base de datos de un estado correcto a otro.

- **Aislamiento:** las actualizaciones realizadas por transacciones simultáneas se deben de aislar de las actualizaciones realizadas por otras transacciones simultáneas.

- **Durabilidad:** una vez que se termina una transacción, sus efectos son permanentes en el sistema.

Los programadores de SQL son los responsables de iniciar y finalizar las transacciones en puntos que exijan la coherencia lógica de los datos.

El programador debe de definir la secuencia de modificaciones de datos que los dejan en un estado coherente en relación con la reglas de la organización. Después, el programador incluye estas instrucciones de modificación en una sola transacción de modo que SQL puede exigir la integridad física de los datos.

SQL proporciona:

- Servicios que aíslan a la transacción.

- Servicios de registro que aseguraran la durabilidad de la transacción, esto se utiliza por si existe un error en el sistema, se deshaga automáticamente las transacciones que no han sido completadas en el momento que hubo un error en el sistema.

- Administra las transacciones que exigen coherencia y atomicidad, esto quiere decir que una vez se inicie una transacción se concluye correctamente o SQL deshará todas las modificaciones que se hayan producido desde que se inició la transacción.

SQL proporciona, servicios de bloqueo que conservan el aislamiento de la transacción, servicios de registro que aseguran la durabilidad de la transacción y características de administración de transacciones que exigen la atomicidad y coherencia de la transacción.

Recuerda 

Las transacciones distribuidas se ejecutan en varios servidores a la vez, cuando una transacción distribuida termina, la propiedad de atomicidad pide que todos los servidores que estén involucrados en la transacción produzcan las modificaciones o no de la misma.

Las transacciones distribuidas se van a ejecutar en dos o más servidores diferentes que serán los administradores de recursos.

SQL soporta las transacciones de base de datos mediante dos sentencias de procesamiento de transacciones, que son **COMMIT** y **ROLLBACK**

Para comenzar una transacción distribuida explicita se utiliza la sentencia **BEGIN TRANSACTION.**

La sintaxis es la siguiente:

```
BEGIN  {TRAN | TRANSACTION}
            [nombre_trasaccion | @nombre_variable_transaccion]
            [WITH MARK [´descripción´]]
```

El nombre transacción es el nombre que se le va a dar a la transacción.

El argumento nombre_variable_transaccion es el nombre de una variable que será definida por el usuario, la variable se debe de declarar con un tipo de datos varchar, nchar, o nvarchar.

WITH MARK [´descripción´] establece que la transacción está marcada en el registro de transacciones, siendo descripción una cadena que describe la marca., cuando se utiliza debemos de determinar un nombre de transacción.

El argumento nombre transacción determinara el nombre de la transacción que fue asignado por la instrucción BEGIN TRANSACTION anteriormente.

El argumento nombre_variable_transaccion determina el nombre de una variable que es definida por el usuario y contendrá un nombre de transacción. La variable se debe de declarar char, varchar, nchar, nvarchar



Para marcar el final de una transacción que se ha realizado correctamente, se utiliza la sentencia **COMMIT TRANSACTION**, con esta sentencia lo que se hará es que se realicen las modificaciones sobre los datos desde el inicio de la transacción y se conviertan en parte permanente de la base de datos, además liberara los recursos que se mantienen por la conexión.

Su sintaxis es la siguiente:

```
COMMIT {TRAN | TRANSACTION}
        [nombre_trasaccion | @nombre_variable_transaccion]
```

La responsabilidad de utilizar **COMMIT** es del programador SQL, solo deberá de utilizarlo cuando todos los datos a los que hace referencia la transacción sean lógicamente correctos, ya que si se ejecuta COMMIT las modificaciones sobre los datos será permanente.

Otro tipo de sentencia **COMMIT** es la **COMMIT WORK**, funciona igual que la **TRANSACTION**, marca el final de una transacción, la única diferencia es que no aceptan nombres de transacción definidos por el usuario.



Para señalar que una transacción se ha realizado sin éxito se utiliza la sentencia ROLLBACK TRANSACTION. Esta secuencia lo que hará será eliminar las modificaciones de datos realizadas desde el inicio de la transacción y también libera los recursos que retine la transacción.

Su sintaxis es la siguiente:

```
ROLLBACK   {TRAN | TRANSACTION}

        [nombre_trasaccion | @nombre_variable_transaccion]

        | nombre_punto_restauracion | @nombre_variable_punto_restauracion]
```

El argumento nombre_punto_restauracion, es el punto de almacenamiento para restaurar una instrucción y permitirá que la transacción retroceda hasta ese lugar.

ROLBACK TRANSACTION va a eliminar todas las modificaciones de datos realizadas desde que comenzó la transacción o hasta un punto de almacenamiento.

UD5
Lo más importante

- La **API** es un grupo de funciones y procedimientos que contiene una biblioteca, que podrá ser usado por otro software como capa de abstracción La función principal de una API es la de suministrar un grupo de funciones que son de uso general, por ejemplo, dibujar un icono. Con una API al programador le será más fácil su trabajo

- **ODBC**, es un estándar de acceso a bases de datos que se creó para facilitar el acceso a los datos desde cualquier aplicación, sin importar que sistema de gestión de datos se utilice.

- **JDBC**, es un derivado que está inspirado en **ODBC**. Gracias a **JDBC**, un programa Java podrá aceptar a cualquier base de datos sin tener que modificar la aplicación., para conseguir esto es necesario instalar un driver que cumpla JDBC.

- Existen varios tipos o niveles de driver **JDBC**:

    - **Puente JDBC-ODEBC**. Este tipo de driver, establece un **puente entre JDBC y ODBC**, transforma operaciones JDBC en llamadas a la API ODBC.

    - **Java parcialmente Nativo (Java/Binario)** Este tipo de driver utiliza la interfaz de métodos nativos de Java para convertir solicitudes de API JDBC en llamadas a bases de datos.

    - **Java nativo –NET**. En este tipo de driver las solicitudes de la base de datos JDBC, estarán traducidas por un protocolo de red que es independiente de la base de datos y que serán dirigidas al servidor de capa intermedia.

- **Java nativo JDBC –Independiente**. En este tipo de driver la comunicación se realiza directamente con el servidor de base de datos utilizando el protocolo nativo del servidor.

– Tres niveles de abstracción:

  - **Nivel interno o físico**. Este nivel describe la estructura física de almacenamiento Emplea un modelo físico de datos y los únicos datos que existen se encuentran en este nivel.

  - **El nivel externo o de visión**, este es el que está más cerca del usuario se especifica varios esquemas externos o vistas de usuario. Cada esquema externo explica la parte de la base de datos que interesa a un usuario o a un grupo de ellos y las que no las oculta.

  - **El nivel Conceptual**, aquí se describirá toda la estructura de una base de datos mediante un esquema conceptual, en este esquema se describe las entidades, los atributos, las relaciones, operaciones de los usuarios y restricciones.

– Las operaciones que se pueden llevar a cabo sobre una base de datos, se basan en la algebra relacional, esto consiste en la aplicación de un grupo de operadores con los cuales se podrán realizar operaciones de inserción, borrado, modificación y consulta de tablas.

– Las operaciones pueden ser **urinarias, binarias y derivadas**.

  - Las **unarias**, son la selección y la proyección.

  - Las **binarias**, son Unión , diferencia y producto cartesiano

  - **Derivadas** son intersección, join y cociente.

– La sentencia por excelencia de consulta de datos es la sentencia **SELECT**. La sentencia **SELECT** lo que va a hacer es obtener filas o registros de una o más tablas, también se podrá utilizar junto a otro sentencia o para realizar una subconsulta dentro de una consulta SQL.

La cláusula SELECT determina las columnas que va a devolver la consulta

– El orden de la clausulas en la sentencia **SELECT** es muy importante, cualquiera de las cláusulas opcionales se pueden suprimir, pero si no se hacen deben de llevar un orden.

- El lenguaje Transact-SQL se compone de varios elementos

  · **Identificadores**:

    Son los nombres de los objetos, cualquier elemento de SQL puede servir de identificador como son las tablas, las columnas, índices, procedimientos, las vistas, bases de datos y servidores.

  · **Tipos de datos:**

    Todos los objetos que contienen datos tienen asociado un tipo de datos en Transact- SQL, el tipo de dato lo que va a hacer es precisar las clase de datos del objeto, que pueden ser, binario, carácter, entero, decimal, etc.

  · **Constantes**

    Una constante también llamada literal, representa el valor específico de un dato.

- **Un procedimiento almacenado** está formado por una serie de instrucciones Transact SQL, que servirán para definir un determinado proceso, este podrá ser ejecutado directamente en el servidor de Base de datos, permitiendo la ejecución de una o varias acciones.

  Acepta parámetros de entrada y devuelve un valor o conjunto de resultados. Pueden tanto recibir como devolver información, a este es a lo que nos referimos con que un procedimiento almacenado emplea parámetros de entrada y salida.

- **Los procedimientos** se utilizan también para obtener por medio de instrucciones condicionales y cálculos más complejos, información de varias tablas. Otro uso de un procedimiento es la validación de datos que integran la estructura del banco de datos, a estos procesos almacenados se les denomina disparadores o tiggers.

- **Una transacción** de base de datos, es una unidad de trabajo realizada por una o varias sentencias SQL que están relacionadas.

- Toda unidad lógica debe de tener cuatro cualidades que se denominan ACID, son, la atomicidad, coherencia, aislamiento y durabilidad.

- **Las transacciones distribuidas** se ejecutan en varios servidores a la vez, cuando una transacción distribuida termina, la propiedad de atomicidad pide que todos los servidores que estén involucrados en la transacción produzcan las modificaciones o no de la misma.

- **Las transacciones distribuidas** se van a ejecutar en dos o más servidores diferentes que serán los administradores de recursos.

UD5
Autoevaluación

1. Las tecnologías más utilizadas para realizar aplicaciones web son:

    a. CGI,ASP,.NET y JSP.
    b. .NET, CGI, JSP, PHP y ASP.
    c. .NET, JSP y PHP.
    d. CGI, JSP y .NET.

2. ¿Cuáles son los elementos que forman una base de datos?.

    a. El hardware y un software.
    b. El hardware.
    c. El hardware, un software y datos.
    d. Ninguna de las anteriores.

3. ¿De que se ocupa el programador de una aplicación?.

    a. Son aquellos que pueden acceder a la información por medio de lenguajes de consulta SQL.
    b. Son aquellos que a través de un lenguaje genera nuevos módulos u objetos capaces de manejar nuevos datos en el sistema.
    c. Son aquellos que utilizan aplicaciones avanzadas para reconocer imágenes, audio y demás.
    d. Son aquellos que interactúan con el sistema gestor de base de datos por medio de aplicaciones que serán permanentes.

4. La sentencia para conceder permisos es:

    a. GRANT.
    b. SELECT.
    c. REVOKE.
    d. CREATE.

5. ¿Cuáles son las operaciones unarias?

    a. Unión y diferencia.
    b. Unión, diferencia y producto cartesiano.
    c. Selección y proyección.
    d. Intersección, join y cociente.

6. La operación de intersección lo que hace es que:

    a. A partir de dos tablas ya existentes dara como resultado una tabla nueva con filas de las dos tablas y columnas de una de ellas.
    b. A partir de dos tablas existentes, se combinan por medio de una condición pero da como resultado una tabla nueva con las filas que cumplan la condición determinada.
    c. A partir de dos tablas existentes dará como resultado una nueva tabla con las filas que están en una pero no están en la otra.
    d. A partir de dos tablas existentes, dará como resultado una tabla nueva con todas las filas de las dos tablas.

7. Para ejecutar una consulta se utiliza la sentencia:

    a. INSERT.
    b. ALTER.
    c. GRANT
    d. SELECT.

8. ¿Qué determina la sentencia SELECT?

    a. Las filas que va a devolver un producto cartesiano.
    b. Las filas que va a devolver una proyección.
    c. Las columnas que a a devolver una consulta.
    d. Las columnas que va a devolver una unión.

9. ¿Qué tipos de procedimientos almacenados existen?

    a. Procedimientos almacenados del sistema y procedimientos almacenados definidos por el usuario.
    b. Procedimientos almacenados del sistema, procedimientos almacenados definidos por el usuario y Procedimientos almacenados extendidos.
    c. Procedimientos almacenados del sistema, procedimientos almacenados definidos por el usuario y Procedimientos almacenados remotos.
    d. Procedimientos almacenados del sistema, procedimientos almacenados definidos por el usuario ,procedimientos almacenados extendidos y procedimientos almacenados remotos

10. Una transacción distribuida es:

    a. Es una unidad de trabajo realizada por una o varias sentencias SQL.
    b. Esta formada por una serie de instrucciones SQL que servirán para definir un determinado proceso.
    c. Es un programa externo y local.
    d. Es un programa externo.

Área: informática y comunicaciones

UD6

Lenguajes de definición de datos

6.1. Conceptos básicos, nociones y estándares

6.2. Lenguaje de definición de datos (DDL) y aplicación en SGBD actuales

6.3. Discriminación de los elementos existentes en el estándar SQL-92 de otros elementos existentes en base de datos comerciales

6.4. Sentencias de creación. CREATE:

- 6.4.1. Bases de datos
- 6.4.2. Tablas
- 6.4.3. vistas.
- 6.4.4. Tiggers o disparadores
- 6.4.5. Procedimientos almacenados

6.5. Sentencia de modificación: ALTER

- 6.5.1. base de datos.
- 6.5.2. Tablas
- 6.5.3. Vistas
- 6.5.4. Disparadores o triggers
- 6.5.5. Procedimientos

6.6. Sentencias de borrado: DROP, TRUNCATE

- 6.6.1. Bases de datos.
- 6.6.2. Tablas.
- 6.6.3. Vistas.
- 6.6.4. Disparadores o triggers
- 6.6.5. Procedimiento

6.1. Conceptos básicos, nociones y estándares



Un sistema gestor de base de datos tiene como función principal realizar consultas que son bastante complejas y que no vienen definidas por el sistema.

Para más información, consulta "Sistema gestor de base de datos" en los anexos al final del libro.

Estas consultas son realizadas por los usuarios y las utilizan como ayuda para realizar consultas complejas, estas se realizarán de forma directa sobre el sistema gestor de base de datos.

Estas consultas se realizan sin que se tenga que elaborar un programa que ejecute esa consulta.



Un sistema gestor de base de datos está formado por niveles y esquemas para trabajar sobre una base de datos, los SGBD deben de conocer la estructura de esta, que entidades va a tener, que tipo de datos va a contener las entidades, que atributos definen a la entidad, las relaciones entre ellas, etc.

Se trata de un conjunto de objetos que se almacenan en una base de datos.



Una base de datos está compuesta por tablas, columnas y tipos de datos.

Las tablas son la estructura básica de cualquier base de datos relacional. Una tabla es una colección organizada de registros que serán las filas de la tabla, y de los atributos que serán las columnas.

dni_cliente	nombre	apellidos
15246985b	Pepito	Roldan Muñoz
0369852n	Manuel	García Contreras
47852369m	Rosa	Domínguez Expósito

Cuando se crea una tabla en la base de datos de SQL, se establece la estructura de tabla mediante la identificación de las columnas que describen los atributos de una tabla, además, todas las columnas de una tabla tienen un tipo de datos que describirá que tipo de datos aceptará esa columna.

```
dni_cliente varchar (9);
nombre varchar (30);
apellidos varchar (50);
```

Para conocer la definición de la base de datos, los sistemas gestores de bases de datos necesitaran la ayuda de un esquema de la base de datos, este esquema siempre se encontrara disponible por el sistema gestor de base de datos.



El esquema es un elemento muy importante de la estructura de un sistema gestor de base de datos, es lo que va a servir para separar el sistema gestor de base de datos de la base de datos, con esta separación lo que se intenta conseguir es que, se puedan realizar en el esquema de la base de datos sin tener que realizar ninguna modificación en el sistema gestor de base de datos.

En temas anteriores hemos visto los niveles para diseñar una base de datos, que eran el nivel conceptual, lógico y físico.

Durante los años 80, ANSI intentaba crear un estándar para bases de datos.



ANSI recomendó que la estructura de los sistemas gestores de bases de datos tuvieran tres niveles para describir la base de datos, dividiendo el nivel lógico en dos, uno seria el nivel externo y otro el nivel conceptual, el nivel interno es el nivel físico.

Con lo que nos encontraremos con tres niveles de esquemas.

- **Nivel conceptua**l, esquema conceptual.
- **Nivel externo**, esquema externo.
- **Nivel interno** o físico, esquema físico.



Esquema conceptual.

Este esquema realizara una descripción lógica de la base de datos tratando de describir e identificar las entidades necesarias, los atributos y las relaciones que se produzcan entre ellas.

Aquí, en este esquema, se deberán identificar las necesidades que una empresa tendrá, intentando distinguir entidades que puedan representarse como objetos de la vida real.



Esquema externo o también llamado de visión.

El esquema corresponde, a como van a ver los usuarios los procesos de forma lógica.

El usuario realizara un esquema externo con las tablas y atributos que mas les interesen.

Se podrán definir datos derivados o volver a definir una entidad, se podrán definir una mezcla de entidades para que parezcan que es una sola, etc.



Esquema físico o interno.

Este esquema corresponderá a la descripción de cómo se va a ser la organización interna de la base de datos, que dispositivos se utilizaran, de qué forma se almacenara, etc.

El nivel interno de la arquitectura de un sistema gestor de base de datos, corresponde a la definición de datos, aquí es donde se va a definir la estructura física de los datos mediante un esquema interno.

Este esquema se determina por medio de un modelo físico y describe todos los detalles para el almacenamiento de la base de datos, así como los métodos de acceso.

Los únicos datos que existen en la base de datos son los que están en el nivel físico, que se encontraran almacenados en algún dispositivo, como un disco duro.

Para realizar el esquema físico se partirá del esquema conceptual y del esquema lógico, que se abra diseñado en la etapa de diseño conceptual y en la etapa lógica.

Para crear una base de datos primero se debe de crear un esquema conceptual, un esquema externo y no necesariamente crear un esquema interno.

Si el esquema interno o físico no se define, el mismo sistema gestor de base de datos, se encargará de todos los detalles de la organización física.

Recuerda 

Todos los esquemas varían si se utiliza un sistema gestor de base de datos relacional, en este caso no existe una separación concreta y precisa de estos tres niveles.

Solo se distingue una esquema llamado Schema, en el que se podrán encontrar descripciones de los tres tipos de niveles.

En el Schema encontraremos elementos como entidades, atributos, relaciones y restricciones y se utilizan también las vistas, es decir, el esquema externo, ya que una vista es el esquema externo.

En la mayoría de los sistemas de informaron de hoy en día se utilizan los sistemas gestores de bases de datos relacionales.

El modelo relacional, es el que se basa en el concepto matemático de la relación.

La estructura es una tabla que esta compuesta de filas y columnas. Los modelos relacionales ofrecen independencia física total de los datos.

dni_cliente	nombre	Apellidos
15246985b	Pepito	Roldan Muñoz
0369852n	Manuel	García Contreras
47852369m	Rosa	Domínguez Expósito

Existen más modelos de datos que no son utilizados por los sistemas gestores de bases de datos. Solo se usuraran durante la creación del diseño y análisis de la base de datos.

Los dos modelos mas conocidos son:

- Modelos semánticos.
- Modelos funcionales.

Estos, son utilizados para describir las estructuras de los datos del mundo real, la semántica y las relaciones entre ellas, pero no se utilizan para realizar operaciones sobre ellas, solo realizan una descripción lógica de los datos o informaciones.

Se utilizan para realizar el diseño de la base de datos, este es el modelo E-R, que ya estudiamos en temas anteriores.



Este modelo es el más utilizado y extendido por los desarrolladores para el análisis y diseño de los datos, el mas extendido también, es el modelo relacional que terminara el diseño y la construcción, a partir de el modelo E-R de la base de datos con un sistema gestor de base de datos.

El modelo relacional es el que se basa en organizar los datos y representarlos en forma de tablas o también llamada relación.

La representación lógica de un objeto se realiza mediante una tabla que esta compuesta por filas y columnas.

La representación física es una tabla que equivaldría a un archivo secuencial, una fila corresponderá a un registro y una columna corresponderá a un campo.

En el modelo relacional una tabla recibe el nombre de relación, una fila recibe el nombre de tupla y una columna recibe el nombre de atributo.+

Por lo que en el modelo relacional la representación será, una relación (tabla), que esta compuesta por tuplas (filas) y un conjunto de dominios (valores de cada una de los atributos).

Campos				Registros
DNI	NOMBRE	APELLIDOS	DIRECCION	
45252235v	Ana	López Benítez	Calle álamos	
74855231c	Javier	Martínez Gómez	Calle Juan de Austria	
89564522b	Domingo	García Ferrer	Avenida de Andalucía	
77788455z	Francisco	Fernández González	Doctor Marañón	

Vamos a ver el esquema conceptual y lógico, del que parte el esquema interno de los sistemas gestor de base de datos.

En este ejemplo vamos a crear el esquema conceptual y lógico de una base de datos en el que:

a. Se quiere diseñar una base de datos para la organización de una academia.

   La academia se identificara con su nombre, un código que identifica de manera única esa escuela, un número de teléfono y la dirección.

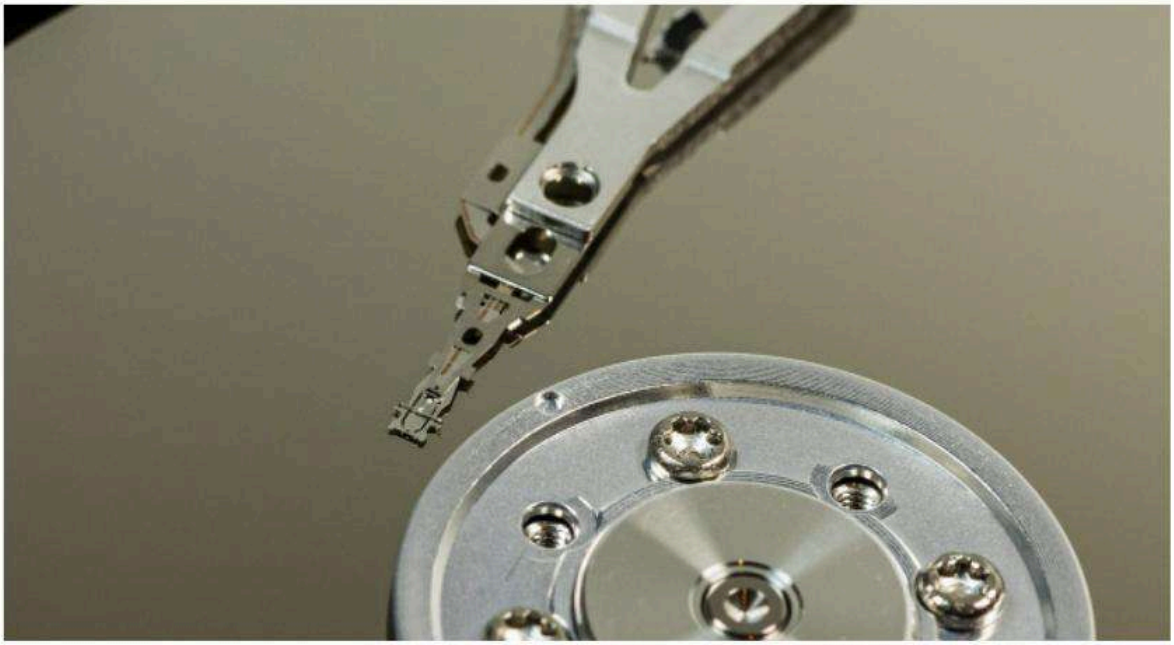
   Esta academia impartirá un curso o varios cursos. Un curso contendrá su nombre y un código de curso.

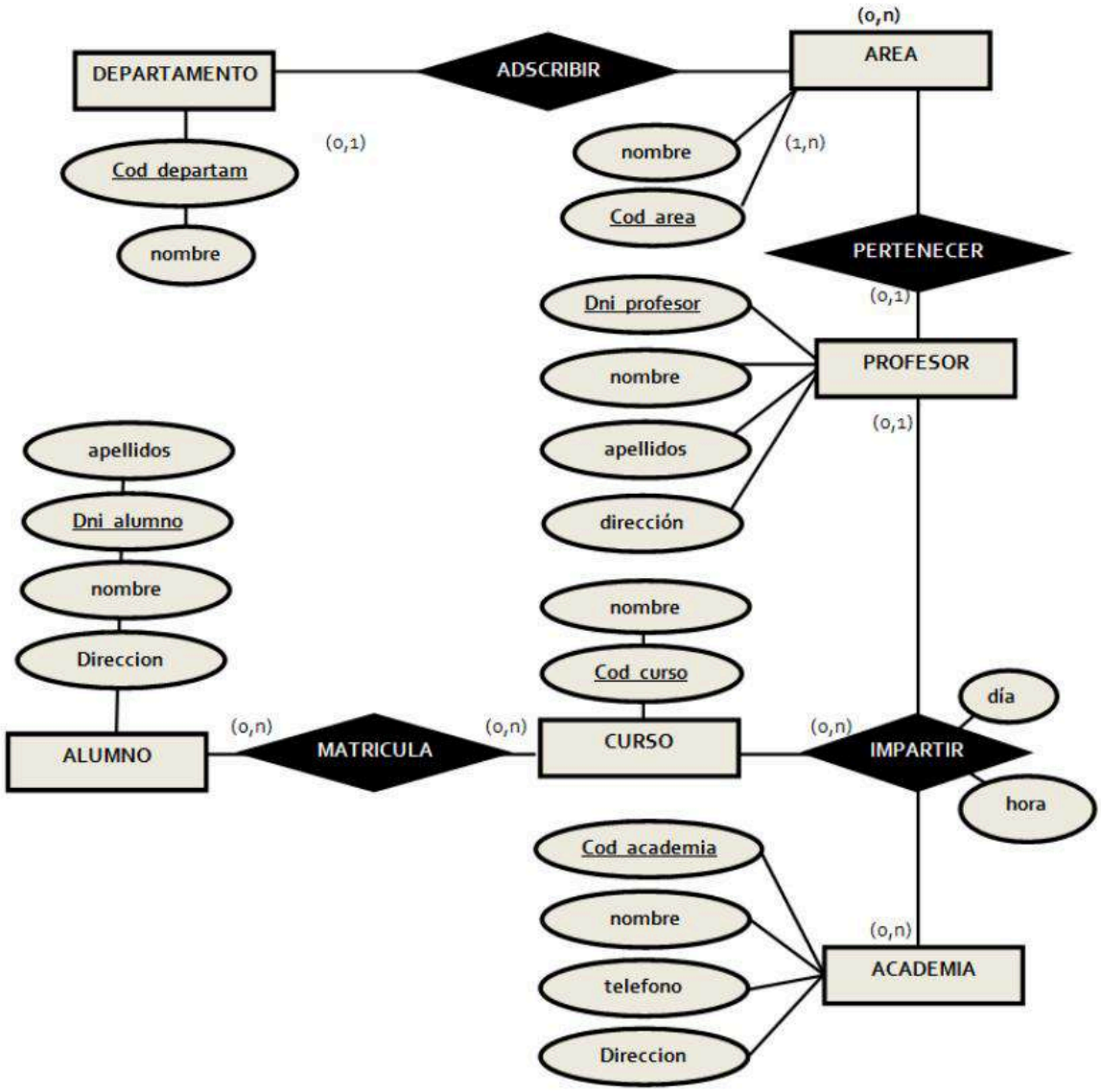
b. A los cursos se matriculan alumnos y los alumnos pueden matricularse en varios cursos a la vez.

   Los alumnos van a contener su dni, nombre, apellidos y dirección.

c. En la academia imparten cursos profesores. Estos profesores contendrán su dni, nombre, apellidos y dirección.

d. Un profesor puede pertenecer a un área, pero esa área es compartida por varios profesores. El área va a contener un código y su nombre. Un área está adscrita a un departamento y un departamento puede estar adscrito a una o varias áreas. Departamento contendrá un código y un nombre.





Importante

SQL es un lenguaje estandarizado por ANSI/ISO, de acceso a base de datos relacionales, por medio de este podremos realizar operaciones de definición, manipulación y gestión de control de las bases de datos relacionales.

Se trata de un lenguaje declarativo, en el que se basa en las matemáticas, se trata de un conjunto de condiciones, afirmaciones, restricciones, ecuaciones y transformaciones que van a describir un problema y la solución, estos lenguajes no se especializan en como se va a llegar a una solución sino en que esta la solución. Solo habría que indicar que se quiere hacer.

SQL es el lenguaje estándar mas utilizado con el que se puede acceder a las bases de datos comerciales.

El lenguaje SQL es el lenguaje relacional estándar de facto basado en el algebra relacional.



SQL esta compuesto por sentencias, clausulas, operadores y funciones agregadas.

- Sentencias.
- Clausulas.
- Operadores.
- Funciones agregadas.

Estos elementos se combinan en las instrucciones para crear, actualizar y manipular las bases de datos.

Este lenguaje tiene una serie de reglas sintácticas básicas:

- Los enter, tabulaciones y espacios en blanco, no tienen ningún significado especial a la hora de ejecutar las sentencias.
- Las listas de objetos se separan por comas ´, ´ y se podrán utilizar paréntesis para agrupar a los objetos.
- Tanto el lenguaje de la aplicación como el lenguaje del sistema gestor de base de datos que permita implementar SQL hospedado o embebido, deberá de utilizar la sintaxis SQL, aunque puede contener algunas variaciones.
- En SQL, las columnas que son clave primaria no pueden aceptar valores nulos.
- SQL, no diferencia entre mayúsculas o minúsculas, menos en los casos en los que queramos introducir una cadena de caracteres como valor y luego queramos compararlos.
- Sí una esta en minúsculas y la otra en mayúsculas, si la comparamos las diferenciara.



Los lenguajes SQL que más se utiliza en el sistema gestor de base de datos se dividen en dos:

- Lenguaje de manipulación de datos (DML).
- Lenguaje de definición de datos (DDL).

Los sistemas gestores de bases de datos deben de ofrecer lenguajes e interfaces apropiados para cada tipo de usuario, que pueden ser: administradores de la base de datos, diseñadores, programadores de aplicaciones y usuarios finales.

El lenguaje de definición DDL.

Está compuesto por las sentencias que se utilizan para crear los objetos dentro de la base de datos y modificar propiedades y atributos de la propia base de datos.

Estos objetos pueden ser tablas, columnas, vistas, procedimientos y disparadores.

Las instrucciones básicas de SQL DDL son:

- CREATE.
- ALTER.
- DROP.

Una base de datos utiliza un esquema, por medio de él se van a definir las tablas de la base de datos, los campos de cada tabla y las relaciones que hay entre cada una de ellas.

Va a ser una representación gráfica de la base de datos. Esto es lo que queremos conseguir con el lenguaje de definición de datos, transformar el modelo entidad –relación a un esquema relacional.

Cuando ya se ha terminado el diseño de una base de datos y ya se ha el elegido el sistema gestor de base de datos, lo que se debería de hacer es elaborar el esquema conceptual y el esquema interno de la base de datos.

En algunos sistemas gestores de bases de datos no separan estrictamente estos niveles por lo que el administrador de la base de datos van a utilizar el mismo lenguaje para realizar los dos esquemas, mediante DDL, que lo hará

SQL dispone de muchos servicios: servicio de notificación, servicio de integración, etc.

6.2. Lenguaje de definición de datos (DDL) y aplicación en SGBD actuales



El lenguaje de definición de datos DDL es un lenguaje proporcionado por sistema gestor de base de datos, lo que va a hacer es permitir que los usuarios puedan realizar estructuras para almacenar los datos y también podrá definir los datos de procedimientos o funciones, básicamente se utiliza para crear estructuras de datos.

DDL no es un lenguaje independiente.

A parir de aquí ya podremos utilizar el lenguaje DDL, que ya veremos en las siguientes lecciones.



Para realizar la comunicación entre el usuario y el sistema gestor de base de datos, se debe de utilizar un lenguaje. Existen multitud de tipos diferentes de lenguajes de programación, desde los mas sencillos hasta los mas complicados, según el tipo de usuario para los que están pensados.

Existirán usuarios que sean expertos y se ocuparan de realizar los procesos mas complejos y para realizarlas necesitara la ayuda de un lenguaje complejo, y existirán usuarios que no tengan conocimientos informáticos y no necesitara realizar procesos tan complejos y solo necesite utilizar un lenguaje mas sencillo.

También existirán usuarios que estén especializados para trabajar con bases de datos y que tienen conocimiento de los lenguajes que se utilizan para su administración. Estos casi siempre son los administradores de las bases de datos.



El lenguaje mas utilizado en los sistemas gestores relacionales es SQL.

SQL dispondrá de una serie de lenguajes que están especializados para realizar las funciones mas importantes de un sistema gestor de base de datos.

Todos los sistemas gestores de base de datos proporcionan lenguajes e interfaces para cada tipo de usuario que pueden ser administradores, diseñadores, programadores y usuarios finales.

La utilización de estos lenguajes va a dar la posibilidad al administrador de la base de datos ,de determinar los datos que componen la base de datos, la estructura, las relaciones que existen entre estos datos, las reglas de integridad, el control de acceso, las características de tipo física y las vistas externas de los usuarios.



SQL dispondrá de una serie de lenguajes que están especializados para realizar las funciones mas importantes de un sistema gestor de base de datos.

Los lenguajes del sistema gestor de base de datos SQL son: el lenguaje de definición de datos DDL y los lenguajes de manipulación de datos DML y lenguaje de cuarta generación.

- Lenguaje DDL.
- Lenguaje DML
- Lenguaje de cuarta generación.

Importante

- **Lenguaje DDL.**

  Se utiliza para especificar el esquema de la base de datos, las vistas de los usuarios y las estructuras de almacenamiento. Es el que desarrolla el esquema conceptual y el esquema interno.

- **Lenguaje DML.**

  Se utiliza para leer y actualizar datos de la base de datos, los usuarios de la base de datos podrán realizar por medio de una serie de sentencias, consultas, añadir, eliminar y modificar datos de la base de datos.

- **Lenguaje de cuarta generación.**

  Se define lenguaje de cuarta generación como un conjunto de herramientas pre-fabricas, que consiguen dar un lenguaje de programación de alto nivel.



A parte existen también otros lenguajes que hemos mencionado en lecciones anteriores, pero las que van a servir para la creación y manipulación de datos van a ser estos tres lenguajes.

Hoy en día los sistemas gestores de base de datos comerciales incluyen un tipo de lenguaje de cuarta generación, que lo que va a hacer es permitir al usuario crear aplicaciones de forma muy sencilla, son las llamadas herramientas de desarrollo.



Los lenguajes de cuarta generación son una evolución de los de tercera generación.

Es un lenguaje de programación más avanzado, es un lenguaje no procedural, ya que el propio lenguaje es capaz de decirle al ordenador como debe de ejecutar el programa, este lenguaje se caracteriza porque son más fáciles de usar que lo de tercera generación.

Los lenguajes de cuarta generación incorporan interfaces gráficas y pone a disposición gestiones avanzadas, pero el problema es que utilizan mucho más recursos que los de la tercera generación.



Los lenguajes de cuarta generación son aquellos que son diseñados para cumplir un objetivo concreto, se utiliza comúnmente para el desarrollo de software comercial, lo que intenta conseguir con estos lenguajes de cuarta generación es producir una mayor abstracción y mayor potencia a las secuencias.

La cuarta generación introduce lenguajes y métodos de más alto nivel, se utiliza sentencias simples que generan una serie de instrucciones que el los lenguajes de tercera generación son mucho más complicadas y laboriosas.

Este lenguaje de alto nivel hace que disminuya también los errores que se puedan producir al programar.

Lo que lleva al avance hacia lenguajes de cuarta generación, es que al programador le proporciona concentración en resolver problemas y no en complicaciones que puedan surgir que son inherentes al lenguaje, por lo que reduce el esfuerzo en la programación, se reduce el tiempo y se reduce el coste.

Todos los lenguajes que están compuestos por una interfaz de desarrollo y un sistema de base de datos, son ejemplos de lenguajes de cuarta generación.



Los lenguajes de cuarta generación de alto nivel, están diseñados con un sistema central de diccionario de datos, una biblioteca, un generador de CRUD, generador de informes, lenguajes de consulta utilizado por el usuario final, un sistema gestor de base de datos y herramientas de diseño e integración de API.

- Sistema central de diccionario de datos.
- Biblioteca.
- Generador de CRUD.
- Generador de informes.
- Lenguajes de consulta.
- Sistema gestor de base de datos.
- Herramientas de diseño.
- Integración de API.

La estructura de bloques de los lenguajes de programación de tercera generación, mejoró en su momento el proceso de desarrollar el software, pero este método de desarrollo llega a ser muy lento y tener tendencia a errores.

Los lenguajes de cuarta generación incluyen.

Lenguajes de presentación, lenguajes de consulta y generador de informes.
Lenguajes especializados, como hojas de cálculo y lenguajes de base de datos.
Aplicaciones para la definición, inserción, modificación y selección de datos de la base de datos.
Lenguajes de alto nivel para producir código de la aplicación.

El lenguaje SQL es un ejemplo claro de lenguaje de cuarta generación.

Hay cuatro tipos deherramientas de cuarta generación:

- Generador de formularios:

  Esta herramienta es utilizada para crear formularios, con los que se podrán introducir o visualizar datos.

  Estos generadores van a permitir que el usuario determine qué y cómo se quiere visualizar la información.

- Generador gráfico:

  Se trata de una herramienta que se utiliza para conseguir visualizar los datos de una base de datos por medio de un gráfico en el que se podrán ver preferencias y relaciones entre datos. Pueden ser de diferentes tipos, lineal, en barras, etc.

- Generador de aplicaciones:

  Esta herramienta es utilizada para crear programas que hagan de intermediario entre la base de datos y el usuario.

  Está compuesto por procedimientos que se ocupan de realizar funciones que se utilizan en casi todos los programas. Esto hace que disminuya el tiempo que se tarda en diseñar un programa de aplicación.

  En este caso se dice lo que tiene que hacer el programa y el generador de aplicaciones se encarga de cómo hacer la tarea.

- Generador de informe:

  Esta herramienta se utiliza para la creación de un informe de los datos de una base de datos.

Se utiliza para visualizar información de los datos de la base de datos, es muy parecido a realizar una consulta, por medio de unas instrucciones, realiza consulta sobre los datos y que se visualice en un informe.

La apariencia de cómo se van a visualizar los datos lo determina el propio usuario o mediante el generador de informe que lo hará de forma automática.

Ventajas de los lenguajes de cuarta generación:

Elaboración de programas en menos tiempo y con menos coste.
Menor problemática a la hora de mantener a los programas.
Menor número de errores.
Elaboración de software menos agotador para el desarrollador.

Desventajas de los lenguajes de cuarta generación:

Dependencia de proveedores externos.
Algunos incumplen las normas ANSI e ISO.
Herramientas más flexibles.



El lenguaje SQL, ofrece lenguajes de 4cuarta generación y las herramientas visuales para facilitar el uso del lenguaje de cuarta generación.

El lenguaje de cuarta generación y las herramientas visuales, traducen lo que realiza el usuario a sentencias e instrucciones SQL, mediante la compilación, se traducirá las sentencias y por medio del interprete que SQL lleva integrado en el sistema gestor de base de datos.



Muchas bases de datos trabajan con un programa de aplicación, este se realizara por medio de cualquier lenguaje de programación.

Estos lenguajes suelen no ser declarativos, como es el caso del lenguaje SQL, y no tienen ninguna instrucción que puedan utilizar para acceder a la base de datos.

Se tendrán dos opciones para realizar el acceso a la base de datos, en la lección anterior ya los vimos.

- Llamada a funciones.
- SQL hospedado o embebido.

Se realizará o por medio de una llamada a funciones, que se realizan por medio de APIs, que son librerías de funciones especializadas en la utilización de bases de datos, por ejemplo ODBC O JDBC, estos suelen ser los estándares utilizados.

O por medio del SQL hospedado o embebido, en el que se incluye las sentencias e instrucciones SQL de nuestra base de datos dentro del lenguaje de programación de la aplicación.

En este caso se necesitará un pre-compilador para transformar estas sentencias e instrucciones en el lenguaje de programación de la aplicación para que lo acepte.

SQL será el lenguaje hospedado y el lenguaje de programación de la aplicación será el lenguaje anfitrión.

Ahora vamos a preocuparnos de definir el lenguaje DDL y en el próximo tema nos ocuparemos del lenguaje DML.



El lenguaje DDL, es el lenguaje utilizado para definir el esquema de la base de datos.

Este lenguaje va a proporcionarnos unas sentencias para realizar las siguientes funciones:

- Definir el esquema de la base de datos.
- Definir los dominios de la base de datos.
- Definir las relaciones, por medio del esquema relación y restricción.
- Definir las restricciones más importantes.
- Definir las relaciones o tablas derivadas, esto se hace por medio de vistas, que es el esquema externo o de vista.
- Definición de permisos a usuarios.



Las sentencias que componen el lenguaje de definición de datos DDL son:

CREATE	ALTER	DROP	TRUNCATE

Estas sentencias pueden crear los siguientes objetos:

- Bases de datos.
- Tablas.
- Vistas.
- Dominios.
- Procedimientos almacenados.
- Disparadores o triggers.



Para realizar la acción de creación se utilizara la sentencia:

CREATE

Para modificar cualquiera de los objetos de la base de datos mencionados anteriormente, se utilizara la sentencia:

ALTER

Si queremos modificar un dato de cualquier objeto , deberemos de utilizar ALTER y el nombre del objeto que queramos modificar.

Por ejemplo.

ALTER CLIENTES

En el que mediante estas sentencias estamos declarando que queremos realizar una modificación en la tabla CLIENTES.

Con las sentencias:

DROP TRUNCATE

Las sentencias **DROP** y **TRUNCATE**, son utilizadas para eliminar tablas y registros respectivamente.

También eliminaran vistas, procedimientos almacenados, bases de datos y disparadores o triggers.

Las **sentencias** son elementos básicos en los que se divide el código de un lenguaje de programación.

Una sentencia es una línea de código en algún lenguaje de programación, en este caso SQL.

Las sentencias determinan su final con un ´;´algunos están separados por enters, por medio del punto y coma separamos las sentencias unas de otras.

Toda sentencia de programación tiene una sintaxis y una semántica.

Una sentencia tiene un formato determinado, la primera línea será la acción que se va a realizar y siempre va en mayúsculas.

El lenguaje SQL contiene una serie de sentencias en forma de verbos

Para realizar las peticiones de los datos se utiliza una sentencia SQL, por la que se podrán obtener filas de una o varias tablas según la petición del usuario.

Recuerda 

En los sistemas gestores de bases de datos se encuentra un usuario muy especial, este es, el administrador de la base de datos.

El administrador de la base de datos es el que se encargara de realizar las tareas de administración y control de la base de datos. Este asegurara la gestión y el mantenimiento para asegurar de que se explote la base de datos de forma correcta y eficiente.

6.3. Discriminación de los elementos existentes en el estándar SQL-92 de otros elementos existentes en base de datos comerciales

SQL fue estandarizado por primera vez en 1986 por el ANSI, ahí dio lugar la primera versión estándar de este tipo de lenguaje, es el llamado SQL-86 o SQL1, una año más tarde también estandarizo SQL la ISO.

El primer estándar, no podía abarcar las necesidades de los desarrolladores, además de contener funciones de almacenamiento que no eran utilices, y así en el año 1992, apareció un nuevo estándar más amplio y examinado, es el llamado “SQL-92” o “SQL2”.

SQL ha seguido revisándose y se le han ido agregando a lo largo de los años.

Hubo una nueva ampliación en el año 1999, en el que se incorporaron expresiones recursivas y regulares, trigger y algunas características de los objetos.

Más tarde se produjo otra ampliación en el año 2003 en el que se incorporaron aspectos de XML, cambio en funciones, estandarización de los objetos sequences y de cadenas auto numéricas.

En 2005 volvió a ampliarse con la introducción de las formas de utilizar SQL y XML conjuntamente. También se introdujeron las formas de importar y guardar datos XML en la base de datos SQL y poder publicarlos el XML.

Y lo mas importante, da facilidades a las aplicaciones para añadir dentro de su código SQL el uso de XQuery, que es el lenguaje de consulta XML publicado por W3C (World Wide Wet Consortium) que se utiliza para el acceso concurrente a datos ordinarios SQL y documentos XML.

La ultima revisión se realizo en 2008, en la cual no se va a utilizar la clausula ORDER BY fuera de las definiciones de los cursores.

Se incorporan los disparadores INSTEAD OFF y se añade la sentencia TRUNCATE.

SQL has sido adoptado como el estándar oficial de la ANSI y de la ISO. La versión original fue desarrollada por IBM a principio de los 70.



SQL es un lenguaje de alto nivel, es un lenguaje sencillo, en el que los programadores no tienen que especificar muchos detalles de la manipulación de datos, como se hace necesario en C, por ejemplo.

Existen sistemas gestores de bases de datos relacionales que utilizan como lenguaje SQL estándar y que son sistemas de gestión de bases de datos comerciales.

Con comerciales, nos referimos a que estos sistemas gestores se venden y también existen sistemas gestores de bases de datos que son libres.

Ejemplos de sistemas gestores de bases de datos comerciales de venta:

- ORACLE.
- POSTGRE ORACLE.
- SQL SERVER
- MYSQLSERVER.
- Microsoft ACCES

Ejemplos de sistemas gestores de bases de datos libres:

- MySQL.
- Open office ACCES

Desde que se creo o se introdujo el lenguaje SQL hemos visto que ha sufrido modificaciones según las exigencias del mercado y de las situaciones de los sistemas de información actuales.

Las modificaciones mas importantes han sido las siguientes:

- SQL 89.
- SQL 99 o SQL3.
- SQL 92 o SQL2.

Modificaciones intermedias.

- SQL 92 o SQL2
- OLB-P8.
- PSM-96.
- CLI-95.

- **SQL 89.**

  Las primeras sintaxis y semánticas de SQL se dieron en 1986.

  En 1989, ANSI definió SQL 89 , que estaba basado en el anterior, la principal aportación a esta versión fue la utilización de SQL de forma interactiva o dentro de una aplicación.

- **SQL 92 o SQL2**

  En 1992 fue modificado por ANSI, en esta versión se mejoro de forma bastante considerable las versiones anteriores.

Luego nos encontramos los estándares intermedios:

- CLI-95.

  Es el estándar SQL/CLI, en el que se incorporo el estándar ODBC, para acceder desde una aplicación a una base de datos relacional, mediante la llamada de una función.

- PSM-96.

  Es el estándar SQL/PSM, se caracteriza por la incorporación de la sintaxis de la lógica de los módulos del servidor SQL.

- OLB-P8.

  Es el estándar SQL/OLB, se caracteriza por la incorporación de comando de SQL en programas Java y esta basado en el driver JCDB

- SQL 99 o SQL3.

  Gracias a esta nueva modificación en el estándar, los sistemas gestores de bases de datos orientadas a objetos y los sistemas gestores de base de datos relacionales, dejaron de estar en conflicto, ya que se mejoraron las facilidades a la hora de soportar un lenguaje de consulta declarativo, creando el lenguaje de OQL, que es muy parecido a SQL3.

  Además de mejorara las funciones ofrecidas por el sistema gestor de base de datos.

En 1999, después de estos estándares intermedios citados con anterioridad, es aceptado y publicado el SQL3 Osql:1999, gracias también a la participación de la ISO, la ISO y ANSI, conjuntamente estandarizaron SQL.

- SQL 2003.

  En el año 2003 una nueva modificación del estándar de SQL, SQL2003 , se caracterizo por la integración de archivos XML y estandariza los generadores de secuencias o valores generados, incluirá también columnas que se van a utilizar como identificadores.

Las diferencias que han surgido des el estándar SQL92 hasta el 2003, han sido sobre todo en relación con la conectividad del acceso a los datos desde una aplicación, con lo que es posible tener acceso a base de datos desde cualquier ordenador del mundo.

Modificación	Aportación
SQL 89	utilización de SQL de forma interactiva o dentro de una aplicación
SQL 92 o SQL2	mejoro de forma bastante considerable las versiones anteriores
CLI-95	incorporo el estándar ODBC
PSM-96.	incorporación de la sintaxis de la lógica de los módulos del servidor SQL
OLB-P8.	incorporación de comando de SQL en programas Java y está basado en el driver JCDB
SQL 99 o SQL3.	mejoraron las facilidades a la hora de soportar un lenguaje de consulta declarativo, creando el lenguaje de OQL
SQL 2003.	integración de archivos XML y estandariza los generadores de secuencias o valores generados

6.4. Sentencias de creación. CREATE:



La creación de una base de datos suele ser responsabilidad del administrador de la base de datos, este se encargara de que el sistema funcione correctamente, de autorizar los accesos a la base de datos, observar, organizar y conseguir los recursos necesarios tanto de hardware como de software.

Será el responsable de la seguridad o problemas de lentitud del sistema.

La creación de una base de datos consiste en la creación de las tablas que son las que la compondrán, antes de proceder a crear las tablas tenemos que crear la base de datos.

La creación de datos es la parte correspondiente al lenguaje de definición de datos DDL, este lenguaje está formado por una serie de sentencias. Una sentencia es una instrucción para que se realice una determinada acción con ciertas informaciones.

Estas sentencias se utilizaran para el desarrollo o construcción de la estructura de una base de datos, gracias a el se podrán definir en gran medida los aspectos relacionados con el nivel interno de una base de datos.

Mediante la creación de datos a través del lenguaje DDL, se podrá definir la estructura de una base de datos, creando para ello distintos objetos, además, a través del lenguaje DDL se podrá mantener dicha estructura.

Las sentencias de creación de datos DDL utiliza un verbo que ira acompañado de los diferentes objetos que se van a poder crear en una base de datos. El verbo es:

```
CREATE
```

Para eliminar se utilizara el verbo (según sea el caso).:

```
DROP
```

```
TRUNCATE
```

Para modificar se utilizara el verbo:

```
ALTER
```

Vamos a crear una base de datos tomando de ejemplo el esquema conceptual que tenemos realizado en al primera parte del tema.

6.4.1. Bases de datos



Para crear una base de datos se utiliza la sentencia **CREATE**.

Su sintaxis es la siguiente:

```
CREATE DATABASE nombre_base_de_datos
```

Esta es la sentencia más sencilla para crear una base de datos.

También está la sentencia más completa de creación de una base de datos.

La sentencia amplificada aporta además, que se le pueda asignar el tamaño que quiere que se ocupe, el tamaño máximo que podrá ocupar y el tamaño de crecimiento que podrá admitir la base de datos.

También esta sentencia podrá asignar la dirección donde quiere que se cree la base de datos.

La sintaxis es la siguiente:

```
CREATE DATABASE nombre_base_datos ON PRIMARY

( name =nombre_base_datos,

Filename=´C:\nombre_base_datos.mdt´,

Size= n MB,

Maxsize = n MB.

File growth= n MB);
```

Antes de crear la base de datos se deberá de realizar una comprobación para ver si existe otra base de datos con el mismo nombre, tanto como para borrarla o crear una nueva, como para saber que ya existe y crear otra diferente.

Si se decidiera eliminarla seria:

```
IF DB_ID(´CLIENTES´ IS NOT NULL)

BEGIN

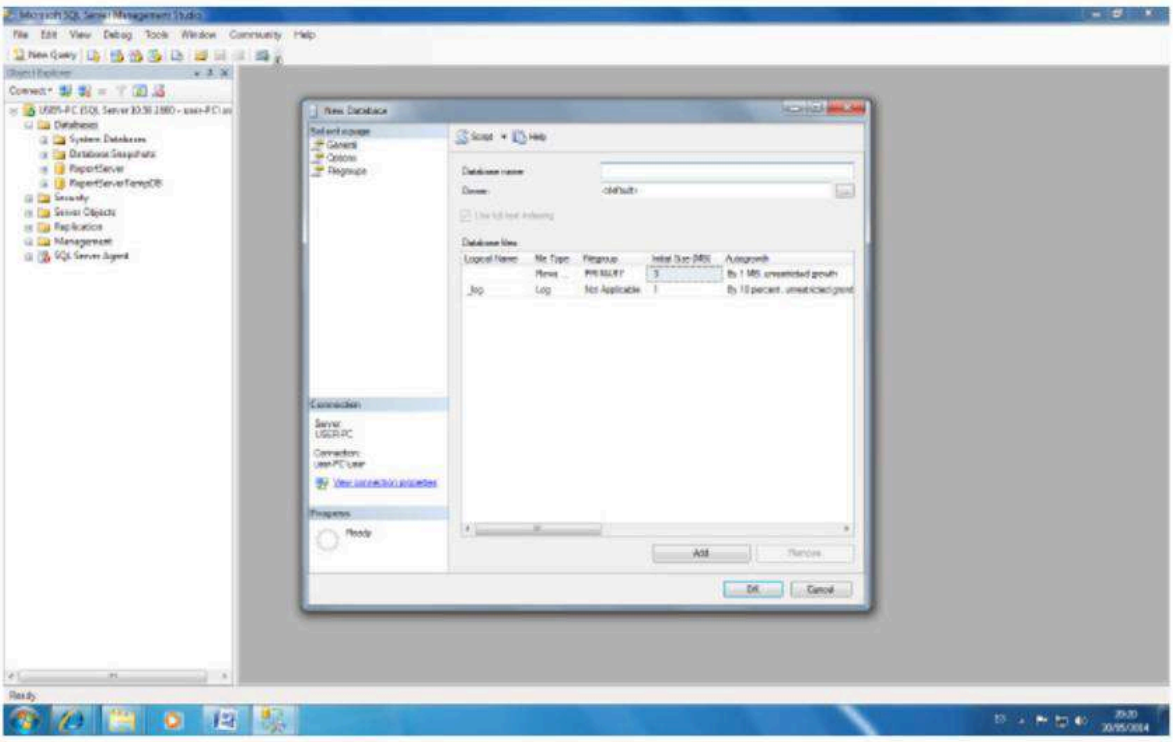
DROP DATABASE CLIENTES

END
```

En este ejemplo, si existe la base de datos **CLIENTES**, la eliminará.

Ahora podemos crear la base de datos con el ejemplo utilizado anteriormente de la gestión de la academia, del cual ya hemos obtenido el modelo E-R y lógico.A la base de datos le pondremos el nombre de **GESTION**.

Se podría crear o utilizando la forma más sencilla o asignando los volúmenes de espacio que van a ocupar.



Cear una base de datos con el admnistrador corporativo

– **Sentencia sencilla:**

```
CREATE DATABASE GESTION
```

– Sentencia completa:

```
CREATE DATABASE GESTION
(
 name =GESTION,
 Filename=´C:\GESTION.mdf´,
 Size= 6 MB,
 Maxsize = 10 MB.
 File growth= 2 MB);
```

En este ejemplo, se crea un archive principal con extensión mdf , con un tamaño inicial de 6 MB, cuando este espacio este ocupado se ampliara 2 MB y cuando se agote otros 2MB mas, así hasta completar los 10 MB.

6.4.2. Tablas

A partir de aquí, una vez que hemos creado la base de datos ya podremos crear las tablas que la componen.



Las tablas son la estructura de datos más básica en cualquier base de datos relacional. Una tabla es una colección organizada de registros (filas), todas ellas con atributos (columnas o campos).

Las columnas describen la estructura de la tabla y las restricciones de integridad de la misma, describen los datos que serán validos dentro de ella.

Cuando se crea una tabla para una base de datos, se establece una estructura de la tabla mediante la identificación de las columnas que describen los atributos de la tabla.

A parte también las columnas de una tabla tienen un tipo de datos que describe que clase de datos aceptar esa columna.



La sentencia CREATE, permitirá la creación de muchos objetos de la base de datos, como son las tablas, vistas, dominios, restricciones y procedimientos.

A parir de aquí lo que vamos a hacer es definir el esquema de la base de datos dentro del sistema gestor de base de datos.



Para realizar la creación de una tabla se utilizará la sentencia:

```
CREATE TABLE nombre tabla
```

Con esta sentencia podremos definir el nombre de una tabla, las columnas que la componen y las restricciones de integridad, que son que son las que van a asegurar que el valor que corresponda con la clave primaria cumpla las reglas de integridad referencial

La sintaxis es la siguiente:

```
CREATE TABLE nombre_tabla

(

Nombre_columna1 tipo_columna [NULL/NOT NULL] | CHECK (expresión_condicional),

[DEFAULT expresionconstante],

Nombre_columna2 tipo_columna [NULL/NOT NULL],

Hasta_columna N,

PRIMARY KEY (campo_clave));
```



Y si contuviera una clave foránea que referenciara a otra tabla se añadiría.

```
FOREING KEY (campo_clave) REFERENCES tabal2 (caompo_clave);
```

Es muy importante tener en cuenta el orden en que se crearan las tablas. No se pueden incumplir las reglas de integridad referencial.

Crearemos todas las tablas, primero se crearan las tablas que no tienen ninguna clave foránea y luego las que si las tengan, para no incumplir las reglas de integridad referencial.

- Nombre_tabla:

  El argumento nombre_tabla, corresponderá al nombre que va a recibir la nueva tabla que se va a crear.

  Los nombres de las tablas se deben de ajustar a las reglas de los identificadores, nombre_tabla solo podrá contener un máximo de 128 caracteres, excepto para los nombres de las tablas temporales locales, que no podrán superar los 116 caracteres.

- Nombre_columna1….nombre_columna n

  El argumento nombre de columna , es el nombre de una columna de la tabla. Los nombres de columna también deben de cumplir las regalas de los identificadores y deben de ser únicos en la tabla, no puede existir en una misma tabla dos columnas con el mismo nombre.

- Tipo_columna:

  El argumento tipo de datos, definirá el tipo de datos que va a contener esa columna. Se permiten los tipos de datos del sistema o los definidos por el usuario

  Es la indicación del tipo de dato que la columna contendrá. Los más utilizados por el estándar SQL son:

  - DATE, TIME y TIMESTAMP
  - CHARACTER VARYING (n)
  - INTEGER
  - SMALLINT
  - FLOAT (p)
  - DECIMAL (p,q)
  - INTERVAL
  - CHARACTER (n)



- CHARACTER VARYING (n)

  Es una cadena de caracteres de longitud variable hasta un máximo que se determina con n. Su abreviatura es VARCHAR.

- INTEGER

  Es un número entero. Su abreviatura es INT.

- SMALLINT

  Es un número entero con signo y con una precisión que no puede ser superior a INTEGER.

- FLOAT (p)

  Es un número con coma decimal móvil, el valor máximo de p dependerá del sistema gestor de base de datos utilizado. Se puede utilizar FLOAT sin p.

- DECIMAL (p,q)

  Es un número con coma decimal fija, por lo menos debe de ser p cifras y signo, con q cifras después de la coma decimal. Su abreviatura es DECIMAL (p).

- INTERVAL

  Es un periodo de tiempo (años, meses, días, horas, minutos, segundos).

- CHARACTER (n)

  Es una cadena de caracteres que fija la longitud por medio de la n. Su abreviatura es CHAR.

- DATE, TIME y TIMESTAMP

  - DATE, permite indicar el año, el mes y el día.
  - TIME, permite indicar la hora, los minutos y los segundos.
  - TIMESTAMP, es la combinación de las dos anteriores.



Clausula defecto (DEFAULT):

Lo que indica es el valor por defecto de la columna si no se le asigna ningún valor a la columna en el momento en que se crea, aunque sea el valor NULL.

La sintaxis es que hay que usar es la siguiente:

```
DEFAULT {valor | NULL}
```



Donde valor, es un valor valido para el tipo con el que la columna se ha definido.

Restricciones de integridad (PRIMARY KEY):

Para diseñar las tablas, será necesario identificar los valores validos para cada columna y decidir como se debe de exigir la integridad de los datos de la columna

Este es un vínculo de integridad que se aplica a cada columna en particular.

Con esto lo que hacemos es concretar que un grupo de valores que son válidos a introducir en una columna concreta de la tabla de una base de datos, esta integridad se comprobara por medio de la validación de los valores y el tipo de datos que se van a introducir, si va a ser numérico, alfanumérico, etc.

Existes varios tipos de integridad: Integridad de la columna, integridad de la tabla e integridad referencial.

- Tipos de restricciones de integridad de la columna.
- Tipos de restricciones de integridad de la Tablas.
- Tipos de restricción de integridad referencial.

Tipos de restricciones de integridad de la columna:

- Por defecto.
- Por validación.
- Por referencia.



- **Por defecto.-**

  Esta restricción lo que hace es asignar un valor por defecto a una columna asignándole un valor al crearla.

La cláusula y la sintaxis son:

```
DEFAULT
```

Ejemplo:

```
CREATE TABLE ALUMNO (ID_ALUMNO INT PRIMARY KEY, EDAD INT DEFAULT 18)
```

En este ejemplo la columna edad si no se introduce nada, se le asignará por defecto 18.



- **Por validación.-**

  Esta restricción lo que va a hacer es un control del valor introducido en una columna. Con lo que se permitirá o no asignar un valor a la columna en función del resultado de la expresión.

La cláusula y la sintaxis es:

```
CHECK (expresión_condicional)
```

Donde expresión_condicional, es una expresión que dará verdadero o falso.

Ejemplo:

```
CREATE TABLE ALUMNO (ID_ALUMNO INT PRIMARY KEY,

EDAD INT DEFAULT 18, SEXO VARCHAR (1) CHECK (SEXO (´F´, M´))
```

En este ejemplo vemos que se declara la columna SEXO con una longitud de uno y con CHECK le ponemos una restricción en la que esa columna solo podrá ser o F o M.



– **Por referencia.-**

Se utiliza para identificar el valor de una columna que hace referencia al valor de la columna de otra tabla diferente.

La cláusula y la sintaxis son:

```
REFERENCES nombre_tabla [(columna1, columna 2….)]
```

Ejemplo:

```
CREATE TABLE CARRERAS ( ID_CARRERA INT PRIMARY KEY NOT NULL, NOMBRE_C VARCHAR (30) UNIQUE)
```

```
CREATE TABLE ALUMNO (ID_ALUMNO INT PRIMARY KEY, EDAD INT DEFAULT 18,SEXO VARCHAR (1) CHECK ( SEXO (´F´, M´)), NOMBRE VARCHAR (30) REFERENCES CARRERAS(NOMBRE_C))
```

En este ejemplo tenemos dos tablas una tabla carreras y una tabla alumno, en la que la columna NOMBRE de la tabla alumnos debe de ser un valor contenido en la columna NOMBRE_C en la tabla CARRERAS.



Vinculos_de_tabla (Primary key, foreing key u unique):

Este es un vínculo de integridad que hace referencia a más columnas de la tabla.

Tipos de restricciones de integridad de la Tablas.

Son:

- Por clave primaria.
- Por valor único.
- Por clave foránea.

– **Por clave primaria.**

Una tabla suele tener una columna o una combinación de columnas cuyos valores identifican de forma única cada fila de la tabla. Estas columnas reciben el nombre de clave primaria o principal de la tabla y exigen la integridad de la tabla.

Ninguna columna que tenga una restricción clave primaria, puede contener el valor NULL, esta restricción además crea un índice único para las columnas de clave principal, este índice permitirá el acceso mas rápido a los datos , eta restricción además asegurara que los usuarios no introduzcan valores duplicados.

Su cláusula y sintaxis es la siguiente:

```
PRIMARY KEY (nombre_de_columna)
Ejemplo:
CREATE TABLE ESTUDIANTE(
                    DNI INT not null,
        Nombre varchar (20) not null,
        Apellidos varchar (30) not null,
                    Dirección varchar (40) not null,
      Primary key (dni));
```

En este ejemplo, hemos creado una tabla donde la columna clave es el DNI del estudiante, el dni identifica de forma única a cada uno de los estudiantes.



- **Por valor único:**

  Con esta restricción lo que se quiere conseguir es asegurar que no se escriban valores duplicados en las columnas que no formen parte de la clave primaria, y tanto esta restricción como la PRIMARY KEY exigen que los elementos sean únicos.

En una tabla se pueden definir varias restricciones de valor único, pero solo podrá existir una PRIMARY KEY y las restricciones de valor único pueden definirse en columnas que acepten valores NULL.

Su sintaxis es la siguiente:

```
UNIQUE
```

Ejemplo.

```
CREATE TABLE ESTUDIANTE(
                    DNI INT not null,
                    Nombre varchar (20) not null,
                    Apellidos varchar (30) not null,
                    Dirección varchar (40) not null,
                   EMAIL VARCHAR (20) UNIQUE not null,
                    Primary key (dni));
```

En este ejemplo, hemos añadido EMAIL, no existirán dos emails iguales en esta columna.



Tipos de restricción de integridad referencial.

- Por clave foránea:

  Una clave foránea o externa es una columna o varias columnas que se utiliza para establecer y exigir una combinación entre los datos de dos tablas.

Cuando realizamos una combinación entre tablas, se incluyen en la misma tabla o en otra tabla nueva la columna o columnas que contienen los valores de la clave principal.

Esta columna se convierte en una clave foránea o externa para la segunda tabla. este tipo de restricción puede contener valores NULL.

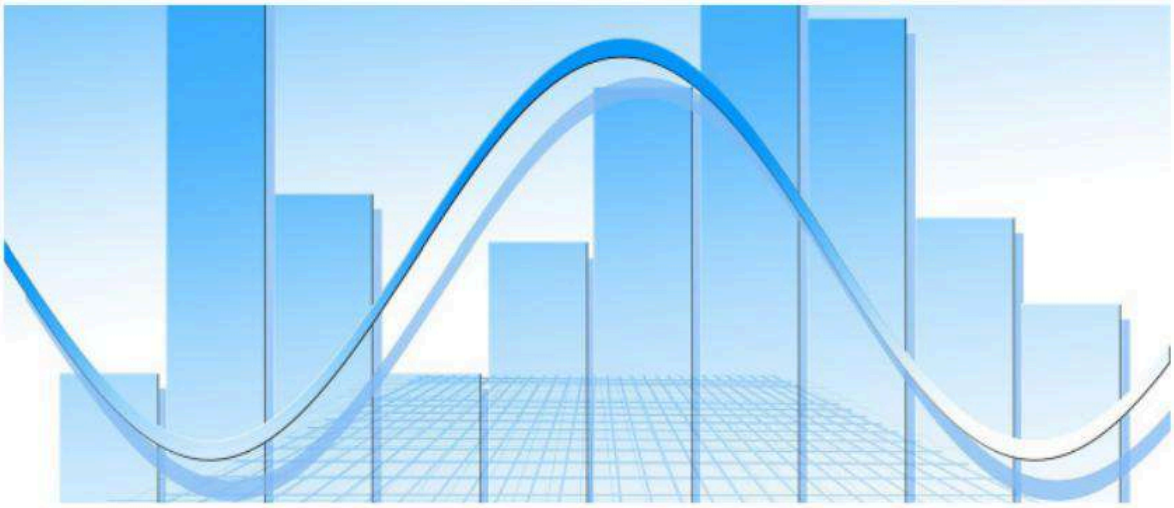
Su sintaxis es la siguiente:

```
FOREING KEY
```

Ejemplo:

```
CREATE TABLE CARRERA(
        ID _CARRERA INT not null,
        NOMBRE VARCHAR (30) not null,
        PRIMARY KEY (id_carrera));
CREATE TABLE ESTUDIANTE(
        DNI INT not null,
        Nombre varchar (20) not null,
        Apellidos varchar (30) not null,
        Dirección varchar (40) not null,
        Primary key (dni),
        FOREING KEY (ID_CARRERA) REFERENCES CARRERA(ID_CARRERA))
```

En este ejemplo, al realizar la combinación o relación de las dos tablas, da como resultado la agregación de la clave primeria de CARRERA en la tabla ESTUDIANTES como clave foránea que hace referencia a la clave primaria de CARRERA.



Recuerda

Si vamos a trabajar con diferentes bases de datos, habrá que poner nombre_nombredelabasededatos.nombre_tabla, para reconocer con que base de datos vamos a trabajar.

Si no se pone es que solo vamos a trabajar en la base de datos en la que estamos.

Así que siguiendo el esquema conceptual seguiremos creando nuestra base de datos.

Creación de tablas de la base de datos GESTION.

Tabla ACADEMIA	Tabla ALUMNO
CREATE TABLE ACADEMIA (cod_academia int NOT NULL, Nombre varchar (30), Dirección varchar (50), Teléfono int, PRIMARY KEY (cod_academia));	CREATE TABLE ALUMNO (dni_alumno varchar (9) NOT NULL, Nombre varchar (30), Apellidos varchar (50), Dirección varchar (50), PRIMARY KEY (dni_alumno));

Tabla DEPARTAMENTO	Tabla AREA
CREATE TABLE DEPARTAMENTO (cod_departamento int NOT NULL, Nombre varchar (30), PRIMARY KEY cod_departamento));	CREATE TABLE AREA (cod_area int NOT NULL, Nombre varchar (30), Cod_departamento int, PRIMARY KEY (cod_academia), FOREING KEY(cod_departamento) REFERENCES DEPARTAMENTO);

Tabla PROFESOR	Tabla CURSO
CREATE TABLE PROFESOR (dni_profesor varchar (9) NOT NULL, Nombre varchar (30), Apellidos varchar (50), Dirección varchar (50), Cod_area int, PRIMARY KEY (cod_departamento) FOREING KEY (cod_area) REFERENCES AREA;	CREATE TABLE CURSO (cod_curso int NOT NULL, Nombre varchar (30), Dni_profesor varchar (9), PRIMARY KEY (cod_curso), FOREING KEY(dni_profesor) REFERENCES PROFESOR);

Tabla MATRICULAR	Tabla LOCALIZAR
CREATE TABLE MATRICULAR (dni_alumno varchar (9) NOT NULL, Cod_curso int, PRIMARY KEY (cod_curso, dni_alumno) FOREING KEY (cod_curso) REFERENCES CURSO, FOREING KEY (dni_alumno) REFERENCES ALUMNO);	CREATE TABLE LOCALIZAR (cod_curso int NOT NULL, Cod_academia int , PRIMARY KEY (cod_curso), FOREING KEY(cod_curso) REFERENCES CURSO), FOREING KEY(cod_academia) REFERENCES ACADEMIA);

6.4.3. Vistas



Una vista es una tabla derivada de otras tablas, se trata de una tabla virtual, no tiene ninguna correspondencia a nivel físico.

Las vistas son parte del esquema externo, con ellos se podrán realizar consultas como si se tratara de una tabla normal, tenemos que tener en cuenta que si realizamos actualizaciones o modificaciones en una tabla, afectará a la tabla original, esto dependerá si el o los usuarios que trabajan con ellas tengan permiso para hacerlo o no.

Una vista se utiliza para crear una tabla con datos de otras tablas, estos datos serán los que al usuario les interese consulta.

Las vista pueden consultar información derivada de la unión de varias tablas, información derivada de realizar una agrupación de filas, en general se utilizan para que nos de información derivada de consultas que son mas complejas y a las cuales se acceden con frecuencia.

También las vistas pueden actuar como mecanismos de seguridad, ya que se pueden crear vistas solo con los atributos de las tablas a los cuales se le permite acceder por medio de permisos del administrador a determinados usuarios.

La consulta que defina la vista puede ser tanto una sola tabla como de varias, pueden ser también tablas de la propia base de datos que se este utilizando o de otras bases de datos.

Es posible realizar consultas distribuidas para, realizar vistas de datos de diferentes bases de datos.

No existen restricciones restricciones para consultar vistas y pocas si se van a modificar los datos de la vista.

Las vistas se utilizan sobre todo para limitar el acceso del usuario a filas o columnas concretas de una tabla, para combinar columnas de varias tablas de manera que aparezcan en una sola tabla.

Las vistas se utilizan como sistema de seguridad ya que periten al usuario acceder a los datos por media de esta, pero no tienen permiso de acceso directo a las tablas base.

La sentencia es:

```
CREATE VIEW
```

La sintaxis completa es la siguiente:

```
CREATE VIEW nombre-vista

AS sentencia_select.

Ejemplo:

        CREATE VIEW VISTA_ESTUDIANTES

         AS

        SELECT DNI,NOMBRE,APELLIDOS

        FROM ALUMNOS.
```

- **CREATE VIEW**, es la sentencia para crear una vista.
- **Nombre_vista** es el nombre que se le va a dar a la vista que se va a crear.
- **La sentencia_select,** es una consulta realizada por medio de la sentencia SELECT.



Las vistas suelen contener los operadores de Unión, intersección, etc., el operador DISTINCT, las funciones agregadas y la clausula GROUP BY.

En las vistas que son actualizables, el sistema traducirá la actualización o modificación sobre la vista en una operación de actualización sobre la tabla base, esto se producirá si no se viola ninguna restricción de integridad sobre la tabla original.



Una modificación realizada con una vista, solo podrá modificar una de las tablas originales involucradas.

Cuando se ha creado ya la vista, se puede consultar los datos como si se tratara de una tabla.

6.4.4. Tiggers o disparadores



Un disparador o también llamado trigger, es una clase especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en la base de datos.

Son un conjunto de reglas que especifican acciones que son activadas automáticamente cuando se produce un evento.

Un evento puede ser :

- CREATE.
- ALTER.
- DROP.
- INSERT.
- DELETE.
- UPDATE.

SQL distingue dos tipos de trigger:

- Trigger DDL.
- Trigger DML.



- **Trigger DDL.**

  Estos se ejecutaran cuando un usuario realice un evento mediante el lenguaje DDL. Los eventos DDL son:

  - CREATE.
  - ALTER.
  - DROP.

- **Trigger DML.**

  Estos se ejecutaran cuando un usuario realice un evento mediante el lenguaje DML. Los eventos DML son:

  - INSERT.
  - DELETE.
  - UPDATE.

Los trigger DDL se ejecutan cuando un usuario utiliza algún evento del lenguaje DDL. Estos son utilizados para comprobar las restricciones de integridad, restauración de la consistencia, para definir las reglas de funcionamiento de una organización o empresa y para mantener la información derivada.

La sentencia para crear un trigger es el siguiente:

```
CREATE TRIGGER
```

La sintaxis completa de esta sentencia es:

```
CREATE TRIGGER nombre_trigger
[BEFORE | AFTER] evento
[granularidad_evento]
[WHEN condición]
BEGIN
Cuerpo_trigger
END nombre_trigger
```



- Con la sentencia **CREATE TRIGGER nombre_trigger**, se creará un disparador con un nombre concreto.
- **BEFORE | AFTER.**

  se pueden tomar estos valores para indicar si el cuerpo del trigger debe de ejecutar antes o después del evento que causa su activación. Estos dos valores se pueden aplicar tanto en las filas como en el orden.

  - BEFORE. Antes.
  - AFTER. Después.

Estas deben de esta acompañando a una operación que cause su activación, estas serán **INSERT**, **DROP** o **UPDATE**. Y se pueden poner mas operaciones añadiendo el operador **OR** y **AND**.

– Granularidad del evento [WHEN].

Este permitirá distinguir si el disparador es para realizarlo en filas o por orden.

- Por filas.

Si es por fila el cuerpo del disparador se realizara fila a fila de la tabla a que se le realice. Este se realizará por medio de la cláusula:

FOR EACH ROW

Esta clausula acompañado de una condición, que debe de cumplir cada una de las filas de una tabla. Esta condición se realiza por medio de la clausula :

WHEN

- Por orden.

Si no se pone nada en la sentencia, se realizara por orden. Si es por orden, se debe de aplicar el cuerpo del disparador a toda la tabla a la vez.

En este caso en la cláusula **WHEN**, se debe de hacer referencia a **NEW** y **OLD**.

– **Con old**. Hacemos referencia al valor de una columna antes de una operación, **UPDATE**, **DELETE** o **NULL**.

– **Con NEW**. Hacemos referencia al valor de una nueva columna después de una operación **INSERT**, **UPDATE** y **DELETE**.



OLD y **NEW** solo se podrán utilizar cuando el trigger se ha definido por filas.



- **BEGIN cuerpo_trigger END.**

  Dentro del cuerpo del trigger podemos poner sentencias de borrado, inserción o modificación y cualquier sentencia SELECT.

6.4.5. Procedimientos almacenados



Un procedimiento almacenado está formado por una serie de instrucciones SQL, que servirán para definir un determinado proceso, este podrá ser ejecutado directamente en el servidor de Base de datos, permitiendo la ejecución de una o varias acciones.

Acepta parámetros de entrada y devuelve un valor o conjunto de resultados.

Pueden tanto recibir como devolver información, a este es a lo que nos referimos con que un procedimiento almacenado emplea parámetros de entrada y salida.

Los procedimientos se utilizan también para obtener por medio de instrucciones condicionales y cálculos más complejos, información de varias tablas.

Un procedimiento almacenado nos aporta simplificación a la hora de ejecutar tareas que son repetitivas, son mucho mas rápidos que ejecutar y elaborar las instrucciones una y otra vez.

Estas son capaces de identificar errores antes de que ellos se ejecuten. Al meter muchas instrucciones en un procedimiento, se ejecutaran todas de una vez con lo que reduciremos el trafico a través de la red.

A través de un procedimiento almacenado podremos almacenar y modificar datos, crear tablas temporales y permanentes, realizar llamadas a otros procedimientos y a hacer referencia a otros objetos de una base de datos.



Otro uso de un procedimiento es la validación de datos que integran la estructura del banco de datos, a estos procesos almacenados se les denomina disparadores o tiggers.

Existen varios tipos de procedimientos que ya vimos en el tema anterior.

La sintaxis para crear unos procedimientos almacenados es:

```
CREATE PROCEDURE nombre_procedimiento [ @ parametro1 <tipo>, …]
AS
[sentencias_del_procedimiento];
```

Ejemplo:

```
CREATE PROCEDURE inserta_cliente @ nombre varchar (50);
    @ primer_apellido varchar (100);
        @ segundo_apellido varchar (100);
    @ dni varchar (20);
AS
INSERT INTO CLIENTES
Nombre, primer_apellido, segundo_apellido, dni) VALUES
(@nombre. @ primer_apellido,@segundo_apellido,@dni);
```

En este ejemplo se va a crear un procedimiento llamado insertar_cliente, en el cual vamos a hacer un procedimiento que al ser llamado insertara un cliente nuevo en la tabla clientes.

6.5. Sentencia de modificación: ALTER

Las sentencias del lenguaje DDL para poder cambiar o modificar el contenido o datos de un registro de una tabla de una base de datos es la sentencia:

UPDATE

Este verbo siempre irá acompañado del nombre de una tabla, mediante esta se podrá realizar modificaciones.

La sentencia UPDATE se utiliza y es de gran utilidad cuando se requiere modificar un número alto de registros y a la vez se encuentran localizados en varias tablas.

Mediante esta sentencia se podrán modificar varios campos simultáneamente.

Las filas que se van a modificar o cambiar tendrán que ser concretadas mediante la utilización de una condición de búsqueda ya que por sí sola la sentencia UPDATE solo podrá funcionar sobre una única tabla.

La única sentencia que puede operar por si misma con múltiples tablas a la vez es la sentencia:

SELECT

En muchas ocasiones puede ser necesario cambiar el valor dentro de una fila, sin que esto produzca un cambio en todos los valores de esta, para estas ocasiones o casos es cuando se utilizara la sentencia UPDATE.

6.5.1. base de datos.



Si después de haber creado una base de datos, queremos realizar alguna modificación sin tener que eliminar toda la base de datos, se utiliza la sentencia:

ALTER DATABASE



Para poder ejecutar esta sentencia se necesitará tener permiso para modificar la base de datos. Se podrá modificar de forma sencilla o de forma más completa, al igual que vimos en la creación.

Si es de la forma sencilla la sintaxis seria:

```
ALTER DATABASE nombre_base_datos

Modificaciones.
```

Ejemplo:

```
ALTER DATABASE UNIVERSIDAD

MODIFI NAME = ESTUDIANTES
```

Si es de la forma completa seria:

```
ALTER DATABASE nombre_base_datos ON PRIMARY

( name =nombre_base_datos,

Filename=´C:\nombre_base_datos.mdt´,

Size= n MB,

Maxsize = n MB.

File growth= n MB);
```

Se podrá cambiar el nombre de la base de datos, cambiar el tamaño y el tamaño máximo.

Ejemplo:

```
ALTER DATABASE GESTION
(
 name =ACADEMIA,
 Filename=´C:\GESTION.mdf´,
 Size= 8 MB,
 Maxsize = 12 MB.
 File growth= 1 MB);
```

En este ejemplo hemos entrado en la base de datos y le hemos cambiado el nombre y el tamaño, si lo comparamos con el ejemplo que utilizamos en la creación, es el mismo, pero con todos sus valores modificados inclusive el nombre.

6.5.2. Tablas



Cuando queremos modificar una tabla con datos que forman parte de una base de datos, se utiliza la sentencia:

```
ALTER TABLE
```

Una tabla puede ser modificada tanto añadiendo un atributo, modificándolo o eliminando atributos como restricciones.

Si vas a modificar atributos:

- **Para añadir atributo.**

  Este se añadirá al final de todos los atributos ya existentes.

Se utiliza la siguiente sintaxis:

```
ALTER TABLE nombre_tabla

ADD definición_atributos;
```

Ejemplo:

```
ALTER TABLE ESTUDIANTES

ADD DIRECCION VARCHAR (40);
```



Este campo se añadirá a la tabla ,en las filas ya existentes aparecerá el valor de esa columna como nulo y si utilizamos la clausula DEFAULT aparecerá el valor por defecto.

- **Para modificar el atributo ya existente.**

  En este caso solo se podrá modificar el tipo de dato que contiene , pero no se podrá modificar el nombre.

Se utiliza la siguiente sintaxis:

```
ALTER TABLE nombre_tabla

    MODIFI DRECCCION VARCHAR (50);
```

En este ejemplo, anteriormente dirección estaba definida con una longitud de 40, ahora lo hemos modificado para que contenga 10 caracteres mas, nunca se podrá modificar la longitud a menos que existan ya columnas que tienen valores, si se quiere que admita valores nulos o por defecto DEFAULT NULL tendrá que añadirse al realizar la modificación.

Por ejemplo:

```
MODIFI DIRECCION VARCHAR (50) DEFAULT NULL o NULL.
```

6.5.3. Vistas



Si se quiere modificar una vista se utiliza la sentencia

```
ALTER VIEW
```

Es muy parecido a la forma de hacerlo en las tablas. Se podrán añadir o eliminar algún campo se realiza en las tablas.

La sintaxis es la siguiente:

```
ALTER VIEW nombre_vista

AS

( sentencia_select);
```

Ejemplo:

```
ALTER VIEW VISTA_ESTUDIANTES

AS SELECT DNI, NOMBRE, APELLIDOS, DIRECCION

FROM ALUMNOS.
```

En este ejemplo, hemos añadido a la vista, un campo más que seria DIRECCION.

Las vistas no contiene información por si mismas, si no que obtienen la información de otras tablas y los refleja.

Si se eliminara una tabla las vistas que estén asociadas a esta tabla se eliminaran también.

Sobre una vista se podrán realizar las mismas operaciones que se realizan con las tablas, menos en algunos casos.

Para actualizar una vista de una sola tabla, se podrán modificar las filas de la vista. Si se modifica una vista se cambiara la tabla sobre la que se basa.

Podrán modificar filas en una tabla a través de una vista-

6.5.4. Disparadores o triggers

Los triggers se tratan de un mecanismo que posee SQL, este se basa en una especialidad de los procedimientos almacenados que se activara en cuanto se produzca la ejecución de alguna sentencia que implique la modificación de datos o estructuras.



Para modificar un desencadenador como es en el caso de los procedimientos almacenados, se deberá de utilizar la sentencia:

```
ALTER TRIGGER
```

La sintaxis completa es:

```
ALTER TRIGGER nombre_trigger [ENABLE|DESABLE]
```

Para habilitar o deshabilitar un trigger o disparador se utiliza la opcion:

- ENABLE.
- DESABLE

- **ENABLE**, es para concretar si se va a habilitar el trigger.
- **DISABLE**, es para concretar si se va a deshabilitar el trigger

También se podrán deshabitar o habilitar los triggers de una tabla.

Se realizara por medio de la siguiente sentencia:

```
ALTER TRIGGER nombre_trigger [ENABLE|DESABLE] ALL TRIGGERS
```

6.5.5. Procedimientos

En ciertas ocasiones los procedimientos almacenados pueden ser modificados debido a que se han realizado cambios en la estructura de las tablas de una base de datos.



Para realizar la modificación de un procedimiento almacenado que ya existe y que se quiere preservar los permisos, se utilizara la sentencia:

```
ALTER PROCEDURE
```

La sintaxis seria la siguiente:

```
ALTER PROCEDURE <nombre_procedimiento> [ “parametro1 <tipo>,..]

AS

[sentencias_del_procedimiento]
```

A través de esta sentencia se podrá cambiar las características de un procedimiento o función almacenada.

Se debe de tener en cuenta que es mejor no modificar ningún procedimiento almacenado del sistema ya que se pueden producir fallos sobre el sistema.

Para poder utilizar un procedimiento almacenado del sistema siempre será mejor crear un procedimiento almacenado por el usuario y copiar las instrucciones del procedimiento del sistema y asi se podrá adaptar a las necesidades de los usuarios

6.6. Sentencias de borrado: DROP, TRUNCATE



Cuando se utiliza una base de datos, además de realizar operaciones de creación, modificación y consulta, también podemos realizar la operación de borrado.

Existen varios tipos de borrado:

- De todos los datos de una tabla.
- De filas que cumplan una series de condiciones determinadas.
- De toda la tabla completa, incluido la estructura de la misma.

Estas sentencias son tres:

- **DELETE.**

  Borrara un determinado numero de filas de una tabla, utilizando para ello la clausula WHERE se podrá limitar el numero de filas a borrar según una condición.

  Su sintaxis es la siguiente:

```
DELETE FROM nombre_tabla WHERE condición.
```

- **TRUNCATE.**

  Borrara todas las filas de una tabla pero no su estructura, por lo que se podrán insertar datos nuevos.

  Su sintaxis es la siguiente:

```
TRUNCATE TABLE nombre_tabla;
```

- **DROP.**

  Eliminara por completo la tabla, desaparecerá de la base de datos tanto su estructura como el contenido de la tabla.

  Su sintaxis es la siguiente:

```
DROP TABLE nombre_tabla;
```

6.6.1. Bases de datos.



Para borrar por completo una base de datos se utiliza la sentencia DROP. Es conveniente preguntar con un IF para evitar eliminar la base de datos equivocada.

La sintaxis completa sería la siguiente:

```
DROP DATABASE [IF EXIST] nombre_base_datos;
```

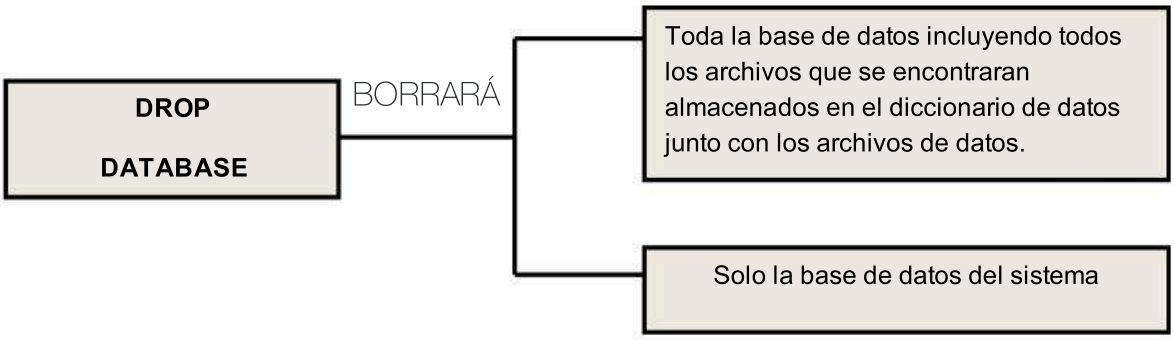
Se podrán eliminar varias bases de datos en una sola instrucción DROP

Con DROP DATABASE se podrán borrar todas las tablas de una base de datos y la base de datos en sí. A la hora de utilizar este comando se debe de tener especial cuidado ya que se puede eliminar una base de datos por error.

Para que no cualquier usuario pueda realizar esta acción, se proporcionara permisos a los usuarios que puedan realizarlo.

Cuando se utiliza esta sentencia se devuelve el número de tablas que se eliminara.

La sentencia DROP DATABASE borrara todo el directorio con los ficheros creados de la base de datos. Mediante la sentencia se podrá eliminar toda la base de datos incluyendo todos los archivos que se encontraran almacenados en el diccionario de datos junto con los archivos de datos, aunque también dará la oportunidad de eliminar solo la base de datos del sistema pero manteniendo los archivos en el diccionario y los archivos de datos intactos.



6.6.2. Tablas.



Para eliminar una tabla se podrá utilizar tanto la sentencia **DROP** como la sentencia **TRUNCATE**.

Las sintaxis de DROP seria la siguiente:

```
DROP TABLE nombre_tabla [RESTRICT|CASCADE]
```

Utilizando la sentencia DROP eliminaremos todos los datos de una tabla ,si ponemos:

- **RESTRICT**, los datos de la tabla no se borraran si está referenciada, como puede ser el caso de una vista.
- **CASCADE**, todos los datos de una tabla se borraran y todos los datos que haga referencia a la tabla se borraran también.

Para eliminar una tabla también podemos utilizar **TRUNCATE**. La sintaxis seria la siguiente:

```
TRUNCATE TABLE nombre_tabla;
```

En este caso se eliminara por completo tanto la estructura como los datos de la tabla, desaparecerá por completo de la base de datos.

Esta sentencia no se puede utilizar en los siguientes casos:

- Tablas a las que se hace referencia mediante una restricción FOREIGN KEY. (Puede truncar una tabla que tenga una clave externa que haga referencia a sí misma).
- Tablas que participan en una vista indexada.

6.6.3. Vistas.



Para eliminar una vista de la base de datos, se realiza a través de la sentencia:

```
DROP VIEW
```

La sentencia completa seria:

```
DROP VIEW nombre_vista;
```

Si se elimina una vista eliminamos su definición y los permisos que estén asociados a ella. Se pueden quitar varias vistas a la vez, separándolas por comas.

Para ejecutar la sentencia **DROP VIEW**, se necesitara permiso ALTER o permiso para poder controlar ese objeto.

Si eliminamos una tabla mediante **DROP TABLE**, se podrán eliminar con DROP VIEW las vistas que estén basadas en esa tabla, no se quitaran por si solas.

Las ventajas y desventajas de utilizar las vistas son las siguientes.

Ventajas:

- Para realizar consultas en las que selección es compleja y con estos se simplifican.
- A través de ellas se van a poder personalizar una base de datos para distintos usuarios, ya que la presentación de la información se hará con una estructura lógica para estos.
- Proporcionaran un control en el acceso de la base de datos, con ello se conseguirá que cada usuario maneje una información determinada.

Desventajas:

- Las restricciones propias a las que están sujetas por las modificaciones.
- Disminución del rendimiento de la base de datos al crear vistas con selecciones complejas.

6.6.4. Disparadores o triggers



Para eliminar un trigger o disparador utilizaremos la sentencia:

DROP TRIGGER

La sintaxis es la siguiente:

```
DROP TRIGGER nombre_trigger;
```

Se podrán eliminar varios triggers a la vez, si ponemos los nombres de los triggers separados por comas.

Los usos de un trigger son:

- Un trigger se utiliza para registrar y ayudar cuando se producen cambios en los datos.
- Se podrá utilizar para validar datos, cambiar o negar operaciones como INSERT, UPDATE y DELETE en una tabla.
- Permitirá conservar y proteger la consistencia de los datos.

Las ventajas que ofrecen los trigger son:

- Proporciona seguridad de los datos.
- Proporciona integridad de los datos.

Las desventajas son:

- Definir previamente las tareas que va a realizar.
- Se debe de programar para cada sistema gestor de base de datos.
- No se pueden utilizar con asiduidad.
- No se puede utilizar en las tablas temporales.

6.6.5. Procedimiento



Para eliminar un procedimiento almacenado se utilizará la sentencia:

```
DROP PROCEDURE
```

La sintaxis completa seria:

```
DROP PROCEDURE nombre_procedimiento;
```

En este caso también se podrán eliminar varios procedimientos almacenados a la vez, se podrán separados por comas unos de otros.

UD6
Lo más importante

- **Un sistema gestor de base de datos** está formado por niveles y esquemas para trabajar sobre una base de datos, los SGBD deben de conocer la estructura de esta, que entidades va a tener, que tipo de datos va a contener las entidades, que atributos definen a la entidad, las relaciones entre ellas, etc.

- **Una base de datos está compuesta** por tablas, columnas y tipos de datos.

  Las tablas son la estructura básica de cualquier base de datos relacional. Una tabla es una colección organizada de registros que serán las filas de la tabla, y de los atributos que serán las columnas.

- **El esquema es un elemento**muy importante de la estructura de un sistema gestor de base de datos, es lo que va a servir para separar el sistema gestor de base de datos de la base de datos, con esta separación lo que se intenta conseguir es que, se puedan realizar en el esquema de la base de datos sin tener que realizar ninguna modificación en el sistema gestor de base de datos.

- ANSI recomendó que la estructura de los sistemas gestores de bases de datos tuvieran tres niveles para describir la base de datos, dividiendo el nivel lógico en dos, uno seria el nivel externo y otro el nivel conceptual, el nivel interno es el nivel físico.

- Este modelo es el más utilizado y extendido por los desarrolladores para el análisis y diseño de los datos es el modelo relacional

- **SQL** es un lenguaje estandarizado por ANSI/ISO, de acceso a base de datos relacionales, por medio de este podremos realizar operaciones de definición, manipulación y gestión de control de las bases de datos relacionales.

- **El lenguaje de definición DDL**. Está compuesto por las sentencias que se utilizan para crear los objetos dentro de la base de datos y modificar propiedades y atributos de la propia base de datos. Las instrucciones básicas de SQL DDL son: **CREATE, ALTER y DROP.**

- **Lenguaje DML**. Se utiliza para leer y actualizar datos de la base de datos, los usuarios de la base de datos podrán realizar por medio de una serie de sentencias, consultas, añadir, eliminar y modificar datos de la base de datos.

- **Lenguaje de cuarta generación**. Se define lenguaje de cuarta generación como un conjunto de herramientas prefabricas, que consiguen dar un lenguaje de programación de alto nivel.

- **Generador de formularios:**

  Esta herramienta es utilizada para crear formularios, con los que se podrán introducir o visualizar datos.

- **Generador gráfico.** Se trata de una herramienta que se utiliza para conseguir visualizar en un gráfico los datos de una base de datos por medio de un gráfico en el que se podrán ver preferencias y relaciones entre datos.

- **Generador de aplicaciones**. Esta herramienta es utilizada para crear programas que hagan de intermediario entre la base de datos y el usuario.

- **Generador de informe:**

  Esta herramienta se utiliza para la creación de un informe de los datos de una base de datos.

- Las **sentencias** son elementos básicos en los que se divide el código de un lenguaje de programación.

- La creación de una base de datos consiste en la creación de las tablas que son las que la compondrán, antes de proceder a crear las tablas tenemos que crear la base de datos.

  Para crear una base de datos se utiliza la sentencia **CREATE**.

- Para realizar la creación de una tabla se utilizará la sentencia: **CREATE TABLE**.

- Una **vista** es una tabla derivada de otras tablas, se trata de una tabla virtual, no tiene ninguna correspondencia a nivel físico.

- Un disparador o también llamado **trigger**, es una clase especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en la base de datos.

- **Trigger DDL**. Estos se ejecutaran cuando un usuario realice un evento mediante el lenguaje DDL. Los eventos

- **Trigger DML**. Estos se ejecutaran cuando un usuario realice un evento mediante el lenguaje DML. Los eventos

- Un **procedimiento almacenado** está formado por una serie de instrucciones SQL, que servirán para definir un determinado proceso, este podrá ser ejecutado directamente en el servidor de Base de datos, permitiendo la ejecución de una o varias acciones.

- La sentencia para crear unos procedimientos almacenados es: **CREATE PROCEDURE.**

- **DELETE**. Borrara un determinado número de filas de una tabla, utilizando para ello la clausula WHERE se podrá limitar el número de filas a borrar según una condición.

- **TRUNCATE**. Borrara todas las filas de una tabla pero no su estructura, por lo que se podrán insertar datos nuevos.

- **DROP**. Eliminara por completo la tabla, desaparecerá de la base de datos tanto su estructura como el contenido de la tabla.

UD6
Autoevaluación

1. ¿De que está compuesta una base de datos?

    a. Por tablas, columnas y tipos de datos.
    b. Por tablas y columnas.
    c. Por tablas.
    d. Ninguna de las anteriores.

2. ANSI recomendó que la estructura de los sistemas gestores de bases de datos tuvieran tres niveles, ¿Cuáles son?

    a. Nivel intermedio, nivel pragmático y nivel interno.
    b. Nivel conceptual, nivel externo y nivel interno o físico.
    c. Esquema conceptual, esquema externo y esquema físico.
    d. B y c son correctas.

3. ¿Cuál es el modelo más utilizado y extendido por los desarrolladores para el diseño y análisis de datos?

    a. Entidad.
    b. Entidad-relación.
    c. Relación.
    d. Modelo orientado a objeto.

4. El lenguaje de datos DDL es un lenguaje:

    a. Proporcionado por el sistema gestor de base de datos, lo que va a hacer es permitir que los usuarios puedan realizar consultas de datos.
    b. Proporcionado por el sistema gestor de base de datos, lo que va a hacer es permitir que los usuarios puedan realizar inserciones de datos.
    c. Proporcionado por el sistema gestor de base de datos, lo que va a hacer es permitir que los usuarios puedan realizar borrados de datos.
    d. Proporcionado por el sistema gestor de base de datos, lo que va a hacer es permitir que los usuarios puedan crear básicamente estructuras de datos.

5. ¿Cuáles son las sentencias que componen el lenguaje de definición de datos DDL?.

    a. CREATE y ALTER.
    b. CREATE, ALTER y DROP.
    c. CREATE y DROP.
    d. CREATE, ALTER, DROP y TRUNCATE.

6. ¿Cuál es la modificación estándar que incorporo el estándar ODCB?

    a. CLI-95.
    b. PSM-96.
    c. OLB-98.
    d. SQL 99 o SQL3.

7. La sentencia CREATE permitirá la creación de :

    a. Tablas, columnas y restricciones.
    b. Tablas, dominios, restricciones y procedimientos.
    c. Dominios y restricciones.
    d. Tablas, dominios y procedimientos.

8. ¿Qué hace la restricción de integridad de columna por defecto DEFAULT?.

    a. Asignar un valor a la columna en función del resultado de la expresión.
    b. No se le asignara ningún valor a la columna.
    c. Asignar el valor de una columna que hace referencia al valor de una columna de otra tabla diferente.
    d. Asignar un valor por defecto a una columna asignándole un valor al crearla.

9. ¿Qué es una vista?

    a. Es un procedimiento almacenado.
    b. Es una tabla derivada de otras tablas, se trata de una tabla virtual, no tiene ninguna correspondencia física.
    c. Es una clase especial de procedimiento almacenado.
    d. Es una tabla normal de una base de datos.

10. Se distinguen dos tipos de trigger:

    a. Trigger DCL y UML.
    b. Trigger DML y DCL.
    c. Trigger DML y DDL.
    d. Trigger DDL y DCL.

Área: informática y comunicaciones

UD7

Manipulacion de los datos

7.1. Lenguaje de manipulación de datos DML

7.2. Consulta de datos SELECT.

7.3. Inserción de datos INSERT

7.4. Modificación de datos UPDATE.

7.5. Eliminación de datos DELETE.

7.6. Agregación de conjuntos de datos para consulta: JOIN, UNION

7.7. Subconsultas.

7.1. Lenguaje de manipulación de datos DML



El lenguaje de manipulación de datos es uno de los lenguajes proporcionado por el lenguaje SQL que forma parte del sistema gestor de base de datos, mediante este lenguaje se permitirá que el usuario o usuarios puedan realizar operaciones de consulta, modificación o eliminación de los datos que están contenido en una base de datos relacional.

Dentro del lenguaje SQL se encuentra el lenguaje DML, que se utiliza para tratar la información, que se encuentra en una base de datos.Las sentencias del lenguaje de modificación de datos DML, se usan en la interrogación y manipulación de datos en esquemas de bases de datos ya existentes.

Para probar estas sentencias, puedes pinchar aquí y ver los requisitos necesarios a instalar SQL server y los pasos a seguir para instalarlo.

> *Para más información, consulta "instalación Sistema gestor de base de datos SQL Server " en los anexos al final del libro.*

Los elementos que conforman este lenguaje son las sentencias:

- SELECT.
- INSERT.
- UPDATE.
- DELETE.

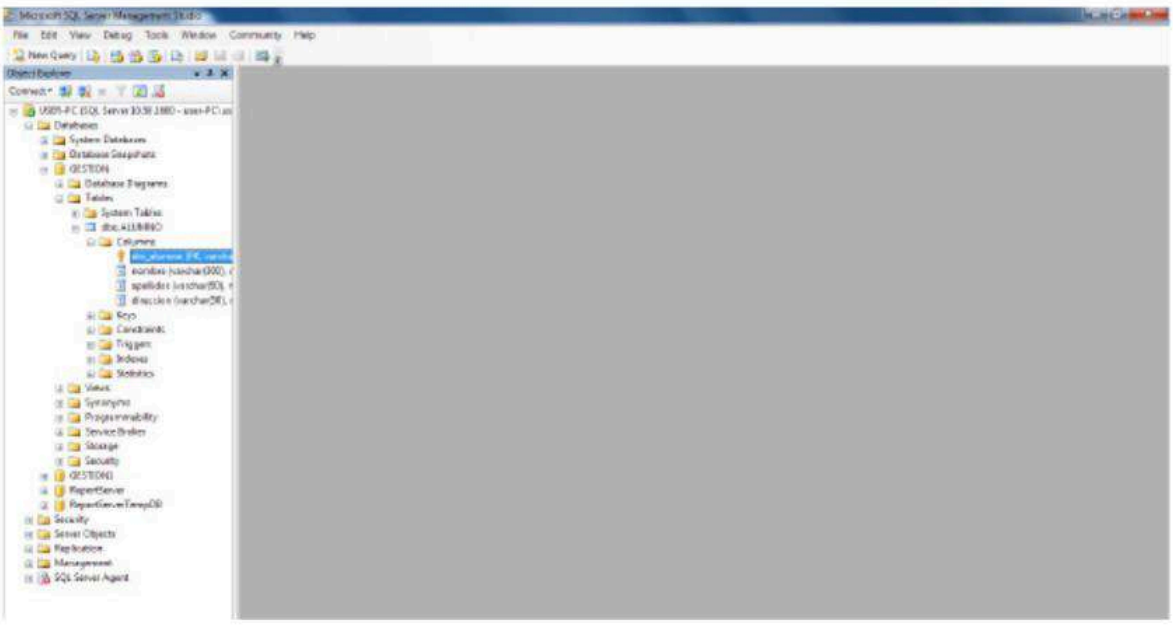


Imagen del programa SQL server 2008

El lenguaje DML se divide en dos grupos:

Lenguajes de consulta procedimentales.	Lenguajes de consulta no procedimentales.

– El lenguaje de consulta procedimental.

  Es en el que los usuarios dan las instrucciones al sistema para que realice una serie de operaciones sobre la base de datos, que dará el resultado buscado.

– El lenguaje de consulta no procedimental

  Es en el que los usuarios hacen una descripción de la información que necesiten sin decidir un procedimiento concreto para conseguir la información.

7.2. Consulta de datos SELECT.

Dentro del lenguaje SQL encontramos un lenguaje de consulta, que nos permitirá basicamnete obtener datos de tablas y especificar la forma en que se presentan.



En primer lugar vamos a ver una consulta simple. Para realizar una consulta se utiliza la sentencia:

```
SELECT
```

Esta sentencia es muy importante y amplia en sus cláusulas, por lo que permitirá realizar consultas bastante complejas sobre una base de datos.

Todas las operaciones de algebra relacional en la que se basa SQL, se pueden realizar mediante la sentencia **SELECT**.

La sentencia **SELECT** lo que hace es recuperar filas de una base de datos y también realiza la selección de una o varias filas o columnas, de una o varias tablas.

La sintaxis completa de la sentencia **SELECT** es bastante compleja y existirán cláusulas que no tienen porque ser obligatorias.

La sintaxis completa es la siguiente:

```
SELECT <nombre/es_columna/as>
INTO <nueva_tabla>
FROM tabla_origen
WHERE <condición_busqueda>
GROUP BY <nombres_columnas>
HAVING  <condición_busqueda_para_GROUP BY>
ORDER BY <nombres_columnas> <ASC| DESC>
```

Las siguientes clausulas son opcionales.
INTO
WHERE
GROUP BY
HAVING
ORDER BY

A continuación vamos a ver detenidamente la sentencia SELECT y cada una de las cláusulas que la pueden acompañar.



La sentencia **SELECT**, nos permitirá realizar consultas de datos sobre cualquier tabla de la base de datos. Nos dará como resultado el listado de las columnas de las tablas seleccionadas.

Su sintaxis es la siguiente:

```
SELECT [DISTINCT | ALL] nombre_columna/s

FROM tabla_consultar
```

Se puede especificar mediante la palabra **DISTINCT** como quiere que sea EL **SELECT**, si se pone esta palabra nos mostrará las filas resultantes sin repeticiones.

Si se pone **ALL**, aparecerán todas y si no se pone actuará como **ALL**.

```
SELECT dni,nombre, apellidos, dirección, edad;
```

En este ejemplo, lo que estamos haciendo es una selección de las columnas que nos interesa de una tabla llamada ALUMNOS.



Existen una serie de clausulas que pueden formar parte de la sentencia SELECT.Una cláusula son condiciones para filtrar y manipular los datos de una o varias tablas.

Las clausulas que se pueden combinar junto a la sentencia SELECT son seis:

- INTO.
- FROM.
- WHERE.
- GROUP BY.
- HAVING.
- ORDER BY.

- **La clausula INTO.**

  Esta cláusula es opcional, que lo que quiere decir es que, se va a crear una nueva tabla con las filas resultantes de la consulta realizada.

  Seria INTO y nombre de la nueva tabla que le va a dar el usuario.

```
INTO <nombr_nueva_tablas>
```

El argumento nueva_tabla especifica el nombre de la nueva tabla que se va a crear, basada en las columnas de la lista de selección SELECT y en las filas seleccionadas por la clausula WHERE.

Las columnas de nueva_tabla se crearan el el orden que especifique la lista de selección SELECT. Cada columna de la nueva_tabla tiene el mismo nombre, tipo de datos y valor correspondiente a los de la lista de selección.

El usuario que ejecuta una instrucción SELECT con la clausula INTO debe de tener permiso CREATE TABLE en la base de datos en la que este trabajando.

Ejemplo:

```
INTO D_ALUMNOS;
```

En este ejemplo lo que estamos haciendo es crear una nueva tabla a la que le hemos dado el nombre de D_ALUMNOS, y en la que aparecerán los resultados finales de los datos de la consulta.

- **La cláusula FROM.**

  Esta cláusula se utiliza para hacer referencia a la tabla o tablas sobre la que se va a realizar la consulta y de donde obtendremos las filas resultantes de la consulta. Esta clausula no puede ser opcional, es completamente obligatoria.

  ```
  FROM <table_origen>
  ```

  Ejemplo:

  ```
  SELECT *

  FROM ALUMNOS
  ```

En este ejemplo hemos realizado una selección de todos las columnas de la tabla ALUMNOS.

- **La cláusula WHERE.**

  La clausula WHERE, se utiliza para realizar criterios de selección. Esta clausula determina mediante una condición de búsqueda, limitando las filas de los datos que se van a devolver.

  Si la consulta que queramos realizar no necesita condiciones, esta clausula se puede omitir, nos es obligatoria, es opcional.

  Para definir las condiciones en la clausula WHERE, podemos utilizar algunos de los operadores con los que cuenta SQL como los operadores de comparación y los operadores lógicos.

  La sintaxis es la siguiente:

  ```
  WHERE <condicion_busqueda>
  ```

El argumento condición_busqueda limita las filas devueltas en el conjunto de resultados mediante el uso de predicados, no existen limites en el numero de predicados que se puedan incluir en una condición de búsqueda.

Ejemplo:

```
SELECT *

FROM ALUMNOS

WHERE dni=´ 47859665b´;
```

En este ejemplo lo que obtendremos es una selección de todas las columnas de la tabla ALUMNOS donde el campo dni es igual a 47859665b.

Cuando vamos a comparar una cifra o una cadena de caracteres concreta siempre se ponen entre ´´.

- **La cláusula GROUP BY.**

Esta clausula determinara como se va a agrupar el conjunto de datos resultantes de la consulta. Permitirá agrupar un conjunto de columnas con valores repetidos y también dejara de utilizar funciones de agregación sobre las columnas con valores no repetidos.

Esta cláusula también es opcional.

Su sintaxis es la siguiente:

```
GROUP BY <nombre/s_columna/s>
```

Ejemplo:

```
SELECT *

FROM ALUMNOS

GROPU BY dni;
```

Este ejemplo nos dará como resultado un listado con todos los campos de la tabla alumnos, pero agrupados por el DNI.

- La cláusula HAVING.

  La cláusula HAVING se utiliza para determinar una condición para la búsqueda de un grupo, normalmente esta clausula es utilizada junto con la clausula **GROUP BY**, que será la clausula que nos dará esos ciertos grupos en los resultados de la consulta.

  Al igual que la cláusula **WHERE**, también se utiliza una condición de búsqueda para concretar los grupos de datos que se desean.

  La clausula **HAVING** se encargará de limitar la selección de los grupos, según los valores de las funciones agregadas que se hayan utilizado, esto es debido a que la clausula **WHERE** solo realizará la selección de las filas individuales.

  Esta clausula también es opcional en la sentencia **SELECT**.Su sintaxis es la siguiente:

```
HAVING <condicion_busqueda_GROUP BY>
```

  Ejemplo:

```
SELECT dni, nombre, salario

FROM PROFESORES

GROUP BY dni

HAVING SUM (salario)>12000
```

  En este ejemplo nos dará como resultado de la consulta un listado e los nombres, dni y salaries de los profesores que estarán agrupados por dni y que aparezcan los empleados que han cobrado mas de 12000.

En esta cláusula se podrán utilizar las funciones agregadas.

- La cláusula ORDER BY.

  Esta clausula determina el orden de la columna o columnas de datos resultantes.

  La clausula **ORDER BY** no se puede utilizar con vistas, funciones, tablas que se originan ni en subconsultas, salvo que se le ponga **TOP**.

  TOP, limitará las filas que se devuelven al primer número N de filas ordenadas.

Esta clausula también es opcional en la sentencia **SELECT**. Su sintaxis es la siguiente:

```
ORDER BY <nombre/s_columna/s><ASC|DESC>
```

Si se pone **ASC** se ordenará de forma ascendente y si se pone **DESC** se ordenará de forma descendente.

Ejemplo:

```
SELECT nombre,apellidos
FROM ALUMNOS
ORDER BY apellidos.
```

Este ejemplo dará como resultado de la consulta, un listado con los nombre y apellidos de los alumnos ordenados por el apellido.



Con algunas cláusulas necesitaremos de la ayuda de operadores para tratar las consultas.

En las cláusulas que van a utilizar condiciones, necesitaremos operadores y funciones agregadas para poder realizarlas.

Los tipos de operadores de condiciones son:

- Operadores de comparación.
- Operadores lógicos.
- Operadores aritméticos.

- **Operadores de comparación:** Comparan un valor con otro, el resultado del operador será TRUE o FALSE.

Los operadores de comparación son:

=	Significa igual a.
>	Significa mayor que.
<	Significa menor que.
>=	significa mayor que o igual que.
<=	Significa menor que igual que.
<>	Significa distinto a.
¡=	Significa no es igual a
¡<	Significa no es menor que
¡>	Significa no es mayor que.

- **Operadores lógicos:** prueban si una condición es cierta o no. Devuelven el tipo de datos

  Booleano True o False.

  Los operadores lógicos son:

  - BETWEEN.
  - IN.
  - LIKE.
  - IS NULL/ NOT IS NULL.
  - ANY, SOME Y ALL.
  - EXISTS/NOT EXISTS.

- **BETWEEN.**

  Se utiliza para expresar una condición en la que se quiere encontrar un valor que se encuentre entre unos límites concretos.

La sintaxis seria la siguiente:

```
SELECT nombre_columna

FROM table_consulta

WHERE BETWEEN limite1 AND limite2
```

- **IN.**

Servirá para comprobar si un valor coincide con los elementos de una lista y para ver que no coincide se utilizara **NOT IN**.

La sintaxis seria la siguiente:

```
SELECT nombre_columnas

FROM table_consulta

WHERE columna [NOT] IN (valor1….,valorn;
```

- **LIKE.**

Servirá para comprobar si una columna de tipo carácter cumple alguna propiedad determinada. Pondremos % para buscar una secuencia de caracteres.

La sintaxis seria la siguiente:

```
SELECT nombre_columnas

FROM table_consulta

WHERE columna LIKE características;
```

- **IS NULL y IS NOT NULL.**

Se utilizara para comprobar si un valor es nulo y también para averiguar si no lo es.

La sintaxis seria la siguiente:

```
SELECT nombre_columnas

FROM table_consulta

WHERE columna IS [NOT] NULL;
```

- ANY, SOME Y ALL

  Para ver si una columna cumple con todas sus filas ALL, o algunas filas ANY YSOME que satisfagan una condición.

  La sintaxis es la siguiente:

```
SELECT nombre_columnas

FROM table_consulta

WHERE columna_operador_comparacion [ALL|ANY|SOME] subconsulta;
```

- EXISTS|NOT EXISTS

  Se utilizara para comprobar si una subconsulta produce alguna fila de resultados. Comprobara si es existe o no existe NOT EXITS algún dato concreto.

  La sintaxis es la siguiente:

```
SELECT nombre_columnas

FROM table_consulta

WHERE [NOT] EXISTS subconsulta;
```

- **Operadores aritméticos:** realizaran operaciones aritméticas. Son:

+	Seria la suma.
–	Sería la resta.
*	Seria la multiplicación.
/	Seria la división
%	Que devolverá el resto entero de una división.



También para realizar estas clausulas necesitaremos la ayuda de las funciones agregados.

Las funciones agregadas son:

- SUM.
- AVG.
- COUNT.
- MAX.
- MIN.

- **MAX y MIN**, con estas funciones agregadas se podrá seleccionar el valor máximo y minimo de una columna.

  Ejemplo de SELECT con la función MAX y MIN:

```
SELECT MAX  (sueldo)
FROM empleados
WHERE sueldo = ´1200´;
```

- **COUNT**, con este tipo de función agregada se podrán cuantificar el número de elementos de una columna, siempre devolverá un número entero.

  Ejemplo de SELECT con la función COUNT:

```
SELECT COUNT(*) AS cantidad_pro
FROM productos
WHERE nombre_producto = ´pantalla´;
```

– **AVG**, con este tipo de función agregada se podrá devolver la media de los valores de una columna, los valores NULL no cuentan.

  Ejemplo de SELECT con la función AVG:

```
SELECT AVG (precio)

FROM producto

WHERE nombre_producto = ´pantalla´;
```

– **SUM**, con este tipo de función agregada se podrán sumar los valores que tienen una determinada columna.

  Ejemplo de SELECT con la función SUM:

```
SELECT SUM (cantidad)

FROM libros

WHERE editorial = ´anaya´;
```

7.3. Inserción de datos INSERT

Una vez creada una base de datos, vamos a poder realizar inserciones, modificaciones y borrados de los valores de las filas de las tablas.



Para insertar datos en una tabla de una base de datos, se utilizara la sentencia:

```
INSERT
```

Antes de realizar consultas de los datos, se necesita introducir datos en las tablas.

La sentencia **INSERT** se utiliza para introducir datos en una o varias filas de una tabla, puede introducir datos en una fila y dejar o no vacío algunos campos.

Existen dos formas de insertar filas en una tabla:

Inserción individual de las filas	Inserción multiple de las filas



– **Inserción individual de las filas.** La sintaxis de la sentencia es la siguiente:

```
INSERT INTO nombre_tabla [(atributos_tabla)] VALUES (lista_valores);
```

- **Con nombre_tabla**, nos referiremos al nombre de la tabla a que la queremos insertar una fila.

- **Con atributos_tabla**, nos vamos a referir a los atributos que tiene la tabla, a cada atributo se le asigna un valor.

  Esto es opcional no tienen porque ponerse obligatoriamente. Pero siempre tienen que estar ordenados igual que en la lista de valores, para que se introduzca correctamente en cada columna a la que corresponde. Si no se enumeran todas las columnas de la tabla , se insertara el valor NULL, o el valor predeterminado si se ha definido anteriormente alguna para la columna.

- **La lista_valores**, contendrá los valores de cada campo de la fila que se quiera insertar, el dato siempre irá entre comillas simples ´´. Se podrán insertar los valores de cada columna de una tabla.

  Los valores suministrados deben de corresponderse con los atributos_tabla. El número de la lista de valores de los datos, debe ser el mismo que el numero de columnas y el tipo de datos, precisión y escala de cada valor de datos deben de coincidir con los de la columna correspondiente.

- La clausula VALUES, se utilizara para especificar los valores de datos para una fila.



La cláusula **VALUES**, especifica los valores de una fila de una tabla. Los valores se especifican, como una lista separada por comas cuyo tipo de datos, precisión y escala deben de ser iguales a la columna correspondiente de la lista de columnas.

Si no se especifica una lista de columnas, los valores deberán especificarse en el mismo orden que las columnas de la tabla.

O podría ser a si:

```
INSERT INTO ALUMNOS

 VALUES (´78950432q´,´Manuel´, ´Gómez Rico´, ´Avenida Velázquez n 8´)
```

O así: (Ejemplo).

```
INSERT INTO ALUMNOS (dni,nombre,apellidos,dirección)

VALUES (´78950432q´,´Manuel´, ´Gómez Rico´, ´Avenida Velázquez n 8´);
```

En este ejemplo lo que hemos hecho ha sido introducir los datos de un alumno en la tabla alumnos, en el que cada valor entre comillas, se asignará a cada una de las columnas en el orden en el que lo hemos escrito.

Si se intenta realizar una inserción tenemos que tener en cuenta si existe una restricción de la clave foránea, si existiera deberíamos primero rellenar la tabal referenciada o eliminar la clave foránea de la tabal.



El propósito de las lista de columnas en la sentencia INSERT es hacer corresponder los valores de los datos en la clausula VALUES, con las columnas que van a recibir estos valores.

Las lista de valores y la lista de columnas deben de contener el mismo numero de elementos y el tipo de dato de cada valor debe de ser compatible con el tipo de datos de la columna correspondiente, o en caso contrario se producirá un error.

El estándar ANSI/ISO, nos va a exigir nombres de columna sin cualificar en la lista de columnas, pero muchas implementaciones permiten nombre cualificados.

No debe de existir ambigüedad ninguna en los nombres de columnas, ya que deben de referenciar todos a columnas de la tabla destino.

Para comprobar que se han introducido todos los datos en la tabla de forma correcta, realizamos un **SELECT**.

```
SELECT * FROM ALUMNOS
```



Cuando se inserta una nueva fila de datos en una tabla, automáticamente asigna el valor NULL a cualquier columna cuyo nombre falte en la lista de columnas de la sentencia INSERT.

Por ejemplo:

```
INSERT INTO ALUMNOS (dni, nombre, apellidos, dirección)

VALUES (´78950432q´,´Manuel´, ´Gómez Rico´);
```

La tabla quedaría de la siguiente manera.

Dni	Nombre	Apellidos	Dirección
8578963b	Benito	Pérez Gómez	Plaza bahía mar s/n
74856859v	Alberto	Guzmán muñoz	Calle alcanzadilla 13
78950432q	Manuel	Gómez rico	NULL

Como podemos ver en la columna dirección, como no hemos especificado ningún valor, se asignara el valor NULL, por defecto.

La asignación de valores NULL, se puede puntualizar, incluyendo las columnas en la lista y especificando la palabra NULL en los propios valores, este seria un ejemplo.

```
INSERT INTO ALUMNOS (dni, nombre, apellidos, dirección)

VALUES (´78950432q´,´Manuel´, ´Gómez Rico´, NULL);
```

Por comodidad el sistema gestor de bases de datos, permitirá introducir la lista de valores de las columnas sin tener que especificar la lista de columnas, esto será mucho mas cómodo, que estar poniendo en cada instrucción todo.

La inserción de los valores se hará de izquierda a derecha.

Ejemplo:

```
INSERT INTO ALUMNOS

 VALUES (´78950432q´,´Manuel´, ´Gómez Rico´, ´Avenida Velázquez n 8´)
```

Lógicamente , cuando no se pone la lista de columnas , la palabra clave NULL, también podrá ser utilizada como en el caso anterior, se le asignara el valor NULL a la columna que se quiere que este vacio o se desconoce el dato.



Inserción múltiple de filas.

Para conseguir insertar varias filas necesitaremos de la ayuda de la sentencia **SELECT**.

La sintaxis es la siguiente:

```
INSERT INTO nombre_tabla lista_columnas

SELECT ( lista de_columnas_tabla)

FROM nombre_tabla
```

Para poder realizar la inserción de muchas filas la lista de campos de las sentencias **INSERT** Y **SELECT** deben ser completamente iguales y no se pueden incumplir las reglas de integridad.

La subconsulta SELECT en la instrucción INSERT, se puede utilizar para agregar valores en una o más tablas. Utilizar la sentencia SELECT, permitirá además que se inserte mas de una fila al mismo tiempo.

La lista de columnas del SELECT debe de coincidir con la lista de columnas de la sentencia INSERT. Si no se especifica ninguna lista de columnas, la lista del SELECT debe de coincidir con las columnas de la tabla en la que se inserta.

Vamos a poner un ejemplo en el que queremos ver el dni, nombre y nota de los alumnos que sea Sobresaliente. La inserción se realizaría de la siguiente manera en una tabla llamada nota_alumnos.

Ejemplo:

```
INSERT INTO NOTA_ALUMNO

SELECT dni, nombre,apellidos,dirección,nota

 FROM ALUMNOS

WHERE dni =´NOTABLE´;
```

Como resultado nos dará una tabla Con el nombre NOTA_ALUMNO, que contendrá el dni, nombre, apellidos, dirección y nota de los alumnos que hayan obtenido una nota de sobresaliente.

- INTO, es una palabra clave opcional que se puede utilizar entre el INSERT y la tabla destino.
- El argumento nombre_tabla, es el nombre de la tabla que va a recibir los datos.
- La instrucción SELECT, que devolverá las filas con los datos que se van a introducir en la tabla.

7.4. Modificación de datos UPDATE.



Para modificar o actualizar datos de una o varias filas de una única tabla de una base de datos, solo podrá cambiar o modificar los datos de una tabla a la vez, se realiza mediante la sentencia:

UPDATE

Con modificación o actualización nos referimos a cambiar el valor de un atributo por otro valor.

La sintaxis mas sencilla de la sentencia UPDATE es la siguiente:

```
UPDATE nombre-tabla

SET columnas=valores,.....n

WHERE condición
```

- **SET**, contiene una lista separada por comas de las columnas que deben de actualizarse y el valor de cada columna, con el formato columna=valor. El valor al que igualamos incluye elementos como constantes, valores seleccionados de una columna de otra tabla o vista o valores calculados mediante un expresión compleja.

- **WHERE**, con esta clausula se especificara la condición de búsqueda que define las filas de las tablas o vistas de origen que estén calificadas para proporcionar valores para la cláusula SET.

La clausula **WHERE** es opcional, y si se omite , entonces se actualizaran todas las filas de la tabla destino.

Ejemplo:

```
UPDATE ALUMNOS

SET apellidos=´González Martínez´

WHERE dni=´75477530R´
```

En este ejemplo modificaremos el valor de apellidos del alumno con el dni que se ha especificado de la tabla **ALUMNOS**.

Una sentencia **UPDATE** sin la clausula **WHERE** es un poco inútil, ya que se efectuara una actualización masiva de toda la tabla.

Cuando se trabaja con la sentencia **UPDATE**, las subconsultas **SELECT** permitirán seleccionar filas para actualizase en función de la información contendida en otras tablas.



Hay que tener en cuenta siempre la integridad de los campos **PRIMARY KEY**. Solo se podrán modificar los registros que no estén referenciados en ninguna tabla.

7.5. Eliminación de datos DELETE.



La forma de eliminar filas de una tabla o vista pueden ser:

- DELETE
- TRUNCATE



Para eliminar filas de datos en tablas o vistas ya existentes la sentencia básica a utilizar es:

```
DELETE
```

La sintaxis simplificada de la sentencia DELETE es la siguiente:

```
DELETE FROM nombre_tabla

WHERE condición
```

– El argumento nombre_tabla, que también podrá ser una vista, nombre la tabla o vista en la que se va a eliminar la fila o filas. Se eliminaran todos los datos de las filas que cumplan la condición **WHERE**. Si no se especifica esta clausula **WHERE**, se eliminarán todas las filas de la tabal o vista.

  La clausula **WHERE** es opcional.

  Ejemplo:

```
DELETE FROM ALUMNOS

WHERE nombre= ´Juan´ AND apellidos =´Benítez Rodríguez´;
```

En este ejemplo lo que haremos será borrar de la tabla alumnos, el alumno con el nombre Juan y de apellidos Benítez Rodríguez.

Si hubiésemos utilizado la sentencia **DELETE** de esta otra forma:

```
DELETE FROM ALUMNOS;
```

En este ejemplo borrara todas las filas de la tabla ALUMNOS. Cualquier tabla a la que se le hayan eliminado sus filas sigue permaneciendo en la base de datos.

La instrucción **DELETE** solo eliminara las filas de la tabla, y esta para ser eliminada por completo, se deberá utilizar la sentencia **DROP**.

La subconsulta **SELECT**, también se podrá utilizar, y esta junto con la sentencia **DELETE**, se podrá utilizar para eliminar valores de más de una fila al mismo tiempo.

A veces, la selección de las filas a borrar contiene datos de otras tablas que se nombrarán dentro de la subconsulta **SELECT**.

Vamos a poner un ejemplo en el que se quiere eliminar un alumno de un curso , porque se ha dado de baja.

Ejemplo:

```
DELETE FROM ALUMNOS
        WHERE dni_alumno=(SELECT cod_curso
                FROM cursos
                WHERE cod_curso=´192´);
```

Las subconsultas en la clausula **WHERE** de la sentencia **DELETE**, pueden anidarse al igual que sucedía en la clausula **WHERE** de la sentencia **SELECT**.

También pueden contener referencias externas a la tabla destino de la sentencia **DELETE**, con la finalidad de implementar las tablas de la subconsulta y la tabla destino de la sentencia **DELETE**.

La única restricción para el uso de las subconsultas en un sentencia **DELETE**, es que la tabla destino no puede aparecer en la clausula **FROM** de la subconsulta.



Solo se podrán borrar datos de una sola tabla, cuando borramos datos una vista, también será borrado de la tabla, ya que las vistas son solo una manera de ver los datos, no se trata de una copia.

Un problema común que se puede dar es que se intente borrar un registro de una tabla referenciada por **FOREING KEY** como tabla maestra, esto producirá un error y no se podrá realizar la sentencia **DELETE** de eliminación.

Eliminar filas con:

```
TRUNCATE
```

La eliminación de filas con la sentencia **TRUNCATE** es un método rápido eliminar todas las filas de una tabla sin registrar las eliminaciones de cada fila individualmente.



Casi siempre es más rápido que utilizando la instrucción **DELETE** sin condiciones, ya que **DELETE** registra cada eliminación de filas y **TRUNCATE TABLE** libera inmediatamente todo el espacio ocupado por los datos de esa tabla.

Como en el caso de la sentencia **DELETE**, la definición de una tabla vaciada con **TRUNCATE TABLE**, permanece en la base de datos, pero seguirán todos sus datos asociados, a diferencia de **DELETE**. Para quietar la definición de una tabla se debe de utilizar la sentencia **DROP**.

La sintaxis de la sentencia **TRUNCATE** es la siguiente:

```
TRUNCATE TABLE nombre_tabla;
```

No se puede utilizar **TRUNCATE TABLE** en una tabla a la que haga referencia una restricción **FOREING KEY**, en su lugar se utiliza la instrucción **DELETE** sin una clausula **WHERE**.

Como **TRUNCATE TABLE** no se registra, no se podrá activar algún desencadenador y no se podrá hacer ninguna recuperación **ROLLBACK**.

TRUNCATE TABLE tampoco puede utilizarse en tablas que formen parte de una vista indizada.

Para que sea mas efectivo y rápido la eliminación de datos de una tabla o vista deberemos utilizar mejor la sentencia **TRUNCATE**.

La siguiente sentencia elimina todas las filas de la tabla ALUMNOS.

```
TRUNCATE TABLE ALUMNOS;
```

7.6. Agregación de conjuntos de datos para consulta: JOIN, UNION



Muchas veces también se hace necesario realizar consultas con más de una tabla, haciendo combinaciones de columnas de tablas diferentes.

Se tendrá que poner en la cláusula **FROM** todas las tablas involucradas en la consulta. La combinación lo que hará es crear una tabla nueva a partir de varias tablas que serán concretadas en la clausula FROM, tendrán que coincidir los valores de las columnas relacionadas de estas tablas.

Si se trabaja con más de una tabla, puede pasar que el nombre de las columnas de las tablas sean iguales, para diferenciarlos se deberá especificar obligatoriamente a que tabla pertenecen, para distinguirlas se haría lo siguiente:

```
Nombre_tabla.nombre_columna
```

Ejemplo:

```
SELECT clientes.cod_cliente, articulos.cod_articulo

FROM clientes, artículos
```

Tipos de combinaciones

- JOIN.
- UNION.

Un **JOIN** es lo mismo que decir consulta combinada. Las consultas combinadas pueden ser de varios tipos:

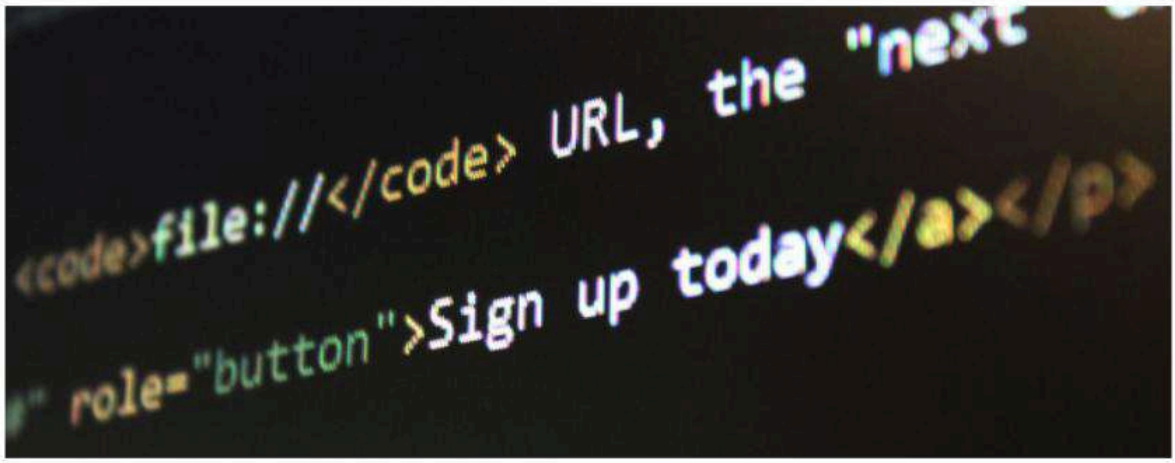
- INNER JOIN.
- JOIN.

Importante

- Combinación interna o INNER JOIN.

  JOIN es una agregación de datos para consultas combinadas, la sentencia **JOIN** tiene la capacidad para que en una sola sentencia pueda consultar múltiples tablas a la vez, esto se lleva a cabo porque existe igualdad entre las columnas de las diferentes tablas que se van a unir.

  Este tipo de **JOIN** lo que realizará es una consulta en la que nos mostrará los datos de dos o mas tablas por medio de una condición **WHERE**.



Para llevar a cabo una consulta combinada entre tablas, tendremos que realizar una consulta **SELECT**, en la que para diferenciar los campos de cada una de las tablas tendremos que poner la tabla de la que proviene seguido de un punto y el campo con el que se va a igualar.

La sintaxis de la sentencia INNER JOIN es la siguiente:

```
SELECT nombre/s_campo/s

FROM nombre_tabla

INNER JOIN nombre_tabla ON condición_combinacion
```

Se pueden poner opcionalmente las clausulas: WHERE, GROUP BY, HAVING y ORDER BY.

Ejemplo:

```
SELECT alumnos.dni.
        Alumnos.nombre,
        Alumnos.apellidos,
        Alumnos.direccion,
        Alumnos.edad,
FROM ALUMNOS , ASIGNATURAS INNER JOIN ALUMNOS ON ALUMNOS
WHERE Alumnos.dni=asignaruras.dni
```

En este ejemplo de combinación de tablas **INNER JOIN**, lo que obtenemos es un listado de los alumnos de la tabla alumnos mas la asignatura que estudia cada uno de ellos proveniente de la tabla asignatura.



- **Combinación externa o JOIN.**

  Este tipo de combinación de tablas, nos permitirá ver los resultados de los campos que son coincidentes en ambas tablas, los campos que no lo son no aparecerán en los resultados.

  La sintaxis de la sentencia JOIN es la siguiente:

```
SELECT nombre/s_campo/s
FROM nombre_tabla
LEFT |RIGHT JOIN nombre_tabla ON condición_combinacion
```

Y se pueden añadir también a la sentencia **JOIN** las clausulas **WHERE, GROUP BY, HAVING y ORDER BY.**



Los JOIN pueden ser de dos tipos:

- LETF.
- RIGHT.

- **Con LEFT JOIN.**

  Lo que hará es que permitirá obtener un listado de los campos que sean tanto coincidentes como si no.

  En caso de que una columna no tenga ningún valor aparecerá como **NULL**.

  Ejemplo:

```
SELECT alumnos.dni.
        Alumnos.nombre,
        Alumnos.apellidos,
        Alumnos.direccion,
        Alumnos.edad,
FROM ALUMNOS , ASIGNATURAS
LEFT JOIN ALUMNOS ON ALUMNOS Alumnos.dni=asignaruras.dni
```

Este tipo de combinación de tablas, nos permitirá ver los resultados de los campos que son coincidentes en ambas tablas y los campos que no lo son.



En caso de que una columna no tenga ningún valor aparecerá como NULL.

- RIGHT JOIN.

En este caso se devolverá todos los valores de las columnas tanto de una como de otra tabla, aunque algunas columnas no tengan valor, también aparecerán con el valor NULL.

Ejemplo:

```
SELECT alumnos.dni.
        Alumnos.nombre,
        Alumnos.apellidos,
        Alumnos.direccion,
        Alumnos.edad,
FROM ALUMNOS , ASIGNATURAS
RIGHT JOIN ALUMNOS ON ALUMNOS Alumnos.dni=asignaruras.dni
```

- UNION:

La sentencia UNION se utiliza para poder combinar el resultado de varias sentencias, como para tratarla como si fuera una tabla única.

La sintaxis de la sentencia UNION es la siguiente:

```
SELECT nombre/s_campo/s
FROM nombre_tabla
LEFT |RIGHT JOIN nombre_tabla ON condición_combinacion
UNION
SELECT nombre/s_campo/s
FROM nombre_tabla
LEFT|RIGTH JOIN nombre_tabla ON condición_combinacion
```

Las clausulas **WHERE**, **GROUP BY, HAVING y ORDER BY** son cláusulas opcionales dentro de la sentencia **UNION**, también podrán aparecer.

Ejemplo:

```
SELECT asignaturas.id_asignatura
        Asignatura.nombre,
        Asignatura.horas,
        Asigantura.dni
FROM ASIGANTURAS
INNER JOIN ALUMNOS ON alumnos.dni=asignatiras.dni
UNION
SELECT profesor.dni
        Profesor.nombre,
        Profesor.apellidos
FROM PROFESOR
INNER JOIN ASIGNATURAS ON profesor.dni=asignaturas.dni
```

En este ejemplo primero nos aparecerá como resultado una tabla con los alumnos de la tabla **ALUMNOS**, con las asignaturas que estudian de la tabla **ASIGNATURAS** y otra columna con los datos de los profesores que imparten esa asignatura.

7.7. Subconsultas.

Importante

Se denomina subconsulta a una sentencia **SELECT** que esta dentro de otra sentencia **SELECT**.

Suele aparecer cuando hay que filtrar datos en las clausulas **WHERE**, **HAVING** y en las listas de selección, estos son claros ejemplos de subconsultas.

Un a subconsulta tiene la misma sintaxis que la sentencia **SELECT**, las subconsultas no pueden tener una clausula **ORDER BY** y no se puede realizar la **UNION**.

A veces dentro del cuerpo de una subconsulta se hace referencia al valor de una columna que esta en el **FROM** de la consulta principal.

Las subconsultas pueden adaptarse a muchas condiciones de comparación.

Dependiendo del operador de comparación que se necesite, estará permitido un tipo de subconsulta u otra, en función del tipo de datos que la subconsulta va a devolver.

Importante

Una subconsulta puede dar como resultado una fila y una columna, muchas filas y una sola columna o múltiples filas con múltiples columnas.





Los operadores de comparación son los siguientes:

- **Simples**, que correspondería a los siguiente operadores: =, ¡=.
- **LIKE**, este tipo de operador lo que hace es realizar una búsqueda basadas en un patrón determinado.

  Los patrones son los siguientes:

  - **`a-z´**, si se pone por ejemplo el carácter ´a´, realizara una búsqueda de los datos que comiencen por la letra a.
  - **`_%´**, si se ponen letras delante del tanto por ciento buscará todos los datos que comiencen por esas letras.
  - **`%_´**, si se ponen letras detrás del % , buscara aquellos datos de una columna que terminen por esas letras.
  - **`%_%´**, si se ponen caracteres entre dos %, buscara los datos que contengan las letras previamente definidas.
- **De pertenencia:** que sería **NOT** o **IN**.
- **De existencia**, que sería **EXIST** y **NOT EXIST**.
- **De rango**, que sería **BETWEN** y **AND**, con este se limita el rango de selección de datos.
- **De agrupación**, que son: **=, ¡=, >,<,>=,<=.**

Los operadores que devuelven una sola fila son : **=, ¡=, >,<,>=,<=, LIKE y BETWEN .**

Los operadores que devuelven múltiples filas son: **IN, NOT IN, NOT EXIST, EXIST.**



La sintaxis de una subconsulta es el siguiente:

```
SELECT nombre/s_columna/s
FROM nombre_tabla
WHERE nombre/s_columna/s operador_comparacion
(SELECT nombre/s_column/s
FROM nombre_tabla2
WHERE (condicion);
```

Ejemplo en que se devuelve una única fila:

```
SELECT *
FROM ESTUDIANTES
WHERE dirección=( SLECT dirección FROM ESTUDIANTES
  WHERE dni = `3387543 a´);
```

Un ejemplo en el que se devuelve múltiples filas:

```
SELECT *
FROM PROFESORES
WHERE dirección IN ( SELECT dirección
      FROM ESTUDIANTES
      WHERE edad > 25);
```

Ejemplo en el que se devuelven múltiples columnas:

```
SELECT *
FROM PROFESORES
WHERE( nombre, apellidos )IN ( SELECT nombre,apellidos
       FROM ESTUDIANTES
         WHERE edad > 25);
```

UD7
Lo más importante

- Los elementos que conforman este lenguaje son las sentencias: SELECT. INSERT UPDATE DELETE

- La sentencia SELECT, nos permitirá realizar consultas de datos sobre cualquier tabla de la base de datos. Nos dará como resultado el listado de las columnas de las tablas seleccionadas.

- Existen una serie de clausulas que pueden formar parte de la sentencia SELECT. Una cláusula son condiciones para filtrar y manipular los datos de una o varias tablas. Las clausulas que se pueden combinar junto a la sentencia SELECT son seis: INTO,FROM,WHERE,GROUP BY,HAVING y ORDER BY.

- La cláusula FROM. Esta cláusula se utiliza para hacer referencia a la tabla o tablas sobre la que se va a realizar la consulta y de donde obtendremos las filas resultantes de la consulta.

- La cláusula WHERE La clausula WHERE, se utiliza para realizar criterios de selección. Esta clausula determina mediante una condición de búsqueda, limitando las filas de los datos que se van a devolver.

- La cláusula GROUP BY. Esta clausula determinara como se va a agrupar el conjunto de datos resultantes de la consulta. Permitirá agrupar un conjunto de columnas con valores repetidos y también dejara de utilizar funciones de agregación sobre las columnas con valores no repetidos.

- La cláusula HAVING. La cláusula HAVING se utiliza para determinar una condición para la búsqueda de un grupo, normalmente esta clausula es utilizada junto

con la clausula **GROUP BY**, que será la clausula que nos dará esos ciertos grupos en los resultados de la consulta.

- **La cláusula ORDER BY.** Esta clausula determina el orden de la columna o columnas de datos resultantes.

- La sentencia **INSERT** se utiliza para introducir datos en una o varias filas de una tabla, puede introducir datos en una fila y dejar o no vacío algunos campos.

- Para modificar o actualizar datos de una o varias filas de una única tabla de una base de datos, solo podrá cambiar o modificar los datos de una tabla a la vez, se realiza mediante la sentencia: **UPDATE**

- Para eliminar filas de datos en tablas o vistas ya existentes la sentencia básica a utilizar es: **DELETE**.

- **TRUNCATE TABLE** libera inmediatamente todo el espacio ocupado por los datos de esa tabla.

- Un **JOIN** es lo mismo que decir consulta combinada. Las consultas combinadas pueden ser de varios tipos **INNER JOIN yJOIN**.

- **JOIN** es una agregación de datos para consultas combinadas, la sentencia **JOIN** tiene la capacidad para que en una sola sentencia pueda consultar múltiples tablas a la vez, esto se lleva a cabo porque existe igualdad entre las columnas de las diferentes tablas que se van a unir.

- **Combinación externa o JOIN**. Este tipo de combinación de tablas, nos permitirá ver los resultados de los campos que son coincidentes en ambas tablas, los campos que no lo son no aparecerán en los resultados.

- **UNION**. La sentencia UNION se utiliza para poder combinar el resultado de varias sentencias, como para tratarla como si fuera una tabla única.

- Se denomina subconsulta a una sentencia **SELECT** que esta dentro de otra sentencia **SELECT**.

  Suele aparecer cuando hay que filtrar datos en las clausulas **WHERE**, **HAVING** y en las listas de selección, estos son claros ejemplos de subconsultas.

  Un a subconsulta tiene la misma sintaxis que la sentencia **SELECT**, las subconsultas, no pueden tener una clausula **ORDER BY** y no se puede realizar la **UNION**.

UD7
Autoevaluación

1. ¿Qué es el lenguaje de manipulación de datos?.

    a. Es el lenguaje mediante el cual uno o varios usuarios pueden realizar operaciones de creación de estructuras de datos de una base de datos.

    b. Es el lenguaje mediante el cual uno o varios usuarios pueden realizar operaciones de consulta.

    c. Es el lenguaje mediante el cual uno o varios usuarios pueden realizar operaciones de consulta y modificación de los datos.

    d. Es el lenguaje mediante el cual uno o varios usuarios pueden realizar operaciones de consulta, modificación y eliminación de los datos.

2. Las sentencias que forman el lenguaje DML son:

    a. SELECT, INSERT y DELETE.

    b. INSERT, DELETE y UPDATE.

    c. SELECT, INSERT, DELETE y UPDATE.

    d. SELECT, INSERT y UPDATE.

3. Las clausulas que se pueden combinar junto a la sentencia SELECT son:

    a. INTO, FROM, GROUP BY, HAVING y ORDER BY.
    b. INTO, WHERE, GROUP BY, CREATE y HAVING.
    c. WHERE, GROUP BY, FROM y HAVING.
    d. INTO, WHERE, FROM, GROUP BY, HAVING y ORDER BY.

4. ¿Qué sentencia se utiliza para insertar datos en una tabla?

    a. UPDATE.
    b. DELETE.
    c. INSERT.
    d. DROP.

5. ¿Qué tipo de operador es IS NULL/ NOT IS NULL?

    a. Es un operador lógico.
    b. Es un operador de comparación.
    c. Es un operador aritmético.
    d. Ninguna de las anteriores es correcta.

6. La función agregada AVG…

    a. Selecciona el valor máximo de una columna.
    b. Selecciona el valor mínimo de una columna.
    c. Cuantificara el número de elementos de una columna.
    d. Devolverá la media de los valores de una columna.

7. La sentencia UPDATE se utiliza para …

    a. Insertar una fila o varias en un una tabla.
    b. Modificar datos de una o varias filas de una tabla.
    c. Actualizar datos de una o varias filas de una tabla.
    d. B y c son correctas.

8. Para eliminar toda la definición de una tabla se utiliza la sentencia:

    a. DELETE.
    b. DELETE y TRUNCATE.
    c. DROP.
    d. TRUNCATE.

9. ¿Qué tipo de combinaciones de tablas existen?

    a. JOIN y UNION.
    b. JOIN.
    c. COMMIT.
    d. COMMIT y UNION

10. La sentencia UNION puede llevar las siguientes clausulas:

    a. WHERE, GROUP BY y HAVING.
    b. ORDER BY, WHERE y GROUP BY.
    c. ORDER BY y HAVING.
    d. WHERE, GROUP BY, HAVING y ORDER BY.

Área: informática y comunicaciones

Glosario

- **Browser o navegador web:** interpretan información de archivos y de sitios web para que estos puedan ser leídos.

- **Pagina web:** se define como un documento electrónico el cual contiene información en forma de texto, de elementos visuales y de elementos sonoros, que se encuentran alojados en un servidor y se pueden acceder a ellas por medio de un navegador.

- **Arquitectura CLIENTE/SERVIDOR**, en el que por un lado se encuentra el cliente que está constituido por browser web, que serán los encargados tanto de mostrar como de solicitar información o documentos a través de una red.

- **Cliente:** Es una aplicación ejecutable que permitirá el acceso a internet y serán capaces de mostrar y solicitar documentos sobre una red. Suelen ser las computadoras o cualquier otro dispositivo móvil que sirva como soporte para la entrada y salida de datos. Se puede decir que el cliente es el consumidor de un servicio.

- **Servidores Web:** Los servidores web son aquellos cuya tarea es alojar sitios web y aplicaciones web, las cuales son accedidas por los clientes utilizando un navegador web que se comunica con el servidor por medio de un protocolo llamado HTTP.

– Arquitectura Cliente/Servidor de tres capas:

  Este tipo de arquitectura se basa en el modelo Cliente/Servidor, pero además se le añade una capa más entre el cliente y el servidor, que será un servidor de aplicaciones, que es donde se encontrara la lógica de la aplicación.

  Se compone de capa de presentación, capa lógica y capa de la base de datos.

– **TCP/IP.** Es el protocolo más básico de internet, suministra todos los servicios necesarios para el transporte de los datos. Cualquier protocolo de internet se basa en IP.

– **Protocolo HTTP:** Es el protocolo por defecto que se utiliza en la arquitectura Web basado en Cliente/Servidor, este protocolo no guarda nada referente a conexiones.

– **Lenguaje HTML (HyperText Markup Language):** Este tipo de lenguaje es el utilizado para definir la estructura y el contenido de una página web. Es un lenguaje de hipertexto que está formado por un conjunto de etiquetas que van a marcar cuando comienza y acaba cada elemento, mediante estas es posible insertarse forma estructurada textos, imágenes, scripts, etc.

– **Objeto:** entidad que tiene un estado, un comportamiento y una identidad propios.

– **Clases:** Podemos definir una clase como una estructura por medio de la cual podemos representar un grupo de objetos del mundo real, los objetos pueden ser cosas, personas, animales, etc.

– **JDBC:** que es un driver para conectar bases de datos con aplicaciones realizadas en Java.

– **ODBC:** que es un driver más común para conectar con bases de datos.

– **Middleware:** actuara como conductor entre sistemas permitiendo que cualquier usuario pueda comunicarse con varias fuentes de información que se van a encontrar conectados por una red. Va a ser el intermediario entre el cliente y el servidor y se ejecutara en ambas partes.

– **Modelo vista controlador MVC:** es un patrón arquitectónico para unificar las aplicaciones web.

- **APPLETS DE JAVA:** Un applet es una programa que se ha escrito en Java y forman parte de los componentes de una agina web.

- **Base de Datos Relacional:** es una base de datos que utiliza el modelo de estructura relacional, este modelo es el más utilizado en la actualidad para poner en funcionamiento unas bases de datos ya planificadas.

- **Entidad:** Es la representación de un objeto en el mundo real, que se puede distinguir del resto.

- **Grado de la relación:** número de entidades que integran una relación .

- **Cardinalidad** es el tipo de correspondencia que se establecerá entre las entidades de la relación. Se refleja el número de ocurrencias de una entidad con las ocurrencias de la otra entidad con la que existe una asociación.

- **La API**, es el interfaz de programación de aplicaciones, en inglés Application Programming Interface, es un grupo de funciones y procedimientos que contiene una biblioteca, que podrá ser usado por otro software como capa de abstracción.

Área: informática y comunicaciones

Área: informática y comunicaciones

Anexo

UF2405: Modelo de programación web y bases de datos

Área: informática y comunicaciones

Tipos de servidores.

- **Servidor de archivos:**

Este tipo de servidor se encarga de almacenar todo tipo de archivos y distribuirlos por la red, son muy típicos en la red de una organización o empresa.

- **Servidor de impresiones:**

Este servidor se ocupara del control de una o varias impresoras de forma remota y también aceptara impresiones de cualquier cliente de la red, estas impresiones se pondrán en cola según el orden de llegada o en orden de importancia.

- **Servidor de correo:**

Este tipo de servidor permitirá enviar correos de unos usuarios a otros, estos usuarios pueden estar tanto dentro como fuera de la red.

- **Servidor de Fax:**

Este tipo de servidor realizara las mismas funciones que el servidor de correo enviara faxes de unos a otros usuarios de una red. Se encargara de transmitir, de recibir y distribuir debidamente los fax.

- **Servidor de telefonía:**

Este tipo de servidor realiza funciones relacionadas con la telefonía, como puede ser la de un contestador de automático, realiza las funciones de respuesta de la voz, almacenando mensajes de voz, etc.

- **Servidor proxy:**

Este tipo de servidor realiza un tipo de funciones a nombre de otros clientes en la red, se realiza para que la actividad sea mayor en ciertas operaciones, también facilita el acceso a internet en una red, permitiendo o no el acceso a ciertos sitios web, además de proporcionar seguridad.

- **Servidor de web:**

Este tipo de servidor almacena contenido como, imágenes, archivos de texto, documentos HTML,etc, que serán distribuidos a los clientes que lo pidan por medio de la red, a través del protocolo HTTP.

- **Servidor del acceso remoto:**

Este tipo de servidor se encargar de controlar las líneas del router o los canales que utilice para comunicarse en la red de una posición remota, responderá a las llamadas telefónicas, entrantes, realizara la autentificación para registrar a un usuario en la red.

- **Servidor de audio/video:**

Este servidor permitirá dar capacidad multimedia a un sitio web, permite ver el contenido multimedia desde el servidor sin interrumpirse, se podrá ver videos o escuchar música sin necesidad de descargar los archivos.

- **Servidor de Chat:**

Este servidor permitirá el intercambio de información entre usuarios , con la posibilidad de poder comunicarse en tiempo real.

- **Servidos FTP:**

Este servidor permitirá el intercambio de datos entre diferentes cliente /servidor, sin importar donde se ubique cada uno de ellos.

- **Servidor Groupware:**

Este tipo de servidor provee herramientas para que colaboren entre si los usuarios, grupos organizaciones o empresas, para que colaboren entre sí de forma útil y eficaz., esto es posible

gracias a la información compartida, que hará que trabajen juntos independientemente de donde estén los usuarios.

- **Servidor IRC:**

Este tipo de servidor es otra opción para los usuarios que buscan discusión en tiempo real, consistirá en distintas redes separadas que va a permitir que los usuarios se conecten unos a otro por medio otra red IRC.

- **Servidor de noticia:**

Este tipo de servidor servirá como una fuente y entrega para los grupos de noticias públicos, se realiza a través de la red de noticias USENET.

- **Servidor de aplicaciones:**

Este tipo de servidor servirá para proporcionar servicios que va a soportar la ejecución de varias aplicaciones desplegadas. Este tipo de servidor proporcionara una estructura de tres capas por lo que el sistema será mucho máseficiente. Esun gran sistema distribuido, lo utilizaran empresas que necesiten un alto rendimiento.

- **Servidor de base de datos:**

Este tipo de servidor dará servicios de base de datos a a otros programas u ordenadores , como el modelo Cliente/Servidor.

- **Servidor de Listas:**

Este tipo de servidor proporcionara una manera mejor de manejar las listas de correo electrónico.

- **Servidor de reserva:**

Es un servidor de reserva, contendrá todo el software de la que está instalada, contara con grandes unidades de almacenamiento servirá para asegurarnos de que si pasa algo con el servidor principal no afecte a la red.

Se pueden dividir en:

  - **servidor dedicado** son aquellos que administran los recursos de la red, atenderá las solicitudes de los clientes.
  -
  - **Servidor no dedicado**, no dedican su potencia a los clientes, procesan solicitudes de un usuario local.
  -

- **Servidores telnet:**

Este tipo de servidor permitirá a los usuarios el acceso a ordenadores huésped y realizar tareas como si estuviera trabajando en ese ordenador.

ARQUITECTURA DE OBJETOS DISTRIBUIDOS

A diferencia del modelo cliente-servidor en un sistema distribuido, tanto los clientes y los servidores tiene funciones diferentes. Los clientes reciben la respuesta por medio de peticiones a los servidores y no de otros clientes.

Los servidores parece que actúan como los clientes al solicitar información de otro servidor, pero no lo son.

Este modelo funciona muy bien para múltiples aplicaciones de negocio. Pero a su vez, lo hace menos flexible para los diseñadores del sistema ya que ellos son los que concretaran donde se suministrar cada servicio.

También tendrán que decidir cómo distribuir la carga entre los servidores.
La arquitectura de objetos distribuidos lo que hace es eliminar las diferencias entre cliente y servidor y diseñar la arquitectura del sistema como una arquitectura de objetos distribuidos.

En una arquitectura de objetos distribuidos el sistema se basa en objetos que son los que van a dar una interfaz a un conjunto de servicios que ellos mismos suministran.

Otros objetos realizan llamadas a estos servicios sin hacer ninguna distinción lógica entre un cliente y un servidor

Estos objetos serán repartidos en varios ordenadores a través de una red y comunicarse a través de un middleware, este actuara de intermediario de las peticiones de objetos. Su misión es proporcionar una interfaz transparente entre los objetos. Suministrara la comunicación entre los objetos y podrá añadirlos y eliminarlos del sistema.

Las ventajas del modelo de objetos distribuidos son las siguientes:

- ✓ Proporcionar al diseñador del sistema la decisión de donde se sitúa la lógica de aplicación de los objetos.
- ✓ Es una arquitectura muy abierta admite añadir nuevos recursos si es necesario.
- ✓ El sistema es flexible y escalable
- ✓ Un objeto que proporciona servicios puede migrar al mismo procesador que los objetos que demandan los servicios, en lo que mejora el rendimiento del sistema.

Una arquitectura de objetos distribuidos se puede utilizar como un modelo de sistema que se puede estructurar y organizar. Se deben identificar los servicios que se quieren prestar utilizando los objetos distribuidos.es un modelo cliente-servidor, pero tanto los clientes como los servidores se implementan como objetos distribuidos que se comunican a través de un bus software.

Otros lenguajes menos utilizados hoy en día:

LENGUAJES	DESCRPCION
CGI	Es el sistema de programación más antiguo para el desarrollo de páginas dinámicas del servidor. Esta caso obsoleto. Es un sistema de comunicación entre el Clientey un servidor web.Las aplicaciones que se ejecutan en el servidor reciben el nombre de **CGIs**. Las aplicaciones CGI fueron las primeras en crea contenido dinámico para las páginas web. En una aplicación CGI, el servidor pasa las solicitudes del cliente a un programa externo. Este programa puede estar escrito en cualquier lenguaje que soporte el servidor, aunque por razones de portabilidad se suelen usar scripts La salida de dicho programa es enviada al cliente.
PERL	Se trata de un lenguaje de programación de no compilación, no necesitará de ningún interprete para ser descifrado. Además, se puede ampliar con subprogramas que se hayan realizados en otros tipos de lenguajes, igualmente es reciproco, otros lenguajes podrán utilizar PERL.
DHTML	En realidad no es un lenguaje en si, sino consiste en una serie de técnicas para crear páginas web. Utiliza una combinación de lenguaje HTML estático (del lado del cliente), el lenguaje de hojas de estilo CSS y la jerarquía de objetos DOM.
XML	Se trata de un lenguaje de marcas muy sencillo que se utiliza para guardar datos. Da soporte a las bases de datos, por eso es muy útil para aplicaciones que necesitan comunicarse entre sí.

TIPOS DE ESTRUCTURAS DE BASE DE DATOS.

- **Estructura jerárquica :**

  Esta corresponde a la que hemos visto en la unidad 4, en el que se basa en la representación en forma de árbol. En el que se va descendiendo a los datos por medio de niveles, desde el mas alto hasta el más bajo.

- **Estructura en Red:**

  Esta estructura también ha sido estudiada en la lección 4, en el que desaparece la jerarquía y aparecen más uniones entre entidades.

- **Estructura transaccionales:**
  Este es un tipo poco utilizado, consiste en el envío y recepción de datos a gran velocidad, se da en el trato de datos de producción o industriales, no importa la duplicación y la redundancia de datos.

- **Estructura de bases de datos relacionales.**
  Este tipo de estructura fue estudiada en la lección 4, es la más utilizada, permitirá resolver los problemas de redundancia, en este tipo de estructura se elimina totalmente las jerarquías entre los campos, pudiendo utilizarlos como clave de acceso.

- **Estructura de datos multidimensionales:**

  Se utiliza para el desarrollo de aplicaciones muy concretas, no se diferencian mucho de la estructura relacional, se diferencial a nivel de diseño conceptual, los tipos de atributos solo pueden ser de dos clases.

- **Estructuras orientadas a objetos:**

  Este, también fue dado en la leccion4, es un modelo muy nuevo, almacena en la base de datos los objetos con un estado y un comportamiento.

- **Estructura de datos deductiva:**

  Se trata de un sistema de base de datos que se basa en reglas y hechos que se almacena en la base de datos, se basa en una lógica matemática, este tipo de estructura aparece como respuesta a las limitaciones con las bases de datos relacional a la hora de manejar consultas recursivas.

Operaciones para conectar una base de datos desde una aplicación creada con JAVA utilizando el driver JDBC.

El Java Database Connectivity (JDBC) API es el estándar industrial para la conectividad de base de datos independiente entre el lenguaje de programación Java y una amplia gama de bases de datos? Bases de datos SQL y otras fuentes de datos tabulares, como hojas de cálculo o archivos planos. El API JDBC proporciona una API de nivel de llamada de acceso de base de datos basada en SQL.

Tecnología JDBC le permite utilizar el lenguaje de programación Java para explotar capacidades para aplicaciones que requieren acceso a los datos empresariales. Con un controlador habilitado para la tecnología JDBC, puede conectar todos los datos corporativos, incluso en un entorno heterogéneo.

El driver JDBC se trata de una API que se utiliza para poder ejecutar sentencias SQL. El driver JDBC se trata de un conjunto de clases e interfaces que han sido programadas en el lenguaje de programación JAVA, mediante el uso de este driver se podrá interactuar con la base de datos.

Se trata de programar el acceso a la base de datos desde JAVA, siendo independiente del tipo de base de datos que se vaya a utilizar. Mediante JDBC se realizan llamadas directamente a SQL.

Existen drivers para la mayoría del los sistemas gestores de bases de datos , se trata de un software totalmente libre y de código abierto.

Para poder utilizar la API de JDBC con un sistema gestor de base de datos , se hará necesario la utilización del driver concreto para ese motor de base de a datos, este servirá de intermediario entre el driver JDBC y la base de datos.

Dependiendo de las especificaciones del driver , podrán estar escritos en diferentes lenguajes de programación con lo que se podrá interactuar por la llamada interface nativa de java o también pueden estar escritos completamente con JAVA.

En la ultima versión que se desarrollo de la API de JDBC también contenía un nuevo puente para poder conectar sistemas gestores de bases de datos ODBC, que va a ser el mas utilizado para en entornos de Microsoft. Todos estos driver son estándares de ANSI e ISO.

Antes de realizar cualquier operación en la base de datos se necesita registrar el controlador.la interfaz de JDBC se encuentra en la librería java.sql, deberemos de importar todas la clases definidas en esta librería a la aplicación JAVA.

La clase DriverManager es la encargada de realizar la conexión, lo hace entregando una clase llamada Conection.el destino de la conexión se concreta especificando la URL de JDBC con el que se podrá acceder a la base de datos, la clase DriverManager se ocupara de cargar todo lo referente a la conexión.

SISTEMAS GESTORES DE BASES DE DATOS.

Los sistemas gestores de bases de datos, se trata de un tipo de software especifico, estará dedicado completamente a ser utilizado de interfaz entre las bases de datos, los usuarios y las aplicaciones que la utilizaban.

Este tipo de software se utiliza para manejar de forma clara y sencilla las bases de datos, para que esto sea así, lo que intentan conseguir es obtener una abstracción de la información, la consistencia , la seguridad o el tiempo de respuesta a las peticiones que se realicen.

Mediante los sistemas gestores de bases de datos se consigue proveer de lenguajes e interfaces que va a simplificar todo lo referente a la recuperación de datos de la base de datos, además nos permitirá manejar gran cantidad de datos.

Existen varios tipos de sistemas gestores de bases de datos: los que son de código abierto o los que son de código propietario.

Como ejemplo de sistema gestor de base de datos con código abierto y multiplataforma es MySQL. Para este tipo de sistema gestor de base de datos existen una multitud de APIs que pueden trabajar con lenguajes de programación diferentes.

Como ejemplo de sistema gestor de base de datos de código propietario se encuentra SQL Server. SQL server es un sistema de administración y análisis de datos relacionales, este también proporciona compatibilidad con documentos XML y HTTP, además de funciones mas avanzadas sobre la administración de la base de datos, que servirán para facilitar tareas que sean habituales.

Con SQL server también se podrán implementar funciones web y de comercio electrónico incluyendo XML enriquecido y soporte estándar de Internet. Mediante estos se obtendrá un acceso seguro y bastante sencillo a través de la web.

Este sistema gestor de base de datos también proporcionará una amplitud de herramientas de análisis integradas, servicios de transformación y duplicación de datos, funciones de programación y herramientas de depuración.

Es un sistema cliente-servidor para administrar bases de datos relacionales con el que se podrá realizar transacciones en línea.

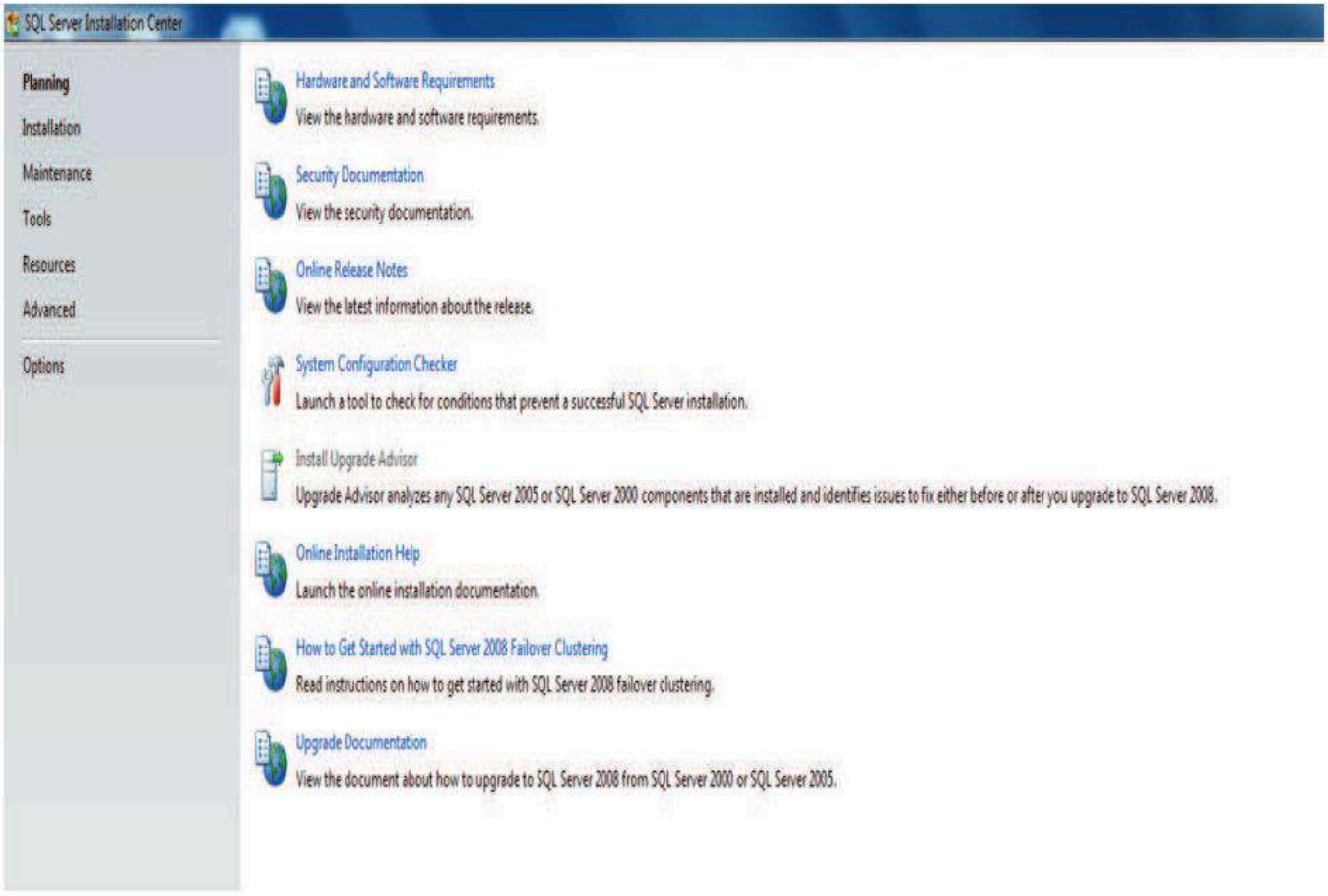
Requisitos y pasos para la instalación del sistema gestor de base de datos SQL.

En primer de instalar SQL server, veremos los requisitos que se necesitan para llegar a instalar el programa con éxito.

- Se necesitara un ordenador o equipo con un procesador mínimo si es x86 de 1,0 GHz o superior. Si es x64, 1,4 GHZ o superior.
- Se necesitara un espacio en memoria RAM dependiendo si son ediciones Express u toras ediciones. Con ediciones Express tendrá que ser de 512 MB como mínimo y en todas las demás ediciones de 1 GB como mínimo. En ambos casos si es mayor mejor será el rendimiento.
- Se necesitara un disco duro con una memoria mínima de unos 32 GB o superior , el máximo puede ser el que se quiera.
- Se necesitara una pantalla para visualizar el programa, necesitara una resolución mínima de 800x600 o superior.
- Los sistemas operativos a utilizar, dependerá del sistema operativo con el que se trabaje, todas las versiones de SQL irán bien con el sistema operativo Windows NT.

Instalación de SQL SERVER 2008.

Para instalar SQL server 2008 se necesitara una unidad de CD ROM,, se introducirá el disco y aparecerá una ventana con el asistente para la instalación . Aparecerán las siguientes opciones:



Aquí encontraremos las opciones para la instalación. Pincharemos en la opción Installation, y se volverá a pinchar en, una instalación nueva de SQL, a partir de aquí ira pidiendo la instalación de los diferentes componentes y se verá , si el proceso se está produciendo con éxito o no en todo momento. Si en algún momento no se pudiera instalar algún componente, el programa de instalación ira advirtiendo de los procesos que han fallado durante la esta.